

Custom Compiler™ Schematic Editor and Text Editor User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	15
Related Products, Publications, and Trademarks	15
Conventions	16
Customer Support	17
Statement on Inclusivity and Diversity	17

1. About the Schematic Editor	19
Schematic Editor Description	19
Schematic Editor User Interface	20
Schematic Editor Tools and Utilities	21
Schematic Editor Assistants	25
Schematic Editor Toolbars	25
Annotation Toolbar	27
Check Toolbar	27
Command Options Toolbar	27
Editing Toolbar	28
Hierarchy Toolbar	28
Mouse Bindings Toolbar	28
Navigation Toolbar	29
Object Info Toolbar	29
Overlay Toolbar	29
PVE Toolbar	29
Schematic Capture Toolbar	30
Selection Toolbar	30
Standard Toolbar	30

2. Getting Started	32
Starting the Schematic Editor	32
Schematic Entry Workflow	32
General Schematic Entry Workflow	34
Example of Schematic Entry Workflow	36

3. Customizing the Environment	38
Setting Options for the Command Use Model	38
Object Orientation	38
Infix Mode	40
Creating Shapes With Infix Mode Enabled	41
Editing Objects With Infix Mode Enabled	41
Angle Mode for Creating and Editing Objects	41
Schematic Editor Options	43
Setting Display Options	44
Setting Design Options	49
Setting General Options	51
Symbol Editor Options	51
Setting Display Options	51
Setting Design Options	52
Setting General Options	53
Managing Label Visibility	54
Specifying Default Preferences	55

4. Creating Schematics	56
Creating Instances	57
Adding Instances	58
Add Instance Toolbar and Dialog Box Options	59
Add Instance Menu Options	62
Naming Instances	63
Stacking Instance Names	63
Naming Iterated Instances	64
Adding Instances Using Arrayed Placement	64
Default Naming	65
Creating Wires	65
Understanding Jog Styles	66
Naming Wires	67
Stacking Wire Names	67
Bus Notation	68
Named Bus Tapping	69
Justification and Orientation	71
Adding Individual Wires	71
Adding Groups of Wires	73

Contents

Many-to-Many	74
One-to-Many	75
Adding Wire Stubs	77
Adding Wire Names	80
Setting Display Mode for Bundle Names	82
Parameterized Connections on Wires	82
Understanding Connectivity With Wires	83
Wire-by-Name	83
Wire-to-Iterated Instance	83
Creating Global Net Names	84
Creating Pins	85
Naming Pins	86
Pin Name Stacking	86
Unnamed Pins	87
Adding Single Pins	87
Adding Multiple Pins Using Name Stacking	87
Adding Multiple Pins Using Iterated Names	88
Using Command Options	88
Add Pin Toolbar and Dialog Box Options	89
Add Pin Menu Options	90
Creating Connection Definitions	90
Creating Connection Assignments	94
Adding a Connection Definition	97
Defining the Label Location	98
Adding a Connection Assignment	99
Creating Shapes	99
Adding Lines	100
Adding Rectangles	100
Adding Polygons	100
Adding Ellipses and Circles	101
Adding Arcs	101
Add Shape Toolbar Options	102
Add Shape Menu	102
Creating Blocks	102
Wiring Blocks	103
Wiring From a Block to a Pin	103
Wiring From a Pin to a Block	104
Add Block Toolbar Options	105
Add Pin to Block Dialog Box Options	105

Contents

Creating Sheets	105
Creating Constraints	111
Annotating Designs	112
Labels	113
Creating Labels	114
Creating Custom Labels	116
Notes	117
Creating Notes	118
Constraint Notes	119
Creating Constraint Notes	119
Images	121
Tables	122
Creating Tables	122
Editing Tables	123
Connecting Components and Nets	124
Wiring Symbol Pins	124
Adding Patchcords	125
Understanding Connectivity With Patchcords	126
Creating Patchcords	127
Drawing Rulers	127
Drawing Linear Rulers	129
Drawing Single-Segment Linear Rulers	130
Drawing Multisegment Linear Rulers	130
Create Ruler Toolbar Options	131
Create Ruler Menu and Shortcut Keys	131
Deleting Rulers	131
5. Using Templates	132
Creating Templates	132
Placing Templates in a Schematic Design	138
6. Creating and Using Schematic Overlays	144
About Schematic Overlays	144
Getting Started With Overlays	146
Schematic Overlay Menu and Toolbar Options	147
Working With Schematic Overlays	149

Editing an Overlay Design	149
Using Masks in Schematic Overlays	151
Creating Area Masks	152
Creating Clip Masks	155
Using a Mask to Create an Editable Copy of the Reference	156
Updating the Reference Design With Changes From an Overlay Design	157
Opening the Reference Schematic Design	158
Comparing the Overlay With the Reference	158
Updating the Overlay Design With the Latest Reference Design	160
Library Management of Overlay Design Views	161
Binding Overlay Config Views	161
Controlling View Bindings With the Hierarchy Editor	162
Clearing Overlay View Bindings	163
Saving Overlay Designs	164

7. Generating Cellviews 165

8. Creating Symbol Views From Designs 167

Creating Symbols From Schematics	167
Specifying Destination Cellviews	172
Creating a Cellview From a Pin List	172
Creating a Cellview From a Component	173
Creating a CellView From a File	174
Updating a Cellview	175
Pin Name Mapping Table	176
Updating Symbols	177
Generator Option Sets	178

9. Creating Symbols 183

Creating a Symbol Manually	184
Using Symbol Editor Toolbars	184
Importing Existing Symbol Graphics	185
Adding Symbol Pins	186
Naming Pins	188
Symbol Pin Name Stacking	188
Creating Bused or Bundled Pins	188

Contents

Selecting Pin Shapes	189
Specifying Connections	189
Updating Pin Labels	190
Adding Selection Shapes	190
Creating Selection Shapes Automatically	190
Creating Selection Shapes Manually	191
Redefining the Symbol Origin	192
10. Editing Schematics	193
Undo and Redo	195
Direct Object Manipulation	195
Moving Objects	196
Moving Objects in the Current Cellview	198
Moving Objects Between Cellviews	199
Stretching Objects	200
Stretch Toolbar Options	203
Stretch Menu	203
Aligning Objects	204
Aligning Preselected Objects	206
Aligning a Set of Preselected Objects	206
Aligning to an Individual Preselected Object	208
Aligning Post Selected Objects	208
Align Toolbar Options	210
Copying Objects	211
Creating Multiple Copies	212
Copy Toolbar Options	213
Copy Menu	214
Using the Clipboard	214
Cutting Objects to the Clipboard	215
Copying Objects to the Clipboard	215
Pasting Objects From the Clipboard	216
Paste Toolbar Options	217
Deleting Objects	217
Editing Canvas Text	217
Editing a Cellview in Place	218

Contents

Rotating and Flipping Objects	219
Connecting and Disconnecting Crossing Wires	219
Connecting Crossing Wires	220
Disconnecting Crossing Wires	220
Chopping Wires	221
Colorizing Nets	223
Colorize Toolbar Options	224
Selecting Objects and Areas	225
Selecting Single Objects	225
Selecting Multiple Objects	226
Selecting Objects By Region	226
Selecting Objects By Name	227
Adding and Removing Selected Objects	228
Selecting All Objects	228
Enabling Partial Area Selection	228
Filtering Selections	229
Enabling Undo and Redo Selection	230
Renumbering Instances	230
Renaming Instances	232
Ignoring Instances	233
Ordering and Synchronizing Ports	234
Ordering Ports	235
Sorting Individual Port Names	235
Sorting Multiple Port Name Collections	236
Sorting All Port Names	236
Expanding and Creating Bus Names	236
Adding Port Names	236
Editing Port Names	237
Moving Port Names Between Lists	237
Synchronizing Port Order Between Views	238
 11. Editing Parameters	 239
On-Canvas Parameter Editing	239
Filtering Parameters	240
Editing Cellview Properties	241
Editing Instance Attributes and Properties	241
Editing Wire Properties	242

Editing Pin Attributes	242
Editing Text Attributes	242
12. Exploring the Design	243
Searching for Objects	243
Reviewing Search Results	244
Color-Coding Search Results	244
Displaying Label Attachments	244
Adding Probes	244
Using the Add Probe Command	245
Add Probe Dialog Box and Toolbar Options	245
Add Probe Menu	249
Removing Probes	250
Highlighting Signals From WaveView	250
13. Assistants for Schematic Designing	251
Deciding Which Assistant to Use	251
AMS Visualization Assistant	253
Opening the AMS Visualization Assistant	254
Highlighting Partitions in View-Based Mode	254
Highlighting Partitions in Simulator-Based Mode	257
Editing Interface Element Parameters	263
Highlighting Partitions in a Text Editor	268
Refreshing Partitions	271
Clearing Partitions	272
Accessing Analog Explanations	272
Device Label Editor Assistant	273
Opening the Device Label Editor Assistant	273
Controlling Device Label Display	273
Editing Attribute Labels	274
Editing Parameter Labels	275
Saving and Loading Device Label Settings	277
Saving Device Label Display Settings	277
Loading Device Label Display Settings	277
Estimated Parasitics Assistant	278
Opening the Estimated Parasitics Assistant	278

Contents

Adding Devices to the Schematic	278
Registering Custom Parasitic Devices and Device Libraries	280
Specifying Parasitic Options to Calculate Capacitance	282
Calculating Capacitance Values and Adding Capacitance Devices	283
Specifying Parasitic Options to Calculate Resistance	290
Calculating Resistance Values and Adding From Layout	291
Calculating Pi Resistance/Capacitance Values	296
Ignoring Estimated Parasitic Devices in a Simulation	299
Adding EMag Devices to the Schematic After Extracting EM Data	299
Marker Browser Assistant	302
Opening the Marker Browser Assistant	303
Viewing Error Markers	303
Showing Error Markers by Severity	304
Showing Error Markers Added by Integrated Tools	305
Updating Error Markers	306
Net Tracer Assistant	306
Opening the Net Tracer Assistant	307
Creating a Probe	308
Viewing a Probed Net	308
Setting Probe Color	309
Clearing Probes	309
Filtering Nets by Name	309
Tracing Through Ignored Devices	310
Tracing Through by Cell Configuration	310
Saving Probe Nets	310
Loading Probe Nets	311
Template Manager Assistant	311
About the Template Manager	312
Displaying the Template Manager Assistant	313
Finding Template Matches	314
Binding Templates to Constraints	316
Managing Bound Templates	317
Recognizing Built-In Circuit Structures in Template Cells	318
Adding Notes to Template Matches	319
 14. Analyzing and Debugging Schematic Designs	 321
Deciding Which Tool or Utility to Use	321

Contents

Annotate Operating Point Utility	323
Compare Designs Utility	327
Using the Compare Designs Utility	329
Using Compare Design Menus	335
Connections Navigator	337
About the Connections Navigator	337
Opening the Connections Navigator	338
Parameter Region	338
Assignments Region	339
Blocking Assignments Region	340
Connections Navigator Toolbar Icons	341
Connections Navigator Toolbar Menus	341
Setting Connections Navigator Options	342
Connections Navigator Dialog Box Options	342
Editing Assignments	342
Traversing the Design	343
Net Colorization Definitions Editor	343
Plot Signal Utility	345
Opening the Plot Signal Utility and Using the Toolbar	345
Choosing Plot Settings	346
Plotting Signals From the Schematic Editor	348
Power Domain Analyzer Utility	350
Opening the Power Domain Analyzer	350
Viewing Power Domain Results	350
Analyzing and Debugging Inherited Connections	352
Configuring View Options	355
Debugging Problems With Inherited Connections	357
Exporting Results to a File	359
Changing Column Display	359
Update Estimated Parasitics Utility	360
15. Editing HDL Formats With the Text Editor	362
Opening an HDL Text Editor View	362
Using Text Editor Toolbars	363
Setting HDL Text Editor View Display Options	364
Setting HDL Text Editor View Design Options	365

Traversing the Hierarchy of HDL Text Editor Views	367
Editing HDL Text Editor Views	368
Annotating HDL Text Editor Views after Running Simulation	369
16. Editing Netlist Format Designs	370
Opening Netlist Views	370
Setting Netlist Editor Design Options	370
Working With a Netlist View	371
17. Checking Designs	373
Electric and Semantic Rule Checking Overview	373
Specifying ERC and SRC Rules	374
Configuring Check Options	375
Managing Markers	378
Setting Up Check Rules	378
Connectivity Checks	379
Physical Checks	381
Namespace Checks	382
Compatibility Checks	383
Parameterized Connection Checks	384
Multisheet Checks	384
Overlay Checks	385
Design Integrity Checks	385
Template Integrity Checks	386
User Checks	386
Running Checks	387
Checking and Saving the Current Design	387
Checking the Current View	387
Checking the Hierarchy	388
Cross-View Checking	388
Cross-View Checking in the Schematic Editor	389
Configuring Check Options	389
Running Checks	390
Cross-View Checking in the Symbol Editor	391
Configuring Check Options	391
Running Checks	392
Viewing Violation Information	392

Contents

Using the Marker Browser	392
Using Probes	393
Working With Hierarchy	393

A. Schematic Editor Shortcut Keys	394
Schematic Editor Function Shortcut Keys	394
Schematic Editor Keypad Shortcut Keys	395
Schematic Editor Number Pad Shortcut Keys	399

About This User Guide

This user guide explains how to use the Custom Compiler Schematic Editor to create electronic schematics.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the Custom Compiler Release Notes on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the Custom Compiler tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to see the documentation for the following related Synopsys products:

- Synopsys® Custom Compiler™
- PrimeSim™ Continuum
- PrimeSim™
- PrimeSim™ Pro
- PrimeSim™ SPICE
- PrimeSim™ HSPICE®
- PrimeSim™ XA

- PrimeSim™ CCK
- PrimeSim™ MOSRA
- VCS® PrimeSim™ AMS
- Synopsys FineSim®
- Synopsys HSPICE®
- Synopsys VCS®
- Synopsys Verdi®
- Synopsys OptoCompiler™
- Synopsys StarRC™
- Mentor Graphics® Calibre®

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.

Convention	Description
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working

environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

About the Schematic Editor

This chapter provides a general overview of the capabilities and features of the Custom Compiler Schematic Editor.

This chapter describes the following topics:

- [Schematic Editor Description](#)
- [Schematic Editor User Interface](#)
- [Schematic Editor Tools and Utilities](#)
- [Schematic Editor Assistants](#)
- [Schematic Editor Toolbars](#)

Schematic Editor Description

This guide describes schematic entry tasks. The Custom Compiler Schematic Editor lets you:

- Place and connect design components in a schematic (schematic entry).
- Edit properties of schematic design components.
- Add pins to define block interfaces for hierarchical designs.
- Verify schematic designs for electrical and semantic correctness.
- Use the Text Editor for language-based design and reuse of text views (Verilog, VHDL).
- Use the Symbol Editor for creating graphical representations of hierarchical designs.

For layout entry tasks, see the *Custom Compiler Layout Editor User Guide*.

For automating routing tasks, see the *Custom Compiler Custom Router User Guide*.

For verification instructions, see the *Custom Compiler™ Design Checking and Physical Verification User Guide*.

For simulation instructions, see the *PrimeWave Design Environment User Guide*.

Schematic Editor User Interface

The Schematic Editor consists of the following components:

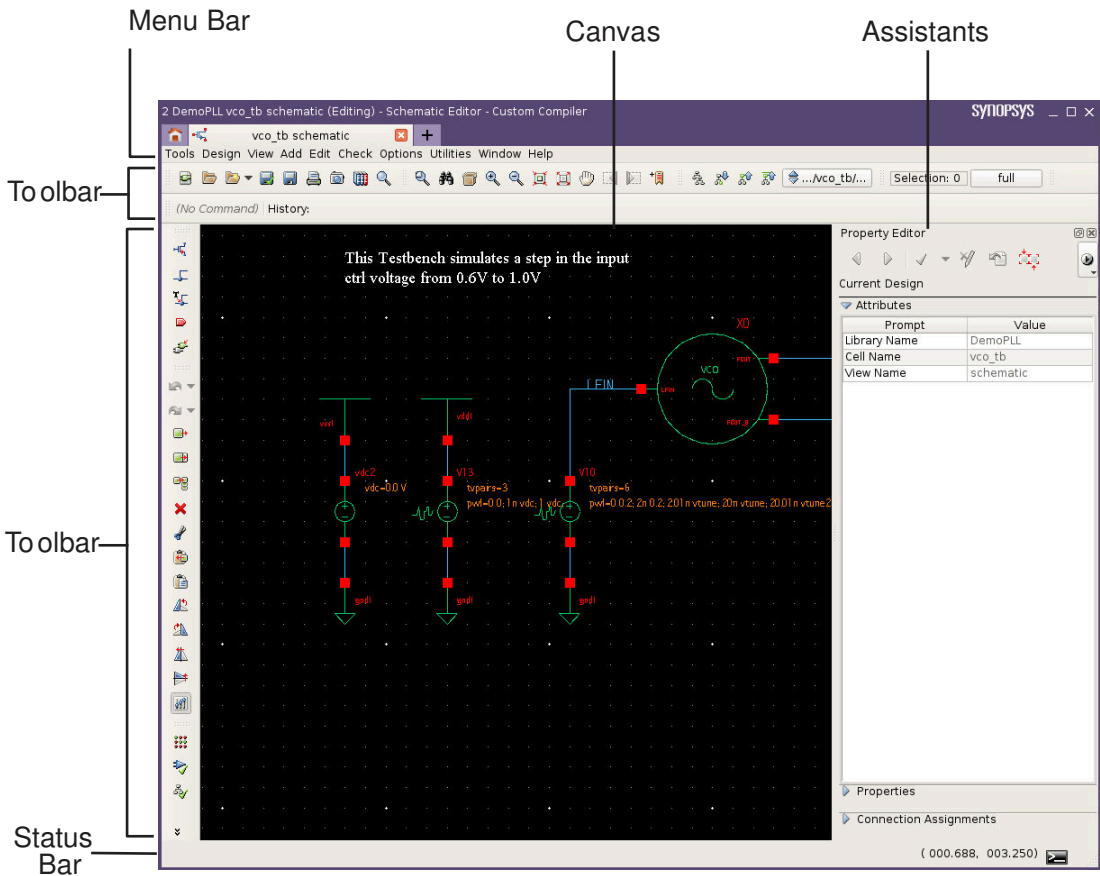


Table 1 lists the components in the Schematic Editor user interface.

Table 1 Schematic Editor Components

Component	Description
Menu Bar	Contains menus of commands that offer quick access to schematic design operations.
Toolbar	Displays button commands and options. For more information, see Schematic Editor Toolbars .
Status Bar	Displays the current state of the software and other information.
Canvas	The area on which you create and route your design.

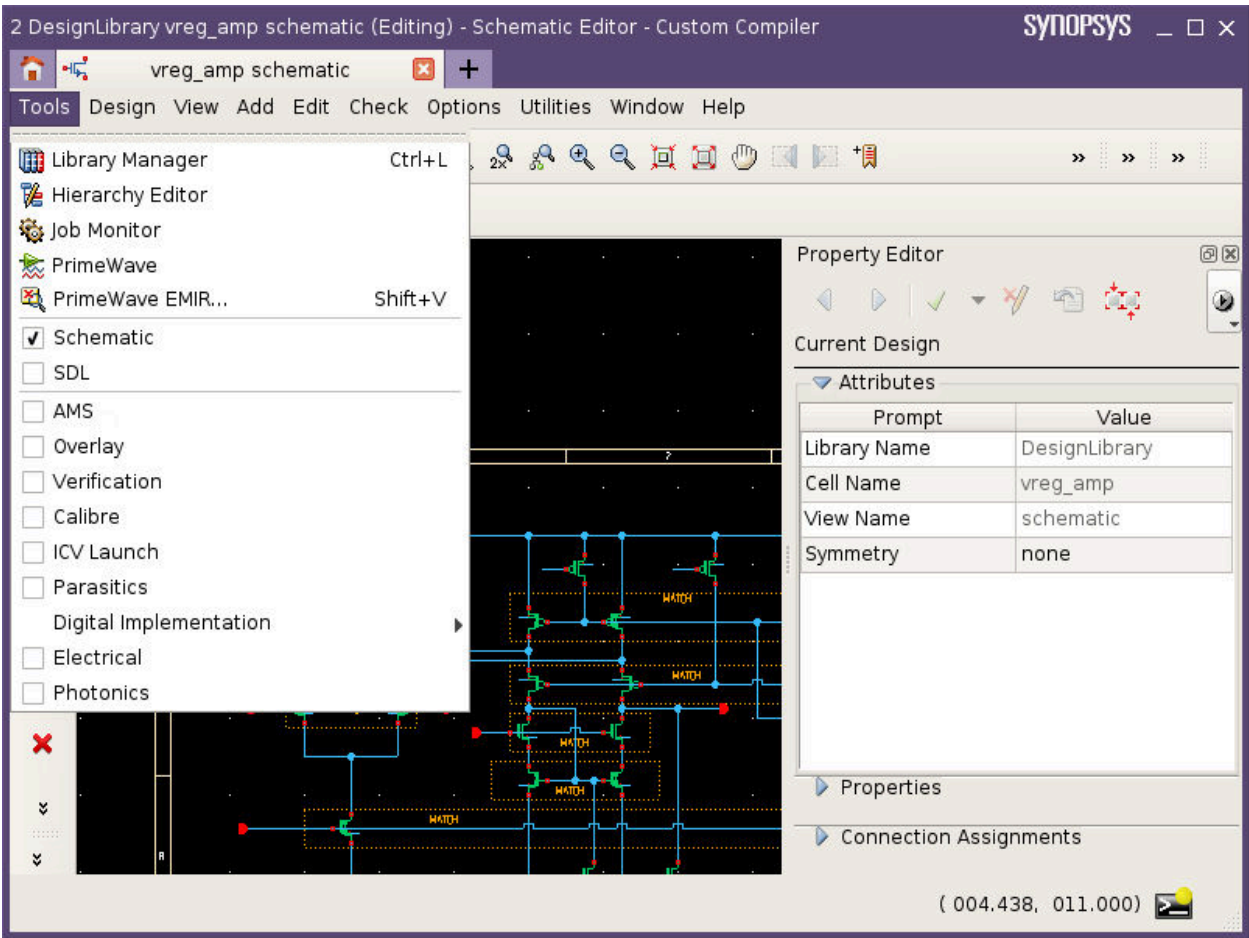
Table 1 Schematic Editor Components (Continued)

Component	Description
Assistants	Aid you with design and routing. For information, see Schematic Editor Assistants .
Tools (Not Shown)	Let you manage or access system-level design components. For information, see Schematic Editor Tools and Utilities .

Schematic Editor Tools and Utilities

The Schematic Editor has a set of tools and utilities that lets you manage or access system-level design components, such as design libraries and technology libraries.

You can display the tools using the commands in the **Tools** menu.



[Table 2](#) lists the tools in the **Tools** menu and briefly describes how each tool assists with designing. Additional tools are described in [Analyzing and Debugging Schematic Designs](#).

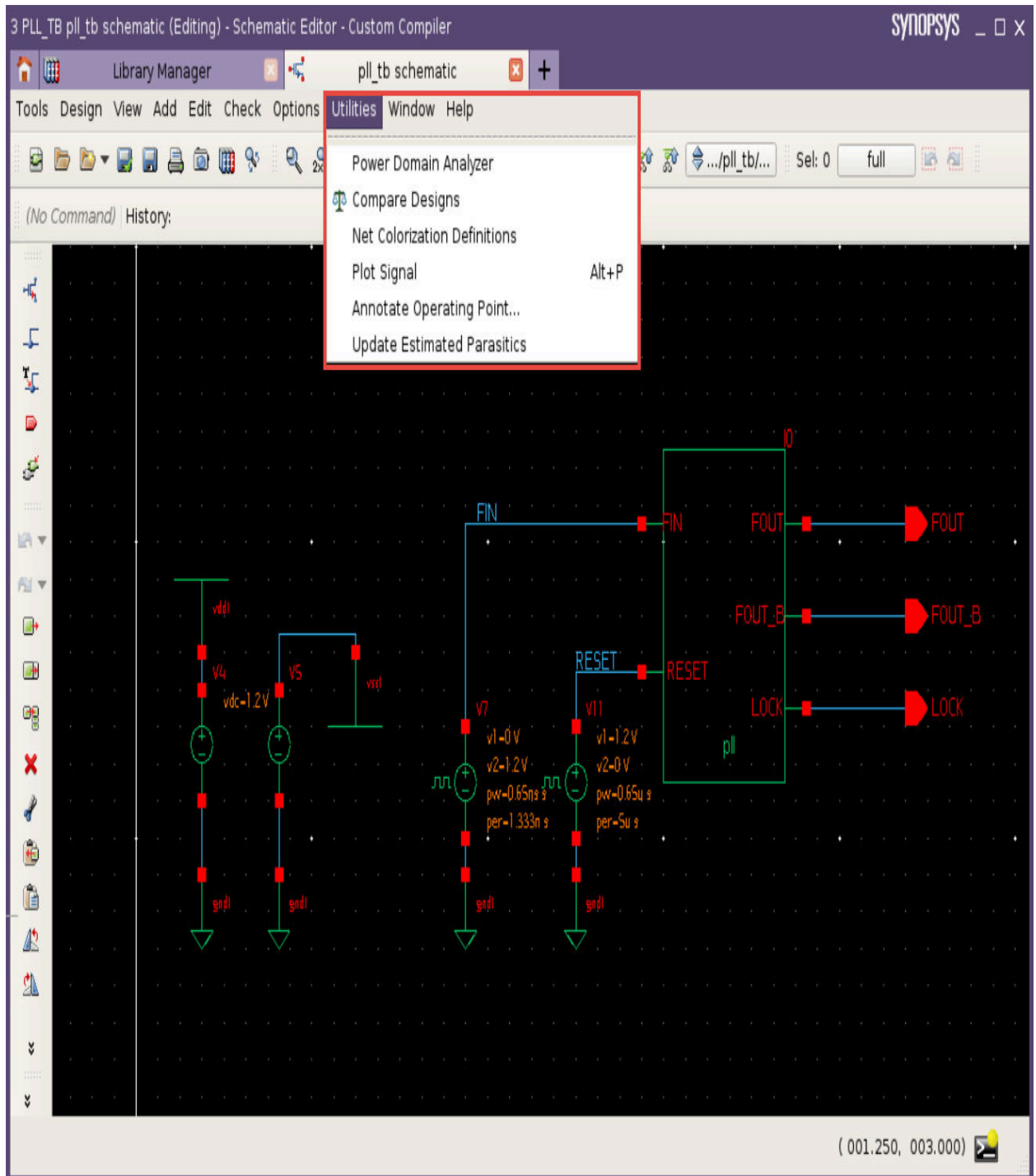
Table 2 *Schematic Editor Tools*

Tool	Description
Library Manager	Creates, and manages, libraries, cells, cellviews, and cell categories. For information, see Library Manager Tool in the <i>Custom Compiler Environment User Guide</i> .
Hierarchy Editor	Defines, manages, and maintains hierarchy configuration views. For information, see Hierarchy Editor Tool in the <i>Custom Compiler Environment User Guide</i> .
Job Monitor	Tracks running job processes. For information, see Job Monitor Tool in the <i>Custom Compiler Environment User Guide</i> .
PrimeWave	Provides a custom environment for netlisting and simulation. Consult the <i>PrimeWave Design Environment User Guide</i> for details on using the PrimeWave Design Environment.
PrimeWave EMIR	Opens the PrimeWave EMIR dialog box for setting up and running Dynamic EMIR analysis and Static Res Check analysis, and viewing the analysis results. Consult the <i>Custom Compiler Electrical Checking and Reporting User Guide</i> for details.
Schematic	
SDL	Opens the Define Physical Target dialog box, with which you can create a layout equivalent of current schematic design. Consult the <i>Custom Compiler SDL and Visually Assisted Automation Guide</i> for details on using SDL.
AMS	Opens the AMS Visualization Assistant. For information, see AMS Visualization Assistant .
Overlay	Adds the Overlay menu to the menu bar. For information, see Creating and Using Schematic Overlays .
Verification	Adds the Verification menu to the menu bar, from which you can perform layout-versus-schematic (LVS) comparisons. Consult the <i>Custom Compiler Design Checking and Physical Verification User Guide</i> for details on LVS checks.

Table 2 Schematic Editor Tools (Continued)

Tool	Description
Calibre	Adds the Calibre menu to the menu bar, from which you can use the Calibre interactive tool to perform layout-versus-schematic (LVS) comparisons. Consult the <i>Custom Compiler Design Checking and Physical Verification User Guide</i> for details on LVS checks.
ICV Launch	Adds the ICV Launch menu to the menu bar, from which you can access the ICV Launch tool to run design rule checks and compare layout versus schematic processes. Consult the <i>Custom Compiler Design Checking and Physical Verification User Guide</i> for details on ICV Launch.
Parasitics	Adds the Parasitics menu to the menu bar, from which you can extract interconnect parasitics with the Synopsys StarRC tool. Consult the <i>Custom Compiler Design Checking and Physical Verification User Guide</i> for details on parasitic extraction.
Digital Implementation	Choose ICC to add the ICC menu to the menu bar, from which you can perform digital and analog co-design using the Custom Compiler Co-design Assistants with IC Compiler I. Consult the <i>Custom Compiler and IC Compiler Co-Design Assistant User Guide</i> for details. Choose IC Compiler II to add the IC Compiler II menu to the menu bar, from which you can perform digital and analog co-design using the Custom Compiler Co-design Assistants with IC Compiler II. Consult the <i>Custom Compiler and Co-Design Assistant User Guide</i> for details. Choose Fusion Compiler to add the Fusion Compiler menu to the menu bar, from which you can perform digital and analog co-design using the Custom Compiler Co-design Assistants with Fusion Compiler. Consult the <i>Custom Compiler and Fusion Compiler Co-design Reference</i> for details.
Electrical	Adds the Electrical menu to the menu bar, from which you can access tools for voltage and current annotation. Consult the <i>Custom Compiler Electrical Checking and Reporting User Guide</i> for details.
Photonics	Adds the Photonics menu to the menu bar. You can access the OptoCompiler user documentation through the product Help menu or on SolvNetPlus .

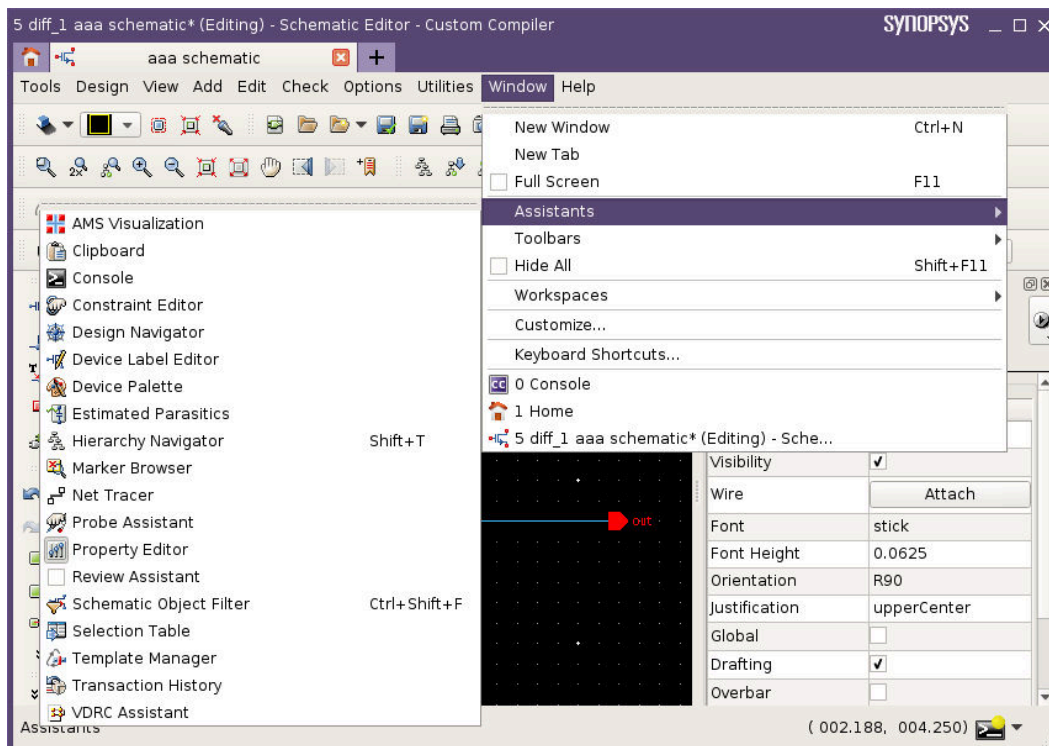
You can display the utilities using the commands in the **Utilities** menu.



[Analyzing and Debugging Schematic Designs](#) covers instructions for using the Schematic Editor utilities.

Schematic Editor Assistants

The Schematic Editor has assistants that aid you with operations on the design in the current design window. You can display or hide the assistants using the commands in the **Window** menu.

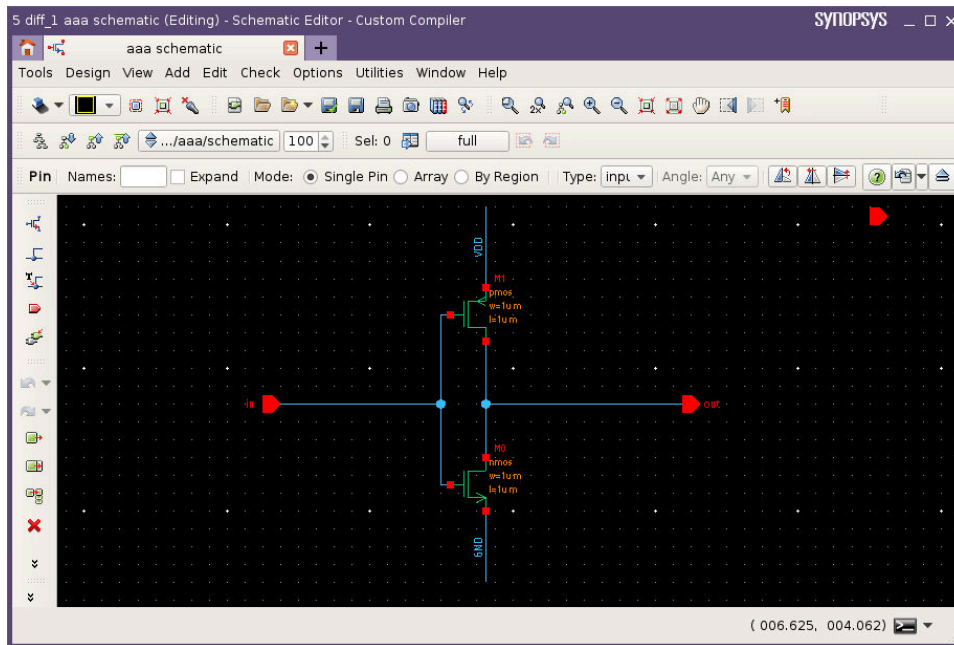


When you display an assistant, it appears on the right side of the window by default.

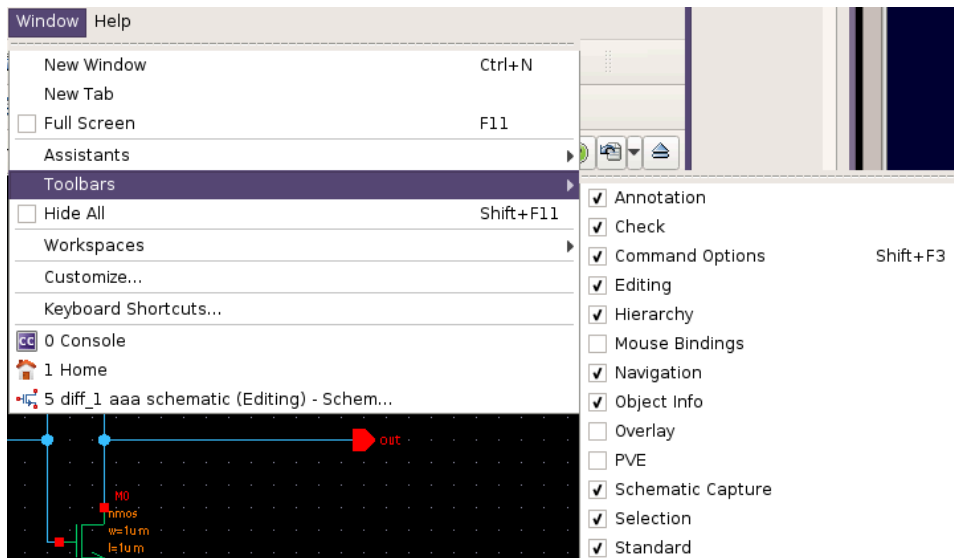
[Assistants for Schematic Designing](#) covers instructions for using the Schematic Editor design window assistants.

Schematic Editor Toolbars

Schematic Editor toolbars offer quick access to schematic design operations. The following figure shows the default locations of the toolbars.



You can display or hide the toolbars using the commands in the **Window** menu.



These topics briefly describe the Schematic Editor toolbars.

- [Annotation Toolbar](#)
- [Check Toolbar](#)
- [Command Options Toolbar](#)

- [Editing Toolbar](#)
- [Hierarchy Toolbar](#)
- [Mouse Bindings Toolbar](#)
- [Navigation Toolbar](#)
- [Object Info Toolbar](#)
- [Overlay Toolbar](#)
- [PVE Toolbar](#)
- [Schematic Capture Toolbar](#)
- [Selection Toolbar](#)
- [Standard Toolbar](#)

Annotation Toolbar

Synchronizes operations you perform in the Design Navigator Assistant.



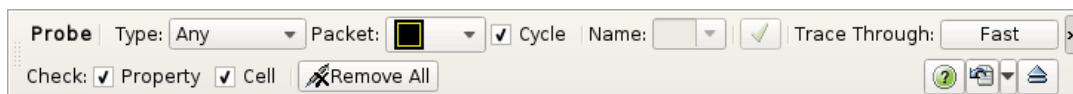
Check Toolbar

Shows check setup options, checks current views and hierarchy, shows or hides labels, and opens the Marker Browser assistant.



Command Options Toolbar

Shows options for the currently enabled interactive command. The following figure shows the command options for the **Add > Probe** command.



Editing Toolbar

Edits properties. Includes editing operations: undo and redo, move, stretch, copy, delete, cut, copy to clipboard, paste, rotate left or right 90 degrees, flip objects vertically and horizontally. Also opens the Property Editor.

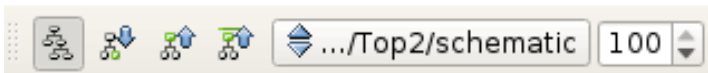


For information, see:

- [Undo and Redo](#)
- [Moving Objects](#)
- [Stretching Objects](#)
- [Copying Objects](#)
- [Deleting Objects](#)
- [Cutting Objects to the Clipboard](#)
- [Copying Objects to the Clipboard](#)
- [Pasting Objects From the Clipboard](#)
- [Rotating and Flipping Objects](#)
- Property Editor Assistant in the *Custom Compiler Environment User Guide*

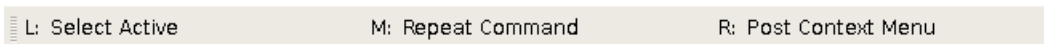
Hierarchy Toolbar

Opens the Hierarchy Navigator, descends in edit or read mode, edits in place, returns to previous hierarchy level or to top, selects any cellview from current hierarchy traversal. For information, see Navigating the Design Hierarchy and Hierarchy Navigator Assistant in the *Custom Compiler Environment User Guide*.



Mouse Bindings Toolbar

Shows the binding for the currently active command at the bottom of the window. For information, see Customizing Keyboard Shortcuts in the *Custom Compiler Environment User Guide* and [Schematic Editor Shortcut Keys](#).



Navigation Toolbar

Zooms areas, in, out, fits designs to viewports, pans viewports, goes to previous or next viewport, adds new bookmarks.



For information, see these sections in the *Custom Compiler Environment User Guide*:

- Zooming
- Fitting
- Panning
- Switching to Previous or Next Viewports
- Bookmark Manager Tool

Object Info Toolbar

Shows shape type and net for shape under cursor. For vias, pins, and shapes, shows the type and net. For instances, shows the cell name and instance name. For markers, shows the category types and text.

A small icon consisting of three horizontal lines, used to toggle the Object Info toolbar.

Marker: criticalError **Text:** m1.s.1/3/4/5/6

Overlay Toolbar

Shows the Overlay toolbar. See [Creating and Using Schematic Overlays](#).



PVE Toolbar

Reruns design rule checks (DRC), layout versus schematic (LVS) comparisons, and layout parasitic extractions (LPE).



Schematic Capture Toolbar

Checks the design for compliance with electrical and semantic rules and then saves it (check and save), adds instances, wires, wire names, and pins. Additionally creates new cellviews from current cellviews.



Selection Toolbar

Enables quick access to frequently used commands for object selection.



Category	Description
Select Count	Lists the number of partially or fully selected objects.
Selection Mode	See Selecting Objects and Areas .
Undo Selection	See Enabling Undo and Redo Selection .
Redo Selection	See Enabling Undo and Redo Selection .

Standard Toolbar

Creates new cellviews, opens existing or recent designs, saves and prints designs, exports images, opens the Library Manager and the Hierarchy Navigator, searches for and replaces cellviews.



For information, see the following sections in the *Custom Compiler Environment User Guide*:

- Creating Designs
- Opening Designs
- Saving Designs
- Printing and Plotting Designs
- Exporting Design Images

Chapter 1: About the Schematic Editor

Schematic Editor Toolbars

- Library Manager Tool
- Search and Replace Tool

2

Getting Started

This chapter explains how to start the Custom Compiler Schematic Editor. This chapter also provides a high-level view of the schematic entry workflow.

This chapter describes the following topics:

- [Starting the Schematic Editor](#)
- [Schematic Entry Workflow](#)

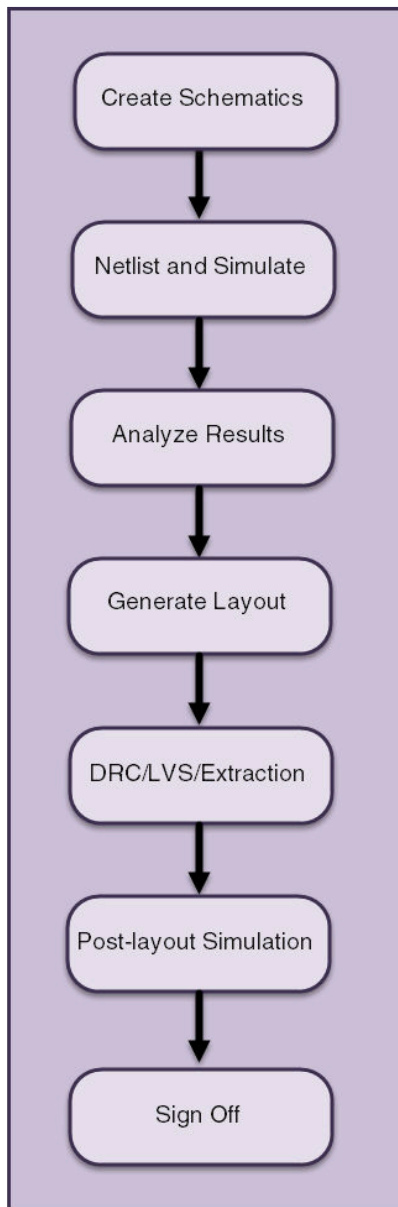
Starting the Schematic Editor

To start a Custom Compiler session and open the Schematic Editor:

1. Start a Custom Compiler session. For instructions, see Starting Sessions in the *Custom Compiler Environment User Guide*.
2. Display the Schematic Editor. For instructions, see General Designing in the *Custom Compiler Environment User Guide*.

Schematic Entry Workflow

The Custom Compiler environment includes various tools you can use to create and verify circuits. The following figure shows a typical design flow.



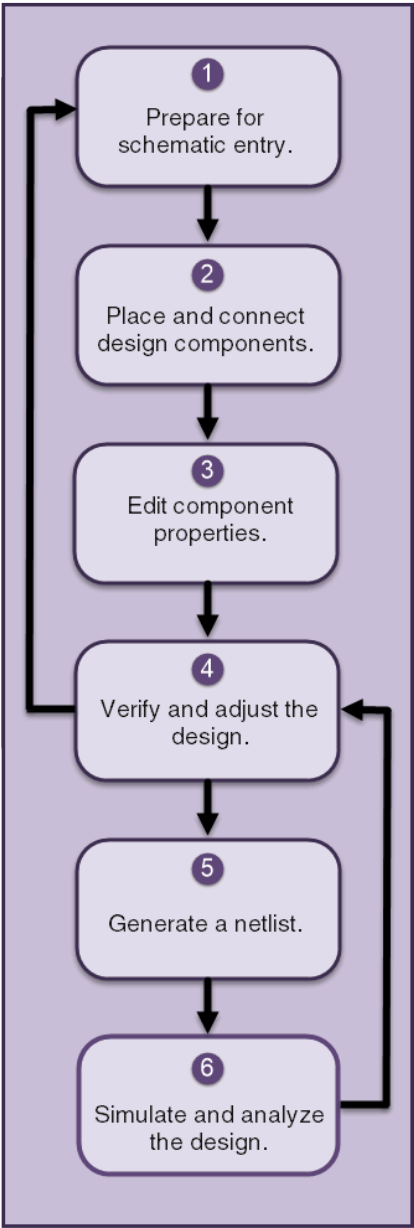
This section describes the following topics:

- [General Schematic Entry Workflow](#)
- [Example of Schematic Entry Workflow](#)

General Schematic Entry Workflow

Creating schematics using schematic design is the first step in the overall design flow. The following figure shows a high-level schematic entry workflow. The workflow does not include detailed tasks within each step. The specific workflow you use depends on your design and application. You might not need to perform all the tasks within the steps shown. In addition, the workflow implies an overall iterative process that involves making incremental adjustments to the design during schematic entry and after netlisting, simulation, and analysis. When performance is satisfactory, the design phase moves to layout.

You can also launch various tools from the Schematic Editor to address other parts of the design flow.



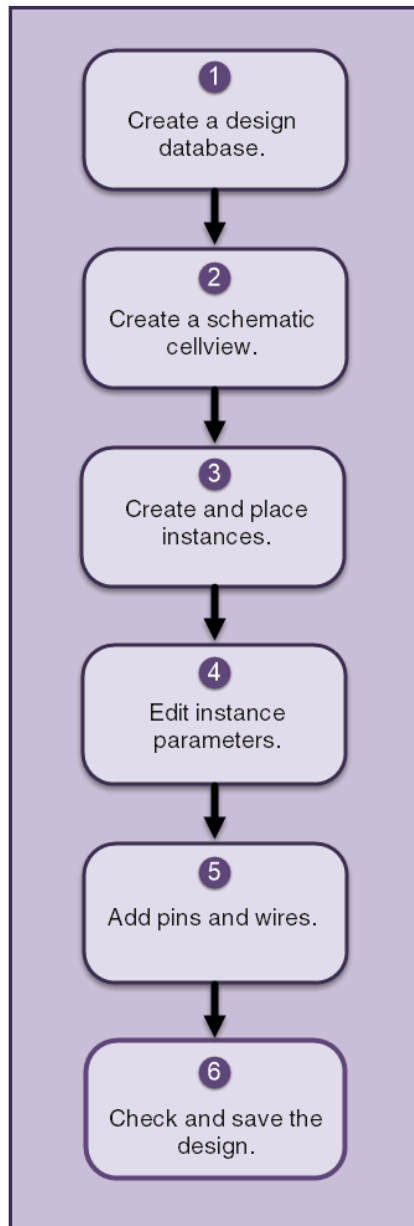
The following table describes each step in the schematic entry workflow.

Table 3 *Schematic Entry Workflow Description*

Step		Description
1	Prepare for schematic entry.	Prepare for schematic entry by checking the technology file for the necessary process-level constraints, check the placement and routing constraints, set up the design environment, and familiarize yourself with how to work with the design and hierarchy levels. For information, see the following topics: • Creating Constraints • Getting Started • General Designing in the <i>Custom Compiler Environment User Guide</i> • Working With Hierarchy
2	Place and connect design components.	Place and connect design components. For more information, see the following topics: • Creating Schematics • Editing Schematics
3	Edit component properties.	Edit component properties. For information, see Editing Schematics.
4	Verify and adjust the design.	Verify the design for electrical and semantic correctness. For more information, see Analyzing and Debugging Schematic Designs.
5	Generate a netlist.	Generate a netlist to use to simulate and analyze the design. For netlisting instructions, see the <i>PrimeWave Design Environment User Guide</i>. For information on working with designs in netlist format, see Editing Netlist Format Designs.
6	Simulate and analyze the design.	Simulate and analyze the performance of the circuit. For simulation and analysis instructions, see the <i>PrimeWave Design Environment User Guide</i>.

Example of Schematic Entry Workflow

The following figure shows a sample workflow that is commonly used for an analog circuit, a CMOS Differential Amplifier. Notice that the sequence of steps are similar to those presented in the [General Schematic Entry Workflow](#).



3

Customizing the Environment

You can customize various Schematic Editor features by setting options using the graphical interface Tcl commands. Automatic loading of preferences can also be done through the Custom Compiler load sequence, as described in the Custom Compiler Environment User Guide. This chapter describes the options you can set using the graphical interface to customize how you work with schematic designs.

This chapter covers the following topics:

- [Setting Options for the Command Use Model](#)
- [Schematic Editor Options](#)
- [Symbol Editor Options](#)
- [Managing Label Visibility](#)
- [Specifying Default Preferences](#)

Setting Options for the Command Use Model

This section explains several options and settings that affect many of the object creation and editing commands:

- [Object Orientation](#)
- [Infix Mode](#)
- [Angle Mode for Creating and Editing Objects](#)

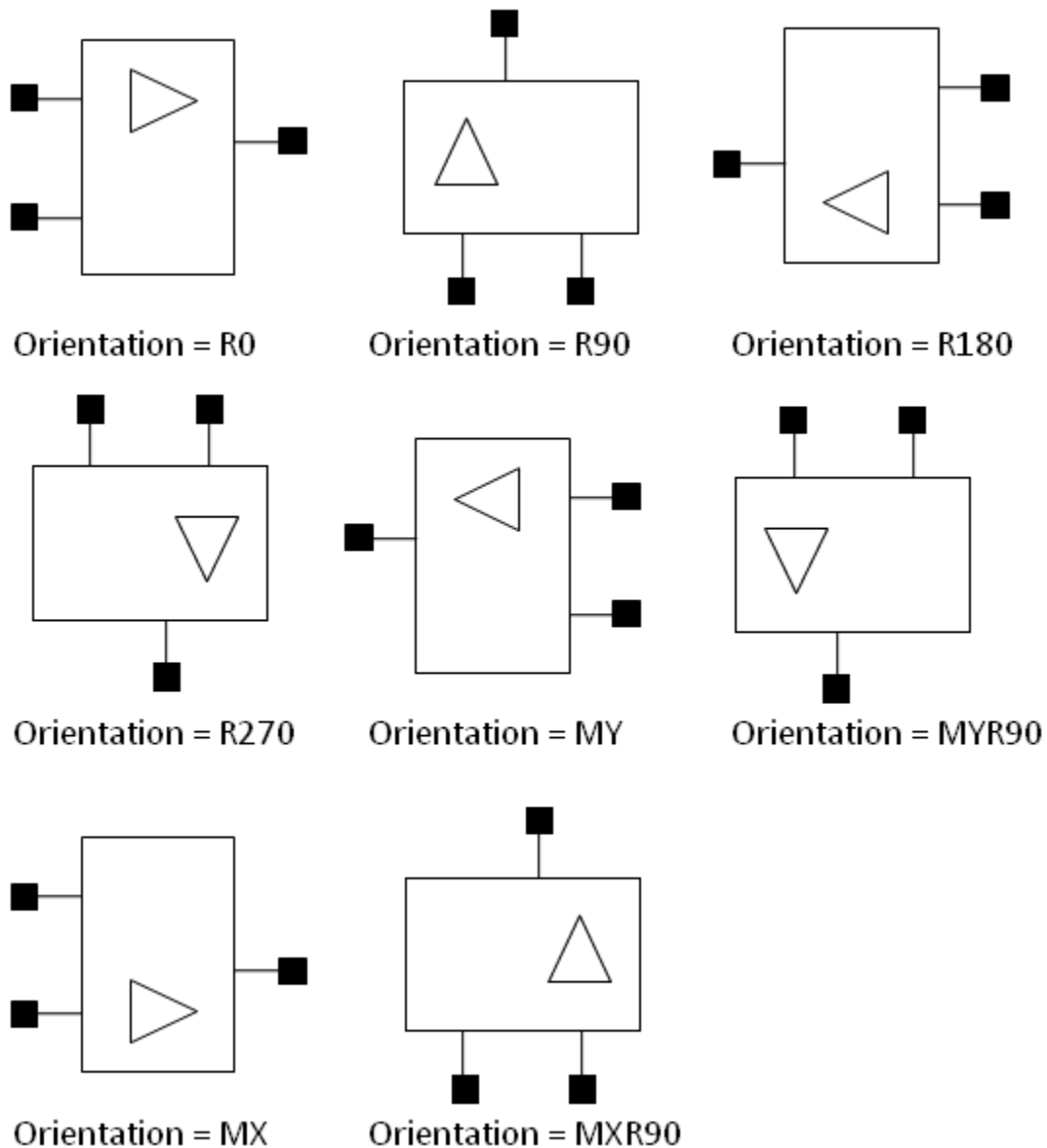
Object Orientation

Object orientation refers to the rotation and mirroring of objects. In the OpenAccess documentation, orientation is called transformation.

Except for wires in a schematic, all other objects can have any orientation.

Orientation can usually be set in the graphical interface when a creation or editing command is active. An orientation option also has a corresponding Tcl preference. Orientation can be set by default by setting the preference at the site, project, user, or local level.

The following figure shows how orientation affects instances.



Infix Mode

Infix mode lets you use fewer mouse clicks for the object creation commands and all interactive editing commands that require a reference point.

In infix mode, when you use a right-click menu or a shortcut key to start a command, the tool generates an implicit click at the current cursor location.

To use infix mode:

- Enable the **Infix Mode** option in the **General Options** dialog box, **Commands** tab.

For more information about the General Options dialog box, see the *Custom Compiler Environment User Guide*.

Note:

Infix mode only affects the first point you enter for a command. When you repeat a command, infix has no further effect.

[Table 4](#) lists the Schematic Editor commands that support infix.

Table 4 Schematic Editor Commands Supporting Infix

Add Menu	Edit Menu	View Menu
Wire	Move	Zoom > Area
Shape	Stretch	
	Copy	
	Select By Region	
	Connect Wires	
	Disconnect Wires	

[Table 5](#) lists the Symbol Editor commands that support infix.

Table 5 Symbol Editor Commands Supporting Infix

Add Menu	Edit Menu
Selection	Define Origin

This section includes the following topics:

- [Creating Shapes With Infix Mode Enabled](#)
- [Editing Objects With Infix Mode Enabled](#)

Creating Shapes With Infix Mode Enabled

For shape creation commands, the cursor location becomes the first point in the object's definition; for example, the first corner of a rectangle, the first point on a line segment, and so forth.

Editing Objects With Infix Mode Enabled

For editing commands, the meaning of the implicit mouse click depends on whether you have already selected the objects to edit:

- If you have preselected one or more objects, the cursor location sets the reference point for the editing command.
- If you have not preselected any objects, the cursor location selects the active figure for editing and also sets the reference point for the command (such as the axis of rotation) if the command requires a reference point.
- If you have not preselected any objects and there is no active figure, the implicit click only sets the reference point. The command prompts you to select the object to edit.

Angle Mode for Creating and Editing Objects

The **Angle** option controls cursor snap locations for all points except the first, while creating or editing objects.

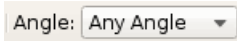
You can change the angle mode using the following methods while an object creation or editing command is active:

- The F6 key

Press F6 to toggle the angle mode settings while creating or editing objects with the following commands:

- **Add > Wire**
- **Add > Pin** (Schematic Editor)
- **Add > Pin** (Symbol Editor)
- **Add > Selection**
- **Edit > Move**

- **Edit > Stretch**
- **Edit > Copy**
- **Edit > Paste**
- The **Angle** option on the command option toolbar or the command option dialog box



These are design-specific option settings that are reset when you close the editor, unless they are set in your `prefs.xml` file, which is loaded at startup. For more information about the `prefs.xml` file, see the *Custom Compiler Environment User Guide*.

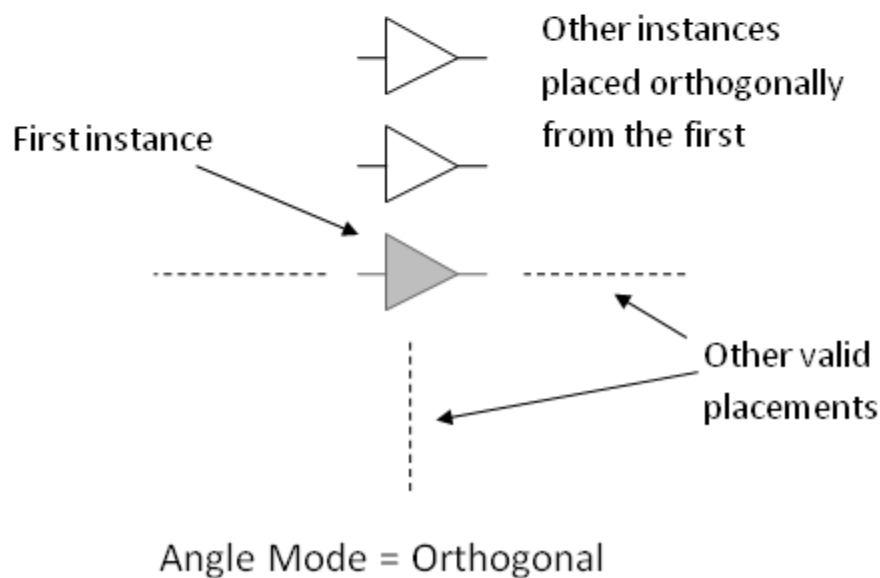
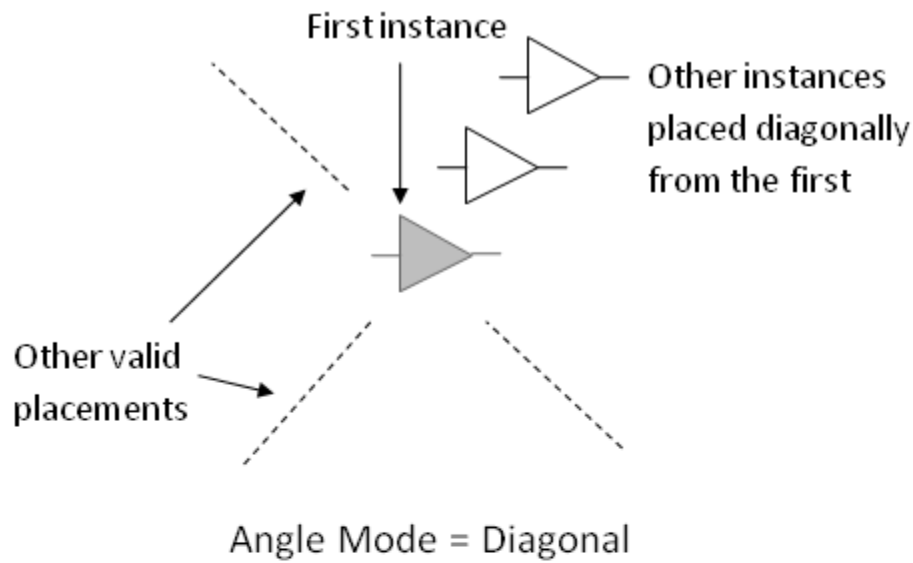
Angle mode can usually be set in the graphical interface when a creation or editing command is active. An angle mode option also has a corresponding Tcl preference. Angle mode can be set by default by setting the preference at the site, project, user, or local level.

[Table 6](#) shows the choices for angle mode.

Table 6 Angle Mode Choices

Mode	Description
Any Angle	Lets you place vertices without any restrictions on the positions of the resulting objects.
Diagonal	Lets you draw objects positioned at multiples of 45 degrees.
Orthogonal	Lets you place objects at multiples of 90 degrees.

The following figure shows how instance creation is affected by different angle mode settings.



Schematic Editor Options

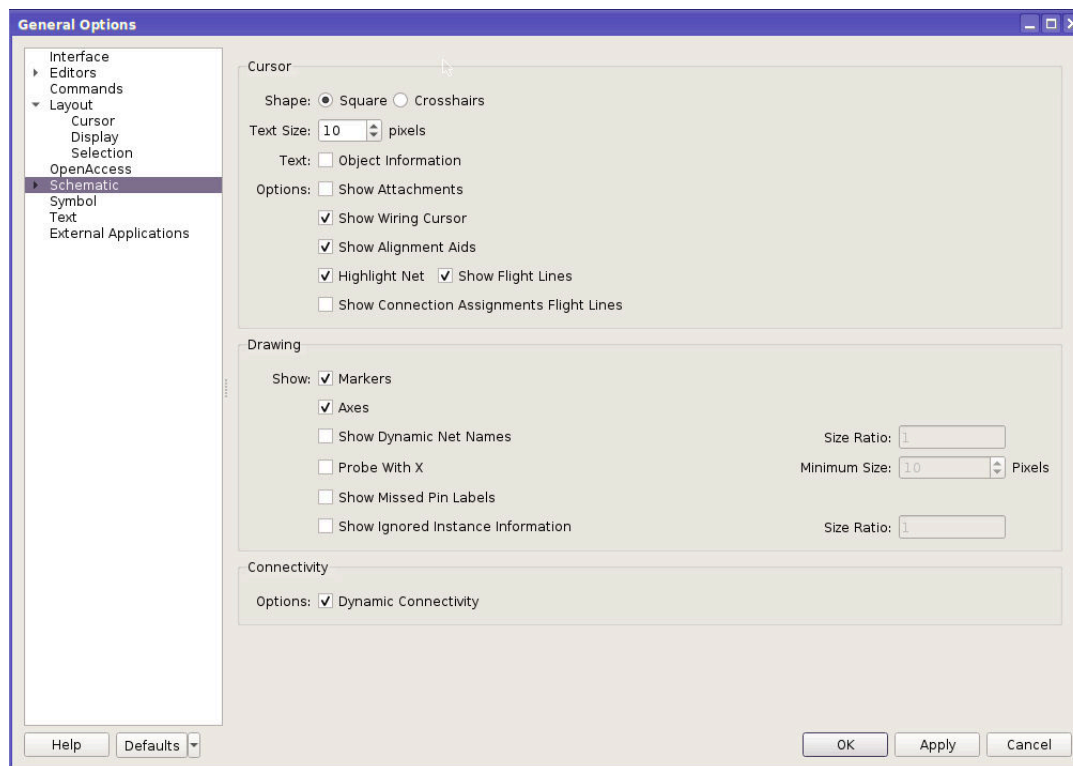
The **Options** menu commands open dialog boxes where you can set most Schematic Editor options interactively. The options are organized into tabs in the **General Options** dialog box.

They are described in the following sections:

- [Setting Display Options](#)
- [Setting Design Options](#)
- [Setting General Options](#)

Setting Display Options

The schematic display options control a variety of user preferences for displaying schematic designs. Use the **Options > General** command to open the **General Options** dialog box, then switch to the **Schematic** tab.



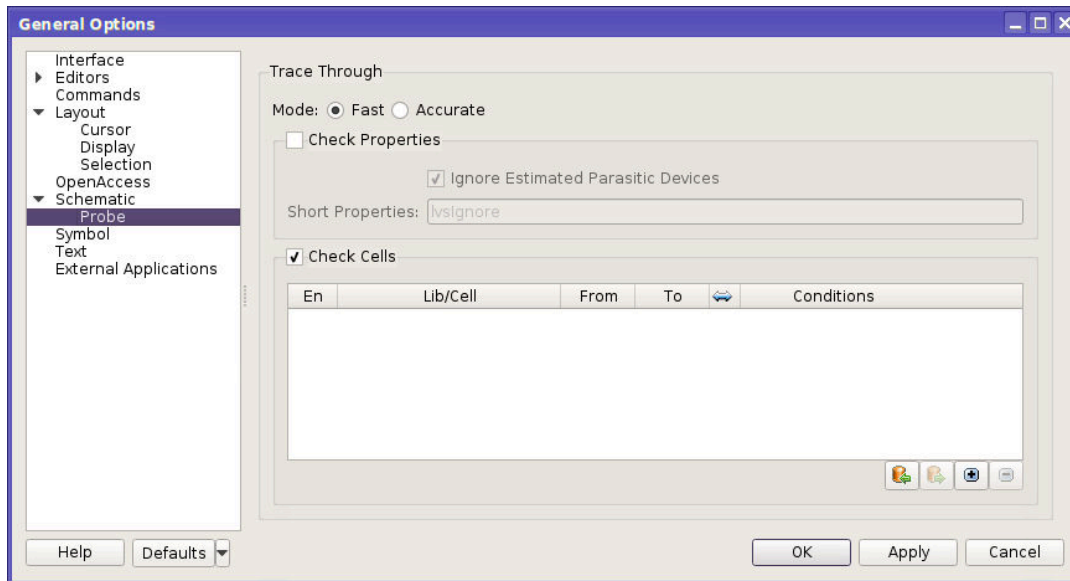
The following table describes the schematic display options.

Option	Description
Cursor	
Shape	Sets the shape of the cursor to Square (the default) or Crosshairs . When set to Crosshairs , the cursor consists of one horizontal and one vertical line drawn across the entire viewport.

Option	Description
Text Size	Sets the text size. The default is 10 pixels.
Text	Object Information displays the net name when hovering over a net. When hovering over an instance terminal, shows the terminal name and the net name.
Options	Show Attachments displays parent/child relationships involving text objects. Show Wiring Cursor highlights (with an X marker) the best possible wiring connection point when drawing a wire. Show Alignment Aids provides alignment lines when stretching, moving, or copying and pasting instances and pins. On by default. Highlight Net displays wiring objects that are connected physically or by name. Show Flight Lines toggles on/off the display of flight lines when Highlight Net is turned on. Show Connection Assignments Flight Lines toggles on/off the display of flight line on instances with oaAssignment. This option is disabled when Highlight Net or Show Flight Lines are deselected.
Drawing	
Show	Markers displays warning and error markers when any violations are made based on the setup options. Axes displays the X and Y axes lines. Show Dynamic Net Names displays temporary labels for nets where the net label (or the label of terminal connected to the net) is not shown in the current viewport (in other words, nets that are not clearly identifiable). Size Ratio is the ratio of the temporary label font size and the default net name label font size. In net highlighting, Probe with X adds a cross of constant size (independent from the zoom factor) to small objects to make them more noticeable. Minimum Size is the threshold in pixels; if the object size is less than Minimum Size, it is highlighted with a cross. Note: 1x objects in net highlighting are not impacted by this option.

Option	Description
	When the Show Missed Pin Labels option is turned on (off by default), you can draw missed pin labels using the <code>dynamicPinName</code> packet. The default color is selected to be slightly different from the pin's red color, so you can figure out that the label is not a real label thus it cannot be modified. When the Show Ignored Instance Information option is turned on (off by default), displays the ignored instances specified from the Ignore Instance command, see Ignoring Instances. Size Ratio is the ratio of the temporary label font size to display the ignore property information under the marker.
Connectivity	Dynamic Connectivity (enabled by default) allows you to disconnect the Schematic Editor's connectivity engine. This is useful to speed up processing when editing large designs because it allows you to make all necessary modifications to a design quickly, then the tool performs connectivity extraction once when you are finished editing the design. When Dynamic Connectivity is on, the tool keeps a connectivity graph of the entire design in order to provide connectivity information quickly when necessary (for example, when netlisting, calculating flight lines, or using DRC checks). When Dynamic Connectivity is turned off, the connectivity engine is disabled and connectivity information is only calculated when it is required (for example, for DRC checks, dynamic net names, the stretch function, flight lines, and so forth).

To set probe options, switch to the **Schematic > Probe** tab.

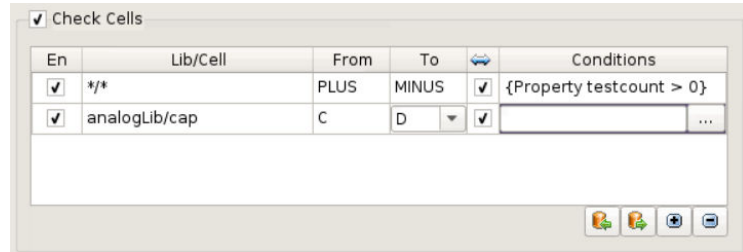


The following table describes the schematic probe **Trace Through** options on the **Probe** tab.

Option	Description
Trace Through	Set up the following trace mode: • Fast: Only finds equivalent nets in the current view context. • Accurate: Traces through up and down the hierarchy contexts to find all equivalent nets. This option is also available on the Probe toolbar and the Add Probe dialog box. See Add Probe Dialog Box and Toolbar Options.
Check Properties	Enables the Ignore Estimated Parasitic Devices and Short Properties options. This option is turned on by default.
Ignore Estimated Parasitic Devices	Enables tracing through parasitic devices marked by the <code>nlParasitic</code> property. This option is turned on by default. For information about ignoring estimated parasitic devices while shorting nets during netlisting, see the <i>Custom Compiler Environment User Guide</i>.
Short Properties	A list of properties. Devices having properties (with a value of non-false) in the list are shorted. The default is lv\$ignore. A device is marked for shorting via the <code>nlParasitic</code> property or through a user-defined short property. The tool then looks for properties listed in <code>nlShortExprPropNames</code> to determine how to short the device (for example, which pins are connected, and so forth).

Option	Description
--------	-------------

Check Cells	Opens a cell configuration table where you can define the library, cell name, and conditions for tracing through devices. Devices matching any cell configuration can be traced through.
-------------	--



Check Cells table options

En: Enables or disables the cell configuration row.

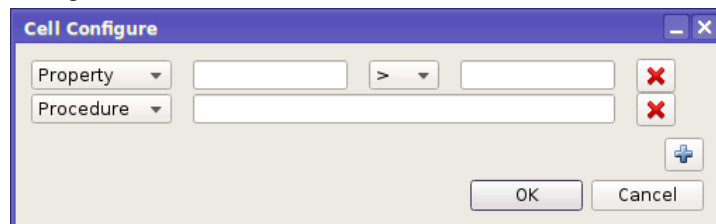
Lib/Cell: The library and cell. Wildcards are supported.

From: The term name from which to start tracing. Wildcards are supported. Only scalars and bit names are supported, not bus names.

To: The term name at which to end tracing. Wildcards are supported. Only scalars and bit names are supported, not bus names.

Bidirectional: Allows bidirectional tracing through devices. When this option is off, there is only unidirectional tracing (From > To). Default is on.

Conditions: Allows you to define conditions for tracing through the device. Click the  button to open the **Cell Configure** dialog box.



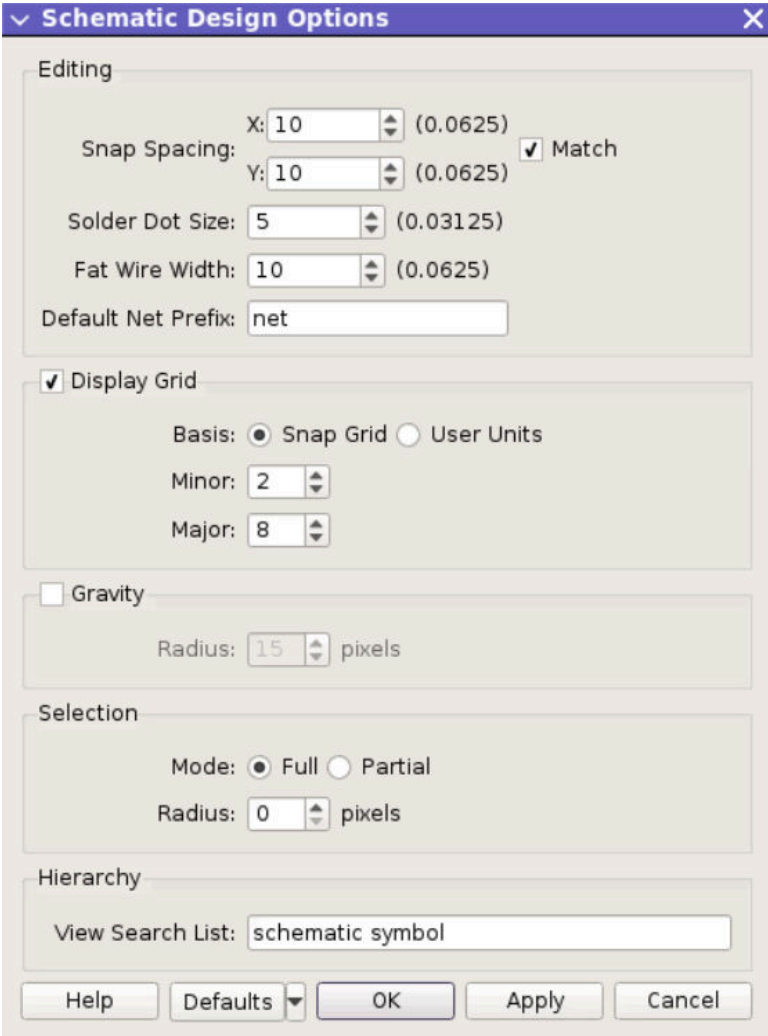
The **Cell Configure** dialog box allows you to add two types of conditions:

- **Property:** Defines the property name, operation, and value. If the device contains this property and its value matches this operation, the device is traced through.
- **Procedure:** Defines the procedure name. The argument of this procedure is the instance term. If the evaluation result is true, the device is traced through.

Option	Description
	• Import: Imports the cell configuration settings from a JSON syntax file. • Export: Exports the cell configuration settings to a JSON syntax file. • Add: Adds a new row in the Check Cells table. • Remove: Removes the selected row in the Check Cells table.

Setting Design Options

The **Schematic Design Options** dialog box controls design-specific preferences. Use the **Options > Design** command to open this dialog box.



Schematic Design Options

Editing

Snap Spacing: X: 10 (0.0625) Y: 10 (0.0625) ☒ Match

Solder Dot Size: 5 (0.03125)

Fat Wire Width: 10 (0.0625)

Default Net Prefix: net

☒ Display Grid

Basis: ☒ Snap Grid ☐ User Units

Minor: 2

Major: 8

☐ Gravity

Radius: 15 pixels

Selection

Mode: ☒ Full ☐ Partial

Radius: 0 pixels

Hierarchy

View Search List: schematic symbol

Help Defaults OK Apply Cancel

The following table describes the schematic design options.

Table 7 *Schematic Design Options*

Option	Description
Editing	
Snap Spacing	• X, Y: The distance between the snap coordinates. Default: 10 DBU. • Match: When enabled (default), whatever you type in the X or Y field also appears the other field. This saves you from having to enter the same data twice.
Solder Dot Size	Specifies the size overlap of solder dots. Default: 5 DBU.
Fat Wire Width	Specifies the fat wire width. Default: 10 DBU.
Default Net Prefix	Specifies the default prefix for system defined net names. Default: net.
Display Grid	
Basis: Snap Grid	• Minor: The number of snap grid points per minor dot. Default: 2. • Major: The number of minor grid points per major dot. Default: 8.
Basis: User Units	• Minor: The number of user units per minor dot. Default: 0.125. • Major: The number of user units per major dot. Default: 1.
Gravity	Snaps the cursor to the active object when the cursor moves within the radius specified.
Radius	Specifies the radius in pixels around an active object in which cursor gravity snapping is enabled.
Selection	
Mode	• Full: Restricts area selection to complete objects. • Partial: Lets you partial-area select the following: wire segments, symbol and note shapes, pin shapes in the Symbol Editor.
Radius	Specifies the radius in pixels around the cursor in which objects can be selected. Default: 0.
Hierarchy	
View Search List	Specifies the search list for design navigation and netlisting.

Setting General Options

For help with other options in the General Options dialog box, click **Help** in that dialog box or refer to the *Custom Compiler Environment User Guide*.

Symbol Editor Options

When working with symbol views, the **Options** menu commands open dialog boxes where you can set most Symbol Editor options interactively.

The options are organized into three dialog boxes that are described in the following sections:

- [Setting Display Options](#)
 - [Setting Design Options](#)
 - [Setting General Options](#)
-

Setting Display Options

The symbol display options control user preferences for displaying symbol designs. Use the **Options > General** command to open the **General Options** dialog box, then switch to the **Symbol** tab.

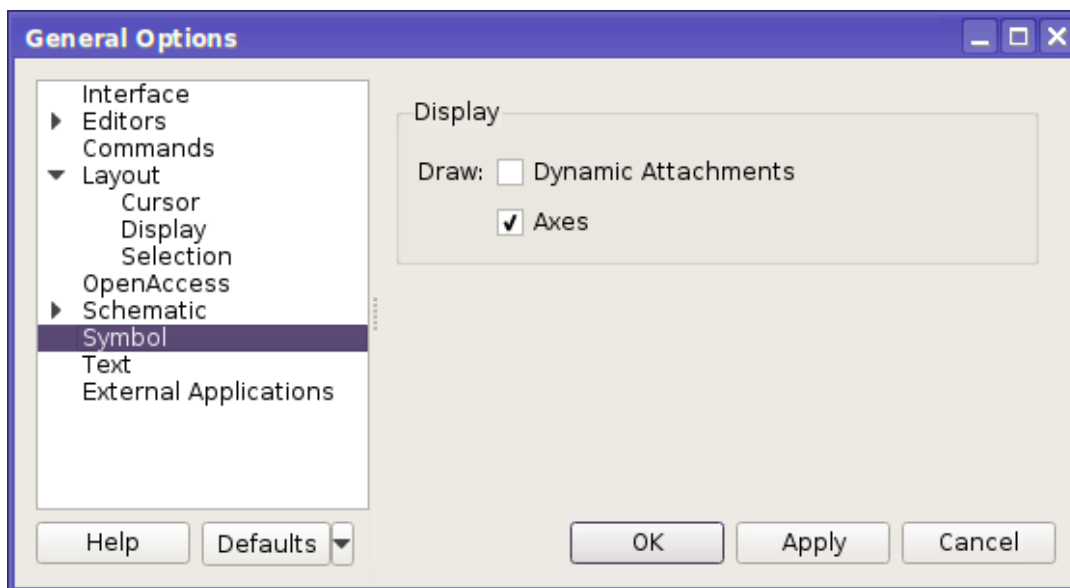


Table 8 Symbol Display Options

Option	Description
Display	
Draw	Dynamic Attachment displays parent/child relationships involving text objects.
	Axes displays the X and Y axes lines. On by default.

Setting Design Options

The **Symbol Design Options** dialog box controls design-specific preferences. Use the **Options > Design** command to open this dialog box.

Symbol Design Options

Snap Spacing

Mode: ☒ Pin ☐ Graphics

X: 10 (0.0625) ☐ Match

Pin: Y: 10 (0.0625)

Graphics: ☐ Value ☒ Ratio 10

X: 1 (0.00625) ☐ Match

Y: 1 (0.00625)

☒ Display Grid

Basis: ☒ Snap Grid ☐ User Units

Minor: 2

Major: 8

☐ Gravity

Radius: 15 pixels

Selection

Mode: ☒ Full ☐ Partial

Radius: 0 pixels

Help Defaults OK Apply Cancel

Table 9 Symbol Design Options

Option	Description
Snap Spacing	
Mode	Specifies Pin (default) or Graphics mode. Allows you to have two distinct grids, your default one for ensuring pin alignment and a graphics grid for advance details required during symbol generation.
Pin	• X, Y: The distance between the snap coordinates. Default: 10 DBU. • Match: When enabled (default), whatever you type in the X or Y field also appears the other field. This saves you from having to enter the same data twice.
Graphics	Specifies Value or Ratio (the default). The graphic grid size can be specified as either specific values or as a ratio to the default pin grid size. The default is 10 times smaller. If Value is chosen, specify X,Y coordinates.
Display Grid	
Basis - Snap Grid	• Minor: The number of snap grid points per minor dot. • Major: The number of minor grid points per major dot.
Basis - User Units	• Minor: The number of user units per minor dot. • Major: The number of user units per major dot.
Gravity	Snaps the cursor to the active object when the cursor moves within the radius specified.
Radius	Specifies the radius around an active object in which cursor gravity snapping is enabled.
Selection	
Mode	• Full: Restricts area selection to complete objects. • Partial: Lets you partial-area select the following: wire segments, symbol and note shapes, pin shapes in the Symbol Editor.
Radius	Specifies the radius around the cursor in which objects can be selected.

Setting General Options

For help with the General Options dialog box, click **Help** in that dialog box or refer to the *Custom Compiler Environment User Guide*.

Managing Label Visibility

The **Manage Labels** command lets you control the visibility of label objects and wire names.

For information on using the `ise::manageLabels` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To control visibility:

1. Choose **Edit > Properties > Manage Labels**.

The **Manage Labels** toolbar appears.

2. The table below describes the **Manage Labels** options.

Table 10 *Manage Labels Options*

Option	Description
Surround Brightness	Controls the level of brightness for displaying objects other than the wire segments and labels. Note: This option also appears in the General Options window. For help with the General Options dialog box, click Help in that dialog box or refer to <i>the Custom Compiler Environment User Guide</i> .
Show All	Displays all wire names.
Hide All	Hides all wire names.

Drag the **Surround Brightness** slider to dim all objects except wire segments and wire labels.

3. Use one of the following methods to specify wire label visibility:
 - Click on individual wire labels to toggle the current visibility setting.
 - Click on a wire segment to toggle all of the segment's wire labels.

In the case where some of a wire segment's labels are invisible and some visible, clicking on the wire segment makes all labels visible.

4. Click the **Apply** button (check mark) to commit the changes.

The command remains active.

5. Press Esc to end the command.

Specifying Default Preferences

You can specify default preferences in the `prefs.xml` file, which is loaded at startup. For more information about the `prefs.xml` file, see the *Custom Compiler Environment User Guide*.

4

Creating Schematics

This chapter describes the features and procedures you can use to create schematic designs, as well as procedures that outline some of the tasks you might perform to create a schematic. The Schematic Editor supports these tasks and many others related to schematic creation and editing.

The following figure illustrates some tasks you might use the Schematic Editor to perform while creating the schematic.



This chapter describes procedures for:

- [Creating Instances](#)
- [Creating Wires](#)
- [Creating Pins](#)
- [Creating Connection Definitions](#)
- [Creating Shapes](#)
- [Creating Blocks](#)
- [Creating Sheets](#)
- [Creating Constraints](#)
- [Annotating Designs](#)
- [Connecting Components and Nets](#)
- [Drawing Rulers](#)

Creating Instances

An instance is a representation of one design or device, as part of another design. The design containing the instance is the parent design and the design that the instance references is the master. You can have several instances of the same master in one design, but they must have different names.

For information on using the `se::createInst` and `ise::createInst` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

You can create the following instance types:

- Scalar
- Iterated

The instance type is determined by the name of the instance. You can specify a name or let the Schematic Editor apply a default name.

The following sections describe:

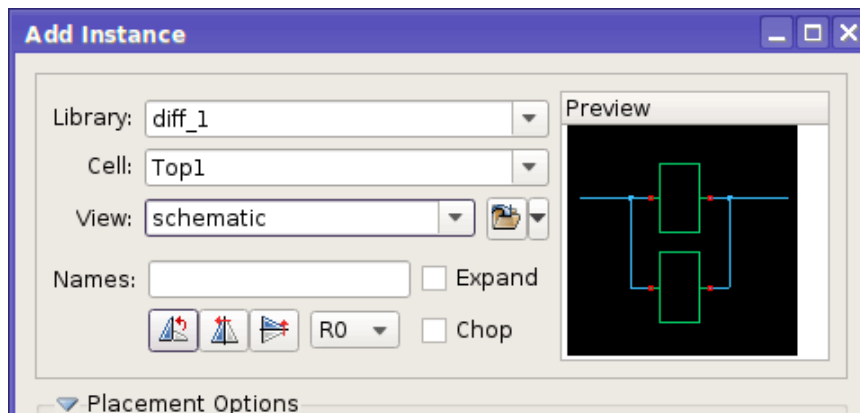
- [Adding Instances](#)
- [Naming Instances](#)

Adding Instances

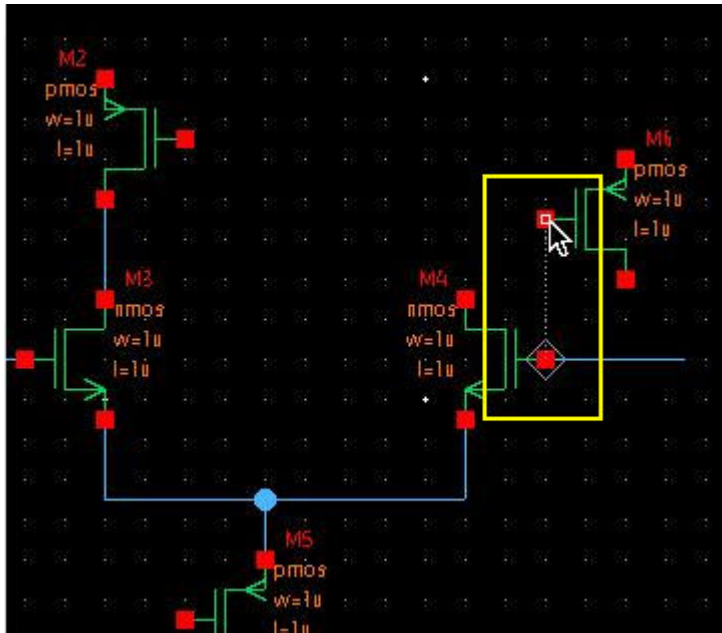
To add an instance:

1. Choose **Add > Instance**. The **Add Instance** dialog box appears. See [Add Instance Toolbar and Dialog Box Options](#).
2. Specify the instance options, instance parameters, and connection assignments.

After you select the LCV information, the tool shows the selected cellview in the Preview area, as illustrated in the following figure.



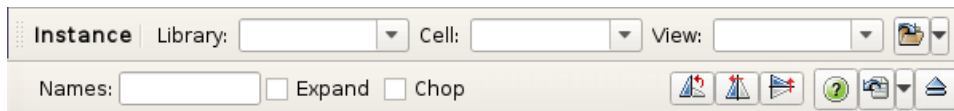
3. (Optional) Press Alt+A to toggle the alignment cursor. When enabled, cross hair cursor lines appear to help guide the placement. These lines only appear when pins from other objects are in the same vertical or horizontal plane as the pins being moved. The aligned pins are highlighted with a diamond shape.



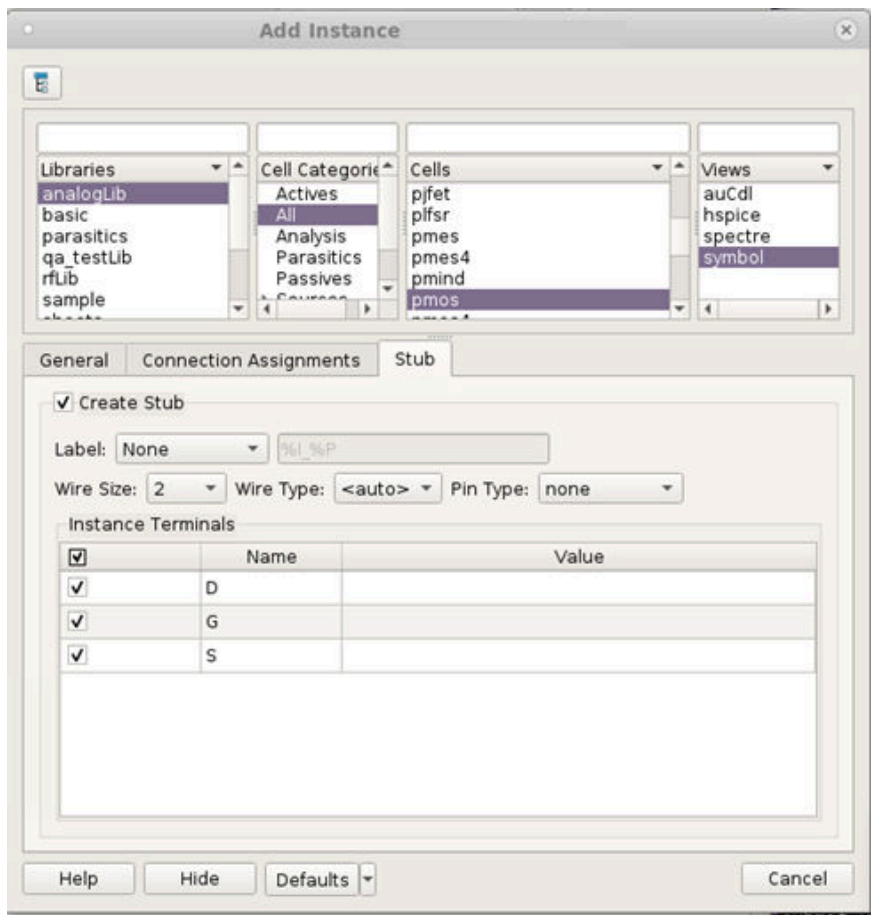
4. Click in the canvas to place the instance. The command remains active.
5. Press Esc to end the command.

Add Instance Toolbar and Dialog Box Options

The following figure illustrates the **Instance** toolbar.



When the **Add > Instance** command is active, pressing the shortcut key F3 or clicking the **Eject** button on the toolbar shows the **Add Instance** dialog box.



The **Hide** button at the bottom of the dialog box closes the dialog box and displays the instance options in a toolbar. The **Eject** button on the toolbar opens the dialog box. You can set the `seCreateInstDialogMode` preference to control whether the dialog box or the toolbar opens by default.

The toolbar and dialog box contains the following options:

Option	Description
Library, Cell, View	Specifies the library, cell, and view (LCV) containing the instance you want to place. Tcl preference: <code>seInstMaster</code>
Cell Category	(Dialog box only) Shows cell categories. Click Show/Hide Cell Category  to show or hide the Cell Categories list box. Tcl preferences: <code>seInstShowCellCategories</code> and <code>seInstShowCellViewTable</code>

Option	Description
<i>General</i>	(Dialog box only) Sets general options.
Names	Specify one or more instance names, an iterated (vectorized) name, or leave blank. When blank, the editor generates a default name. See also Naming Instances . Tcl preference: <code>seInstName</code>
Expand	When enabled, iterated names are expanded to scalarized names for each instance you add. Tcl preference: <code>seInstExpand</code>
Chop	When enabled, creates a gap in the existing wires and inserts the instance into this gap with one action. See Step 1 for more details. After the instance is added (that is, after the instance pins are already connected to the wire ends), the shorting wires are chopped under the instance area according the following rules: • Only the wire segments of nets that are connected to more than one pin of this instance are affected. • Only the wire segments that are completely located in the instance area (including the area borders) are deleted. The instance area is the rectangular union of instance's selection shape and all its pins' bounding boxes. • Wires partially located in this area are not affected. • Net name labels are processed the same way as wire segments. • All net renaming actions caused by aforementioned deletions are made as expected, similar to how renaming occurs after any other object deletion. Tcl preference: <code>seInstChop</code>
Transform icons	These options let you rotate or flip the instance before placement.
Orientation	(Dialog box only) Sets orientation of instances by R0 , R90 , R180 , R270 , MY , MYR90 , MX , and MXR90 . Tcl preference: <code>seInstOrientation</code>
Cols/Rows	(Dialog box only) These options let you place an array of instances. Tcl preferences: <code>seInstNumColumns</code> and <code>seInstNumRows</code>
Angle	(Dialog box only) Specifies how to place an array of instances. The options are: Orthogonal , Diagonal , or Any Angle . See Angle Mode for Creating and Editing Objects . Tcl preference: <code>seInstAngle</code>
Preview	(Dialog box only) Shows a snapshot image of the selected cellview. If the snapshot image does not exist, show an empty area.
Parameters	(Dialog box only) Specifies cell-specific parameters.
<i>Connection Assignments</i>	(Dialog box only) Specifies connection assignments.

Option	Description
Stub	To create a new stub.
Create Stub	Enable to create a new stub on the instance terminal. When disabled, no instance terminal is listed in the Instance Terminals table. The associated preference is <code>seInstCreateStub</code> . Default: <code>false</code> .
Label	Label type. Select a label type from the menu: None , Pin Name , or User Defined . If you select User Defined , enter the user label in the text box.
Wire Size	The wire size of stub. Select a value from the menu: auto , 1 , 2 , 3 , 5 , 10 . Default: 2 .
Wire Type	The wire type. Select a wire type from the menu: auto , thin , fat . Default: auto .
Pin Type	The pin type. Select a pin type from the menu: none , noConn Pin , auto , input , output , inputOutput , switch , tristate . Default: none
Instance Terminal	When library cell view is selected and the Create Stub is selected, the terminal names are listed in the table. The table has the following columns: • Enabled: Select row to create or not create stub on this instance terminal. Default: enabled. You can click the header to disable/enable all rows. The header shows the status: ◦  : depicts that all instance terminals are enabled. ◦  : depicts that some instance terminals are enabled and some are disabled.. ◦  : depicts that all instance terminals are disabled. • Name: Terminal Name of this instance terminal. • Value: The label text of this stub. If the value is empty, create the label text in the general stub label setting. Default is empty.

Add Instance Menu Options

The following figure illustrates the **Add Instance** menu.

Remove Point	Backspace
 Rotate 90	R
 Flip Horizontal	Shift+R
 Flip Vertical	Ctrl+R

The menu contains these options:

Option	Description	Default Shortcut Key
Remove Point	Deletes the previously entered point.	Backspace
Rotate 90	Rotates the instance 90 degrees counter clockwise	R
Flip Horizontal	Flips the instance about the Y axis.	Shift+R
Flip Vertical	Flips the instance about the X axis.	Ctrl+R

Naming Instances

Use the **Names** text field to add one or more instance names. You can also specify an iterated instance name (a vectored name) to create an iterated instance; for example, `M1<0:6>`. A default instance name is generated if this field is empty at the time that you define a point on the canvas.

Stacking Instance Names

An instance name stack consists of a list of space-delimited instance names in the **Names** field.

When a stack of names exists, the first name in the stack is processed first; for example, when you add an instance to the canvas, the first instance name and trailing space are removed from the **Names** field. As you add instances, the corresponding instance names and trailing spaces are removed from the **Names** field. This process repeats until the **Names** field is empty. If you continue to place instances after the name stack is empty, the Schematic Editor names the instances.

Instance name stacking behavior is dependent on the **Cols/Rows** settings in the dialog box. By default, these fields are each set to 1, which specifies that a single click represents a single instance creation.

When either of the **Cols/Rows** fields has an integer greater than 1, you must define two points on the canvas:

- The first point defines the position for the first instance.
- The second point defines the step distance.

The direction of placement is from the first point to the second. After you define the first point, a runtime instance array appears on the cursor showing the location of instances using the current snapped cursor location as the step distance from the first point.

When both the **Cols/Rows** fields have integers greater than 1, the result is a two dimensional array and you must specify three points:

- The first point defines the origin of the array.
- The second point defines the distance between columns.
- The third point defines the distance between rows.

When defining the first point, a single instance is attached to the cursor; when defining the second point, a row of instances is shown and the spacing between them is interactively shown as the cursor moves; when defining the third point, the complete array is shown and the row spacing is interactively adjusted.

Naming Iterated Instances

If the left-most name in the field is an iterated instance name, for example `I11<0:10>`, and the **Expand** option is not selected, an iterated instance (an `oaVectorInst` object) is added to the cell view.

The **Expand** option enables the use of scalarized placements for iterated instance names. When the **Expand** option is selected and the current instance name is vectored, a single instance bit is placed and the range of the instance name in the toolbar is reduced. After placing the last instance bit, the iterated instance name is removed from the instance name stack in the toolbar.

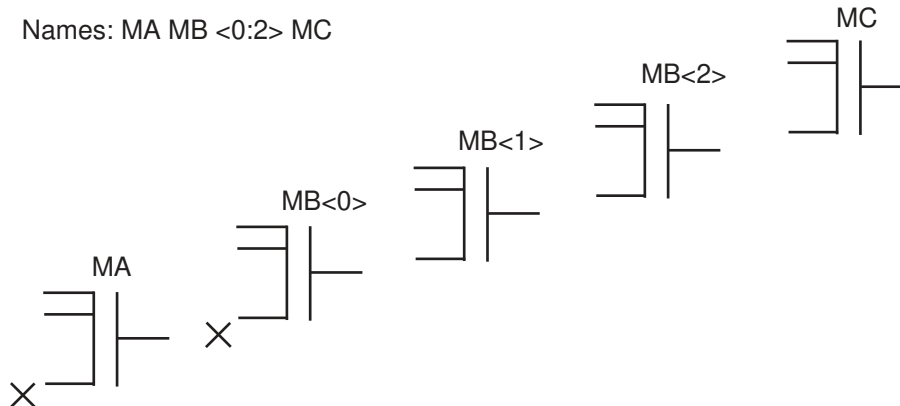
Adding Instances Using Arrayed Placement

A list of instance names is used to name, in sequence, an array of created instances. If the number of instance names is smaller than the array, the names for remaining instances are automatically generated (see [Default Naming](#)).

The order of creation and naming is row based: the first instance at the origin receives the first name, the next instance in the first row gets the second name, and so on.

When there are invalid names in the instance name field, the instance image disappears from under the cursor and the focus changes to the instance name field where the invalid instance name is automatically selected. A name is regarded as invalid if a placed instance with that name already exists or when the name contains an invalid character.

The following figure shows an instance array addition for the case when the **Columns** options is set to 5 and the **Expand** option enabled.



With the exception of Library, Cell, and View names and entered instance names, the **Defaults** button resets all the dialog box and toolbar control settings to their defaults, which are retrieved from the tool configuration. The **Defaults** button also resets all properties of the current temporary instance to their defaults.

All dynamic ERC/SRC checks are executed after each instance addition.

Default Naming

If the **Names** field is empty when you click in the canvas, the editor creates a unique instance name by applying a default prefix and appending an integer to the prefix.

The default prefix is determined using the following rules:

- Derived from the cellview property `instNamePrefix` of the selected master.
- If the cellview property `instNamePrefix` is not defined, the editor checks the Prefix selection, which is specified in the **General Options** dialog box.
 - If Prefix is selected (default), the string in the text field is used as the prefix.
 - If Cell Name is selected, the cell name of the selected master is used as the prefix.

Creating Wires

The Schematic Editor lets you route a wire between any two points using the shortest possible loop-free path, while optionally avoiding any or all of the following conditions:

- Intersections with schematic and symbol pins
- Routes over symbols
- Name shorts

The Schematic Editor has a run time connectivity engine that assigns net names to wires and other connectivity objects if they are not explicitly named at creation time.

This section describes the following topics:

- [Understanding Jog Styles](#)
- [Naming Wires](#)
- [Adding Individual Wires](#)
- [Adding Groups of Wires](#)
- [Adding Wire Stubs](#)
- [Adding Wire Names](#)
- [Setting Display Mode for Bundle Names](#)
- [Parameterized Connections on Wires](#)
- [Understanding Connectivity With Wires](#)
- [Creating Global Net Names](#)

Understanding Jog Styles

When you start to route a wire on the canvas, there are two possible routes to any destination point:

- Horizontal followed by vertical (H+V)
- Vertical followed by horizontal (V+H).

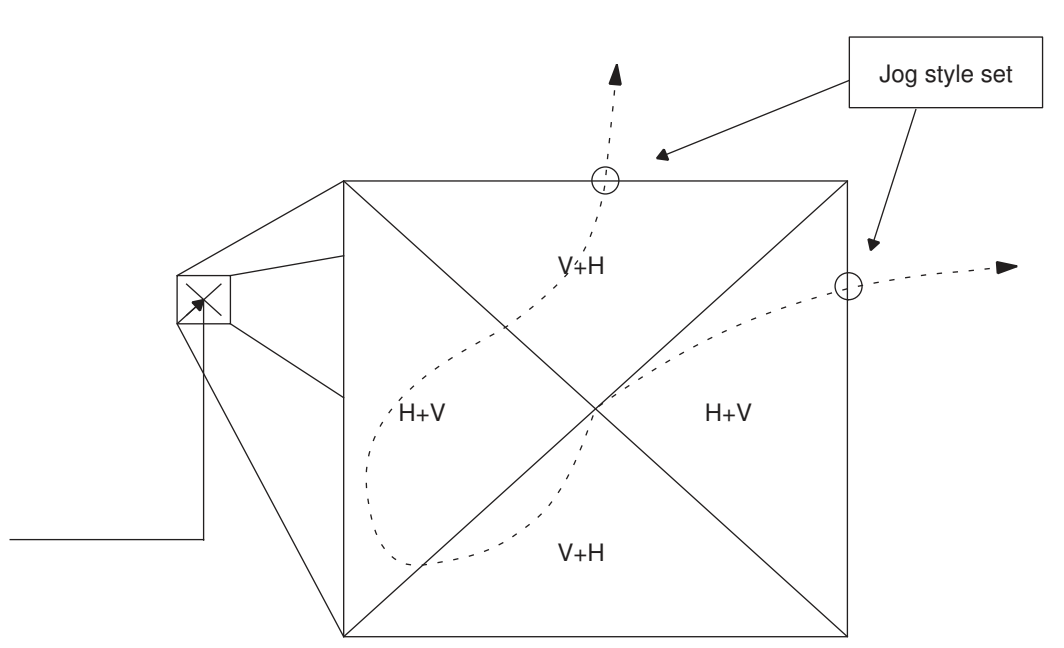
This distinction is called the *jog style*.

A jog style is set, or reset, whenever your cursor intersects the edge of the jog reset bounding box that surrounds the last entered point of an active **Add > Wire** command.

- If the crossover point intersects either the upper or lower edge of the bounding box, the jog style is set to V+H.
- If the crossover point intersects the left or right edges of the bounding box then the jog style is H+V.

[Figure 1](#) shows the possible routes and set points of a jog reset bounding box.

Figure 1 Jog Style Routes and Set Points



Naming Wires

Wires can be labeled during creation or afterwards. Label during creation by entering one or more space-delimited names in the **Nets** field of the **Add > Wire** command. If no names are entered, the Schematic Editor assigns a default name.

For information on using the `se::createWireName` and `ise::createWireName` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

For information about labeling wires after they are created, see [Creating Symbols](#).

This section describes the following topics:

- [Stacking Wire Names](#)
- [Bus Notation](#)
- [Named Bus Tapping](#)
- [Justification and Orientation](#)

Stacking Wire Names

A name stack consists of a list of space-delimited wire names in the **Names** field.

When a stack of names exists, the first name in the stack is processed first; for example, when you add a wire name to the canvas, the first name and trailing space are removed

from the **Names** text field. As you add additional wire names to the canvas, the corresponding names and trailing spaces are removed from the **Names** text field. This process repeats until the **Names** text field is empty.

Bus Notation

The Schematic Editor supports notation for creating complex bus signals. Using specific patterns in the naming of wires, you can repeat individual bits or entire ranges. For example, the pattern `BUS<start:stop*n>` repeats each bit in the range n times, and the pattern `BUS<(start:stop) *n>` repeats the entire range n times.

Examples

The following example repeats the entire range `A<0:2>` four times.

`A<(0:2) *4>`

The expanded sequence of bit nets is:

`A<0>,A<1>,A<2>,A<0>,A<1>,A<2>,A<0>,A<1>,A<2>,A<0>,A<1>,A<2>`

The following example repeats each bit in the range between `A<0>` and `A<15>` four times.

`A<0:2*4>`

The expanded sequence of bit nets is:

`A<0>,A<0>,A<0>,A<0>,A<1>,A<1>,A<1>,A<1>,A<2>,A<2>,A<2>,A<2>`

You can also use combinations of the above two basic patterns to create additional sequences.

The following example repeats the specified pattern twice.

`A<(0:2*2) *2>`

The expanded sequence of bit nets is:

`A<0>,A<0>,A<1>,A<1>,A<2>,A<2>,A<0>,A<0>,A<1>,A<1>,A<2>,A<2>`

The following example specifies two patterns.

`A<0:2*2,2:0*2>`

The expanded sequence of bit nets is:

`A<0>,A<0>,A<1>,A<1>,A<2>,A<2>,A<2>,A<2>,A<1>,A<1>,A<0>,A<0>`

The following example increments indexes by 2.

`A<0:6:2>`

The expanded sequence of bit nets is:

$A<0>$, $A<2>$, $A<4>$, $A<6>$

Named Bus Tapping

Bus tapping lets you make connections to a subset of bits from a bus.

The Custom Compiler Schematic Editor supports two bus tapping methods:

- [Wire Labeling](#)
- [Unnamed Tapping](#)

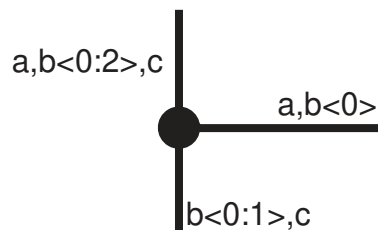
Wire Labeling

With this method, you can tap a subset of bits from a bus or bundle through wire labeling. Two wire objects with touching vertices can be labeled such that one is a subset of the other. In general, all wiring objects that interact together must be labeled, either explicitly or by their subnet, such that all labels are part of the same net family.

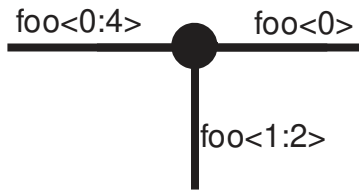
Figure 2 Two Wire Objects With Touching Vertices



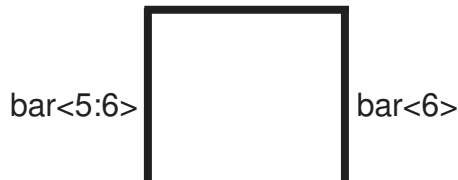
When three or more labeled segments join at a single point, each wire name must be a subset of at least one of the other wire names, or else a name short occurs. That is, there must be a parent label which is a superset of all other labels from the connected wiring. For example, a wire label of 'a,b' can be connected to a wire labeled 'a,b,c' and to a wire labeled 'b,c'. In this case, the 'a,b,c' label is the parent label containing all bit/scalar nets in the wire, see the following figure.



The following figure shows how three line objects join together, each with a different net, but all part of the same net family of $foo<0:4>$. A net family is a group of two or more nets or bundles in which each net or bundle is related to at least one other net or bundle in the group. For two bundles to be related one must be a subset of the other. In this example, both $foo<0>$ and $foo<1:2>$ are subsets of $foo<0:4>$.



In the following figure, the unlabeled wiring object has two names to choose from, but chooses the widest net; that is, the net with the most bits.



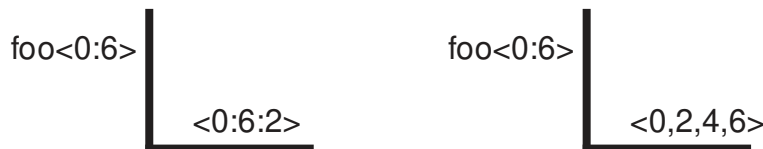
Unnamed Tapping

With this method, you supply only the bit syntax. This allows for a general tapping behavior without having to explicitly reference the base name or bundle.

The following figure shows how you can tap the `foo<4:0>` bus without supplying the base name of `foo`. Note that the bit syntax does not require the angle brackets (`<>`); only a valid bus range syntax. For example, `foo<0:1>` can be tapped by using a label of `0:1`.



The Custom Compiler Schematic Editor also supports skip counts and explicit indexing in unnamed tapping (see the following figure). For example, `foo<0:6>` can be tapped using `<0:6:2>` or `<0,2,4,6>` to rip off just the even numbered bits of the vector.



Note:

You can also draw fat wires instead of thin wires to visually distinguish buses and bundles from single bits.

Justification and Orientation

The default settings for justification and orientation, respectively, are lowerCenter and R0 for the wire name. If you move the cursor over a vertical wire segment, the orientation of the ghost name automatically switches to R90. If you move the cursor away from a vertical wire, the orientation reverts to R0.

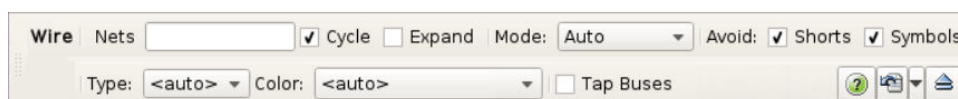
Adding Individual Wires

The **Wire** command lets you draw a polyline; that is, a list of points, or coordinates, that represent the beginning and end points of straight-line wire segments.

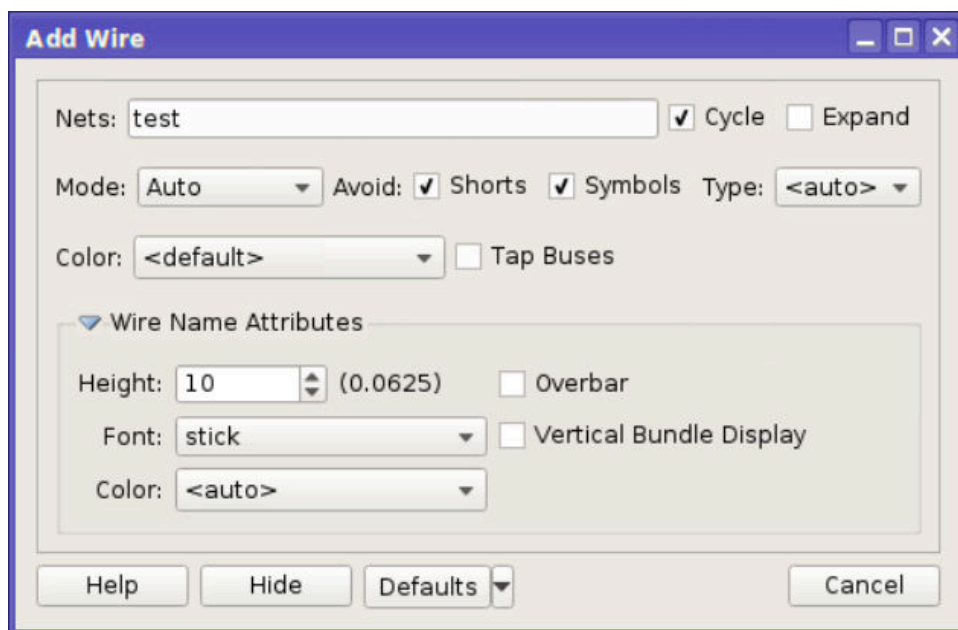
For information on using the `se::createWireName` and `ise::createWireName Tcl` commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To add a wire:

1. Choose **Add > Wire**. The **Wire** toolbar appears.



To open the **Add Wire** dialog box, use the F3 key or click the **Eject** button on the toolbar after initiating the interactive command.



2. Specify the wire options.

The dialog box and toolbar contain the following options:

Option	Description
Nets	Specifies a net or a list of nets.
Cycle	Specifies to delete names from the Nets list when the associated wires are committed. When deselected, the first name in the list is reused.
Expand	When enabled, bus names are expanded to bit net names for each wire you add.
Mode	Specifies how to route the wire. If Auto routing is not feasible, the router defaults to Any Angle to connect the source and destination points with a flight line. See Angle Mode for Creating and Editing Objects .
Avoid	◦ Shorts: The router avoids overlapping existing wiring object endpoints that would cause a short between two or more nets, whether the nets are named or not. ◦ Symbols: The router uses the object's selection box (or if that does not exist, the bounding box) as the avoidance region.
Type	Specifies the wire type by <auto> , thin , or fat .
Color	Specifies the wire color.
Tap Buses	Lets you tap buses when starting, or committing to, a multi-bit wire. When you click on a bus or bundle wire segment, the Select Net Names dialog box opens so you can specify the nets to tap.
Wire Name Attributes	(Dialog Box only) A toggle that expands and collapses the dialog box to reveal or hide the text attributes, such as text height, font, color, you can define. When Overbar is selected, displays an overline above the text. When Vertical Bundle Display is selected, sets the display mode for bundle names to vertical. See Setting Display Mode for Bundle Names.

The menu can be shown during the **Add Wire** command by right-clicking. The menu contains these options:

Command	Description	Default Shortcut Key
Commit	Completes the wiring operation.	Return
Remove Point	Deletes the previously inserted point.	Backspace

Command	Description	Default Shortcut Key
Select Bits	Lets you tap buses when starting or committing a wire. When you click on a bus or bundle wire segment, the Select Net Names dialog box opens so you can specify the net to tap.	

3. Click multiple points on the canvas to draw the wire.
4. To complete the wire.
 - Double-click to end the wire in free space
 - Click a valid connection point (for example, an instance pin, a schematic pin, a wire segment or endpoint, or a solder dot).
 - Press S to route the wire to the snap cursor.The command remains active.
5. Press Esc to end the command.

Adding Groups of Wires

Instead of adding wires one at a time, you can use the **Route** command to add groups of wires between sets of pins or sets of wire stubs (see [Adding Wire Stubs](#)).

Two modes are provided:

- [Many-to-Many](#)
- [One-to-Many](#)

The **Route** command automatically names all created nets unless you specified names in the **Net** field of the **Route** toolbar. If you specify names, they are applied in order until none are available.

For information on using the `ise::route` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

Many-to-Many

To add groups of wires between sets of pins:

1. Choose **Add > Route**.

The **Route** toolbar appears. It contains the following options:

Option	Description
Nets	Specifies a net or a list of nets.
Cycle	Specifies to delete names from the Nets list when the associated wires are committed. When deselected, the first name in the list is reused.
Expand	When enabled, bus names are expanded to bit net names for each wire you add.
Commit Wires	Commits the preview routes.
By Name	When enabled, specifies to create logical connections between the objects by creating stubs with labels. This option applies to both many-to-many and one-to-many routes.
Avoid	◦ Shorts: The router avoids overlapping existing wiring object endpoints that would cause a short between two or more nets, whether the nets are named or not. ◦ Symbols: The router uses the object's selection box (or if that does not exist, the bounding box) as the avoidance region.
Type	Specifies the wire type.
Color	Specifies the wire color.

2. Specify the route options.
3. Select a source pin or pins.
4. (Optional) To add additional pins to the set, use Shift+click or Shift+region-select; to remove pins, use Control+click or Control+region-select. Alternatively, the Backspace key undoes all target pins first, and then all source pins with one click.
5. Select the destination pins.

A preview shows how the wires will be drawn between the source and destination sets.

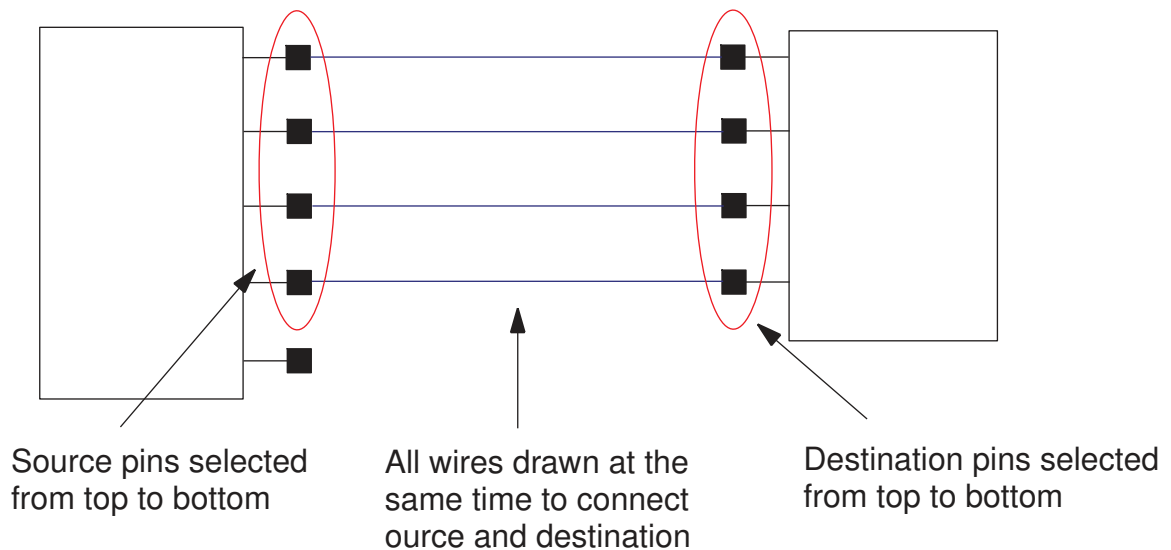
Note:

If the set of wires is not possible to route as a group (due to the topology not being supported, insufficient channel space, etc.) then pins are routed

one by one and independent of each other. Individual nets can be routed by name if there is no possible way to route the net.

6. (Optional) If the quality of the route is not acceptable there are two options:
 - From the **Route** toolbar, select **By Name** to create named wire stubs to complete the connectivity.
 - If selecting multiple pins, you can route in smaller groups.
7. Press Enter or select the **Commit Wire** button in the toolbar to complete the route.
8. Press Esc to end the command.

The following figure shows groups of wires drawn between two sets of pins.



One-to-Many

[Show me](#) how to add different types of routes in a design schematic.

You can use the **Add > Route** command in cases where you want to wire a single pin or wire to many pins. Your source or destination in the selection process can be a single pin or net. If a pin or net is an array, the tool routes that pin or net according to its size. When routing to a bus, auto tapping occurs as each pin (per order selected) connected to the bus is assigned a bit (or bits, according to its width). The following figures provide examples.

Examples

Figure 3 Trunk Routing (Supply Rail With Taps to Instances)

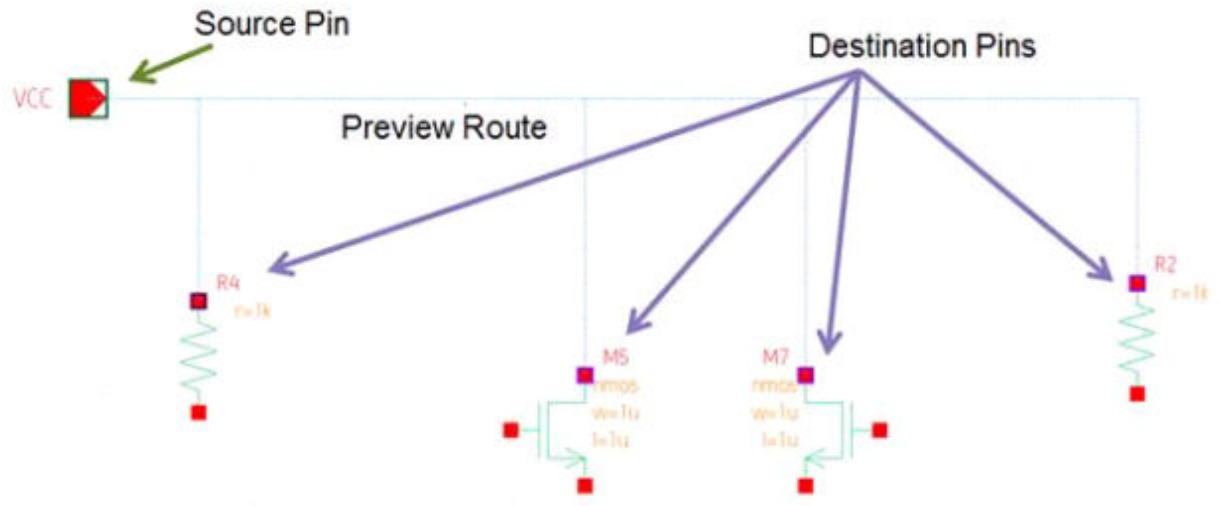
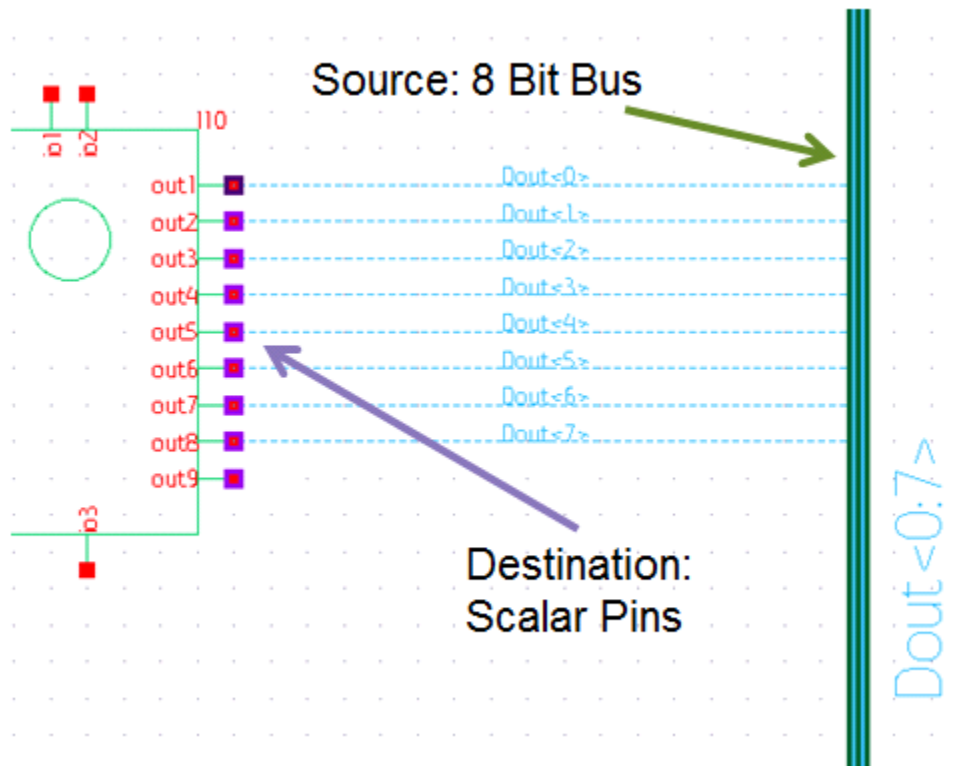


Figure 4 Bus Routing (Auto Bus Tapping to Scalar Bits)



Adding Wire Stubs

The **Add > Stub** command lets you quickly add wire stubs to existing pins. The wire stubs provide an easy way to probe and annotate the signals on those pins. The wire stubs also let you connect two nets by name or connect a number of pins to the same net.

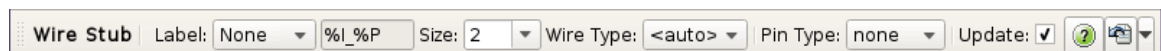
For information on using the `se::createWireStub` and `ise::createWireStub` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

[Show me](#) how to add wire stubs.

To add wire stubs:

1. Choose **Add > Stub**.

The **Wire Stub** toolbar appears.



The toolbar contains the following options:

Option	Description
Label	When the value of the wire label option is None, the stub is created with no wire label and it is auto-named. When the value of the net name option is Pin Name, a wire label with the name of the pin is created. When you choose User Defined, specify the name of the net in the edit box. If you leave the net name field empty, it is treated as if the None option has been chosen. The User Defined option also supports parametrized names with predefined variables %I and %P, where %I means instance name and %P means pin name. The %I and %P parameters are replaced by the corresponding instance and pin names.
Size	Specifies the stub length in grids or enables wire segment length adjustment to corresponding label length (auto).
Wire Type	Specifies a wire type with one of the following: <auto> , thin , or fat .
Pin Type	Specifies a pin type for pins that are added to the wire stubs. The options are: ◦ none (no pin) ◦ noConn Pin (adds a "no connection" symbol to the end of each wire stub in the design, which prevents "Floating Instance Pins" connectivity rule violations. Note: To create the noConn Pin in the general terminal, use the <code>seStubNoConnRef</code> preference. The default is set to <code>basic/noConn/symbol</code>. To create the noConn Pin in the optical terminal, use the <code>seStubOpticalNoConnRef</code> preference. The default is set to <code>photonicLib/opticalNoConn/symbol</code>.) ◦ auto (adds pins according to pin direction) ◦ input ◦ output ◦ inputOutput ◦ switch ◦ tristate
Update	When enabled (default), updates already existing stubs for missed pins or labels according to the specified label or pin creation policy. When disabled, do not process any existing stubs.

2. Specify the wire stub options.

3. Use one of the following methods to select pins:

- Click individual pins to add stubs one at a time.
- Click and drag to select a set of pins to add stubs all at once.
- Click in the middle of a symbol to add multiple stubs to unconnected pins on all sides of the symbol. A preview is shown for the stubs as you hover in the different regions of the symbol.
- Click closer to a symbol edge to add stubs only to the pins on one side of the symbol (top, bottom, left, or right). A preview is shown for the stubs as you hover in the different regions of the symbol.
- Click closer to a symbol corner to add stubs to the pins on adjacent sides of the symbol. A preview is shown for the stubs as you hover in the different regions of the symbol.

After clicking, a wire stub extends from each pin.

If you selected a label option, the labels appear with the stub.

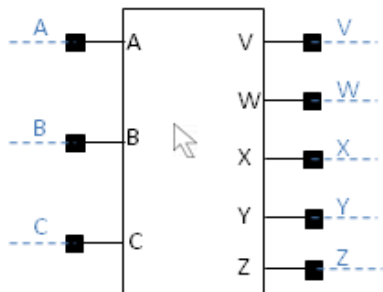
Note:

- Pins already attached by a wire are excluded from the wire stub command.
- The command does not create net labels for the instance terminal connected to other instance terminals or connected to multiple I/O pins.

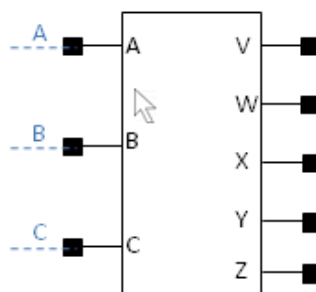
When creating on overlapped pins, the first pin of the overlapped pins is used as the default wire/pin name, which can be renamed after creation.

4. Press Esc to end the command.

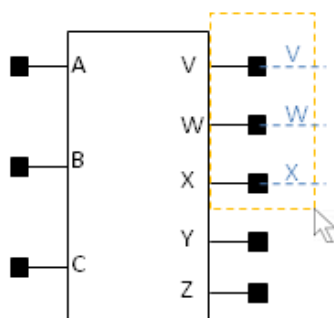
The following figure shows various wire stub creation modes.



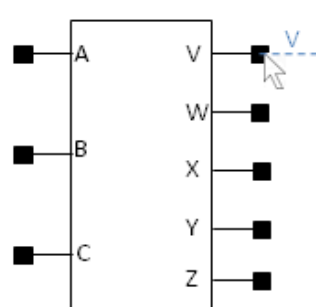
Mouse towards center of instance draws wire stubs on all pins



Mouse towards one edge of instance draws wire stubs on pins on that edge



Drag left mouse button to select pins to add stubs



Click on a single pin to draw a wire on it

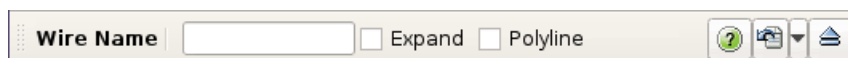
Adding Wire Names

For information on using the `se::createWireName` and `ise::createWireName` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

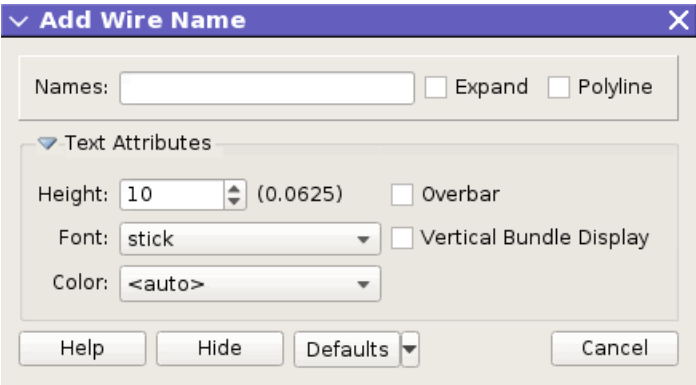
To add a wire name:

1. Choose **Add > Wire Name**.

The **Wire Name** toolbar appears.



To open the **Add Wire Name** dialog box, use the F3 key or click the **Eject** button on the toolbar after initiating the interactive command.



2. (Optional) Specify the wire name options.

The toolbar contains the following options:

Option	Description
Names	Specifies the wire names. You can also enter bus and bundle names. See also Stacking Wire Names .
Expand	When enabled, bus and bundle names are expanded to bit net labels for each wire name you add; for example, if bus name <code>n2<0:10></code> is entered in the Name field, the expanded bit net labels are <code>n2<0></code> , <code>n2<1></code> , ..., <code>n2<10></code> . As you place each bit net label, the bus range is decremented in the Names entry field until the last bit is placed, at which point the bus name disappears from the Name field along with any trailing spaces. You can use expanded names in conjunction with the polylines.
Polyline	When enabled, you must enter a sequence of two or more points defining a polyline. Intersection points of the polyline with existing wiring objects are determined and labels are assigned to the intersecting wires at the intersection points, sequentially along the polyline. The intersecting wire segments are grouped with the newly created label for that intersection point.
Text Attributes	(Dialog Box only) A toggle that expands and collapses the dialog box to reveal or hide the text attributes, such as text height, font, color, you can define. When Overbar is selected, displays an overline above the text. When Vertical Bundle Display is selected, sets the display mode for bundle names to vertical. See Adding Wire Names .

When you move your cursor over the canvas the wire name text appears on the cursor.

3. Click a wire segment in the canvas.

The wire name appears in the canvas, grouped with the wire segment. See also [Justification and Orientation](#).

Note:

If you click anywhere other than on a wire segment, the wire name appears at that location, but a ghost line is displayed from the wire name origin to your cursor, and you are prompted to select a wire segment. Once you click a wire segment, the wire name is grouped with the wire segment, and the ghost line disappears.

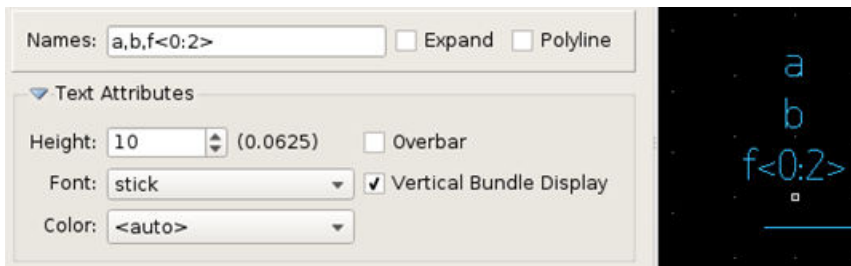
The command remains active.

4. Press Esc to end the command.

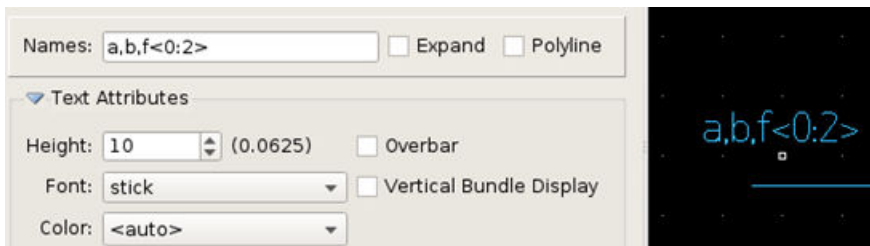
Setting Display Mode for Bundle Names

You can set the **Vertical Bundle Display** option to separate bundle name members by new line instead of comma.

In the following example, the bundle name `a,b,f<0:2>` is displayed vertically when **Vertical Bundle Display** is on.



When **Vertical Bundle Display** is off, the bundle name `a,b,f<0:2>` is displayed as follows.



Parameterized Connections on Wires

You can label wires using net expressions in order to assign a parameter that can be reconnected to another net at higher levels of hierarchy. These are called *connection*

definitions and are typically used to pass connections through a hierarchy without adding pins. For more information about creating parameterized connections see [Creating Connection Definitions](#).

Understanding Connectivity With Wires

The following sections describe wiring connectivity:

- [Wire-by-Name](#)
- [Wire-to-Iterated Instance](#)

Wire-by-Name

When more than one physically unconnected wire exists for the same signal, then that signal is said to be *connected by name*. Assigning the same name to each wire connects the instance pins, without physically connecting the wires.

Wire-to-Iterated Instance

For schematic and symbol pins, a width comparison is made to ensure the assigned net has the correct width. The width for a schematic pin is defined by the width of the terminal associated with the pin.

For symbol pins, these are the valid connection widths:

- The product of the master terminal width and the associated instance dimension.
For example, for a symbol pin whose terminal is 'A<0:2>' (width of 3) from a vectored instance with dimension 2, the width of the instance specific symbol pin is 6 (3 * 2).

- The width of the terminal

From the previous example, the value would be 3.

The Schematic Editor lets you connect a wire to an iterated instance based on the following three cases:

Case 1: Wire net width equals the connection width

When the following condition is true, a connection can be made directly.

```
wire net width = instance width X symbol pin terminal width
```

Case 2: Net width equals the symbol pin width

When the following condition is true, the tool accounts for the iterated instance by placing a multiplier on the wire net that is equal to instance dimension.

```
net width = symbol pin width
```

For example, if the pin is a<2:0> and the instance name is I1<3:0>, the wire net name remains b<2:0>, but the associated instTerm is connected to <*4>(b<2:0>).

Note:

For scalar instances, the dimension is 1, and Case 1 applies.

Case 3: Wire net width value is 1

When the following condition is true,

```
wire width = 1
```

a multiplier is added to the net connecting to the symbol pin, where

```
multiplier = instance dimension X symbol pin width
```

Note:

For scalar instances, the dimension is 1, so the multiplier would be the pin width.

Creating Global Net Names

Global nets make implicit connections across the design hierarchy without using pins and wires. Global nets are used primarily for power and ground connections.

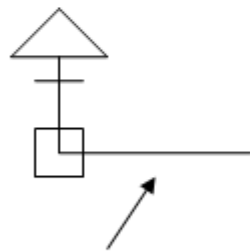
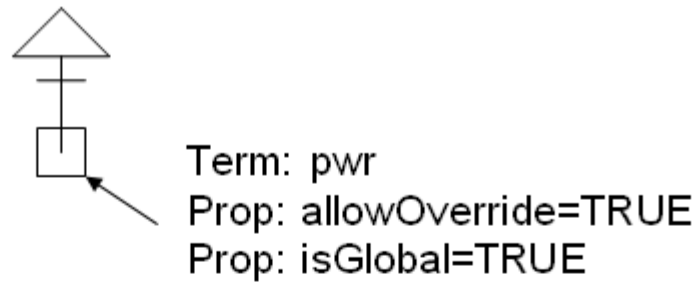
You can create a global net by adding a "!" to a net name or by connecting the net to a global pin or a global instance pin.

Note:

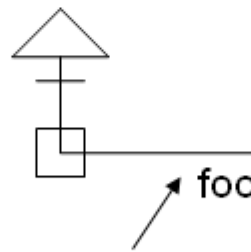
The net must be marked global in all designs in the design hierarchy, either by adding "!" to the net name in each cellview, or by connecting the net in each cellview to a global pin.

The Schematic Editor treats schematic nets with names ending with a "!" as global nets, but the Schematic Editor does not require ending a global name with a "!". The *basic* library, which comes with the Custom Compiler installation, has various global symbols that you can use to create global nets.

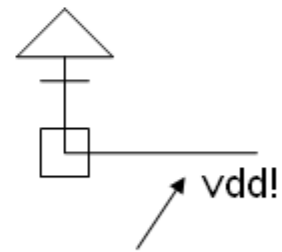
The Schematic Editor supports the concept of a global symbol that promotes any net to which it is attached to be a global net, as illustrated in the following figure. If a named net, global or local, connects to this symbol pin, the connecting net name is used and marked global.



Assigned net
pwr marked
global



Assigned net
foo marked
global



Assigned net
vdd! marked
global

Creating Pins

Pins are used as the physical interfaces on a block that allow connections through hierarchy. You can add pins to the canvas using the **Create > Pin** command. You can create scalar or bus pins by naming the pins appropriately (see [Naming Pins](#)). The pin creation command also supports name stacking (see [Pin Name Stacking](#)).

For information on using the `se::createPin`, `ise::createSchematicPin`, and `ise::createSymbolPin` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

The following preferences let you specify the pin symbols for creating the input, output, or input-output pins:

- `seSchematicPinInput`
- `seSchematicPinOutput`
- `seSchematicPinInputOutput`

For information about setting these preferences, see the *Custom Compiler Tcl Reference Manual*.

This section includes the following topics:

- [Naming Pins](#)
- [Pin Name Stacking](#)
- [Unnamed Pins](#)
- [Adding Single Pins](#)
- [Adding Multiple Pins Using Name Stacking](#)
- [Adding Multiple Pins Using Iterated Names](#)
- [Using Command Options](#)

Naming Pins

Pins can be named in the **Pin** toolbar using scalar or bus notation. For example, a pin named `a` or `a<1>` is a single-bit scalar pin and a pin named `a<3:0>` is a 4-bit bus pin. Pin names can also be bundled by listing different names separated by a comma. For example, `a,b,c` is a 3-bit bundled pin.

Pin Name Stacking

A pin name stack consists of a list of space-delimited pin names in the **Names** field.

When a stack of names exists, the first name in the stack is processed first; for example, when you add a pin to the canvas, the first name and trailing space are removed from the **Names** text field. As you add additional pins to the canvas, the corresponding names and trailing spaces are removed from the **Names** text field. This process repeats until the **Names** text field is empty. If you undo a pin addition, the name is returned to the top of the stack.

Unnamed Pins

When unnamed pins are placed on labeled wires they automatically take the name of the net on the wire.

Adding Single Pins

To add a single pin:

1. Choose **Add > Pin**.

The **Pin** toolbar appears.

2. (Optional) Specify a pin name.

Note:

If you place an unnamed pin on an existing wiring object, the pin inherits the name of the current net.

3. (Optional) Specify the remaining pin options.
 4. (Optional) Press Alt+A to toggle the alignment cursor. When enabled, cross hair cursor lines appear to help guide the placement. These lines only appear when pins from other objects are in the same vertical or horizontal plane as the pins being moved. The aligned pin is highlighted with a diamond shape around the pin.
 5. Click the canvas to place the pin.
The command remains active.
 6. Press Esc to end the command.
-

Adding Multiple Pins Using Name Stacking

A pin name stack consists of a list of space-delimited pin names in the **Names** field. With each click on the canvas, a pin is placed, and the corresponding name is removed from the stack.

To add multiple pins using a name stack:

1. Choose **Add > Pin**.

The **Pin** toolbar appears.

2. Specify a name stack.
3. Specify the remaining pin options.

4. (Optional) Use the shortcut keys [and] to switch among types of general pins such as input, output, and inputOutput.
5. (Optional) Press Alt+A to toggle the alignment cursor. When enabled, cross hair cursor lines appear to help guide the placement. These lines only appear when pins from other objects are in the same vertical or horizontal plane as the pins being moved. The aligned pin is highlighted with a diamond shape around the pin.
6. Click the canvas to place one pin for each name in the stack.

The command remains active.
7. Press Esc to end the command.

Adding Multiple Pins Using Iterated Names

Iterated names are for use with a bus or a bundle. You can scalarize them as they are placed by using the **Expand** option.

To add multiple pins using a name stack:

1. Choose **Add > Pin**.

The **Pin** toolbar appears.
2. Specify a bus name.
3. Select the **Expand** option if scalarization is needed.
4. Specify the remaining pin options.
5. (Optional) Press Alt+A to toggle the alignment cursor. When enabled, cross hair cursor lines appear to help guide the placement. These lines only appear when pins from other objects are in the same vertical or horizontal plane as the pins being moved. The aligned pin is highlighted with a diamond shape around the pin.
6. Click on the canvas to place one pin for each name in the bus.

The command remains active.
7. Press Esc to end the command.

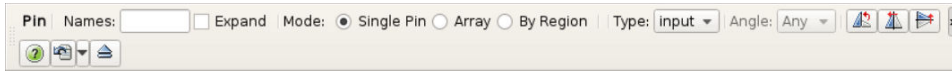
Using Command Options

The following sections describe the options on toolbar, dialog box, and menu:

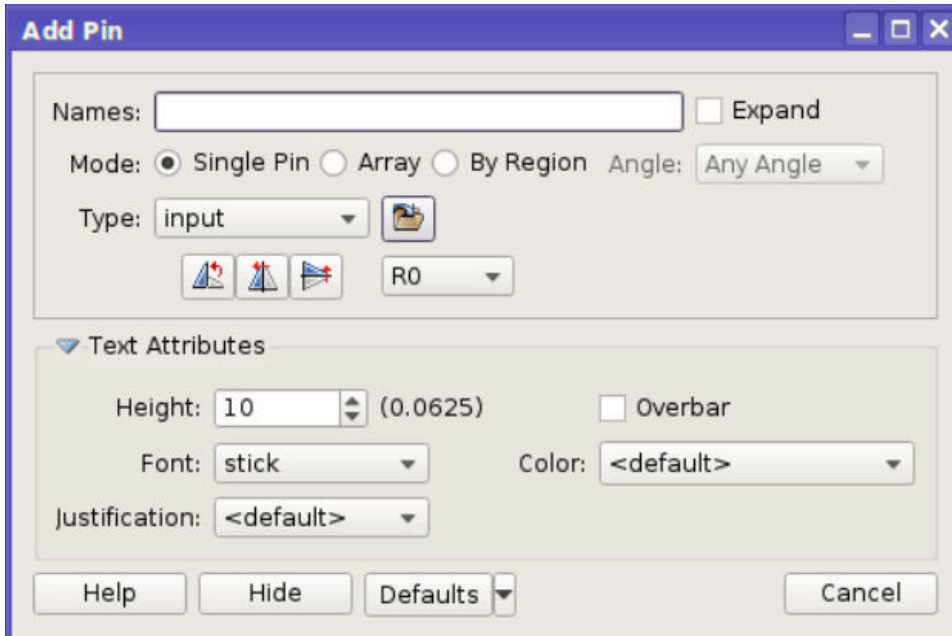
- [Add Pin Toolbar and Dialog Box Options](#)
- [Add Pin Menu Options](#)

Add Pin Toolbar and Dialog Box Options

The following figure illustrates the **Pin** toolbar.



When the **Add > Pin** command is active, pressing the shortcut key F3 or clicking the **Eject** button on the toolbar shows the **Add Pin** dialog box.



The dialog box and toolbar contain the following options:

Option	Description
Names	Specifies the pin names. You can also enter bus and bundle names.
Expand	When enabled, bus and bundle names are expanded to individual terminal names or bit terminals.
Array	Specifies that the pins in the Names field form an array.
Type	Specifies a pin type for all pins listed in the Names field. The options are: input , output , inputOutput , switch , and tristate .
Angle	Specifies how to place an array of pins. The options are: Orthogonal , Diagonal , or Any Angle . See Angle Mode for Creating and Editing Objects .
Transform Icons	These options let you rotate or flip the pin before placement.

Option	Description
Text Attributes	(Dialog box only) A toggle that expands and collapses the dialog box to reveal or hide the text attributes you can define.

Add Pin Menu Options

The menu can be shown during the **Add Pin** command by right-clicking. The menu contains these options:

Command	Description	Default Shortcut Key
Rotate 90	Rotates the pin 90 degrees counter clockwise	R
Flip Horizontal	Flips the instance about the Y axis.	Shift+R
Flip Vertical	Flips the instance about the X axis.	Ctrl+R

Creating Connection Definitions

A connection definition lets you parameterize a pin or wire, which lets you reassign the connection to the pin or wire at higher levels of hierarchy. Connection definitions are also known as parameterized connections. Connection definitions enable you to do the following:

- Use multiple power supplies in a design
- Override substrate connections
- Use parameterized power and ground symbols

You can add definitions to symbol pins (see [Adding Symbol Pins](#)) as well as to schematic nets and pins (see [Adding a Connection Definition](#)). For terminals, you specify the connection definition on an attached pin.

Connection definitions result in less wiring in a design because the pins already have a net connection. Consequently, when you draw a wire to a pin that has a connection definition, the net on the wire is automatically assigned the default net on the connection definition. If the wire is labeled, the connection definition does not apply and the name of the net is derived from the label.

A connection definition consists of two parts:

- The connection definition name
- The default net name

Figure 5 through Figure 11 show the use of connection definitions.

Figure 5 shows a connection definition that has been added on a symbol pin. The connection definition name is *invpow* and the default net name is *vdd*.

Figure 5 Connection Definition Added to a Symbol Pin



The tool assigns the default net name to the pin when the symbol is placed in the design. Figure 6 shows an asterisk (*) next to the net name indicating that a connection definition exists.

Figure 6 Asterisk Indicating a Connection Definition Exists

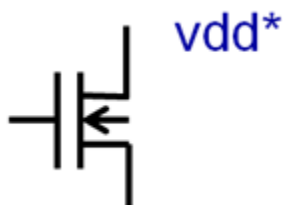


Figure 7 shows that the netlisting view of the MOSFET has a connection definition assigning the bulk node to connect to the source node.

Figure 7 Netlisting View of the MOSFET

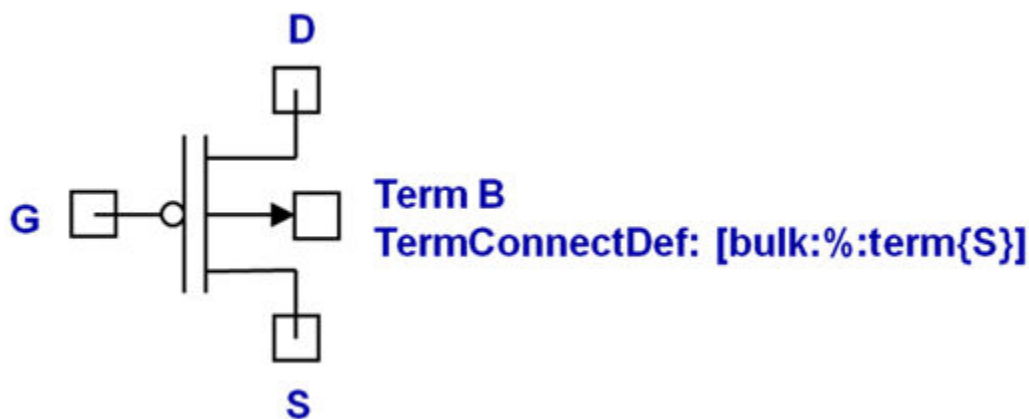


Figure 8 shows that the placed view of the MOSFET in the schematic does not have a pin for the bulk node, which means you do not have to wire it. You can still reassign the bulk node to connect to another net.

Figure 8 *Placed View of the MOSFET*

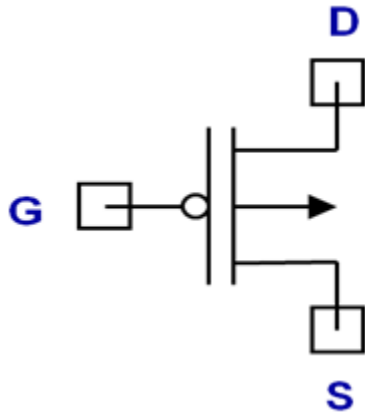


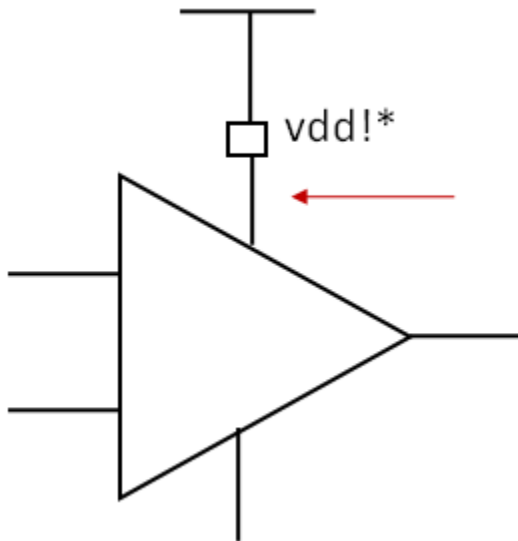
Figure 9 shows a power symbol with a connection definition assigning the default net vdd!.

Figure 9 *Assigning a Net to a Power Symbol*



Figure 10 shows that a net connected to the power symbol is automatically assigned net vdd!.

Figure 10 Assigning a Net From the Power Symbol



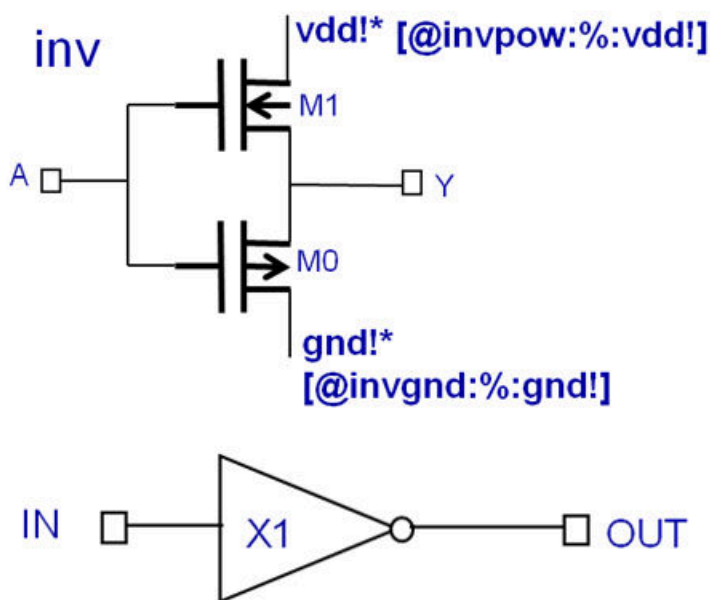
Connection definitions are netlisted as extra ports in the subcircuit definition with the default form `<nlVirtualTerminalsPrefix>_<connection definition name>_<default net name>`. `nlVirtualTerminalsPrefix` has a default value of `vt`. You can control the format of the virtual terminal name with the Tcl preference `nlVirtualTerminalsFormat`.

When the value of the `nlPromoteTerms` Tcl preference is true (the default), the top-level terminal names for connectDefs will be changed to match the default net names. This is used for CDL to ensure that the top-level pins match the layout net names.

For example, assume a connectDef `@power:%:vdd!` that can exist anywhere in the hierarchy. When the Custom Compiler tool creates a terminal for this connectDef, if it is unassigned, the terminal is created in the top-level design. If `nlPromoteTerms` is false, the standard naming convention is applied: `vt_power_vdd!`. If `nlPromoteTerms` is true, the default net name is applied instead: `vdd!`

Figure 11 shows how an inverter subcircuit with connection definitions is netlisted.

Figure 11 Inverter Subcircuit With Connection Definitions



```
.subckt inv Y A vt_invpow_vdd! vt_invgnd_gnd!
M1 Y A vt_invpow_vdd! net1 pmos
M0 Y A vt_invgnd_gnd! net2 nmos
.ends inv
X1 OUT IN vdd! gnd! inv
```

To add connection definitions, use the **Connection Definition** command or insert a net expression label (see [Adding a Connection Definition](#)).

To connect to nets other than the default, you can create connection assignments on instances that contain connection definitions (for more information, see [Creating Connection Definitions](#)).

This section includes the following topics:

- [Creating Connection Assignments](#)
- [Adding a Connection Definition](#)

Creating Connection Assignments

To connect to different nets other than the default, you can create connection assignments on instances that contain connection definitions.

There are two methods for adding connection assignments:

- Override the connection definition by assigning a new net (Assign value).
- Redefine the connection definition by assigning a new connection definition name (Assign assignment).

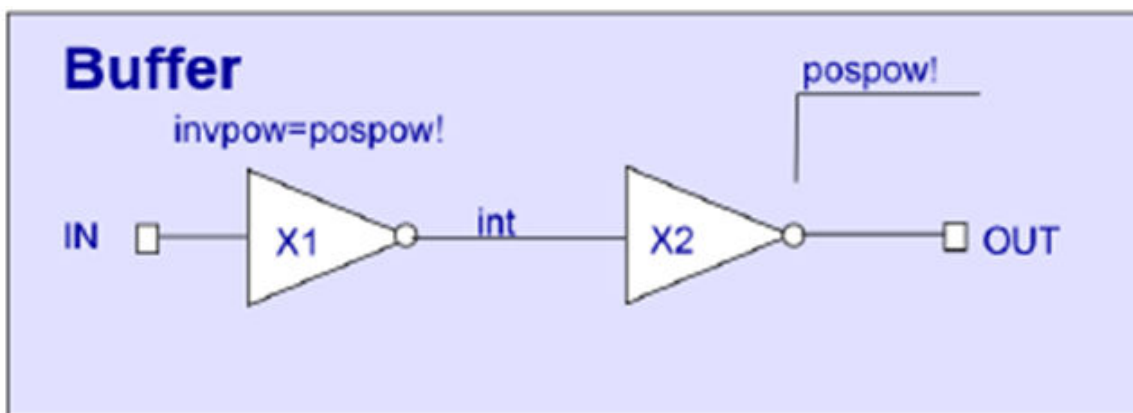
The hierarchy above which the connection definition is being defined, is searched for an oaAssignment object with the same name as the connection definition handle, and the value assigned to that object (oaAssignValue) is used as the value of the connection. You can define oaAssignments as parameters or properties on their instances.

You can redefine connection definitions with oaAssignValues that are themselves connection definitions, thereby redefining the connection definition for points further up the hierarchy. This means that an assignment can redefine a connection definition by supplying a new connection definition name and a default net name. These are Assign Assignments and the new definition is then used up through the hierarchy. The Custom Compiler Schematic Editor also supports cell-view-wide oaAssignments.

The Schematic Editor allows connection definition default net names that differ from the net name upon which the definition is placed. You can also use parameterized connections for non-global signals.

The Schematic Editor also lets you specify a default connection definition that references a terminal elsewhere on that same device. In other words, a designer could set a connection definition on a MOSFET bulk pin that has the default connection corresponding to whatever net ultimately connects to that same device's source pin.

The following figure shows a buffer circuit with a connection definition and [Example 1](#) shows how it is netlisted.

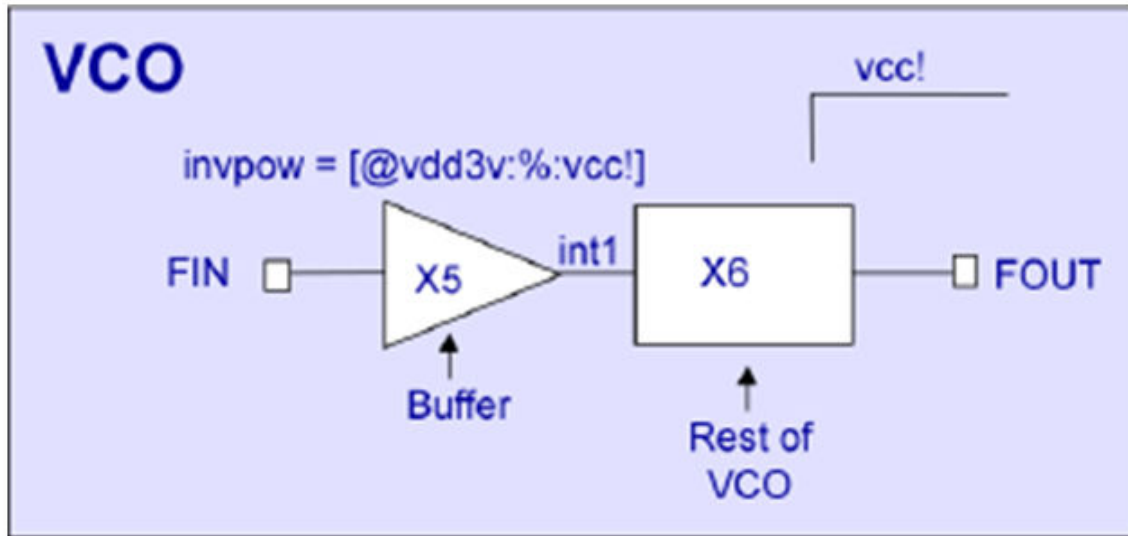


Example 1

```
.subckt buffer IN OUT vt_invpow vdd! vt_invgnd_gnd!
X1 IN int pospow! vt_invgnd! inv
```

```
X2 int out vt_invpow_vdd! vt_invgnd_gnd! inv
ends buffer
```

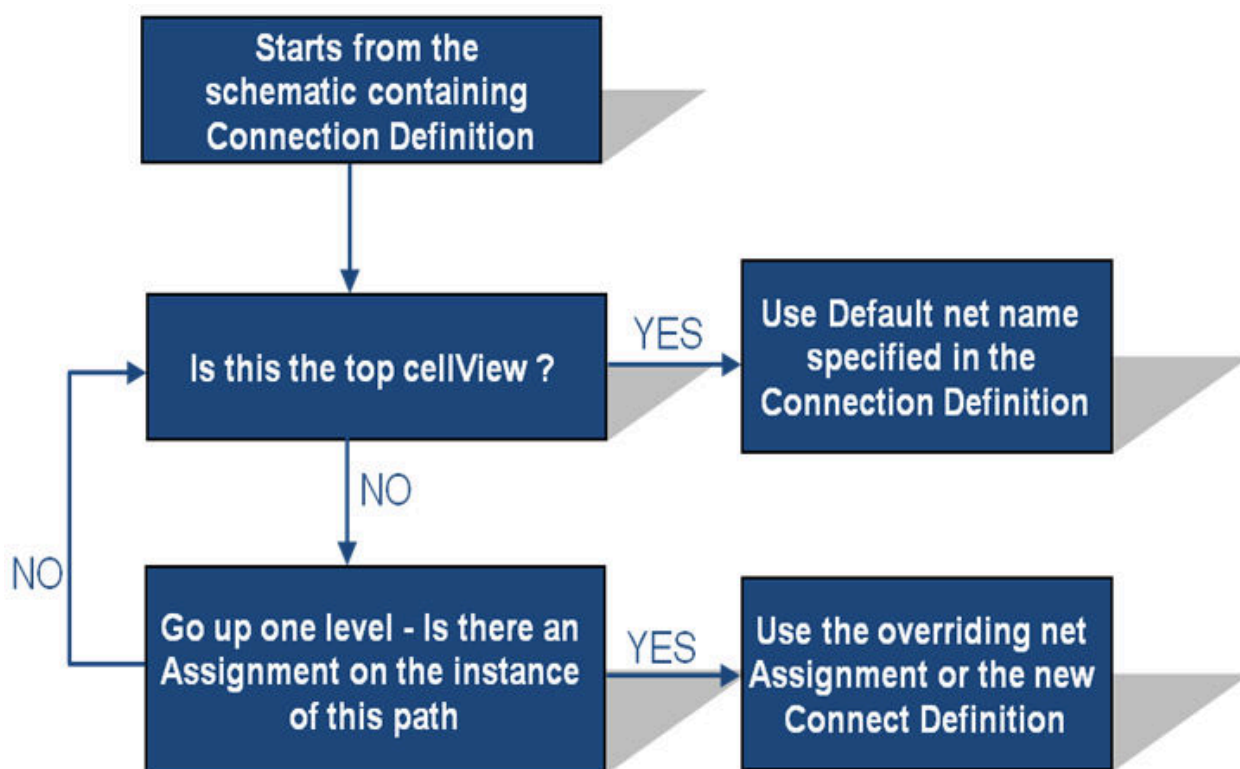
The following figure shows a VCO circuit with a connection definition and [Example 2](#) shows how it is netlisted.



Example 2

```
.subckt VCO FIN FOUT vt_vdd3v_vcc! vt_invgnd_gnd!
X5 FIN int1 vt_vdd3v_vcc! vt_invgnd_gnd! buffer
X6 int1 FOUT <rest of VCO circuit>
ends VCO
```

The following figure shows how the tool evaluates connection definitions.



Adding a Connection Definition

As you create a connection definition label, you must also select the pin or wiring object with which you want to associate the connection definition.

For information on using the `se::createConnectDef` and `ise::createConnectDef` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To add a connection definition label:

1. Choose **Add > Connection Definition**.

The **Connection Definition** toolbar appears.

The **Eject** button on the toolbar opens the dialog box, which lets you access the Text Attributes options. The **Hide** button at the bottom of the dialog box closes the dialog box and re-displays the toolbar.

The dialog box and toolbar contain the following options:

Option	Description
Name	Specifies the net expression name.
Default Net	Specifies the net expression default net.
Text Attributes	(Dialog box only) A toggle that expands and collapses the dialog box to reveal or hide the attributes you can apply.

2. Specify the connection definition name and a default net.

The connection definition name is used to create subsequent net assignments on instances or within designs higher in the design hierarchy.

The default net must exist in the design containing the connection definition.

Together, the **Name** and the **Default Net** comprise the net expression; for example, the **Name** “power” and **Default Net** “vdd!” are expressed as [!power:%:vdd!].

3. Move your cursor over the canvas.

The connection definition label follows your cursor.

4. Click the canvas to place the label.

The evaluated net name label (for example, vdd!*) appears on the canvas.

5. Click a pin or wire object to associate the label with the object.

Note:

The result depends on whether a pin or wire object is active (see [Defining the Label Location](#)).

The command remains active.

6. Press Esc to end the command.

This section includes the following topics:

- [Defining the Label Location](#)
- [Adding a Connection Assignment](#)

Defining the Label Location

Placement of the connection definition label on the canvas depends on whether the active object is a wire or a pin.

If either a wire or a pin is active when you click to define the label location:

- The active object becomes the parent object.
- The associated net or terminal is retrieved from the object.

The terminal or net must be a single bit net. If the terminal or net is a single bit, then a check is made for an existing definition. If there is an existing definition, you are prompted to replace the current connection definition or cancel the operation. If you choose to replace the current definition, the original label is deleted; if you cancel, you must select another object.

If no pin or wiring object is active when you click to define the label location:

- The tool prompts you to select a parent object for the connection definition label.

As you drag the cursor across the canvas, a line is drawn from the connection definition label to the cursor. As you move the cursor over a pin or wiring object, that object becomes active. If you click the active object, the object becomes the parent object.

You can modify the label attributes at anytime before the final object is selected.

Adding a Connection Assignment

Use one of the following methods to specify a connection assignment:

- Set the name and net under the Connection Assignments group in the Property Editor (see the *Custom Compiler Environment User Guide*).
- Add the assignments using the Connections Navigator (see [Editing Assignments](#)).

Creating Shapes

The following sections describe how to use shape types to annotate your schematics or to create symbol graphics:

- [Adding Lines](#)
- [Adding Rectangles](#)
- [Adding Polygons](#)
- [Adding Ellipses and Circles](#)
- [Adding Arcs](#)

For information on using the `ise::createShape` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

Adding Lines

For information on using the `se::createLine` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To create a line:

1. Choose **Add > Shape**.

The **Shape** toolbar appears.

2. Choose the line icon in the toolbar.
3. Specify the shape options.
4. Enter a sequence of two or more points defining a polyline.
5. Double-click or press the Enter key to complete the shape.

The command remains active.

6. Press Esc to end the command.

Adding Rectangles

For information on using the `se::createRectangle` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To create a rectangle:

1. Choose **Add > Shape**.

The **Shape** toolbar appears.

2. Choose the rectangle icon in the toolbar.
3. Specify the shape options.
4. Click and drag across the canvas to draw the rectangle, or click twice on the canvas to specify the two opposing corners.

The command remains active.

5. Press Esc to end the command.

Adding Polygons

For information on using the `se::createPolygon` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To create a polygon:

1. Choose **Add > Shape**.

The **Shape** toolbar appears.

2. Choose the polygon icon in the toolbar
3. Specify the shape options.
4. Click multiple points on the canvas to draw the polygon.
5. Double-click or press the Enter key to complete the polygon.

The command remains active.

6. Press Esc to end the command.

Adding Ellipses and Circles

For information on using the `se::createEllipse` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To create an ellipse or a circle:

1. Choose **Add > Shape**.

The **Shape** toolbar appears.

2. Choose the ellipse or the circle icon in the toolbar.
3. Specify the shape options.
4. Click and drag across the canvas to draw the ellipse or the circle, or click twice on the canvas to specify the center and extents points.

The command remains active.

5. Press Esc to end the command.

Adding Arcs

For information on using the `se::createArc` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To create an arc:

1. Choose **Add > Shape**.

The **Shape** toolbar appears.

2. Choose the arc icon in the toolbar.
3. Specify the shape options.
4. Click twice on the canvas to specify the diagonal of the arc.
5. Click a third time to specify the size and complete the shape.

The command remains active.

6. Press Esc to end the command.

Add Shape Toolbar Options

The toolbar contains the following options:

Option	Description
Shape Icons	Specify the shape you want to draw.
Angle	Specify the drawing angle.
Style	Use the three Style menus to specify the color, fill style, and line style for the shape you want to draw.
Width	Specify the line shape width in user units. A width of 0 (zero) creates an <code>oaLine</code> object. A width greater than 0 creates an <code>oaPath</code> object.

Add Shape Menu

The menu contains these options:

Command	Description	Default Shortcut Key
Commit	Completes the shape creation operation	Return
Remove Point	Deletes the previously inserted point.	Backspace

Creating Blocks

The **Add > Block** command supports "top-down" methodology by letting you create and place blocks (such as new cells) in the canvas prior to creating text or schematic views.

For information on using the `se::createBlock` and `ise::createBlock` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

 [Show me](#) how to add blocks.

To create a block:

1. Choose **Add > Block**.

The **Block** toolbar appears.

2. Specify the library, cell (either an existing cell or a new cell name), and view for the block.

Note:

A symbol view type is automatically created for new cells and added to the library after you place it.

3. Click the canvas to specify the first point of the rectangle.
4. Click the canvas to specify the second point and complete the rectangle.

The command remains active with a duplicate block attached to the cursor.

5. (Optional) To place additional instances of the new block just click at the desired locations on the canvas.
6. Press Esc to end the command.

Wiring Blocks

You can use the **Add > Wire** command to automatically add pins to blocks when you add wiring, as described in the following sections:

- [Wiring From a Block to a Pin](#)
- [Wiring From a Pin to a Block](#)

Wiring From a Block to a Pin

Use the following procedure:

1. Choose **Add > Wire**.

The **Wire** toolbar appears.

2. Specify the wire options (see [Creating Wires](#)).
3. Without clicking the canvas, move your cursor from the edge of a block to the block boundary line.

A pin appears on the boundary line.

4. Click the pin shape.

The **Add Pin to Block** prompt opens and displays the default pin name and type (see [Add Pin to Block Dialog Box Options](#)).

5. (Optional) Change the pin name.

Individual pin names, buses, and bit names are valid.

6. (Optional) Change the pin type.

7. Click **OK** to commit the pin.

The **Wire** command remains active.

8. Continue wiring.

Wiring From a Pin to a Block

Use the following procedure:

1. Choose **Add > Wire**.

The **Wire** toolbar appears.

2. Specify the wire options (see [Creating Wires](#)).

3. Start wiring from a pin to a block.

4. Move your cursor over the edge of the block boundary line.

A pin appears on the boundary line.

5. Click the pin shape.

The **Add Pin to Block** dialog opens and displays the default pin name and type (see [Add Pin to Block Dialog Box Options](#)).

6. (Optional) Change the pin name.

Individual pin names, buses, and bit names are valid.

7. (Optional) Change the pin type.

8. Click **OK** to commit the pin and terminate the wire.

The **Wire** command remains active.

Add Block Toolbar Options

The toolbar contains the following options:

Option	Description
Library	Specifies the library for the block you are creating. The default is the library for the current schematic.
Cell	Specifies the cell name for the block you are creating
View	Specifies the view name for the block you are creating. The default is symbol .

Add Pin to Block Dialog Box Options

Option	Description
Pin Name	Specifies the pin name.
Type	Lists the available terminal types.

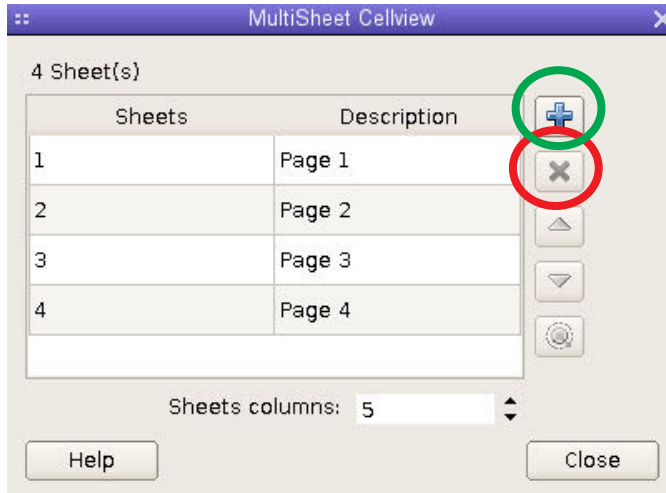
Creating Sheets

The **Add > Sheets** command supports working with multiple areas of a large design by letting you create multisheet cell views. Each design or section of a design in multisheet cell view is emulated as a sheet using sheet symbols for each page in the overview schematic.

To create a sheet:

1. Choose **Add > Sheets**.

The **MultiSheet Cellview** dialog box appears.

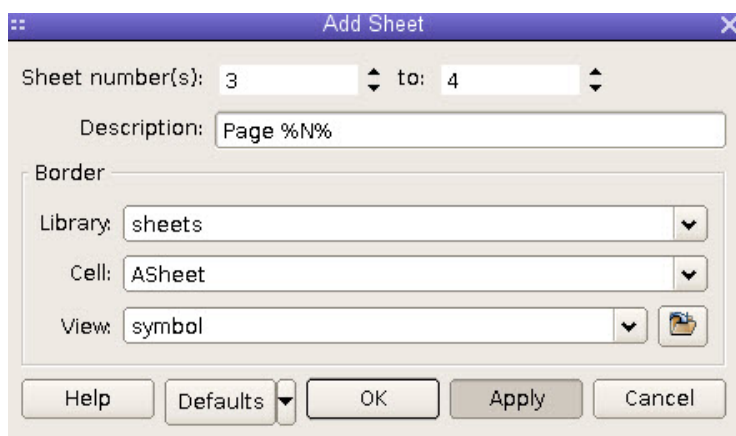


In the **MultiSheet Cellview** dialog box you can:

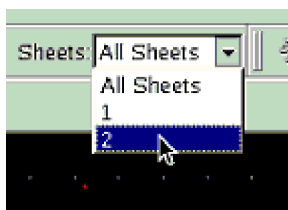
- Specify **Sheets columns** (the number of columns in the overview schematic where the sheets are distributed).
- Add or delete a sheet.
- Update sheets by repositioning or resizing them.
- Change a sheet's description.

A cell view automatically becomes a multisheet cell view when a sheet is added to it using the **Add new sheets** button (circled in green in the figure above). A multisheet cell view is reverted to a regular cell view whenever the last sheet is deleted from the list using the **Delete selected sheets** button (circled in red in the figure above).

Clicking the **Add new sheets** button opens the **Add Sheet** dialog box, in which you can specify where to insert sheet(s), a description for the new sheet, and the source of a sheet symbol view (see the following figure).



Creating a multisheet view adds the **Sheets** drop-down menu to the Schematic Editor navigation toolbar. Use this to change the sheet view, either by scrolling through individual sheets or choosing **All Sheets** to see all sheets at once (see the following figure).



2. Use **Add > By Name Connector** to add a by-name connector to mark graphically the connections between multiple sheets. You can connect nets from different sheets of the same cell using by-name connectors, schematic pins, global signals, or connect-by-name. A by-name connector provides a more clear way to show connect-by-name connections. By-name connectors identify the link between sheets and support same-name net connections in multiple sheets.

ERC rules (accessible using **Check > Setup > Rules > Multisheet** from the Schematic Editor window) are available to check multisheet designs (see the figure below). These rules include:

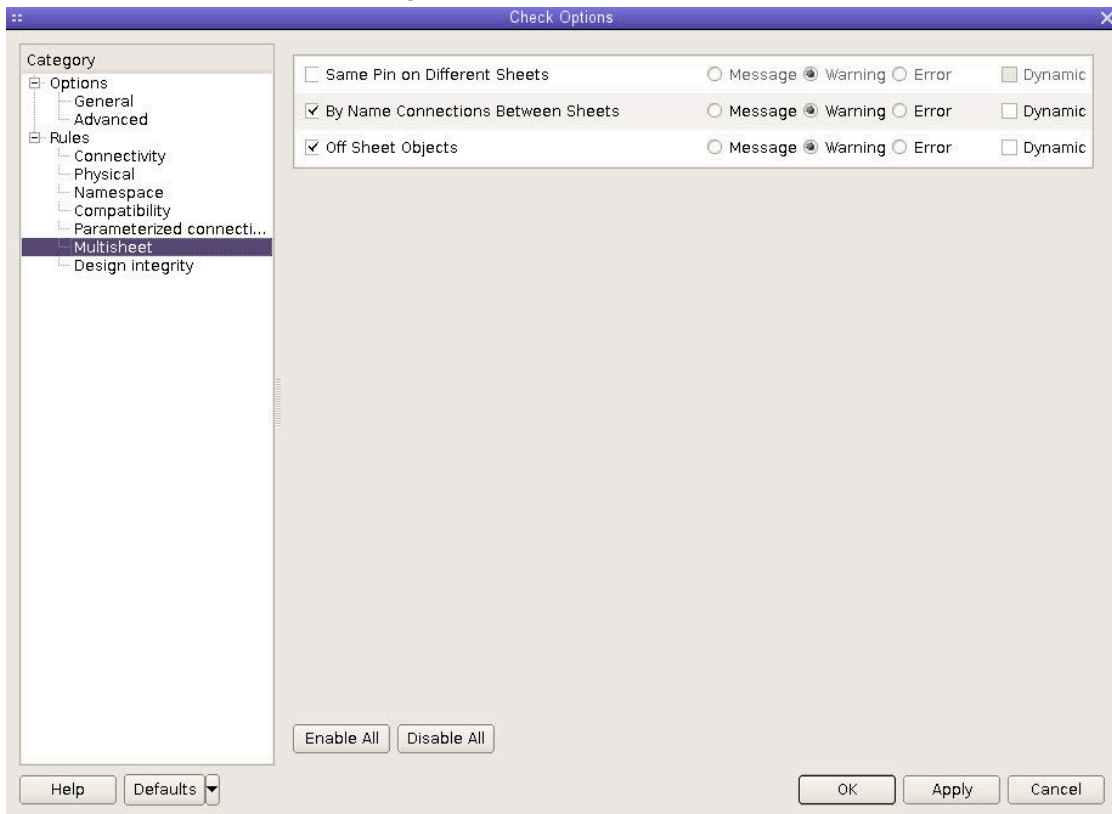
- **Same Pin on Different Sheets.** This is legal and off by default. When enabled, this check identifies pins of the same name on multiple sheets, which might not have been the user's intent. Multiple occurrence of the same pin results in multiple pins on the symbol, which may or may not be desired. Likewise, multiple pins of the same name, similar to **By Name Connections Between Sheets**, might result in undesired logical connections between sheets.
- **By Name Connections Between Sheets.** Returns net labels (names) found in more than one sheet that are not global signals or have sheet connectors

associated with them. This implicit form of connection leads to unexpected shorts. The rule is on by default. The default severity is "Warning."

A by-name connector symbol () indicates such connections between sheets.

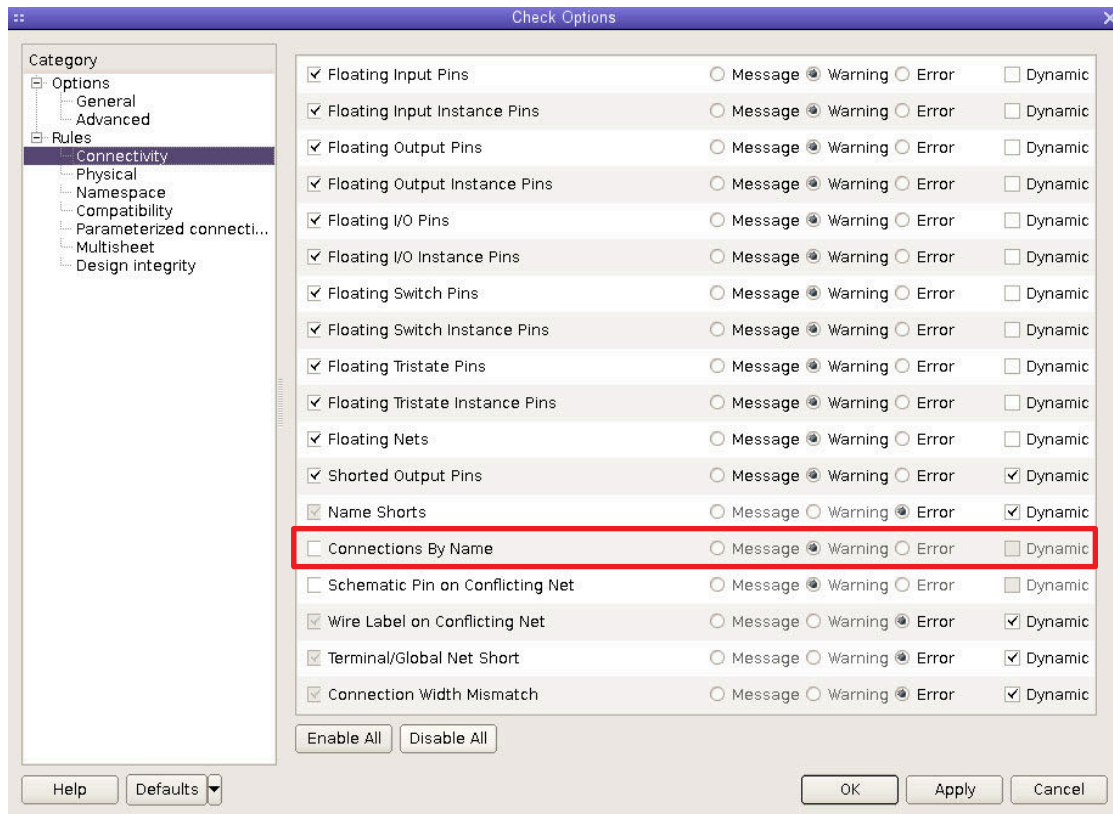
- **Off Sheet Objects.** In order for an object to be considered part of a sheet it must fully reside within the borders of the sheet symbol. This check finds objects outside of these borders so they can be fixed on the **All Sheets** page.

ERC Rules for Multisheet Designs

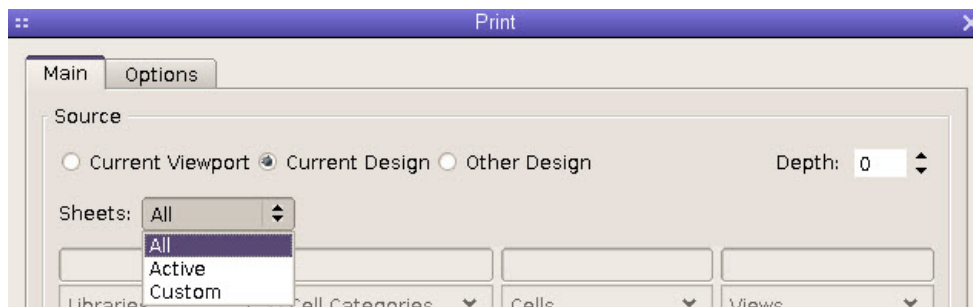


You can choose whether to report **Connections By Name** using **Check > Setup > Rules > Connectivity** from the Schematic Editor window.

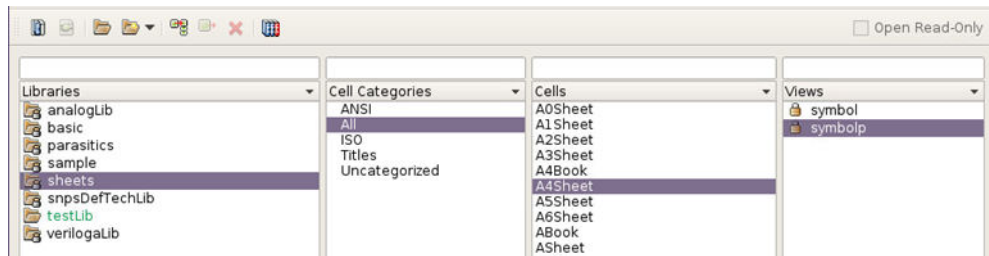
Connections by Name Rules



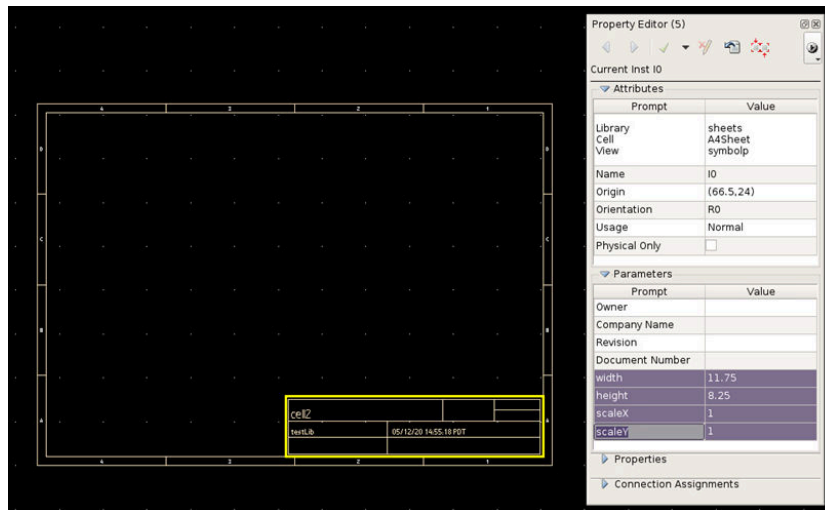
- To print a multisheet design use **Design > Print**. The **Print** dialog box shows the **Sheets** drop-down menu. Choose **All** to print all sheets in order and the whole design on the last page; **Active** to print only the current sheet; and **Custom** to specify a custom range of sheets to print (for example, 1, 3, 5-6).



- You can create a PCell cellview (symbolp) for each sheet symbol in the sheets library. This view allows you to adjust the sheet size using four additional parameters (width, height, scaleX, scaleY).



The symbolp view has four additional parameters (width, height, scaleX, scaleY) to control the size of the sheet symbol.



Adjust the values of the width, height, scaleX, and scaleY parameters to change the size of the sheet symbol.



Creating Constraints

By default, the software uses the constraint values in the technology file. However, you can create design-specific constraints to further restrict placement, routing, and electrical variance.

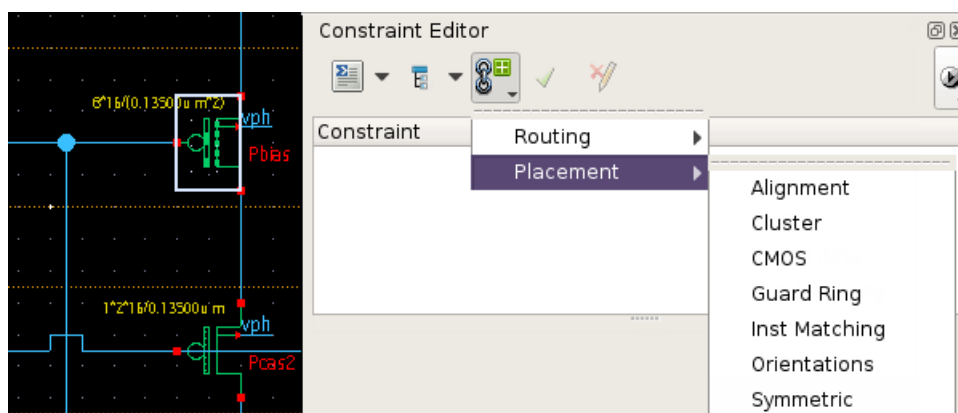
You can use the Constraint Editor to create, delete, modify, and highlight some design-specific Placement and Routing constraints in the schematic. These constraints are retrained with the design when you generate or open the corresponding layout using Schematic-Driven Layout (SDL). See *Generating Layout*.

For general information about the Constraint Editor Assistant, see the *Custom Compiler Environment User Guide*.

For information on the process constraints that apply to all designs, see the Constraints Overview in the *Custom Compiler Technology and Display Resource File Reference Manual*.

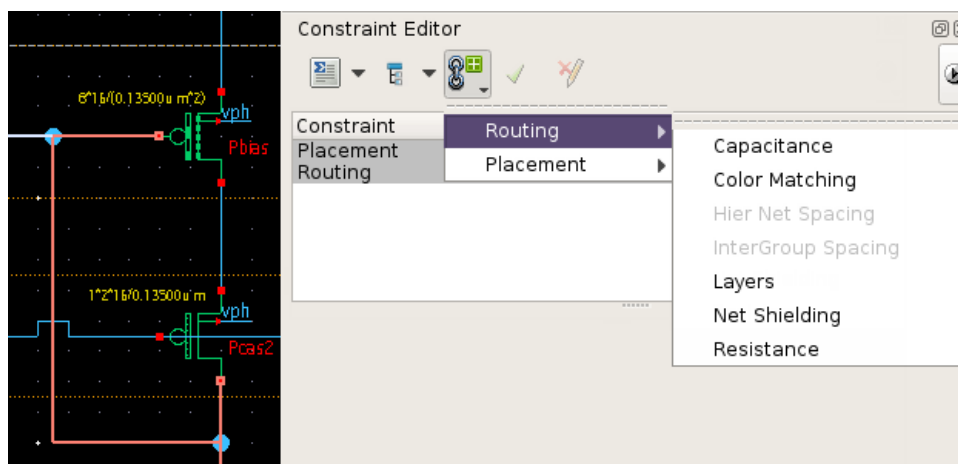
To create a new constraint:

1. Choose **Window > Assistants > Constraint Editor**. The Constraint Editor appears on the right side of the Schematic Editor window. For further instructions on how to use the Constraint Editor, see the *Custom Compiler Environment User Guide*.
2. Depending on the type of constraint you are creating:
 - On the canvas, select the objects to which you want to apply the constraint.
 - Display the Design Navigator, and select the instances or nets to which you want to apply the constraint.
3. In the Constraint Editor, click **Create Constraint** ().
4. To create a placement constraint, choose **Placement > Alignment, Cluster, CMOS, Guard Ring, Inst Matching, Orientations, or Symmetric**.



The newly added constraint appears in the **Constraint** list and is highlighted in the canvas if highlighting is on.

5. To create a routing constraint, choose **Routing > Capacitance, Color Matching, Hier Net Spacing, InterGroup Spacing, Layers, Net Shielding, or Resistance**.



The newly added constraint appears in the **Constraint** list and is highlighted in the canvas if highlighting is on.

For information about placement constraints, see Placement Constraints in the *Custom Compiler SDL and Visually-Assisted Automation User Guide*.

For information about routing constraints, see Routing Constraint Reference in the *Custom Compiler Custom Router User Guide*. The constraint data is transferred with the design to the layout via SDL.

For information about electrical constraints, see About Electrical Checking and Reporting in the *Custom Compiler Electrical Checking and Reporting User Guide*.

Annotating Designs

You can use the following to annotate your schematics:

- [Labels](#)
- [Notes](#)
- [Constraint Notes](#)
- [Images](#)
- [Tables](#)

For more information about annotating EM currents and VDRC voltages, see [Measuring Terminal Current Data Using the Schematic Editor and Annotating Schematics With Current Data in the Custom Compiler Electrical Checking and Reporting User Guide](#).

Labels

Labels are text objects that you can use to display:

- Static information, such as a description of the block that you want visible in higher level schematic designs.
- Object attributes, such as cell names.
- Application specific annotations, such as net names.
- Parameter values, such as the width and length of a MOSFET.

For information about working with labels, see [Managing Label Visibility](#).

The Schematic Editor supports the label types shown in [Table 11](#).

Table 11 Supported Label Types

Label Type	Description
Normal	When selected, static text that does not require any evaluation is created.
IL, SEL	When either is selected, the Functions menu lists the titles of currently registered label evaluation functions.

[Table 12](#) lists the predefined functions registered into the tool during startup.

Table 12 Predefined Functions

Type	Function	Function Notation
Normal	n/a	*
IL	Device annotation	cdsParam(%number%)
IL	Pin annotation	cdsTerm("\%termName%\")
IL	Instance annotation	cdsName()
IL	Net expression	cdsNetExpr()
IL	<none>	*

Table 12 Predefined Functions (Continued)

Type	Function	Function Notation
SEL	Instance label	[@instanceName]
SEL	Cell name	[@cellName]
SEL	Library name	[@libName]
SEL	<none>	*

Creating Labels

For information on using the `ise::createLabel` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To create a label:

1. Choose **Add > Label**.

The **Add Label** dialog box opens.

2. Enter the label text.

The text follows your cursor.

3. Complete the dialog box options.

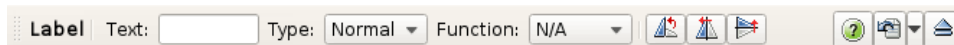
4. Click the canvas.

The label appears in the design and the command remains active.

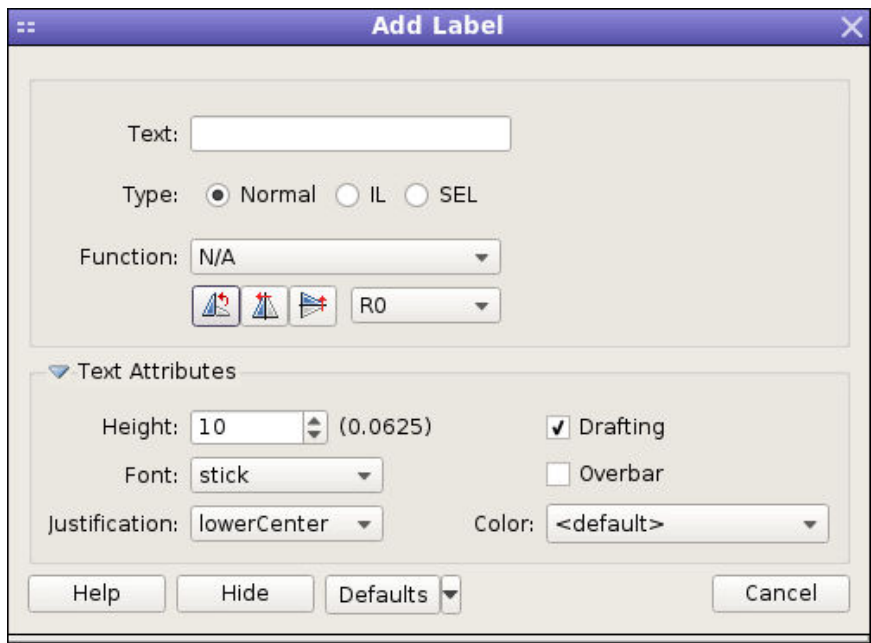
5. Press Esc to complete the command.

Add Label Toolbar and Dialog Box Options

The following figure illustrates the **Label** toolbar.



When the **Add > Label** command is active, pressing the shortcut key F3 or clicking the **Eject** button on the toolbar shows the **Add Label** dialog box.



The **Hide** button at the bottom of the dialog box closes the dialog box and displays the options in a toolbar. The **Eject** button on the toolbar opens the dialog box. You can set the `seCreateLabelDialogMode` preference to control whether the dialog box or the toolbar opens by default.

The toolbar and dialog box contains the following options:

Option	Description
Text	Enter the label text.
Type	Select the label type. Valid values are: Normal , IL , and SEL .
Function	You can select a label function when the type is IL or SEL . When you select Type IL and Function Net Expression , the following fields appear: • Name: Enter the expression name. • Default Net: Enter the default net name.
Transform	These options let you rotate or flip the label before placement.
Text Attributes	(Dialog box only) A toggle that expands and collapses the dialog box to reveal or hide the attributes you can apply.

Creating Custom Labels

You can create your own label functions to customize your display. For example, you could display the width and length of your MOSFET device "width/length" rather than as discrete labels.

The following procedure shows how to:

- Define a set of Tcl procedures for your label function.
- Register the function.
- Define a set of procedures for validating entries made during on canvas editing.
- Define a procedure to be run when you complete on canvas editing.

The following procedure shows how to create such a function using the `se::registerLabelFunction` Tcl command and how to use the new function with your devices.

Note:

For a new function, you must specify a Tcl default evaluation procedure that has the same name as the new function in notation. If you do not specify a procedure, the evaluation is not performed and the function notation is displayed directly in the label.

1. Define the return value (that is, the text you will see on the canvas for this label):

```
proc customMosWLDisplay {} {return "[labelName] = [labelValue]"}
```

2. Define how the value of the label is calculated.

```
proc labelValue {} {  
    set width [db::getParamValue w -evalType full -context  
    db::getCurrentContext] -of [db::getCurrentRef]]  
  
    set len [db::getParamValue l -evalType full -context  
    [db::getCurrentContext] -of [db::getCurrentRef]]  
  
    return $width|$len}
```

3. Define how the name appears on the canvas. In this example: "w|l"

```
proc labelName {} {return "w|l"}
```

4. Define how validation happens when you edit the value of this label with on-canvas editing. In this example, the label must consist of two values separated by a "/" character:

```
proc labelValidate {string} {  
    if {[regexp {././} $string]} {
```



```
    return true
  }
  return false
}
```

5. Commit the changes after on-canvas editing. In this example, the label string that you enter is parsed, and the values of *width* and *length* that you entered are used to set the parameter values on the instance:

```
proc labelCommit {string} {
  if {[regexp {(.*)/(.*)} $string match width len]}
    db::setParamValue w -value $width -of [db::getCurrentRef]
    db::setParamValue l -value $len -of [db::getCurrentRef]
  }
}
```

6. Register the label for use:

```
se::registerLabelFunction mosWLDisp -evalType IL -expr
{customMosWLDisplay()} -layer 237 -purpose -1 -nameEvalProc labelName
-valueEvalProc labelValue -validateProc labelValidate -commitProc
labelCommit
```

Registered labels appear in the **Function** drop-down menu in the **Add Label** dialog box when the type is set to **IL**.

Note in this step that `mosWLDisp` (the first argument of the `se::registerLabelFunction` command) specifies the name that appears in the **Function list**; `customMosWLDisplay` (the `-expr` argument) specifies the text that appears in the **Text** field and on your cursor when you start to place the label.

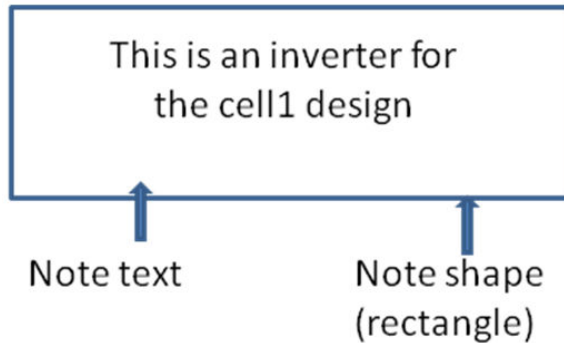
Note:

You can use any procedure names that you wish, although it is a good idea to enclose your procedures in a custom namespace so as to not conflict with other procedure names.

Notes

A note is a static text string containing one or more lines of text and exists on the Layer Purpose Pair (LPP) of the *text* view.

Optionally, you can associate a note with a shape object. The boundary of the shape represents the containing region for displaying the text. For example, you might draw a rectangle and associate it with a note (oaText object). The text is then rendered inside the rectangle.



This section includes the procedure for [Creating Notes](#).

Note:

For information about creating labels for symbols, see [Labels](#).

Creating Notes

For information on using the `ise::createNote` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To add a note:

1. Choose **Add > Note**.

The **Add Note** dialog box appears.

2. Enter the note text in the **Text** field.
3. Complete the dialog box options.

The **Hide** button at the bottom of the dialog box closes the dialog box and displays the note options in a toolbar. The **Eject** button on the toolbar opens the dialog box.

4. Move your cursor over the canvas.

The note text follows your cursor.

5. Click the canvas.

The note appears in the design and the command remains active.

6. Press Esc to end the command.

Add Note Dialog Box Options

The dialog box contains the following options:

Option	Description
Text	Enter the text message you want to add to your design.
Shape	Select the shape you want to associate with your note.
Editor	Opens your local text editor.
Transform Icons	These options let you rotate or flip the note before placement.
Attributes	A toggle that expands and collapses the dialog box to reveal or hide the text attributes you can define.

Add Note Toolbar Options

The toolbar contains the following options:

Option	Description
Text	Enter the text message you want to add to your design.
Editor	Opens your local text editor.
Shape	Select the shape you want to associate with your note.
Transform Icons	These options let you rotate or flip the note before placement.

Constraint Notes

Note:

This functionality is deprecated and will be removed in a future release. See instead [Creating Constraints](#).

A constraint note is a specially formatted schematic note defining a layout constraint. The Schematic Editor provides commands to help create constraint notes with the proper formatting.

Creating Constraint Notes

For information on using the `ise::createConstraintNote` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To add a constraint note:

1. Choose **Add > Constraint Note**.

The **Create Constraint Note** dialog box appears.

2. Choose the constraint type in the **Type** field.
3. Choose the object selection mode, by **Name** or by **Region**, in the **Objects** section.
 - a. For **Name** mode, enter the names of devices in the field. The names are case insensitive and wildcards are supported

- b. **Region** mode allows you to place the note inside a rectangle that contains the instances to constrain. If the rectangle does not already exist, click **Create Rectangle**, which is a shortcut to start `ise::createShape` of type rectangle.

After you create the rectangle(s) needed for the intended constraint notes, cancel the **Create Rectangle** command to return to `ise::createConstraintNote`.

4. Complete the dialog box options.
5. Move your cursor over the canvas.

The constraint note text follows your cursor.

6. Click the canvas.

The note appears in the design and the command remains active.

7. Press Esc to end the command.

Add Constraint Note Dialog Box Options

The dialog box contains the following options:

Option	Description
Type	Choose the type of constraint you want to create: Matching , Cluster , Ignore , Symmetry , Chain , or Variation .
Objects	Select the way you want to select objects for your constraint, by Name or by Region . If you choose Region and the rectangle does not already exist in the design, click Create Rectangle , then create the rectangle on the canvas.
Filters	Filter the specified objects based on the Device Type and Model Name you select here. Model Name supports wildcards.
Options	The optional parameters in the Options section vary according to the constraint type you choose in the Type section. Ignore and Chain types do not have any additional options.

Option	Description
	For Matching, enter the Pattern (for example, ABBA) and the X/Y Spacing for matching constraints. A single value indicates spacing in the X direction. Two values specify the X and Y spacing, respectively. When selected, Generate Guard Ring indicates that a guard ring is created for matching constraints. For Cluster, choose from the drop-down menu of Pattern options: <none>, col, col_right, col_left, row, row_top, or row_bottom. Select the X/Y Spacing for cluster constraints. A single value indicates spacing in the X direction. Two values specify the X and Y spacing, respectively. Select Attract objects to symmetry line to attract objects to the symmetry line. Select Keep Order to maintain the order of objects. When selected, Generate Guard Ring indicates that a guard ring is created for cluster constraints. For Symmetry, choose whether the axis is Vertical (the default) or Horizontal. Select Swappable if the symmetry constraint should allow swapping of objects across the symmetry line. Enter the Expression used to determine objects that should be paired for symmetry. For example: $M_1 * M_2 \Rightarrow M_{11}$ paired with M_{21}, M_{12} paired with M_{22} etc. For Variation, enter the list of allowable number of fingers for MOS devices in the MOS Fingers field. Enter the list of allowable number of resistor segments for MOS devices in the Resistor Segments field. Enter the list of allowable aspect ratios for capacitors in the Capacitor Aspect Ratios field. Orientations allows you individually to enable/disable the three supported orientations: R0, R90, and R180.

Images

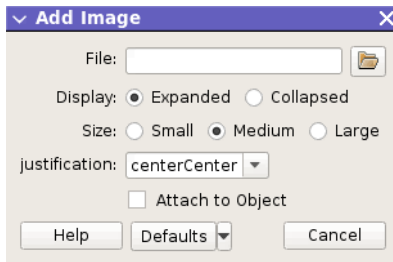
You can embed images, such as waveforms, in a schematic or symbol design. Editing image attributes can be done by selecting an image and editing it in the Property Editor.

When you print the design, the image is also printed.

To add an image:

1. From a schematic or symbol editor window, choose **Add > Image**.

The **Add Image** dialog box appears.



2. Browse to the image file you want to add. Supported formats include .bmp, .gif, .png, .jpg, .jpeg, .pbm, .pgm, .ppm, .xbm, and .xpm.
3. Select the **Display** mode: **Expanded** displays the image on the canvas, while **Collapsed** displays a small icon on the canvas, which you can click to expand the image.
4. Select the **Size** of the image: **Small**, **Medium** (the default), or **Large**.
5. Select the **Justification** of the image. Options include: **upperLeft**, **centerLeft**, **lowerLeft**, **upperCenter**, **centerCenter** (the default), **lowerCenter**, **upperRight**, **centerRight**, and **lowerRight**.
6. Attach an image file to any object (including, for example, instances, wires, pins, text notes, other images, and so forth) using **Attach to Object**. When you attach an image to an object, for example to a wire segment, if you move that object, the image also moves.
7. Click **Cancel** to exit the command.

Tables

A table object in the schematic and symbol view is represented by a table grid and text objects, which have a text layer number and drawing purpose number.

This section includes the procedures for [Creating Tables](#) and [Editing Tables](#).

Creating Tables

For information on using the `ise::createTable` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To add a table:

1. From a schematic or symbol editor window, choose **Add > Table**.
The **Add Table** dialog box appears.
2. Enter the name and title of the table in the **Name** and **Title** fields. The name must be a unique name, but the title does not need to be unique.

3. Adjust the location and justification of the name and title using the drop-down menus.
4. Complete the dialog box options for border and text.
5. Move your cursor over the canvas.

The table text follows your cursor.

6. Click the canvas.

The table appears in the design and the command remains active.

7. Press Esc to end the command.

Add Table Dialog Box Options

The dialog box contains the following options:

Option	Description
Name	Enter the name of your table in the text box. This must be a unique name. Choose the name's position from the drop-down menu.
Title	Enter the title of your table in the text box. Choose the title's position from the drop-down menu.
Border	Select the Width , Height , Color , and Line Style you want for your table. Alternatively, you can select Auto Size for the Width and Height .
Text	Select Height , Justification , Font , and Color of the text in your table. Choose Rotate First Row Text 90 to indicate a header row for your table.
Rows and Columns	Adjust the number of rows and columns in the table. Type the table cell text directly in the cells in the Add Table dialog.

Editing Tables

For information on using the `ise::editTable` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To edit a table:

1. Put the cursor on the table text and double-click or click Return.

The **Edit Table** dialog box appears.

2. Alternatively, the Q key opens the **Edit Table** dialog box rather than the Property Editor when working with table objects.
3. Edit the dialog box options. Table content can be edited using the add, delete, left, right, up, and down buttons next to the table widget. Table content (table texts) can also be edited also using the on-canvas editing functionality.

Connecting Components and Nets

This section describes the procedures for:

- [Wiring Symbol Pins](#)
- [Adding Patchcords](#)

Note:

When the **Dynamic Connectivity** option is enabled in the **General Options** dialog box on the **Schematic** tab, the connectivity engine keeps a connectivity graph of the entire design to provide connectivity information quickly when necessary (such as when connecting components and nets). When **Dynamic Connectivity** is turned off, the connectivity engine is disabled and connectivity information is only calculated when required.

For information on using the `se::createInst` and `ise::createInst` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

Wiring Symbol Pins

There are three cases where one of the new wire endpoints overlaps a symbol pin with a connection width value greater than 1. If any of the three cases exist, then the `instTerm` associated with the symbol pin is connected to the appropriate net, as described in the following sections.

Case 1: Wire net width equals the connection width

When the following condition is true, a connection can be made directly.

$\text{wire net width} \equiv \text{instance width} \times \text{symbol pin terminal width}$

Case 2: Net width equals the symbol pin width

When the following condition is true, the tool accounts for the iterated instance by placing a multiplier on the wire net that is equal to instance dimension.

$\text{net width} \equiv \text{symbol pin width}$

For example, if the pin is `a<2:0>` and the instance name is `I1<3:0>`, the wire net name remains `b<2:0>`, but the associated `instTerm` is connected to `<*4>` (`b<2:0>`).

Note:

For a scalar instance, the dimension is 1 and Case 1 applies.

Case 3: Wire net width value is 1

When the following condition is true

$\text{wire width} \equiv 1$

a multiplier is added to the net connecting to the symbol pin, where

$\text{multiplier} \equiv \text{instance dimension} \times \text{symbol pin width}$

Note:

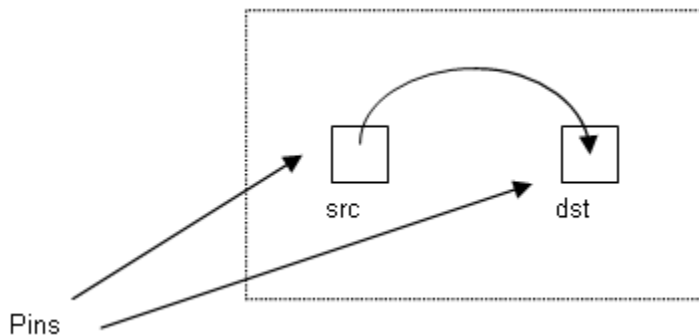
For scalar instances, the dimension is 1, so the multiplier would be the pin width.

Adding Patchcords

To join two or more nets, you place a component called a patchcord. You can use a patchcord to copy one net to another or to map bits of one net to different bits of another net. You can also use patchcords to merge two buses or to connect multiple nets. Patchcords can be connected between nets of unequal widths.

The patchcord symbol has two pins and a property `schType` with a value of `patchCord`. There is a CDF parameter named `schPatchExpr` with a default value of `0=0`. The members of the nets that physically touch the symbol pins are considered to be shorted together based on the value of the `schPatchExpr` parameter. You can use an instance property value override for `schPatchExpr`.

The symbol pin names must be `src` and `dst`, which indicate the order in which to apply the patch expression mapping.



This section includes the following topics:

- [Understanding Connectivity With Patchcords](#)
- [Creating Patchcords](#)

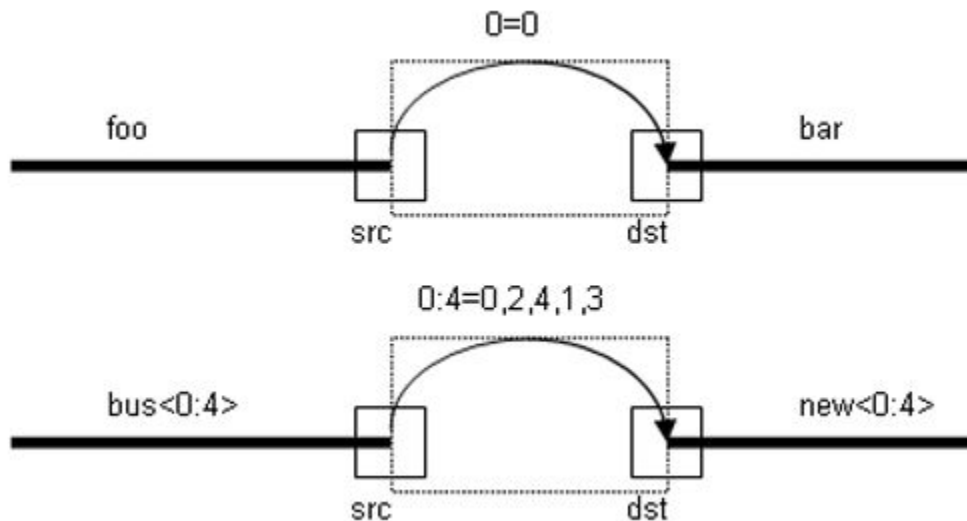
Understanding Connectivity With Patchcords

The connection expression describes a bitwise mapping between the net connected to one of the pins and to the net connected to the other.

A simple example is when two single bit nets are connected to either side of a patch symbol. The default connection expression of $0=0$ means that bit 0 of one net maps to bit 0 of the other net. This is a simple name short or aliasing.

You can also use a patchcord to reorder bus bits. For example, a connection expression of $0:1=1:0$ means to switch bits 0 and 1. So if the buses $a<0:1>$ and $b<4:5>$ connected to this patch, the net equivalence of $a<0>=b<5>$ and $a<1>=b<4>$ would be created. Note that the mapping is positional and not relative to the actual bit number describing the bit net.

The following figure shows how you can use a patchcord to alias net foo and net bar. It also shows a more complex bus alias with bit reordering using the connection expression of $0:4=0,2,4,1,3$.



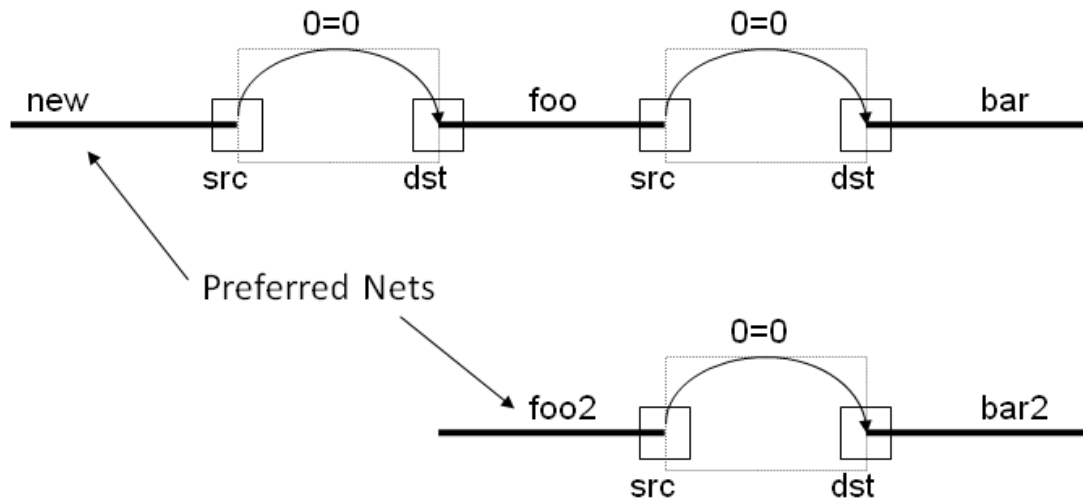
The syntax for connection expressions uses the bus notation syntax. So $0:4$ indicates a range starting from 0 and ending with 4. The expression can also be written in a more verbose, but correct, syntax of $0,1,2,3,4=0,2,4,1,3$. In this expanded form, it is easy to see how each position in the sequence of numbers in the left and right side of the expression maps to the corresponding bit indexes. The expression aliases $bus<0>$ to $new<0>$, $bus<1>$ to $new<2>$, $bus<2>$ to $new<4>$, $bus<3>$ to $new<1>$, and $bus<4>$ to $new<3>$.

Only bit indexes explicitly represented in the expression are aliased. If the bottom patch in above figure had an expression of $0=0$, only `bus<0>` and `new<0>` are aliased.

When two or more nets are aliased, or marked equivalent, one of them is set as the preferred net. The rule precedent for selecting which net in an equivalent set is the preferred net is as follows:

- The net marked global
- The net that does not connect to a dst pin of a patchcord.

See the following figure for an example.



Creating Patchcords

To create a patchcord:

1. Choose **Add > Instance**.
2. Select **Library** (basic), **Cell** (patch), and **View** (symbol).
3. In the **Parameters** region of the dialog box, specify the patchcord expression before placing it.
4. Click the canvas to place the patchcord.

Drawing Rulers

The Symbol Editor lets you draw linear and circular rulers in any design. Rulers are objects, so they are saved with the design. You can select rulers and then move, stretch, or delete them.

For information on using the `ise::createRuler` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

Rulers are labeled in user units measured from the ruler origin. Linear rulers have one or more segments.

[Figure 12](#) and [Figure 13](#) show examples of linear rulers. [Figure 14](#) shows examples of circular rulers.

Figure 12 Orthogonal Linear Ruler

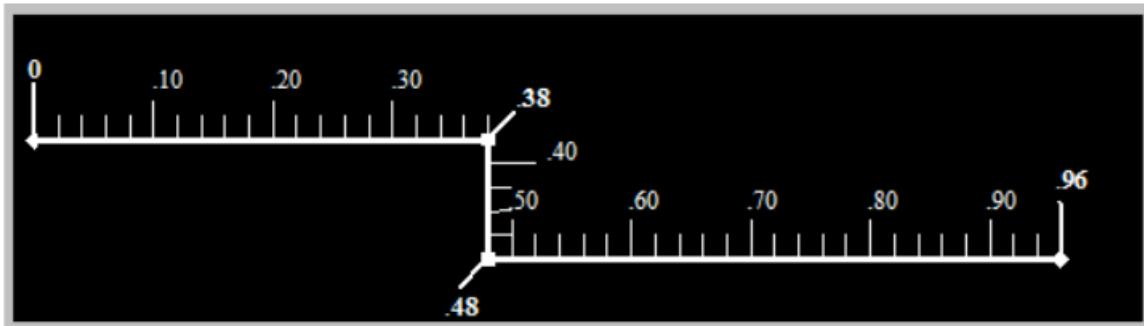


Figure 13 Any-Angle Linear Ruler

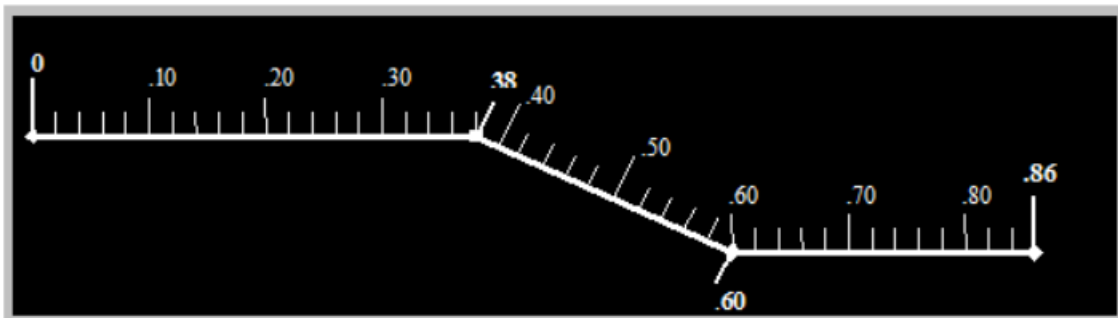
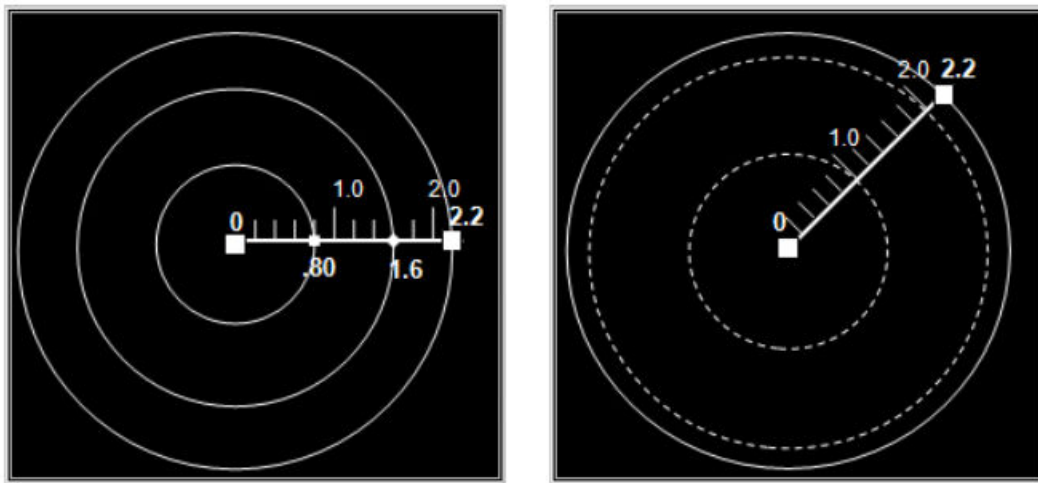


Figure 14 Circular Rulers



Drawing Linear Rulers

You can simultaneously draw both the horizontal and vertical rulers between the edges of the nearest selectable objects and measure them.

To draw the horizontal and vertical rulers:

1. Choose **Add > Ruler**.

The toolbar appears.

2. Choose the **Linear** shape type.
3. Choose **Auto** from the **Points** menu.

The other options become disabled.

4. Move your cursor between two selectable objects.
5. Click to specify the ruler origin.

The tool draws the horizontal and vertical rulers between the nearest edges.

The command remains active.

6. Press Esc to complete the command.

Note:

If there are no selectable objects in either the horizontal or vertical direction, the corresponding ruler is not drawn.

Drawing Single-Segment Linear Rulers

To draw a single-segment ruler:

1. Choose **Add > Ruler**.

The toolbar appears.

2. Choose the **Linear** shape type.
3. Choose **Two** from the **Points** menu.
4. (Optional) Change any of the other available options.
5. Click to specify the ruler origin.
6. Click to specify the end of the segment.

The command remains active.

7. Press Esc to complete the command.

Drawing Multisegment Linear Rulers

To draw a multisegment ruler:

1. Choose **Add > Ruler**.

The toolbar appears.

2. Choose the **Linear** shape type.
3. Choose **Multiple** from the **Points** menu.
4. (Optional) Change any of the other available options.
5. Click once to specify the ruler origin.
6. Continue to click to specify additional segments.

Note:

You can press the Backspace key to delete an incorrectly placed segment endpoint.

7. Double-click to complete the ruler.

The command remains active.

8. Press Esc to complete the command.

Create Ruler Toolbar Options

The toolbar contains the following options:

Option	Description
Type	Specifies the type of ruler. • Linear • Circular
Angle	Controls the orientation of segments.
Points	Sets the number of points to use to draw the ruler. • Two: Creates a single segment ruler. • Multiple: In Linear mode, lets you avoid objects. In Circular mode, the first click defines the origin; successive clicks define anchors.
Color	A list of available color packets.
Cycle	In Circular mode with Multiple points set, lets you cycle through the color packets as you add anchor points.
Delete All	Deletes all rulers from the design.

Create Ruler Menu and Shortcut Keys

The menu contains the following options:

Command	Purpose	Default Shortcut Key
Commit	Completes the ruler if the shape is valid.	Return
Angle Mode	Controls the orientation of segments	
Delete All Rulers	Deletes all rulers from the design.	

Deleting Rulers

To delete individual rulers, select them and press the Delete key, or use the **Delete** command. To delete all rulers, click **Delete All** in the toolbar while the **Create Ruler** command is active.

5

Using Templates

Schematic templates offer many of the benefits of hierarchy, without the burden of dealing with artificial hierarchical interfaces. This chapter describes the features and procedures you can use to create and work with templates.

This chapter describes the workflow for using templates in the Schematic Editor:

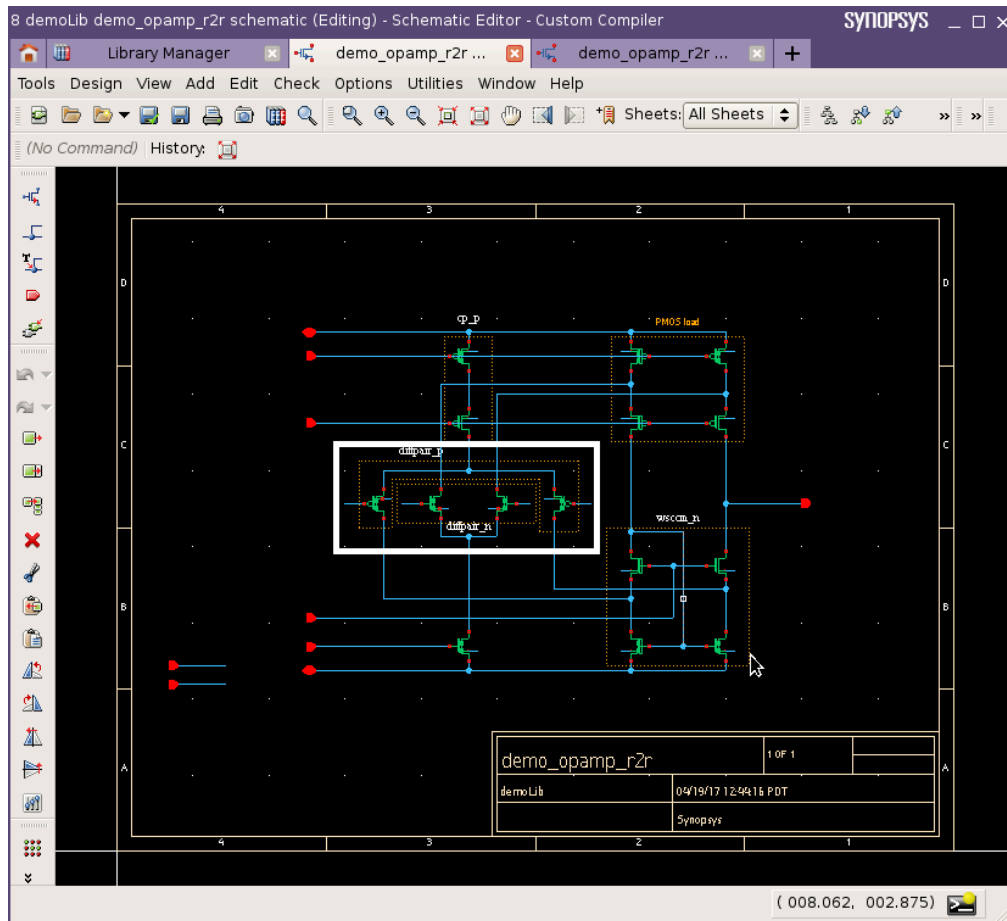
- [Creating Templates](#)
- [Placing Templates in a Schematic Design](#)

For information about Using Templates and the Template Manager in SDL, see the *Custom Compiler SDL and Visually-Assisted Automation Guide*.

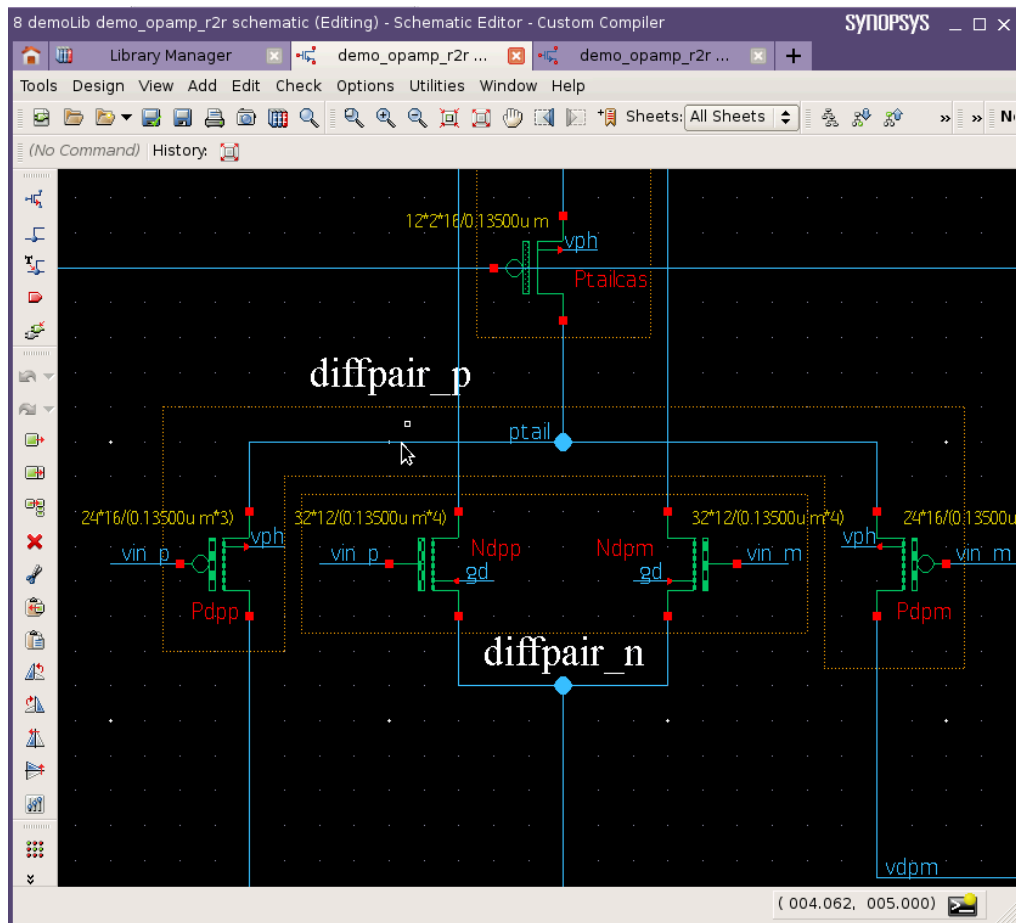
Creating Templates

The **Create Template** feature provides the capability for you to construct your own template library. You can create a new template cell or add views to an existing template cell by selecting components of an existing design and then invoking **Create Template**.

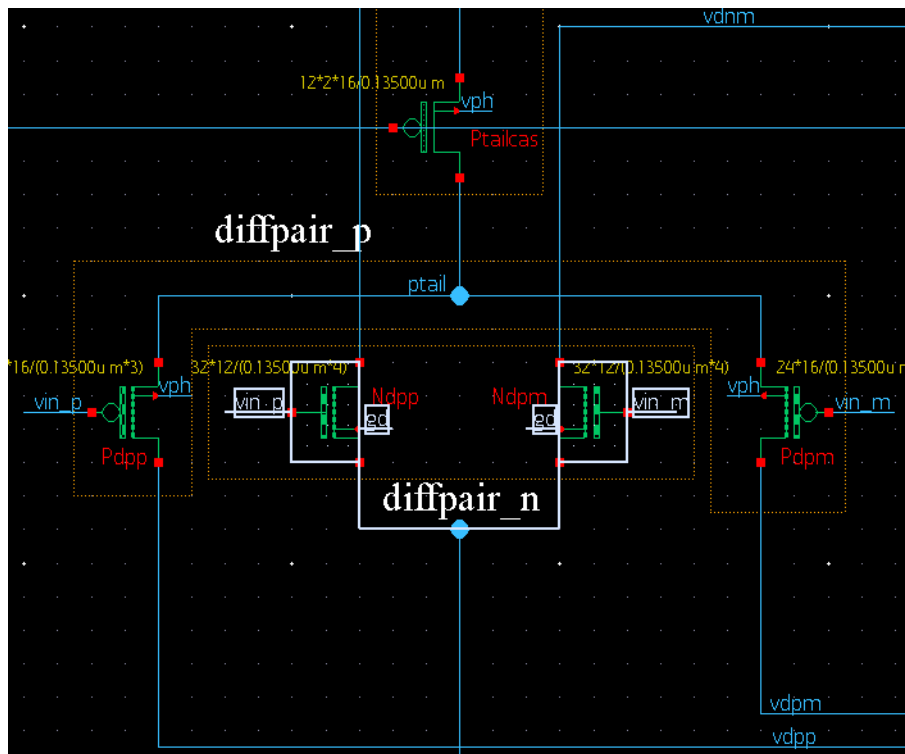
1. Open a schematic design. For instructions, see [Opening Designs](#).
2. Review the design for key structural elements that could be candidates for a template. Note that you can use the Constraint Editor to explore the design for instance matching constraints. See [Creating Constraints](#) for details.
3. Click and hold the mouse button on the canvas, then drag to select a rectangle containing the design elements you would like to use to create a template.



4. Release the mouse button to zoom on the rectangle shown in the figure below. This example shows two NMOS transistors that form a differential pair structure.



5. Select all devices and interconnect wiring to add to the template.



6. Choose **Edit > Create/Add to Template** to open the **Create Template** dialog box.

Create Template

Create

Template Cell, including Schematic View

Storing To

Library:

Cell:

View: schematic

☐ Constraint:

Enter a description for this template (optional)

N-type Differential Pair

☒ Open View For Edit

Help Defaults OK Apply Cancel

7. Enter settings as described in [Table 13](#) and click **OK**.

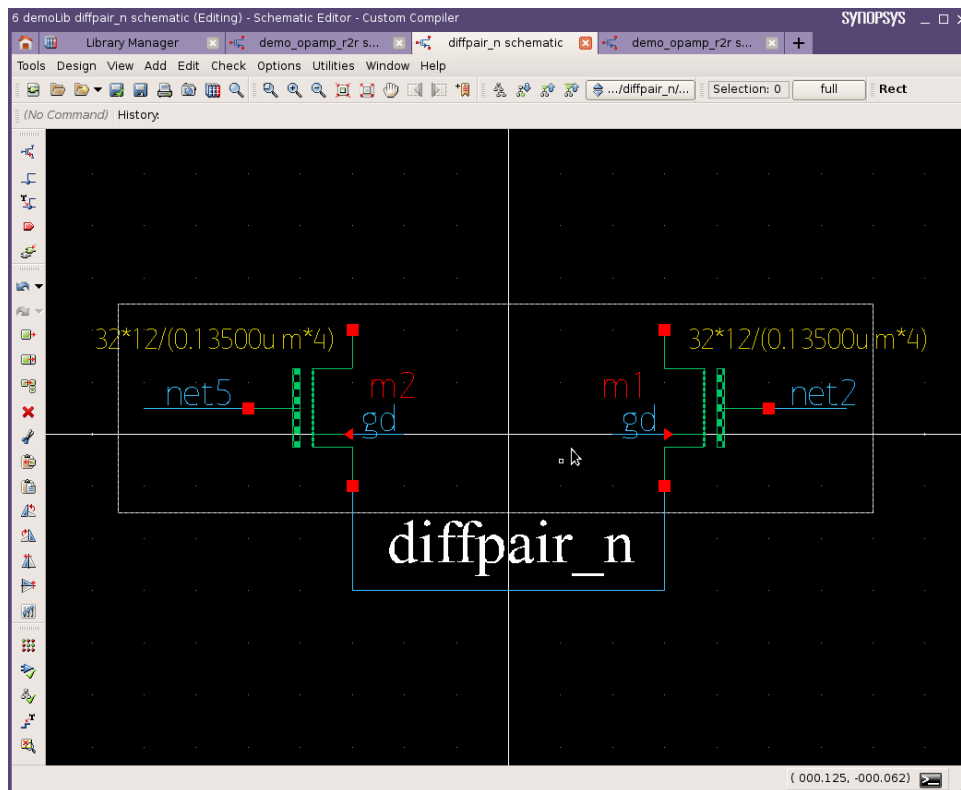
Table 13 Create Template Dialog Box Options

Setting	Description
<i>Create</i>	
Template Cell, with Connectivity Only	Creates a connectivity-only template cell.
Template Cell, including Schematic View	Creates a template cell including a schematic view.
Schematic View in Existing Template	Adds a schematic view to a previously existing template.
<i>Storing To</i>	
Library	Specifies the library name for the new template cell.

Setting	Description
Cell	When creating a new template, specifies a template cell name which does not exist in the library.
View	Specifies a view name for the template schematic. The default view name is "schematic."
Constraint	Saves the constraint information as part of the template. Enter a name for the constraint view if you choose this option.
Description	Specifies the description for the new template cell (such as the purpose of the cell, the type of design it is meant for, components used, and so on). The description is on a per-cell basis.
Open View for Edit	Opens the generated template schematic cell view after it has been created.

If the template cell already exists, a prompt appears asking whether to overwrite the existing cell with the new one.

8. The newly created schematic template cell opens in a new tab. In this example, the template contains the NMOS differential pair, which has the two NMOS transistors (now named m1 and m2), and some local net names (net2, net5).



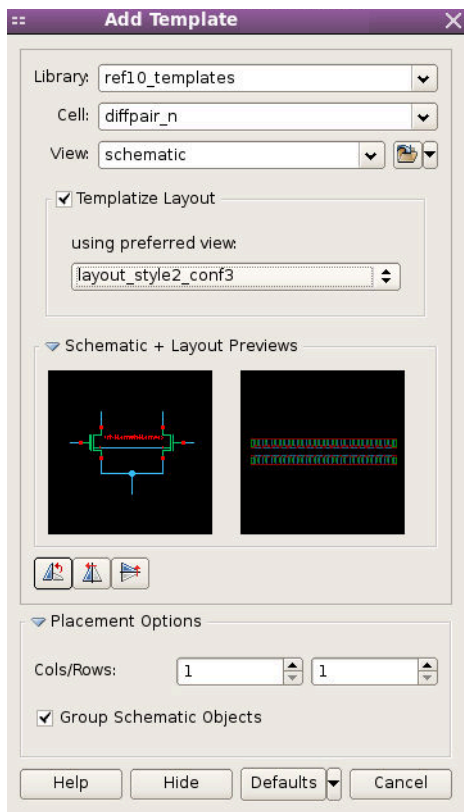
You can also view the new cell in the Library Manager. The template cell contains three views: constraint (if you chose the **Constraint** option in [Step 7](#)), schematic, and template. The presence of the template view identifies this cell as a template cell.

Placing Templates in a Schematic Design

The **Add Template** feature allows you to add templates to a design at any stage during the design creation process. You can use **Add Template** to complete a partial schematic, or you could start with an empty design, add templates first, then add all the other elements of the design.

Add Template functions much like the **Add Instance** and **Add Wire** features.

The screenshot displays a complex schematic diagram in the Synopsys Hierarchy Editor. The circuit is a differential pair, with a central core labeled 'diffpair_p' and 'diffpair_n'. The inputs are labeled 'vph', 'pliar', 'pliar', and 'vph'. The outputs are labeled 'vdam', 'vdpp', and 'wscem_n'. The schematic includes several PMOS load transistors, with one labeled 'PMOS load'. The circuit is implemented using a 12V/12/0.13500u m*3 transistor model. The schematic is titled 'demo_opamp_r2r schematic_no_mirror (Editing) - Hierarchy Editor - Synopsys'.



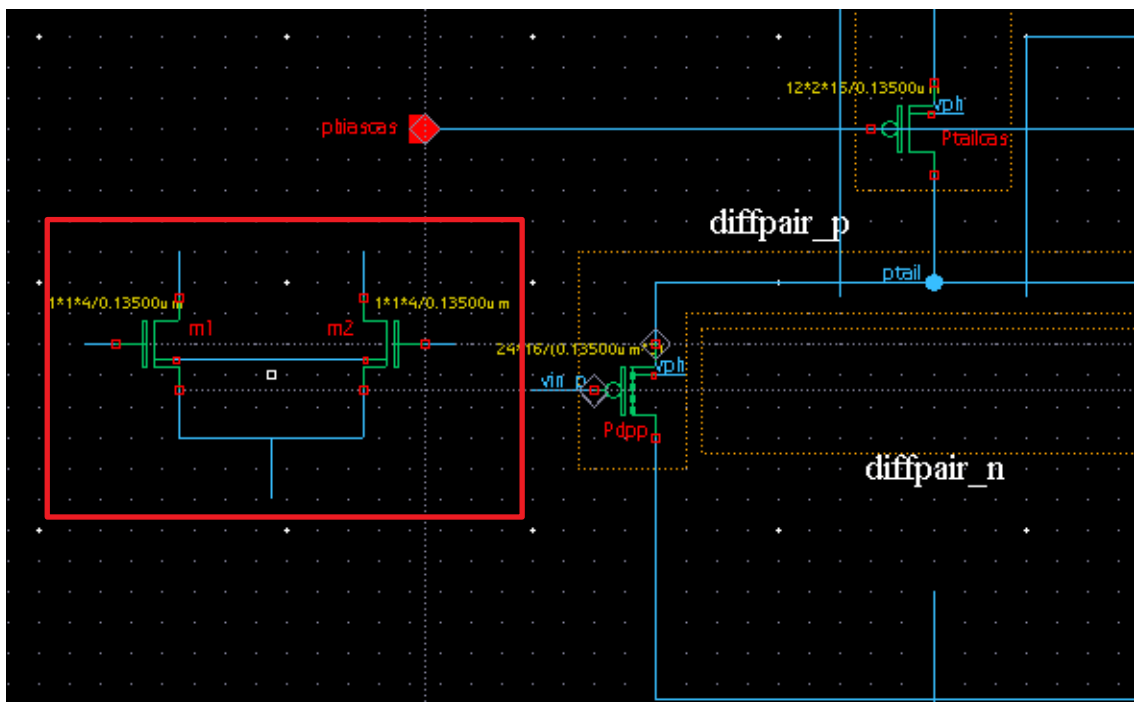
3. Enter settings as described in [Table 14](#).

Table 14 Add Template Dialog Box Options

Setting	Description
Library	Specifies the library name of the template cell. Only libraries containing template cells are available in the drop-down menu.
Cell	Specifies the template cell name. Only template cells are available in the drop-down menu.
View	Specifies the view name for the template cell. The default view name is "schematic."

Setting	Description
Templatize Layout	Provides information to the layout engineer about how best to implement the template elements of this schematic design. Choose a preferred layout view from the using preferred view menu. ◦ <Any> restricts the layout engineer to using one of the template views of this cell for implementing the template devices in the layout, but allows the layout to use any of the available layout views within this template cell (there may be more than one). ◦ A specific preferred view (such as layout style2_conf3, shown in this example) restricts the layout engineer to using only that particular layout view of the template cell for the template transistors (diff pair in this example). When the layout engineer creates the layout, the layout for the template devices must use the information in the specified layout view, and only that view.
Schematic and Layout Previews	Previews the schematic and layout views of the template cell.
 Rotate	Rotates the instantiated template 90 degrees counter-clockwise.
 Flip Horizontal	Mirrors the instantiated template across the Y-axis.
 Flip Vertical	Mirrors the instantiated template across the X-axis.
Placement Options	Allows you to place multiple instances of a template in a user-defined number of rows or columns.
Group Schematic Objects	Determines whether or not to group schematic objects. This gives many of the benefits of hierarchy, without the necessity of creating a symbol view and working through that hierarchical interface. When Group Schematic Objects is off, template elements behave as if they were simply copied or pasted from one design into the current design and must be operated on as single elements, not groups of objects.

4. Hover the pointer over the design canvas and notice that the template is bound to the pointer, ready to be placed in the schematic. The following figure shows the template outlined in red. As you move the pointer around, the template image moves with it, and alignment aids dynamically appear.

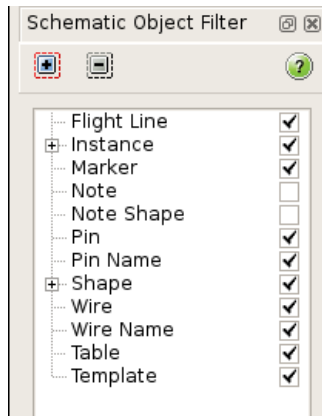


5. Move the mouse pointer until the template lines up exactly where you want it in the schematic, and then left-click to place the template. Use the dynamic alignment aids to help you line up the template in the schematic.

You can rotate or flip the templates before placement using the **Rotate**, **Flip Horizontal**, or **Flip Vertical** buttons in the **Add Template** dialog box.

Multiple templates can also be placed in columns or rows, just as with the **Add Instance** command.

6. Repeat [Step 4](#) and [Step 5](#) to continue placing more template instances. Press Esc or click **Cancel** in the **Add Template** dialog box to terminate the **Add Template** command.
7. (Optional.) Open the Hierarchy Navigator assistant (**Window > Assistants > Hierarchy Navigator**) and notice that no additional schematic hierarchy was introduced upon adding the template cell to the design. For more information about the Hierarchy Navigator Assistant, see the *Custom Compiler Environment User Guide*.
8. (Optional.) Use the Schematic Object Filter assistant (**Window > Assistants > Schematic Object Filter**) to filter template objects in the design for further editing. Be sure that the **Template** option is checked (the default). Unchecking **Template** disables the selection of templates as grouped entities, forcing the selection of only the constituent individual devices.



For more information about the Schematic Object Filter assistant, see the *Custom Compiler Environment User Guide*.

9. Make any necessary edits to place and connect the design's components using the usual schematic editing features.

Grouped templates can be operated upon as a single entity and, at the same time, the objects within a template (instances, wires, and so forth) still operate as they normally would without any templates being present.

10. Save and close the design.

6

Creating and Using Schematic Overlays

The Schematic Overlay feature allows you to create variations of existing designs without modifying the original schematics. This chapter describes the features and procedures you can use to create and work with schematic overlays.

This chapter describes the workflow for creating and using overlays in the Schematic Editor:

- [About Schematic Overlays](#)
- [Getting Started With Overlays](#)
- [Schematic Overlay Menu and Toolbar Options](#)
- [Working With Schematic Overlays](#)

About Schematic Overlays

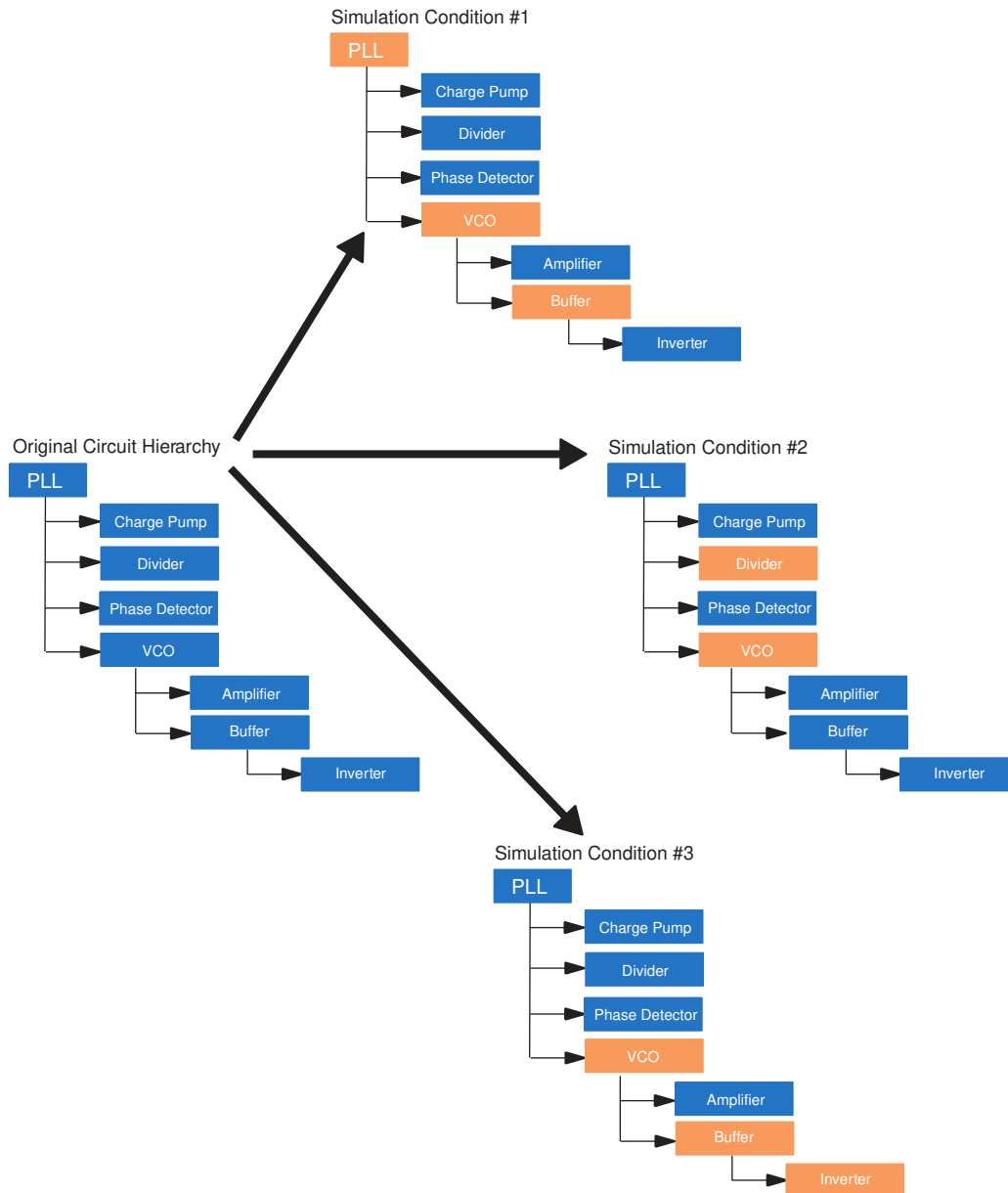
Overlays provide a way to create a variant of the schematic design without modifying the original data. You can use schematic overlays over a base (reference) schematic design to generate a modified circuit design for the purpose of design exploration, what-if analysis, verification, and custom flows. Overlays can be used for any operation requiring a schematic design, such as:

- creating testbenches for simulation and verification
- debugging and experimental analysis on designs
- estimating parasitics for read-only schematics
- performing LVS

The modification or creation of the schematic data to form an overlay could include, for example, adding circuit objects, deleting circuit objects, or changing parameters associated with the circuit objects.

Test circuitry can be modified or inserted in the form of an overlay schematic data without changing the reference schematic data. Overlay modifications are visually distinguished

from the original reference objects. The following figure illustrates schematic modifications with overlay views shown in orange.



You can track the overlay view bindings at the hierarchical level.

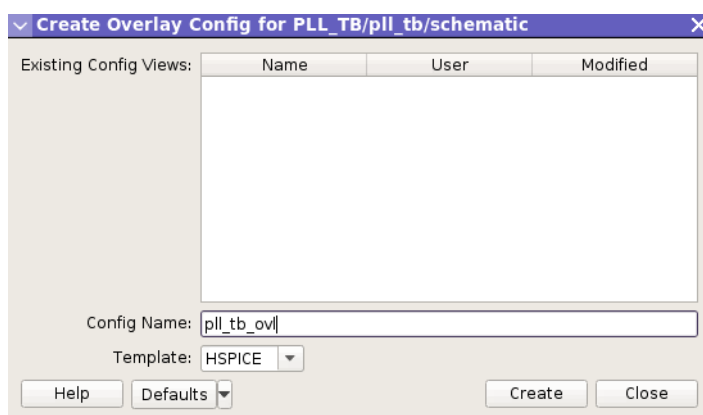
Getting Started With Overlays

Overlays are based on Custom Compiler hierarchy configuration (*config*) views. An overlay config view stores design modifications (overlay objects) and hierarchical rules ("normal" config objects) for binding the design. An overlay config view is necessary for the top cell in order to use schematic overlays.

[▶ Show me](#) how to create a schematic overlay and work with it.

To create an overlay hierarchy configuration (*config*) view:

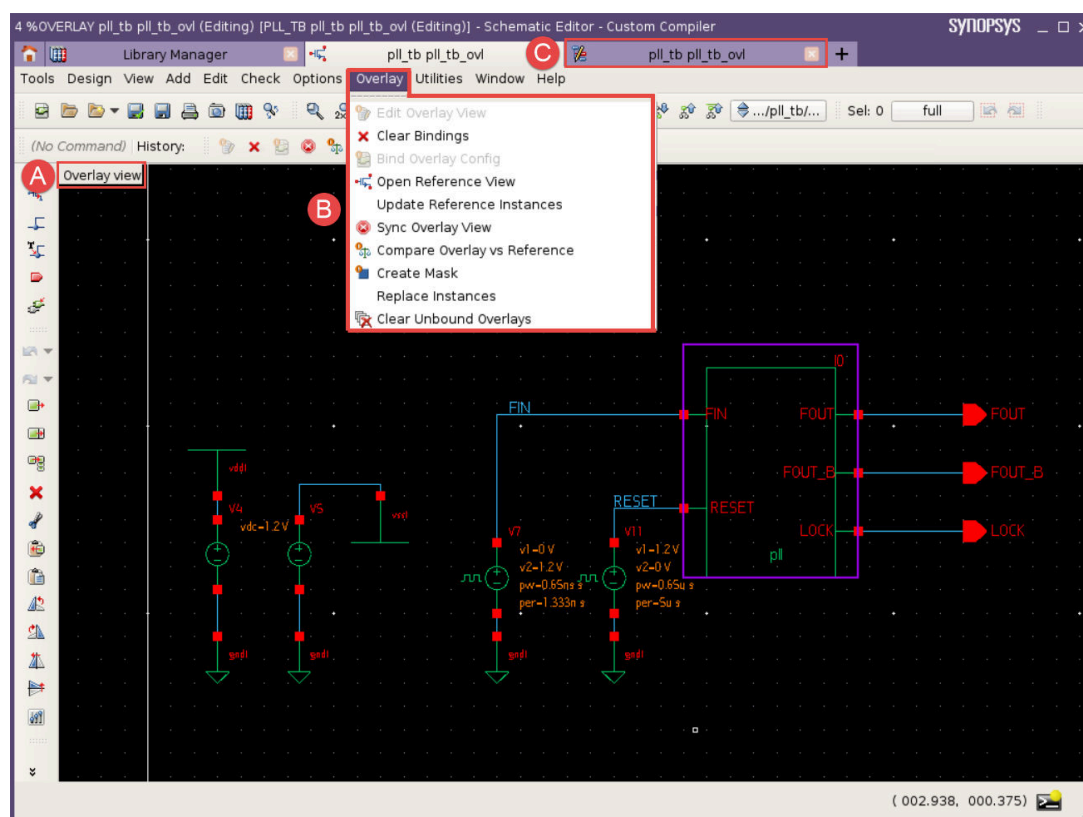
1. Open a schematic design. For instructions, see [Opening Designs](#).
2. From the Schematic Editor, choose **Tools > Overlay**. The **Create Overlay Config** dialog box opens.



3. Select a config view from the list of **Existing Config Views** or enter the **Config Name** to create a new overlay config view.
4. Select a simulation **Template** for your simulator of choice. This uses the existing Hierarchy Editor config templates and is primarily useful as a quick way to set the **View Search** and **View Stop** lists.
5. Click **Create** to create the overlay config view. The dialog box closes and the following results occur (outlined in red in the figure below):
 - a. The Schematic Editor canvas view switches to the overlay design you created, indicated in the upper left corner of the canvas.
 - b. The **Overlay** menu and toolbar are added to the Schematic Editor.
 - c. The Hierarchy Editor opens in a separate tab, showing the detailed hierarchy of the overlay config view. Here you can check which instances are bound to which views. (See the *Custom Compiler Environment User Guide* for information about the Hierarchy Editor.)

Chapter 6: Creating and Using Schematic Overlays

Schematic Overlay Menu and Toolbar Options



Additionally, the new overlay config view is visible in the Library Manager.

Schematic Overlay Menu and Toolbar Options

When you enable the **Overlay** menu (**Tools > Overlay**), the following options are available.

Option	Toolbar Button	Description
Edit Overlay View		Turns on edit mode. When the overlay config is created, an overlay view for the top design is also created so that it is immediately editable. However, lower-level schematics are read-only until you enter edit mode using Edit Overlay View. Read-only cells of the hierarchy are indicated as such by the message in the upper-left corner of the canvas R/O view in overlay config. When in a read-only view, you can switch to edit mode by choosing Edit Overlay View. This makes the current level of the design editable. All changes made are stored with the overlay config. This option is also available from the Design menu when in a read-only overlay view. See Editing an Overlay Design.
Clear Bindings		Clears bindings for the current branch of the hierarchy. See Clearing Overlay View Bindings.
Bind Overlay Config		Opens the Bind Overlay Config dialog box, allowing you to bind an occurrence to an existing overlay config. See Binding Overlay Config Views.
Open Reference View		Opens the view of the reference schematic design in a new tab.
Update Reference Instances		Opens the Update Reference Instance dialog box. Saves changes from the overlay view to the reference design. See Updating the Reference Design With Changes From an Overlay Design.
Sync Overlay View		Syncs the overlay view with the latest reference design. If changes were made to the reference design, this option gets the latest changes in the reference design to make sure the overlay view has the most current version of the reference. The icon changes depending on the state of the design: the "revert" icon is the default and the X icon appears when the tool detects a need to update the design. See Updating the Overlay Design With the Latest Reference Design.
Compare Overlay vs Reference		Opens the Schematic Overlay vs Reference Logical Compare dialog box. See Comparing the Overlay With the Reference.
Create Mask		Opens the Mask toolbar. See Using Masks in Schematic Overlays.
Replace Instances		Opens the Overlay Instance With Mask toolbar. See Using Masks in Schematic Overlays.

Option	Toolbar Button	Description
Clear Unbound Overlays		See Clearing Overlay View Bindings .

Working With Schematic Overlays

After you have created an overlay config (as described in [Getting Started With Overlays](#)), you can create and modify schematic overlays.

This topic describes the following tasks:

- [Editing an Overlay Design](#)
- [Using Masks in Schematic Overlays](#)
- [Updating the Reference Design With Changes From an Overlay Design](#)
- [Opening the Reference Schematic Design](#)
- [Comparing the Overlay With the Reference](#)
- [Updating the Overlay Design With the Latest Reference Design](#)
- [Library Management of Overlay Design Views](#)
- [Binding Overlay Config Views](#)
- [Controlling View Bindings With the Hierarchy Editor](#)
- [Clearing Overlay View Bindings](#)
- [Saving Overlay Designs](#)

Editing an Overlay Design

When working with overlays, only the top-level schematic design is editable, by default. All other occurrences are read-only. Use the **Overlay > Edit Overlay View** command to make a lower-level design editable. When in edit mode, you can make modifications such as:

- create new schematic objects
- modify the parameters and inherited connections on existing objects
- mask out existing schematic objects

The edits you make are associated with the occurrence. For example, if you have two instances of an inverter and descend down one path and then invoke **Edit Overlay View**,

your changes are associated with the descent lineage. The other occurrence will not be modified. Object modifications are data managed with the overlay config.

When in an editable overlay view, you can use the Property Editor to modify device parameters and inherited connections.

Note:

Reference instances cannot be moved, remastered, or deleted. Therefore, the Property Editor cannot be used to modify reference instances attributes such as LCV, instance name, origin, and so forth. However, it can be used to modify reference instance parameters, properties, and inherited connections. To change the master, use the **Replace Instances** command.

See the *Custom Compiler Environment User Guide* for information on the Property Editor.

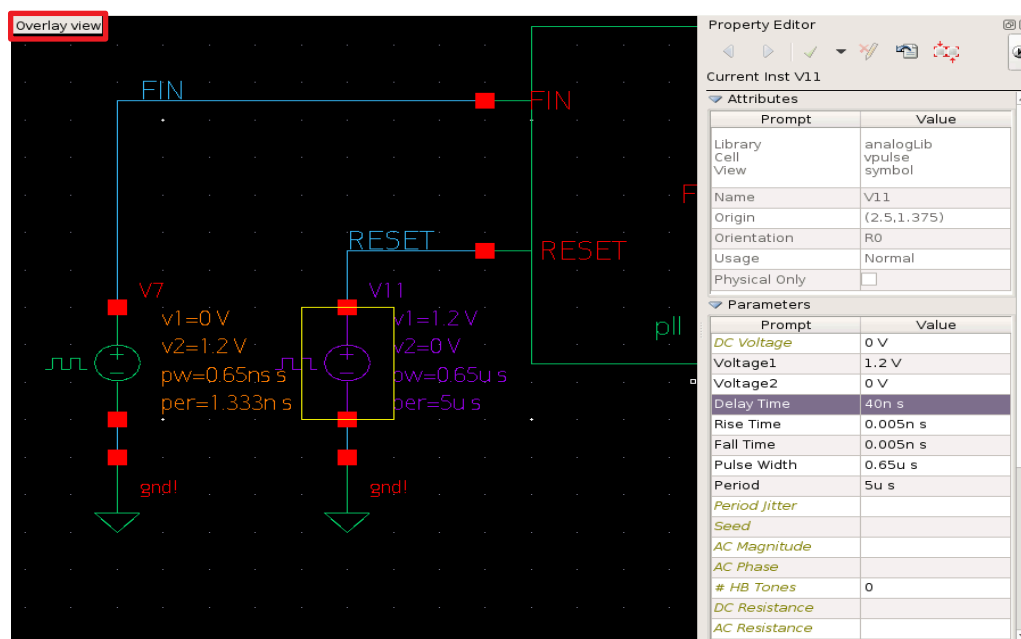
To edit a schematic overlay using the Property Editor:

1. In the top-level overlay config view, descend into the design by double-clicking an object on the canvas or clicking **Descend Edit**  on the toolbar. The tag in the upper-left corner of the canvas reads **R/O view in overlay config**, which indicates you are in a read-only view.
2. Choose **Overlay > Edit Overlay View**. The tag in the upper-left corner of the canvas now reads **Overlay view**, which indicates you have created a new overlay view and it is open to edit.

Note:

Original schematic objects are not editable, so you cannot perform **Move/Stretch/Delete** on original wires or instances.

3. Select an instance in the schematic design.
4. Press Q to open the Property Editor. (Alternatively, choose **Window > Assistants > Property Editor**.)
5. Modify a property of the selected instance and click  to apply the change. In the following figure, the **Delay Time** parameter of instance V11 has been changed. The change is indicated on the canvas by the instance's color change from green to purple.



6. Click **Save** in the editor to save the overlay view.

The overlay view is now available for use in the flows described in the following sections. If you want to propagate instance parameter changes back to the reference design, you can update the reference design using **Overlay > Update Reference Instances** (see [Updating the Reference Design With Changes From an Overlay Design](#)).

You can change a parameter in an overlay view to a design variable, which is then available in the PrimeWave Design Environment for debugging operations such as sweeps, corners, Monte Carlo analysis, and so forth.

Using Masks in Schematic Overlays

Overlay masks are used to remove instances, wires, and labels from the logical overlay design. Masks can be useful for opening wires where you want to place new circuitry, or for hiding areas of the design before netlisting for incremental debugging.

There are two types of masks:

- Area masks (the default) hide everything under the selected area.
- Clip masks function as the inverse of an area mask, hiding everything outside the clipped rectangle.

The precedence of mask effects is such that if an area mask is placed on an instance that is inside a clip mask, the instance is not included because the area mask takes precedence over the clip effect.

Objects under the mask are removed from the logic netlist. Instances do not show up when netlisting, in the Hierarchy Navigator, and so forth. Wires under a mask are cut at the boundary of the mask. All portions of the wire underneath the mask are removed from the logical netlist. This can be useful for splitting a single net into multiple nets and for inserting serial devices into an existing net. Logical connectivity is computed in real time as you create, stretch, and move masks (*dynamic connectivity*).

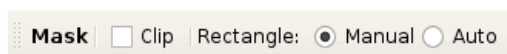
This section contains the following topics:

- [Creating Area Masks](#)
- [Creating Clip Masks](#)
- [Using a Mask to Create an Editable Copy of the Reference](#)

Creating Area Masks

To create an area mask:

1. In an editable overlay view, choose **Overlay > Create Mask**. The **Mask** toolbar appears.

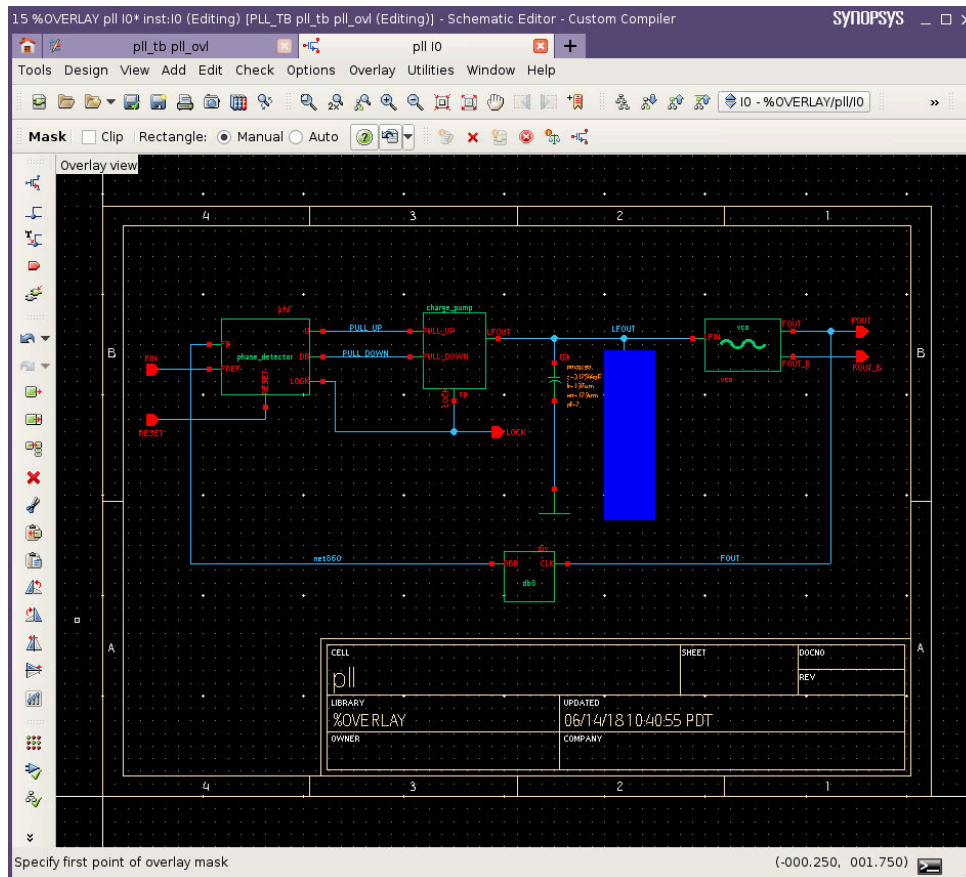


2. Select whether you want to create a mask using **Clip** or not. The default is to draw an area mask. See [Creating Clip Masks](#) for the steps to create a clip mask.
3. Choose whether to draw the mask rectangle manually (**Manual**) or automatically (**Auto**).

To remove an instance from a design it must be completely covered by a mask. If an instance is only partially covered by a mask, it remains as part of the logical design. Partially masked instances are drawn "above" the mask to give a visual indication that the instance is still part of the logical design. An SRC/ERC check automatically flags partially masked instances as violations.

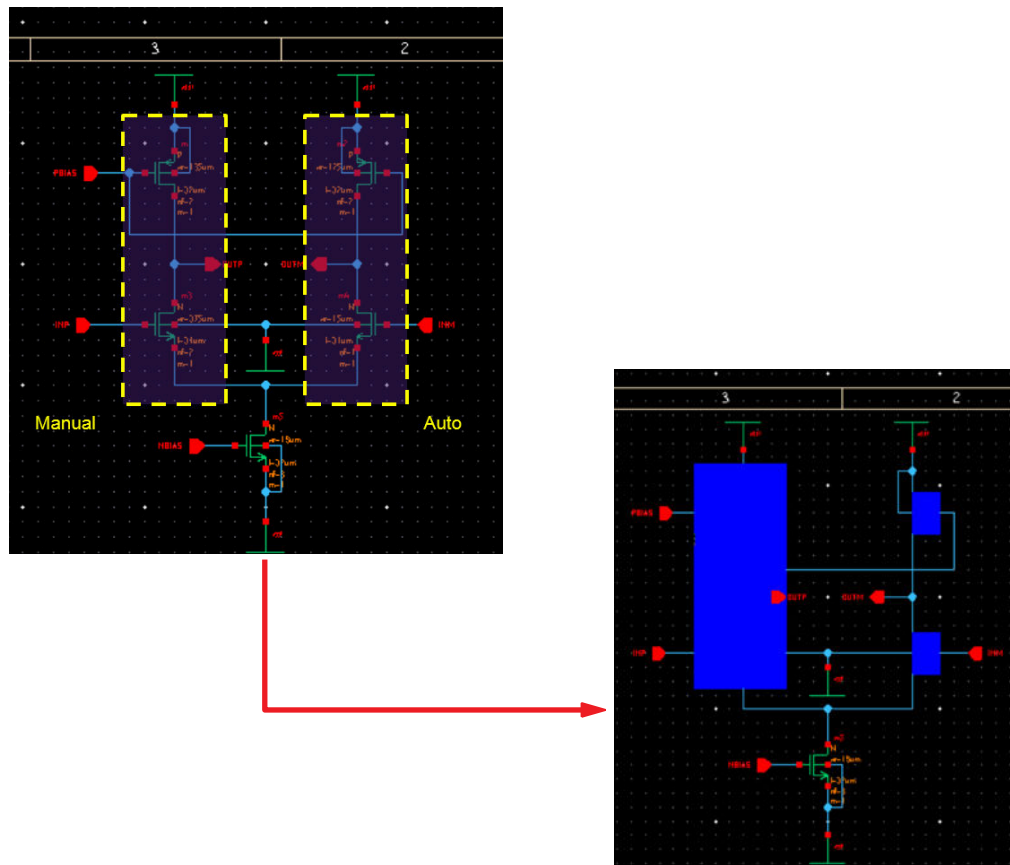
Auto creates a minimum size mask over each instance within the specified rectangle.

4. If you chose **Manual** (the default), click and drag on the canvas around the object you want to block with the mask. In the following example, the res-cap net is masked, which is indicated by the blue rectangle.



5. If you chose **Auto**, click an instance or drag and click to specify instances within the rectangle to process.

The following figure illustrates the difference between mask selection in **Manual** versus **Auto** mode.



6. Save the design.

Now you can use the overlay design for testing or debugging.

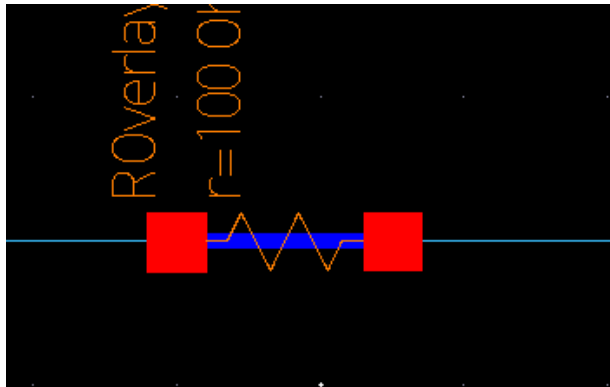
7. Press Esc to exit the **Create Mask** command.

At this point, you could add new objects over the mask, such as instances and wires, or leave the design as is. You can connect them to objects outside the mask as usual.

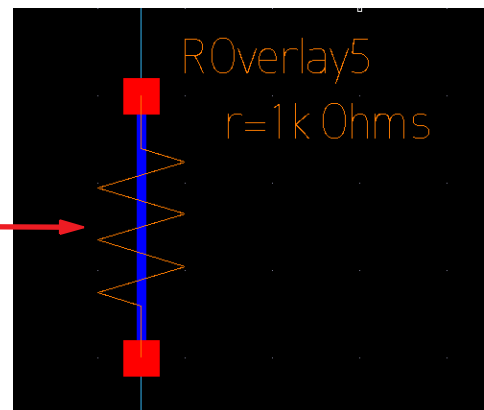
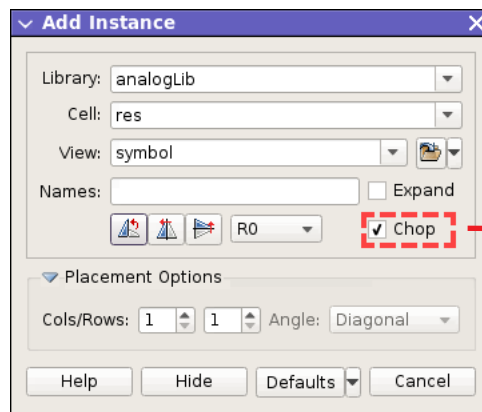
For example, to change the master of an instance (which is an attribute), you could mask (remove) the reference instance and create a new instance on top of the mask. (Alternatively, you could use **Overlay > Replace Instances**.) You are then able to change the master (which is an attribute) of the newly created instance.

Area masks are automatically applied when you insert instances into existing circuitry using the **Estimated Parasitics Assistant** or the **Add Instance** dialog box.

- The **Estimated Parasitics Assistant** applies the minimum-sized mask to properly insert an instance. (See [Estimated Parasitics Assistant](#) for information about the Estimated Parasitics Assistant.)



- The **Add Instance** dialog box applies a mask when you enable the **chop** option. (See [Creating Instances](#) for information about adding instances.)

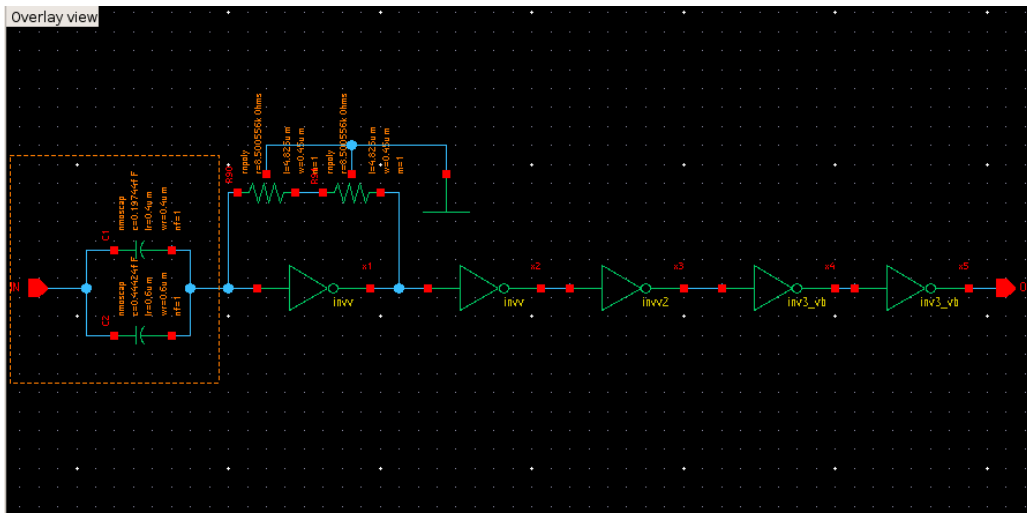


Creating Clip Masks

Clip masks are the inverse of normal masks. All objects within the clip mask are retained. All objects outside of clip mask(s) are removed from the logical design.

To create a mask using **Clip** mode:

1. In an editable overlay view, choose **Overlay > Create Mask**, then select **Clip** from the **Mask** toolbar.
2. On the canvas, click and drag around the objects you want to keep in your design. In the example below, the nmoscap is clipped, which is indicated by the orange dotted rectangle. The rest of the design is now removed from the logical design.



3. Press Esc to exit the **Create Mask** command.
4. Save the design.

You can create more than one clip mask in a design. If you perform an operation on this overlay schematic design (for example, netlisting) the only objects that will be operated on are the ones inside the clip area.

Using a Mask to Create an Editable Copy of the Reference

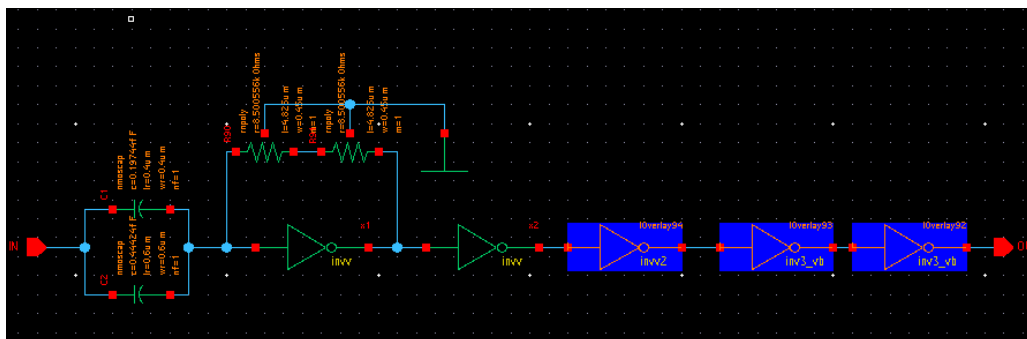
You can quickly create an editable copy of a reference instance:

1. In an editable overlay view, choose **Overlay > Replace Instances**. The **Overlay Instance with Mask** toolbar appears, indicating the active command.

Overlay Instance With Mask (no options)

2. On the canvas, click and drag around the instances you want to replace.

In the example below, three inverters are masked, as indicated by the blue rectangles, and the same instance has been added over each mask, indicated in orange.

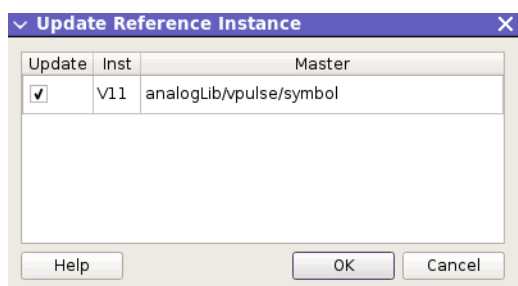


3. Make changes as necessary on the individual mask instances. For example, you could select the overlay instance IOverlay94, open the Property Editor and change the instance master Library/Cell/View.
4. Apply the changes and save the schematic overlay.

Updating the Reference Design With Changes From an Overlay Design

After making changes to parameters in an overlay view, you might decide to keep some of those changes and back-annotate them to the reference design.

1. In an overlay view, choose **Overlay > Update Reference Instances**. (Alternatively, right-click the modified instance and choose **Update Reference Instance** from the menu.) The **Update Reference Instance** dialog box opens.



2. Check the instances you want to update and click **OK** to update the reference design with the changes from the overlay.

After the reference instance is updated and in sync with the overlay, the color of the overlay instance changes back to green from purple.

Note:

Update Reference Instances is disabled if the reference schematic is already open as read-only (R/O). Additionally, the dialog box opens only if there are

reference instances with modified parameters (newly created instances are not processed).

Opening the Reference Schematic Design

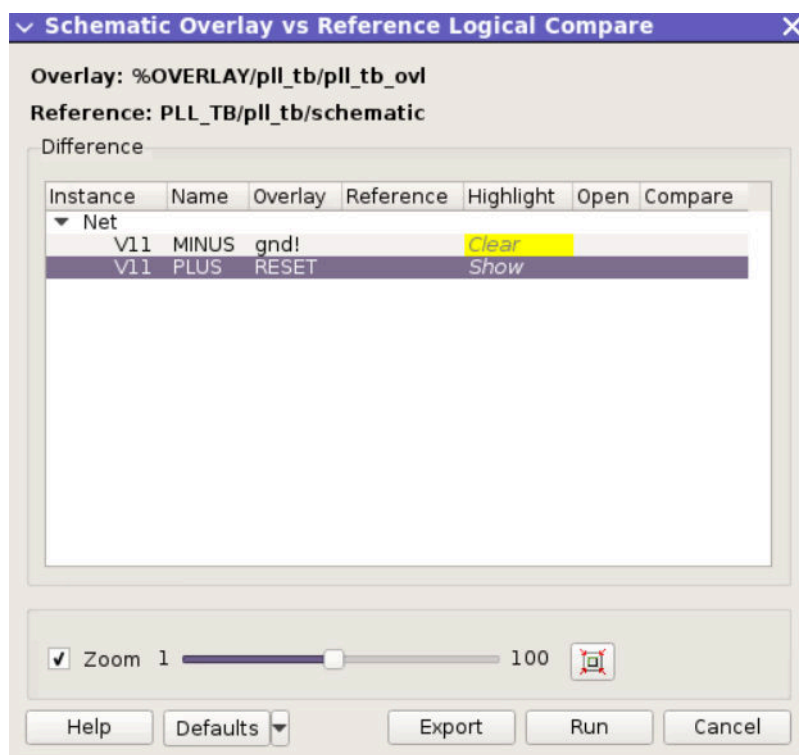
To open the reference schematic design, choose **Overlay > Open Reference View** or

click the  button in the toolbar.

Comparing the Overlay With the Reference

After you have created a schematic overlay, you can compare the overlay with the reference. In an overlay view:

1. Choose **Overlay > Compare Overlay vs Reference** to compare the overlay with the reference. The **Schematic Overlay vs Reference Logical Compare** dialog box opens, listing any differences between the two views.

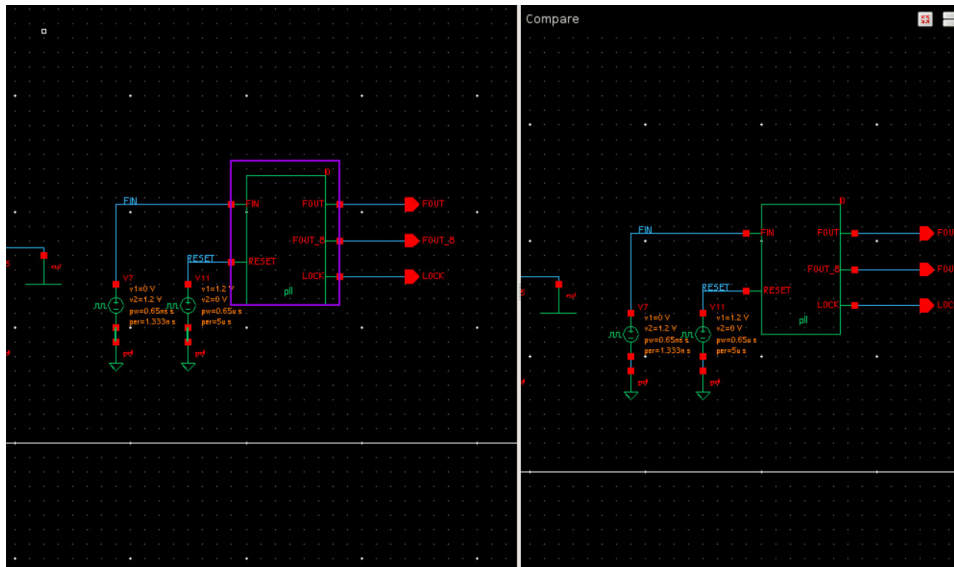


Note:

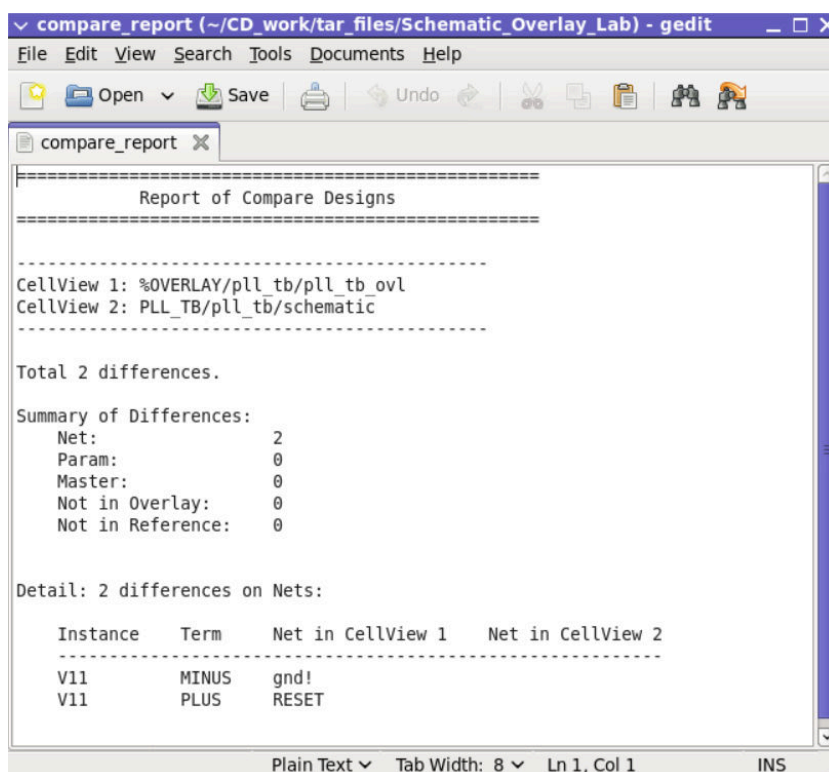
Keep in mind that changing one parameter in the design could result in other parameters it references also being changed via callback. These changes

are reflected in the **Schematic Overlay vs Reference Logical Compare** dialog box.

2. Double-click **Show** in the **Highlight** column of the **Difference** section to open the two views side by side.



3. Double-click **Clear** in the **Highlight** column to clear highlights.
4. Click **Export** to create a text file reporting the design changes, such as the one shown in the following figure.



5. In the **Schematic Overlay vs Reference Logical Compare** dialog box, click **Cancel** to close the side-by-side view and return to the overlay config view.

Updating the Overlay Design With the Latest Reference Design

Changes might be made to the reference schematic design after you take a "snapshot" with your overlay. To update the overlay view so that it has the latest reference instances, choose **Overlay > Sync Overlay View**. After syncing the overlay view with reference, the instance color changes back to green from purple.

The icon for this command changes depending on the state of the design: the "revert" icon



is the default and



appears when the tool detects a need to update the design.

If the overlay view is already up-to-date with the latest reference instances, nothing happens when you choose **Sync Overlay View**.

The Tcl preference `seOverlayUpdateMode` controls sync behavior while opening an overlay view. The default, `explicit`, is to allow you to explicitly choose when to sync the overlay view. Other values are `auto`, which implicitly syncs the overlay on opening if it is out-of-date, and `prompt`, which prompts you with options on how to sync the overlay view.

The Tcl preference `seOverlayAutoUpdateInNetlisting` controls the netlisting of an out-of-date overlay view. If true (the default), the out-of-date overlay design is updated from the reference automatically when netlisted.

For further information about Schematic Editing Tcl preferences, see the *Custom Compiler Tcl Reference Manual*.

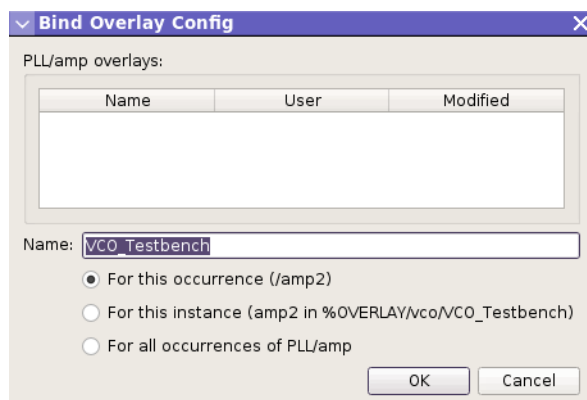
Library Management of Overlay Design Views

The overlay config view is visible in the Library Manager as soon as you create it. Any schematic overlays you create at lower levels in the hierarchy are data-managed with the parent overlay config and are copied, moved, and deleted in the Library Manager together with the parent.

Binding Overlay Config Views

Edit Overlay View creates an occurrence-based binding; only the current occurrence is bound to the overlay view. **Edit Overlay View** stores all modifications throughout hierarchy in the parent overlay config. This is very useful for managing edits that are specific to one design hierarchy.

However, you can create a reusable overlay that can then be referenced from multiple other overlay configs. Do this manually by creating an overlay config for each level of the hierarchy. The overlay configs can then be referenced in other hierarchies using the Hierarchy Editor. Use **Overlay > Bind Overlay Config** to accelerate this process.

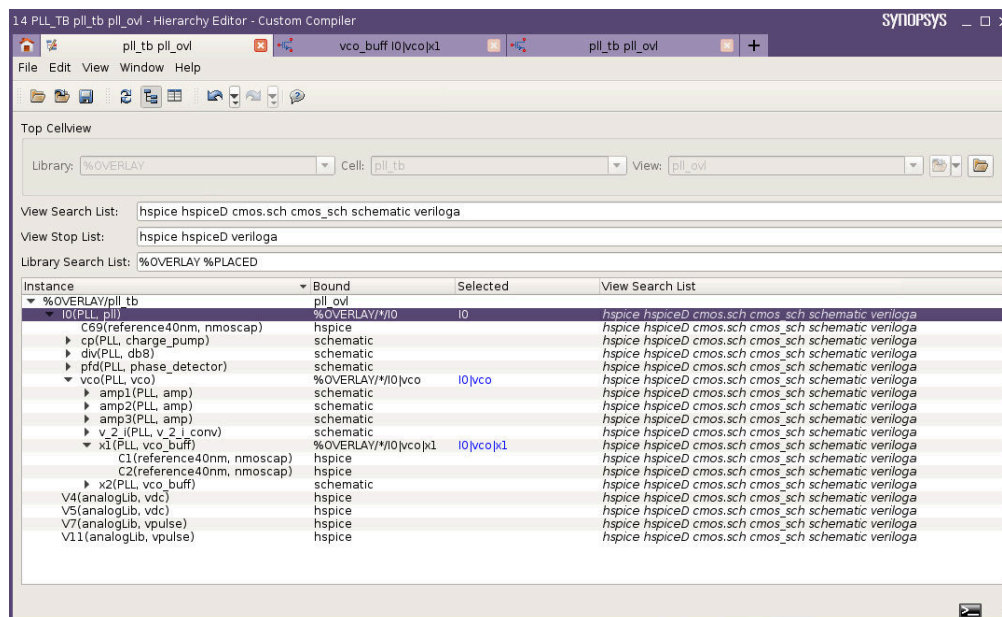


You can choose to bind to the overlay config only **For this occurrence**, **For all occurrences** of that cell, or only **For this instance**.

When you click **OK**, the tool creates the named view if it does not exist. The view is bound into the current hierarchy using the specified scope (occurrence, instance, or for all occurrences (which matches the "Cell" scope in Hierarchy Editor terms)).

Controlling View Bindings With the Hierarchy Editor

View bindings can be managed from the Hierarchy Editor. You can change view bindings based on what needs to be netlisted.



For information about how to use the Hierarchy Editor, see [Hierarchy Editor](#).

After manually removing a binding to an overlay view, it can only be restored using an undo operation or (if the Hierarchy Editor session was closed) by descending into that occurrence and selecting **Overlay > Edit Overlay View**. You are prompted to **Recover** the overlay view, **Overwrite** it, or **Discard** the changes.

You can switch the bindings to the original schematic at any time using one of two methods:

- In the Hierarchy Editor, using the **Clear occurrence scoped view** selection option
- With the command **Overlay > Clear Bindings** (see [Clearing Overlay View Bindings](#))

After using one of these methods, the view binding switches to the original schematic view. Later, you can switch the binding back to the overlay view. Any changes made to the overlay view are still there and can be recovered if you decide to overwrite the view.

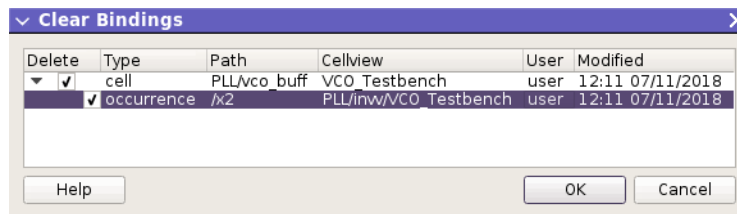


If you do not select **Recover**, the overlay view is no longer visible in the view selector and it is no longer available in the Hierarchy Editor.

Clearing Overlay View Bindings

To clear view bindings for the current branch of the hierarchy, choose **Overlay > Clear**

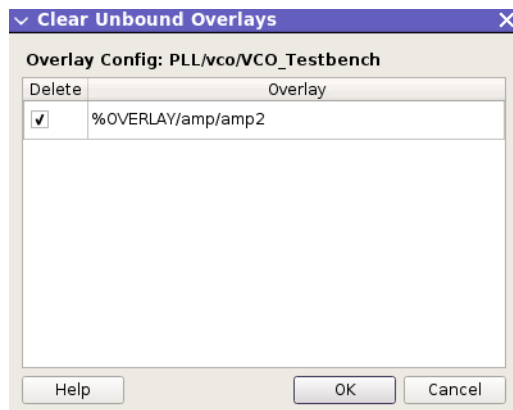
Bindings or click . This option is only valid if there are bound overlay views in lower levels of the hierarchy.



All bound overlay views in lower levels of the hierarchy appear in the **Clear Bindings** dialog box. The rows of the table in the dialog box reflect the hierarchy of the nested overlay configs.

Overlay views whose bindings are cleared become unbound views, but they are not deleted immediately. These views can be restored later (see [Binding Overlay Config Views](#)) or deleted using **Overlay > Clear Unbound Overlays**.

If **Delete** is checked, those views cannot be recovered. They are immediately deleted.



Note:

Unbound views are deleted along with the overlay config for which they were created when the overlay config is deleted.

Saving Overlay Designs

You can save the overlay design as a separate schematic design using **Design > Save as Schematic**.

7

Generating Cellviews

This chapter describes the automatic cellview generators you can use to create new cellviews from existing cellviews.

You can generate new cellviews from different cellviews, such as:

- Creating symbol views from schematics, see [Creating Symbol Views From Designs](#).
- Creating soft abstract views from layouts, see [Creating Soft Abstract Views From Layouts](#) in the *Custom Compiler Layout Editor User Guide for Creating and Editing Objects*.
- Creating layout views from symbols, see [Creating Layouts From Symbols](#) in the *Custom Compiler Layout Editor User Guide for Creating and Editing Objects*.
- Creating abstract views from layouts, see [Creating Abstract Views From Designs](#) in the *Custom Compiler Layout Editor User Guide for Connecting Objects*.

Automatic cellview creation lets you easily generate designs from information in a source design. The Custom Compiler tool supports the source-to-destination cellview type pairings shown in the following table.

Table 15 *Source/Destination Cellview Type Pairings*

Source Cellview	Destination Cellview
schematic	layout, symbol, schematic, Verilog-A, Verilog, VerilogAMS, SPICE, SystemVerilog, VHDL, CDL
symbol	layout, symbol, schematic, Verilog-A, Verilog, VerilogAMS, SPICE, SystemVerilog, VHDL, CDL
Verilog	symbol, schematic, Verilog-A, Verilog, VerilogAMS, SPICE, SystemVerilog, VHDL, CDL
netlist	symbol, schematic, Verilog-A, Verilog, VerilogAMS, SPICE, SystemVerilog, VHDL, CDL
SPICE	symbol, schematic, Verilog-A, Verilog, VerilogAMS, SPICE, SystemVerilog, VHDL, CDL

Table 15 Source/Destination Cellview Type Pairings (Continued)

Source Cellview	Destination Cellview
Verilog A	symbol, schematic, Verilog-A, Verilog, VerilogAMS, SPICE, SystemVerilog, VHDL, CDL
Layout	symbol, schematic, Verilog-A, Verilog, VerilogAMS, SPICE, SystemVerilog, VHDL

8

Creating Symbol Views From Designs

This chapter describes the features and procedures you can use to generate new cellviews from existing cellviews or pin lists.

You can generate symbol cellviews by:

- [Creating Symbols From Schematics](#)
- [Creating a Cellview From a Pin List](#)
- [Creating a Cellview From a Component](#)
- [Updating a Cellview](#)
- [Generator Option Sets](#)

Creating a cellview manually. For information about creating a cellview manually, see [Creating Symbols](#).

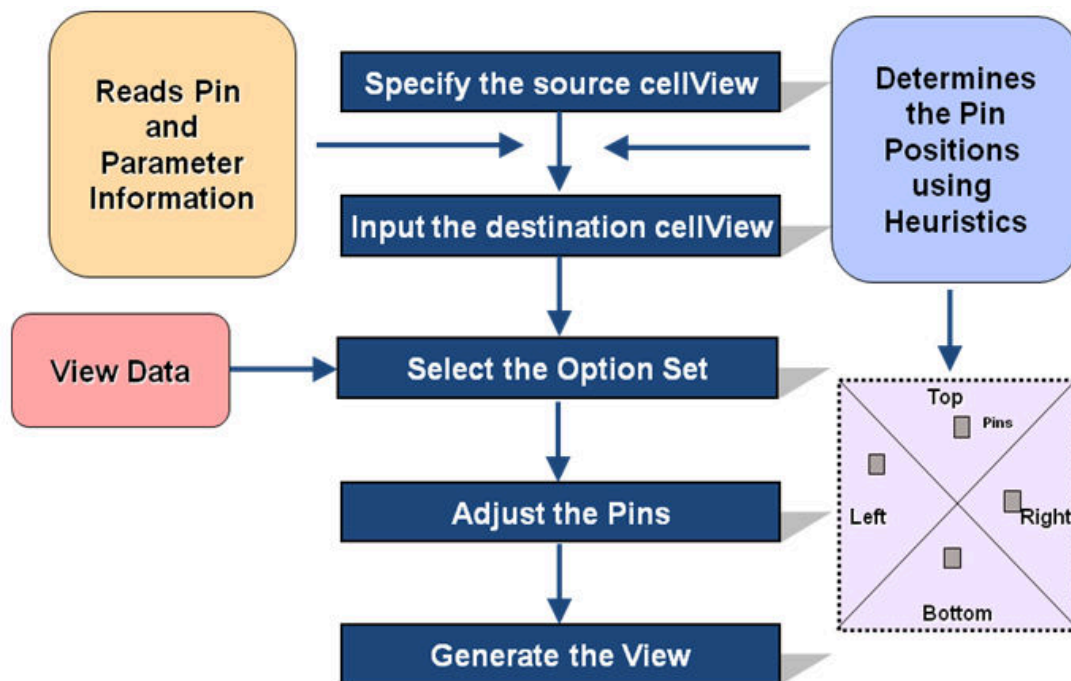
For information about creating abstracts from layouts, see [Creating Abstract Views From Designs](#) in the *Custom Compiler Layout Editor User Guide for Connecting Objects*.

For information about creating layouts from symbols, see [Creating Layouts From Symbols](#) in the *Custom Compiler Layout Editor User Guide for Creating and Editing Objects*.

Creating Symbols From Schematics

To generate a symbol view from a schematic design, you open a source design and then generate a destination cellview using information derived from the source design.

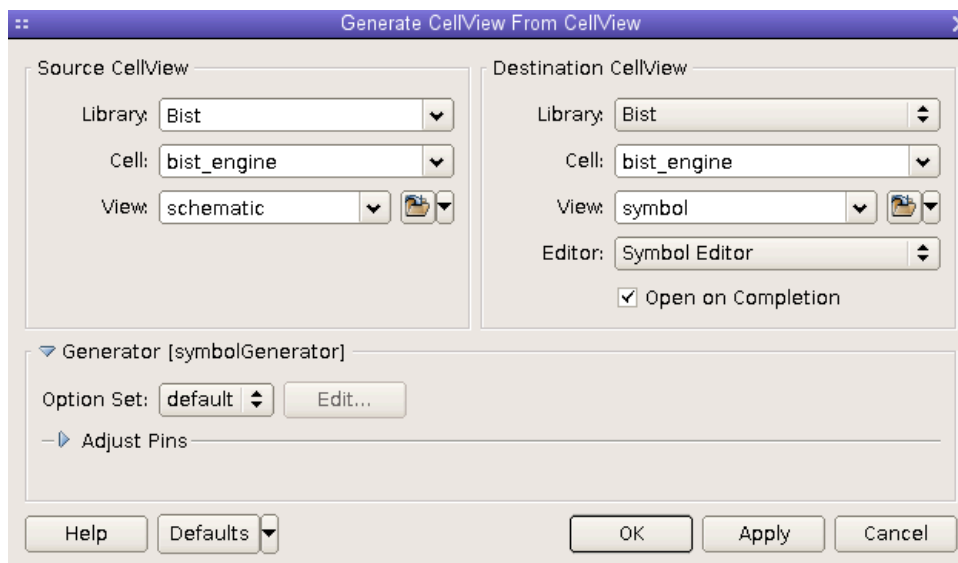
The following figure shows the flow for generating views from a cellview.



To generate a symbol cellview from a schematic cellview:

1. In the Schematic Editor, choose **Design > New CellView > From CellView**.

The **Generate CellView From CellView** dialog box appears.



When the dialog box opens, the tool:

- Populates the **Source CellView** region with the current design cellview information.
- Populates the **Library** and **Cell** fields of the **Destination CellView** region with the current design information.

The **View** field of the **Destination CellView** region is determined by the `dbGenerateCellViewDefaultViewName` preference (see [Specifying Destination Cellviews](#)).

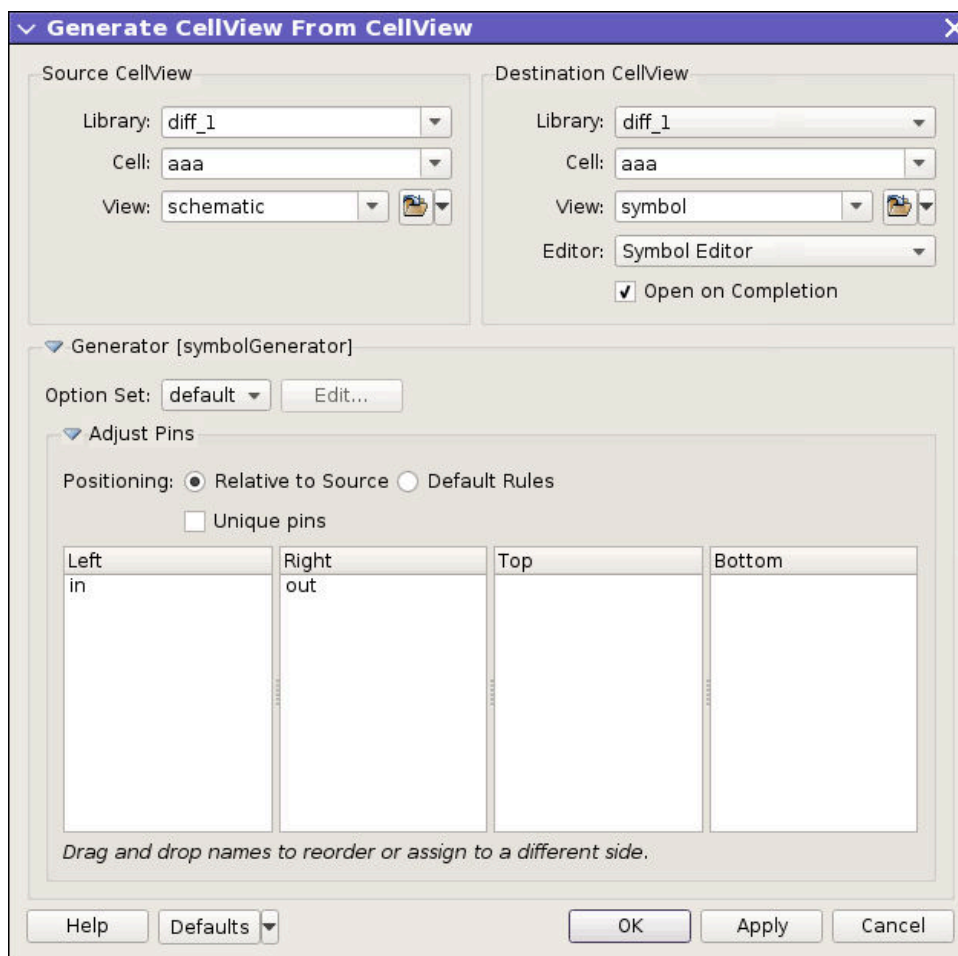
2. (Optional) Specify an **Option Set**.

You can specify an option set if you have registered one for the destination view type. If you do not specify an option set, the default **Option Set** applies. For information about creating option sets, see [Generator Option Sets](#).

Note:

If the **View** in the **Source CellView** section is a text view and the **Editor** of the **Destination CellView** is the Schematic Editor, the **Generate Schematics** option appears in the **Generator** section (shown in figure below). See the Importing Design Netlist Data as Text Views in the *Custom Compiler Environment User Guide* for details.

3. (Optional) Expand the **Adjust Pins** region. Select an option and then drag and drop pin type labels from one column to another.

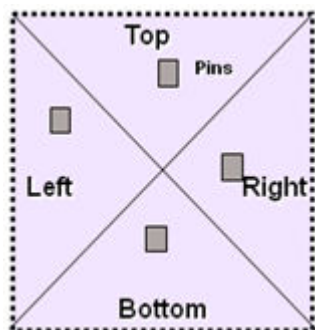


When you provide the source cellview information, the symbol generation utility reads the pin and parameters.

When creating a cellview from another source cellview, such as generating a symbol from a schematic, the tool lets you specify pin positions in the destination design using either of the following two options:

- **Relative to Source:** The tool determines pin positions using heuristics.
 - a. Pins are placed into one of the four regions shown in the following figure based on their position in the cellview relative to the data extents.

By deriving the initial positions from the relative pin placement in the source cellview the source cellview is divided into quadrants and pins are identified as being in the top, bottom, left, or right quadrant, based on the terminal type.



The data extents are the bounding boxes of all the following components: pin, wiring object, or a non-border instance. The regions are defined by connecting the opposite ends of the data extents bounding box with straight lines.

- b. The pins in the top and bottom regions are then sorted based on the X coordinate of their center point in ascending order. Pins with the same X coordinate are ordered relative to their Y coordinate in ascending order.
 - c. The pins in the left and right regions are then sorted based on the Y coordinate of their center point in descending order. Pins with the same Y coordinate are ordered relative to their X coordinate in ascending order.
- **Default Rules:** The tool places pins according to the default rules, which are:
 - Inputs on the left
 - Outputs on the right
 - InputOutputs (bidirectional) on top
 - All other pins on the bottom

The **Unique pins** option lets you filter duplicate pins from the **Left**, **Right**, **Top**, and **Bottom** lists. The associated Tcl preference is `dbGenerateCellViewUniquePins`.

4. Click **OK** or **Apply** to create the cellview.
 - If you click **Apply**, the dialog box remains open and the command remains active.
 - If you click **OK**, the dialog box closes and the command completes.

For information on using the `db::generateCellView`, `db::getCellViewInterface`, `db::showCellViewFromCellView`, `db::showCellViewFromPinList`, `db::showGenerateSchematic`, and `db::showUpdateCellView` Tcl commands, see Database Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

For information on using the `se::generateSchematic` and `se::generateSymbol` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

Specifying Destination Cellviews

You can use the `dbGenerateCellViewDefaultViewName` preference to specify default destination cellview types for particular source view types. [Example 3](#) shows how to specify the destination view to be:

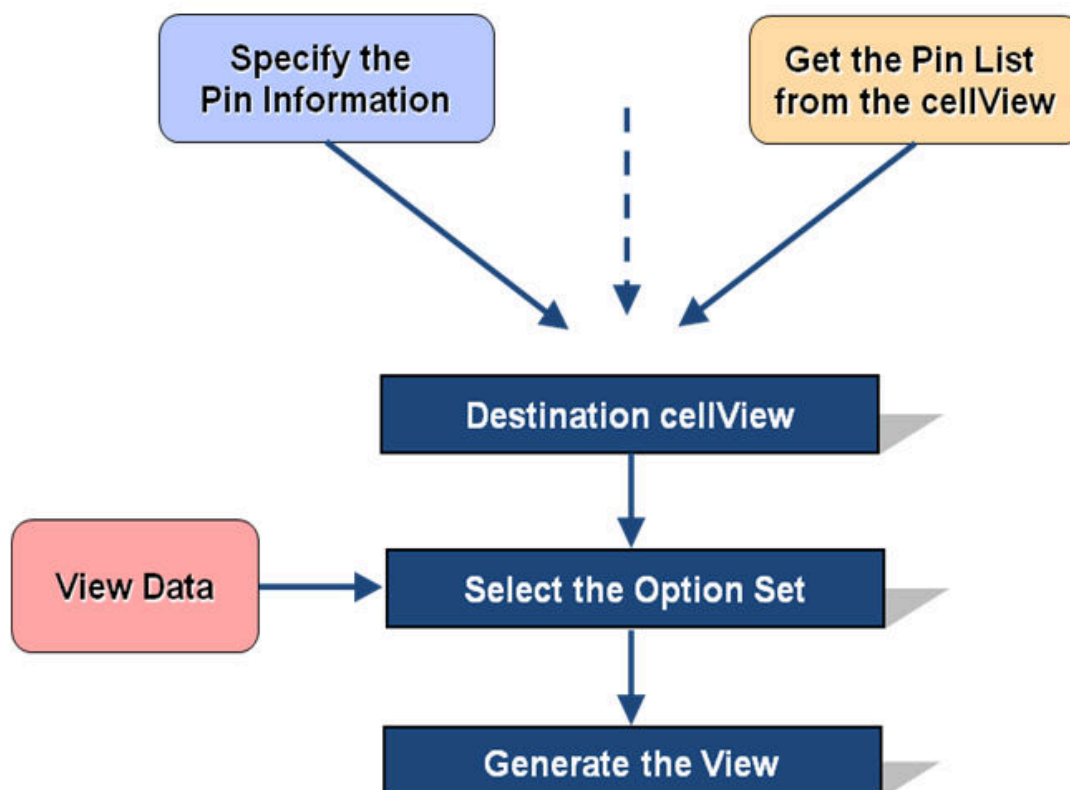
- symbol, when the source is schematic
- abstract, when the source is layout
- schematic, when the source is symbol

Example 3 Setting the Preferred Source and Target Cellview Names

```
db::setPrefValue dbGenerateCellViewDefaultViewName -value {{schematic  
symbol} {layout abstract} {symbol schematic}}
```

Creating a Cellview From a Pin List

The following figure shows the flow for generating views from a pin list.



During symbol creation, the symbol can inherit the information read in from a source cellview or from the information specified in a table.

1. Provide the user-defined pin information or the pin list from the cellview or both. Here *user-defined pin information* means that you can specify your own pin name, type, and the side information. When the pin list is imported from the cell view, the side information is determined by the tool, which you can adjust.
2. Specify the destination cell view and select the available **Option Set**.

Once the destination view type is known, the supported **Option Set** type is retrieved.

When the source is a pin list, you can specify the name, terminal type, and position for each pin.

To create a cell view from a pin list:

1. Choose **Design > New Cellview > From Pins**.

The **Generate CellView From Pin List** dialog box appears.

2. Retrieve the pins from a cell view or specify the pins in the table.
3. Specify the cell view options.
4. Click **OK** or **Apply** to create the cell view.
 - If you click **Apply**, the dialog box remains open and the command remains active.
 - If you click **OK**, the dialog box closes and the command completes.

For information on using the `db::generateCellView`, `db::getCellViewInterface`, `db::showCellViewFromCellView`, `db::showCellViewFromPinList`, and `db::showUpdateCellView` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

For information on using the `se::generateSymbol` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

Creating a Cellview From a Component

To create a cellview from a component:

1. Select the source instance (component) in the schematic.
2. Choose **Design > New CellView > Component**.

The **Generate CellView From CellView** dialog box appears.

3. Specify the cellview options.

4. Click **OK** or **Apply** to create the cellview.
 - If you click **Apply**, the dialog box remains open and the command remains active.
 - If you click **OK**, the dialog box closes and the command completes.

For information on using the `db::generateCellView`, `db::getCellViewInterface`, `db::showCellViewFromCellView`, `db::showCellViewFromPinList`, and `db::showUpdateCellView` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

For information on using the `se::generateSymbol` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

Creating a CellView From a File

*In a case where you need to create a symbol view with more than 2000 pins, it is not easy to do so. This can be achieved by using the **Import CSV** option.*

To create cellView from a file:

1. Select the source cellview in the schematic.
2. Choose **Design > New CellView > Component**.

The **Generate CellView From CellView** dialog box appears.

3. Specify the Destination Cellview options.
4. Select **Import CSV** in the **Adjust Pin** box.

The file browser field appears under the option.

Note:

The **Unique pins** option and the adjust pin list table are disabled when the **Import CSV** option is selected.

5. Select the required CSV file that contains the pin definition. For more information, see [Pin Definition Rule](#).
6. Click **OK** or **Apply** to create the cellview.
 - If you click **Apply**, the dialog box remains open and the command remains active.
 - If you click **OK**, the dialog box closes and the command completes.

Pin Definition Rule

The CSV file must contain the pin definition. The order of the definition cannot be changed

- Pin Name: Any characters and digital numbers
- Pin Side: Left | left | Right | right | Top | top | Bottom | bottom
- Pin Spacing (optional): positive integer value, zero, or empty
- Pin Type (optional): Input | input | Output | output | InputOutput | inputoutput or empty

If parser detects any invalid definition, the symbol generation is disabled and outputs error messages

The following figure is an example of pin definition file.

	A	B	C	D
1	In1	Left	5	Input
2	In2<0:2>	Left	2	
3	In3<0>	Left	2	
4	O1	Right	2	
5	O2	Right	2	

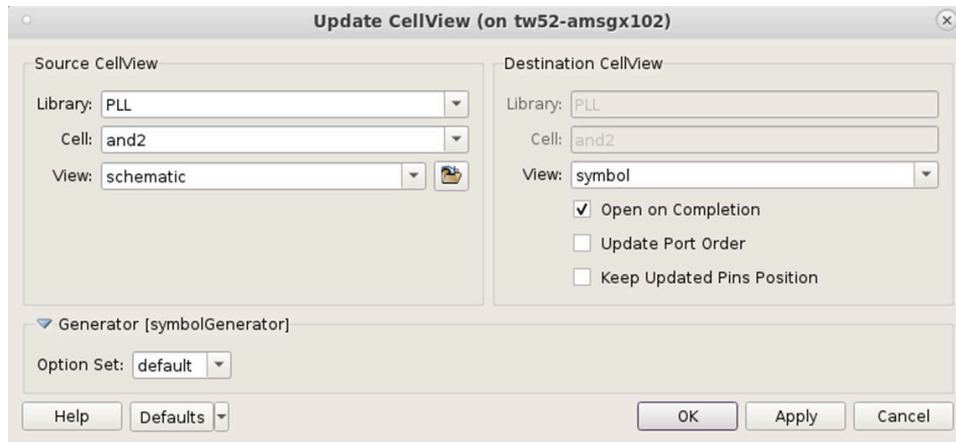
Updating a Cellview

Update CellView is typically run when you add, remove, or rename terminals in a schematic and want to update the associated symbol. You still must manually edit the updated symbol/schematic to place the added terms where you want them.

To update a destination cellview:

1. Choose **Design > Update CellView**.

The **Update CellView** dialog box appears.



2. Specify the cellview options. If invoked from a design, the source **Library**, **Cell**, and **View** fields are autopopulated from the current cellview. You are prompted to choose a destination view. The destination **Library** and **Cell** are read-only, because the update refers only to the same design as the source.

Open on Completion opens the cellview after the tool updates it.

Update Port Order updates port order properties in the schematic or symbol view when updating the cellview.

The **Keep Updated Pin Position** option is shown if destination cellview is **symbol**. Selecting this option shows the **Pin Name Mapping Table**.

See [Pin Name Mapping Table](#)

3. Click **OK** or **Apply** to update the cellview.
 - If you click **Apply**, the dialog box remains open and the command remains active.
 - If you click **OK**, the dialog box closes and the command completes.

For information on using the `db::generateCellView`, `db::getCellViewInterface`, `db::showCellViewFromCellView`, `db::showCellViewFromPinList`, and `db::showUpdateCellView` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

For information on using the `se::generateSymbol` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

Pin Name Mapping Table

Select the **Keep Updated Pin Position** option to display the **Pin Name Mapping Table**.

The pin name mapping table automatically shows pin names in which the symbol pin name does not exist in source cellview or the source pin name does not exist in symbol cellview. If the symbol pin name does not exist in source cellview, then symbol pin name is placed in the Original Pin Name column. If the source pin name does not exist in symbol cellview, then it is placed to the Updated Pin Name column. The pin name mapping table automatically map the symbol pin name and the source pin name on default display. You can drag and drop the pins based on your requirement.

Here is the mapping behavior:

- If the **Original Pin Name** (symbol pin) has no mapping name, then the symbol pin is destroyed.
- If the **Updated Pin Name** has no mapping name, then the pin is placed beneath the bounding box in symbol cellview.
- If the original pin name is mapped with the updated pin name, then it keeps the original position and change the pin name with the updated pin name in symbol cellview.
- If bus splitting names map to the same updated bus name, then keep these bit pins position and change the base names with updated bus name in symbol cellview. (If original total bit and updated total bit are the same.)

Here is the drag and drop behavior:

- Only bus splitting name can drag and drop to the same cell and map to the same original pin name.
- Non-bus splitting or bus splitting name cannot drag and drop to the cell which has non-bus splitting name.
- Different column cell cannot drag and drop to each other. The cell of **Original Pin Name** cannot drag and drop to the **Updated Pin Name** cell.
- **Original Pin Name** cell is editable for separate and multiple bus splitting name to another cell.
- **Updated Pin Name** cell can only have one pin name.

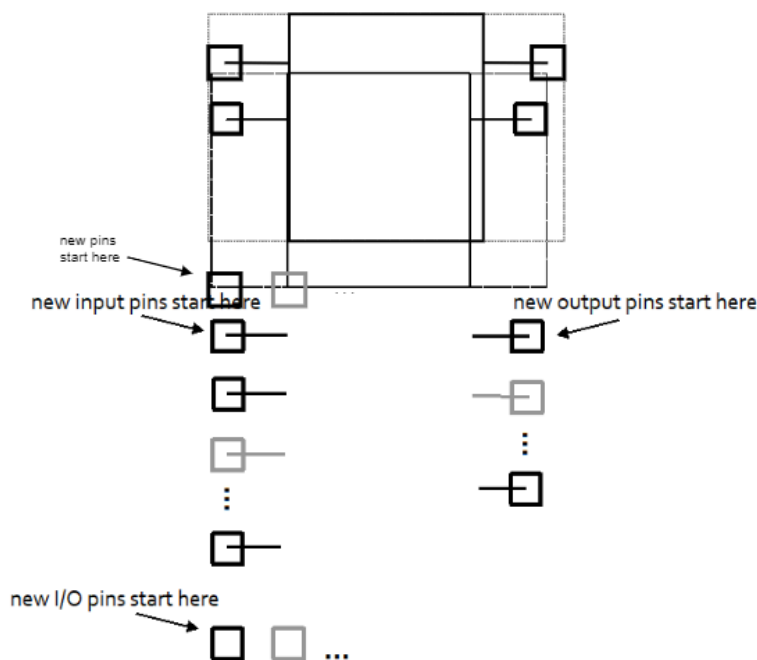
Updating Symbols

When the `se::generateSymbol` command sets the `-update` argument as *true*, the destination cellview is updated instead of recreated. The destination cellview must exist if `-update true` is specified, else a Tcl error is reported. The update process opens the destination cellview in append mode. If this fails, a Tcl error is reported.

The passed in the interface set object is then processed to synchronize the pins and parameters found in the set.

Pins that exist in the cellview but not in the interface set pin list are removed. Pins (from the interface set) that are found in the cellview have the terminal type and connection definitions updated to match the pin from the interface set. Pins from the interface set pin list not found in the cellview are created just below the lower bounds of the cell. The location of extra pins is determined based on the pin type. Input pins are placed starting from the lower left corner of cell bounds and going down. Output pins are placed starting from the lower right corner of cell bounds and going down. After input and output pins are placed, the other type of pins is placed starting from the lower left corner of the updated cell's bound and going in the right direction. The spacing between pins is the value of the pinSpacing SCP. The extra input and output pins that are placed on sides also have stubs created for them with a length equal to the value of the pinStubLength SCP. Note, the cell's bound is calculated based on the available selection box. In case if the cell has no selection shape, the merged box of shapes is used excluding text and pin objects.

The following figure shows an example of updating symbols with new pins.



Generator Option Sets

The Custom Compiler Cellview Generator uses customizable option sets (Generator Option Sets) to determine how a target cellview is to be constructed. The option sets shipped with the Custom Compiler tool can be found at this location:

`install_dir/auxx/cellview_generators`

[Example 4](#) shows the contents of the default generator option set, which is used for generating symbol views from schematics, and vice versa:

Example 4 *Contents of the Default Generator Option Set*

```
namespace eval ::startup {
#for default symbol generator
set scp(unitSize) 0.0625
set scp(aspectRatio) 0.6
set scp(pinShape) square
set scp(pinSize) 90
set scp(pinStubLength) 2
set scp(pinSpacing) 2
set scp(pinToEdgeSpacing) 2
set scp(rectangleMinWidth) 6
set scp(rectangleMinHeight) 6
set scp(pinAnnotationText) cdsTerm("\%p\")
set scp(label1Text) cdsName()
set scp(label2Text) \[@cellName\]
set scp(iteratedLabelText) cdsParam(%n)
set scp(compatibilityProperties) true
db::createGenOptionSet default -type symbolGenerator \
-value [join [array get scp] " "]
unset scp

#for default schematic generator
set sccp(unitSize) 1
set sccp(pinSpacing) 1
set sccp(oppositePinsMinSpacing) 5
set sccp(compatibilityProperties) true
db::createGenOptionSet default -type schematicGenerator \
-value [join [array get sccp] " "]
unset sccp
}
```

Each symbol construction parameter (scp) setting is expressed in units of the unitSize value, which in turn is in user units. For example, the default pin spacing for generated symbols is 2*unitSize, or 0.125.

To omit a specific construction parameter from your generator option set (for example, if you want to generate a symbol that does not include annotation or label text), define the parameters to have empty values as shown in [Example 5](#).

Example 5 *Omitting Specific Parameters*

```
set scp(pinAnnotationText) {}
set scp(label1Text) {}
set scp(label2Text) {}
set scp(iteratedLabelText) {}
```

The following figure shows how symbol construction parameters appear on the canvas.

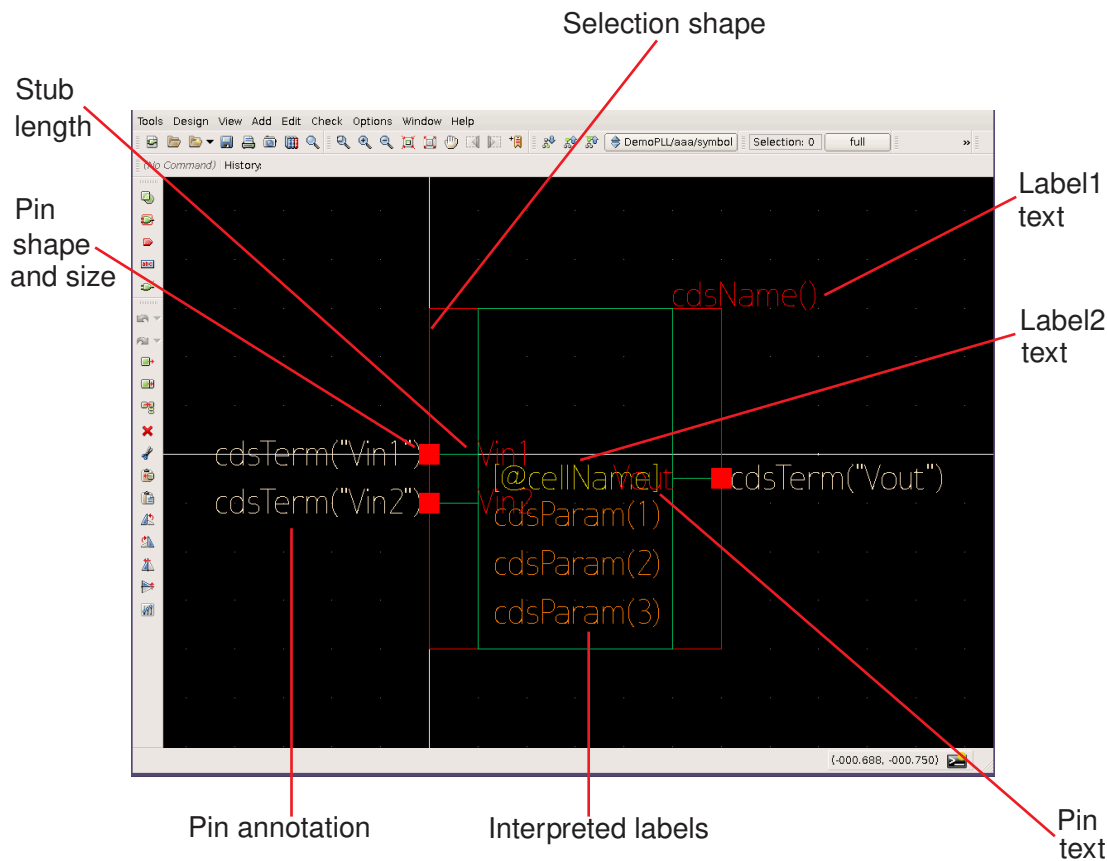


Table 16 lists the symbol construction parameters.

Table 16 Symbol Construction Parameters

Name	Type	Hard Coded Value	Description
unitSize	<float>		Definition of base unit length.
aspectRatio	<float>	0.6	Definition of aspect ratio. Must be greater than zero. By default, width is 60% of the height. The aspectRatio is applied <i>after</i> the minimum height and minimum width calculation. Then, depending on the aspectRatio's value, either width or height is increased to meet the specified aspect ratio.

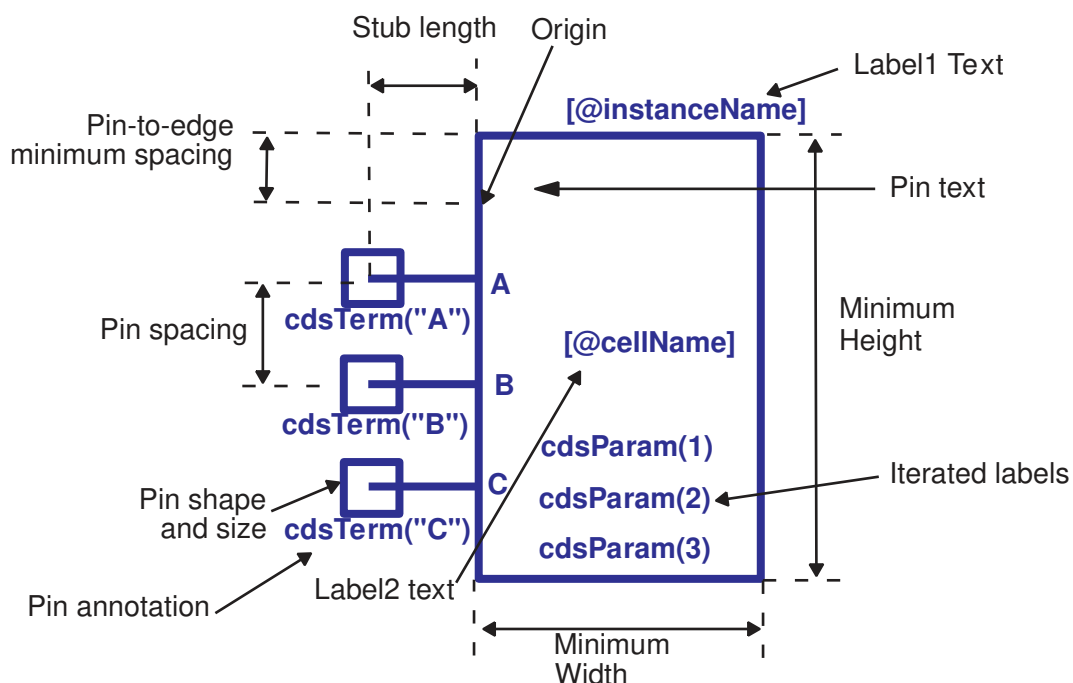
Table 16 Symbol Construction Parameters (Continued)

Name	Type	Hard Coded Value	Description
pinShape	<enPinShape>		Type of shape to create for pins. Valid enum values are square, circle, triangle, and custom.
pinSize	<int>		Value should be from 1 to 100 and represents the percentage of unitSize.
pinStubLength	<int>		Length of line drawn from pin center to core rectangle.
pinSpacing	<int>		Pin spacing.
pinToEdgeSpacing	<int>		Pin to edge spacing.
rectangleOrigin	<enCorner>	upperLeft	Corner of core rectangle defining location where pin addition begins.
cellviewOriginPin	<enOriginPin>	leftFirst	Defines which pin is created at cellview coordinate {0, 0}.
rectangleMinWidth	<int>		Rectangle minimum width.
rectangleMinHeight	<int>		Rectangle minimum height.
pinAnnotationText	<string>		Pin annotation text.
label1Text	<string>		Label1 text.
label1Origin	<enJust>	upperRight	Label1 origin position relative to core rectangle border.
label1Just	<enJust>	lowerLeft	Label1 justification relative to label origin.
label2Text	<string>		Label2 text.
label2Origin	<enJust>	centerCenter	Label2 origin position relative to core rectangle border.
label2Just	<enJust>	centerCenter	Label2 justification relative to label origin.
iteratedLabelText	<string>		Iterated label text.
iteratedLabelCount	<int>	3	Indicates the number of iterated labels to create.

Table 16 Symbol Construction Parameters (Continued)

Name	Type	Hard Coded Value	Description
iteratedLabelPos	<enLabelPos>	bottom	Iterated label starting position.
compatibilityProperties	<bool>		Indicates if properties important to interoperability should be created in the design.

The following figure shows symbol components.



9

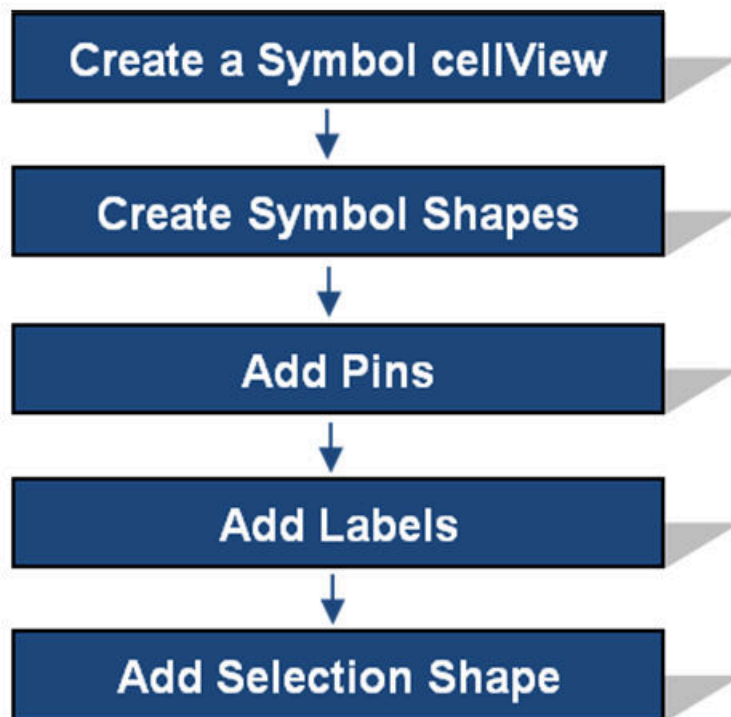
Creating Symbols

This chapter describes the features and procedures you can use to create symbols manually.

The Symbol Editor allows you to:

- Enable generation of schematic containers for hierarchical schematic capture.
- Provide commands for shape/label creation, pin creation, and object manipulation.

The following figure shows the flow for manual symbol creation.



Note:

For information about creating shapes, see [Creating Shapes](#).

This chapter describes procedures for:

- [Creating a Symbol Manually](#)
- [Using Symbol Editor Toolbars](#)
- [Importing Existing Symbol Graphics](#)
- [Adding Symbol Pins](#)
- [Updating Pin Labels](#)
- [Adding Selection Shapes](#)
- [Redefining the Symbol Origin](#)

Creating a Symbol Manually

To create a symbol:

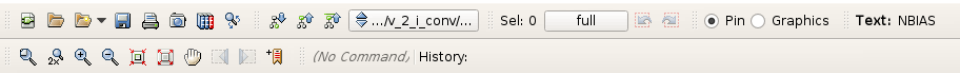
1. From the Console, select **File > New > CellView**.
The **New CellView** dialog box opens.
2. Specify the library and cell name of the symbol.
3. Specify **symbol** as the **View Name**.
4. Click **OK**.

A new Symbol Editor window opens.

Using Symbol Editor Toolbars

Symbol Editor toolbars offer quick access to symbol design operations.

You can display or hide the toolbars in the Symbol Editor window by choosing **Window > Toolbars**.



The following table lists and describes the Symbol Editor toolbars.

Toolbar	Use this toolbar to...
Command Options	Show options for the currently enabled interactive command.
Create	Create symbols.

Toolbar	Use this toolbar to...
Editing	Add shapes, selection shapes, pins, and labels. Import symbols.
Hierarchy	Descend in edit or read mode, edit in place, return to previous hierarchy level or to top, select any cellview from current hierarchy traversal.
Mouse Bindings	Show the binding for the currently active command at the bottom of the window.
Object Info	Show shape type, and net, for shape under cursor. For vias, pins, shapes, show type and net. For instances, show cell name and instance name.
Selection	List the number of partially or fully selected objects.
Snap Spacing Mode	Specify Pin (default) or Graphics mode. Allows you to have two distinct grids, your default one for ensuring pin alignment and a graphics grid for advance details required during symbol generation.
Standard	Create new cellviews, open existing or recent designs, save, print designs, export images, open the Library Manager and the Hierarchy Navigator, and search for and replace cellviews.
symbolNavigation	Zoom areas, in, out, fit designs to viewports, pan viewports, go to previous or next viewport, add new bookmarks.

Importing Existing Symbol Graphics

For information on using the `se::importSymbol` and `ise::importSymbol` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

You can reuse certain characteristics of one symbol when creating another symbol. The characteristics that you can reuse include:

- Shapes (graphical objects)
- Pins
- Text
- Selection Regions
- A combination of these

To import characteristic data from another symbol cellview:

1. Choose **Add > Import Symbol**.

The **Import Symbol** dialog box opens.

2. Specify the cellview that you want to import.

An image of the selected symbol follows your cursor.

3. Select the data options.

By default, all the data check boxes are selected. Click a check box to toggle an option.

The **Hide** button at the bottom of the dialog box closes the dialog box and displays the import options in a toolbar. The **Eject** button on the toolbar opens the dialog box.

4. Click the canvas to place the symbol.

The command remains active.

5. Press Esc to end the command.

Adding Symbol Pins

Pins are used as physical interfaces on a block to allow connections through hierarchy. Pins can be added in the Symbol Editor using the **Add Pin** command. You can create scalar or bus pins by naming the pins appropriately. The pin creation command also supports name stacking. The symbol used for the pins can be set using the preferences `seSymbolPinInput`, `seSymbolPinOutput`, and `seSymbolPinInputOutput`.

For information on using the `ise::createSymbolPin` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To add a single pin:

1. Choose **Add > Pin**.

The **Pin** toolbar opens.

2. Specify a pin name.

This name defines the terminal for the pin.

3. Specify the remaining pin options.

The **Eject** button on the toolbar opens the dialog box. The **Hide** button at the bottom of the dialog box closes the dialog box and displays the **Pin** toolbar. The `seCreateSymbolPinDialogMode` preference controls whether the dialog opens by default.

The dialog box contains the following options:

Option	Description
Names	Specifies the pin names. You can also enter bus and bundle names.
Expand	When enabled, bus and bundle names are expanded to bit net labels for each pin you add.
Array	When enabled, places a one-dimensional array of pins equal to the number of pin names.
Type	Specifies the pin type.
Angle	Specifies the drawing angle. See Angle Mode for Creating and Editing Objects .
Shape	Specifies a pin shape. If you choose custom , a folder icon appears and you can browse to select a symbol to use as a custom pin shape.
Transform	These options let you rotate or flip the pin before placement.
Text Attributes	(Dialog box only) A toggle that expands and collapses the form to reveal or hide the font, height, color, and overbar options.
Parameterized Connections	(Dialog box only) A toggle that expands and collapses the form to reveal or hide the following fields: ◦ Name: A name you define for referencing the connection. ◦ Default Net: An existing net in the design. This net is used if a corresponding assignment is not found up the hierarchy. For more information, see Creating Connection Definitions .

4. Click the canvas to place the pin.

The command remains active.

5. Press Esc to end the command.

This section contains the following topics:

- [Naming Pins](#)
- [Selecting Pin Shapes](#)
- [Specifying Connections](#)

Naming Pins

For information on using the `se::createPin`, `ise::createSchematicPin`, and `ise::createSymbolPin` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

This section contains the following topics:

- [Symbol Pin Name Stacking](#)
- [Creating Bused or Bundled Pins](#)

Symbol Pin Name Stacking

A pin name stack consists of a list of space-delimited pin names. With each click in the canvas, a pin is placed, and the corresponding name is removed from the stack. Performing an undo operation during the command restores the pin name to the top of the name stack.

To add multiple pins using a name stack:

1. Choose **Add > Pin**.

The **Pin** toolbar opens.

2. Specify a list of names (space-delimited).
3. Specify the remaining pin options.
4. Click the canvas to place one pin for each name in the stack.

The command remains active.

5. Press Esc to end the command.

Creating Bused or Bundled Pins

You can specify iterated names to create bused or bundled pins.

- To create a multiple-bit pin, use the same naming syntax as you would to create a bundled net. For example, to create an 8-bit pin with a base name of A, use the pin name `A<0:7>`. You can then place the pin as a bused pin or you can place discrete members of the bus. Select the **Expand** option to scalarize the iterated names as they are placed. To place all discrete members of the bused pin all at once, select the **Array** option as well, and then specify a point to begin the array and a point to end the array.
- To create a bundled pin or an array of pins, use the same procedures as described above. But for bundles, use a comma-delimited member list; for example: `a,b,c,d`. For an array of pins, use a space-delimited member list; for example: `a b c d`.

Selecting Pin Shapes

When placing symbol pins, you can select predefined shapes or represent a pin by a specific instance master.

- To use one of the predefined shapes, use the **Shape** command to select a shape, and then follow the command prompts to create the shape.
- To use a pin instance to represent the pin, select *custom*. A preference value corresponding to the pin type determines the instance master that is used. For example, if you wish to use the cellview basic/block pin/symbol to represent your input pins, set the Tcl preference `seSymbolPinInput` to use that library, cell and view, as shown below.

```
db::setPrefValue seSymbolPinInput -value {basic blockipin symbol}
```

The following table lists the preferences for the other pin types.

Preference	Pin Type
seSymbolPinInput	input
seSymbolPinOutput	output
seSymbolPinInputOutput	inputOutput
seSymbolPinSwitch	switch
seSymbolPinTristate	tristate

Specifying Connections

While placing a pin, you can also specify a terminal connection definition for that pin that defines the net connected to the pin. That is, after placing the symbol with this pin in a design and then drawing a wire from the pin, the wire is on the net defined by the connection definition.

Connection definitions have a name as well as a default net. The default net is used as the connection name unless there is an assignment using the connection definition name somewhere higher in the design hierarchy. For more information on using connection definitions, see [Creating Connection Definitions](#).

Note:

Terminal connection definitions are only available for creation from the **Add Pin** dialog box. You can specify connection definitions only for scalar pins.

Updating Pin Labels

For information on using the `se::updatePinLabels` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To update or add a pin label in a symbol design:

1. Select a pin from the canvas and press Q to open the Property Editor.
2. In the Property Editor, modify the terminal name as you want and then press Enter. The tool updates the `cdsTerm` label associated to the pin.

If the `cdsTerm` label is not updated accordingly, choose **Edit > Update Pin Labels** to reflect the change.

Adding Selection Shapes

Use the **Add > Selection** command to create a selection rectangle or polygon. This is the shape that the Schematic Editor uses to define the selectable, or active, region of the resulting instance. When you select an instance, the selection shape is what you see rendered along with the shape itself.

For information on using the `se::createSelectionShape` and `ise::createSelectionShape` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

The following sections describe how to create and overwrite selection shapes:

- [Creating Selection Shapes Automatically](#)
- [Creating Selection Shapes Manually](#)

Creating Selection Shapes Automatically

The **Auto Create** option lets you generate a selection shape automatically. A generated shape is a rectangle of the minimum size needed to enclose all pin centerpoints and shapes in the symbol.

To create a selection shape automatically:

1. Choose **Add > Selection**.
The **Selection Shape** toolbar opens.
2. Click the **Auto Create** button.

The shape appears that encloses all pin centerpoints and shapes in the symbol.

The command remains active.

3. Press Esc to end the command.

Creating Selection Shapes Manually

To create a selection shape manually:

1. Choose **Add > Selection**.

The **Selection Shape** toolbar opens.

The toolbar contains the following options:

Option	Description
Shape icons	Specifies a selection shape.
Angle	Specifies the drawing angle: Orthogonal , Diagonal , or Any Angle . Only available for polygon shapes. See Angle Mode for Creating and Editing Objects .
Auto Create	Automatically creates the selection shape.
Overwrite	Overwrites any existing selection shape.

2. Specify the shape type.

You can specify either a rectangle or a polygon by clicking on the appropriate icon.

3. If you select a polygon shape, specify the drawing angle. See [Angle Mode for Creating and Editing Objects](#).
4. Click to specify two or more points on the canvas that define the shape geometry.

Note:

You can use the Backspace key to remove the last point entered.

5. Double-click or press Enter to complete the shape.

The command remains active.

Note:

You can click the shape icons to toggle between the shape types before you end the command.

6. Press Esc to end the command.

Redefining the Symbol Origin

The cellview origin setting defines where on the canvas the { 0, 0 } coordinate is located on the canvas.

For information on using the `ide::defineOrigin` Tcl command, see Design Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To redefine a cellview origin:

1. Select **Edit > Define Origin**.
2. Click the canvas to specify a new location for the cellview origin.

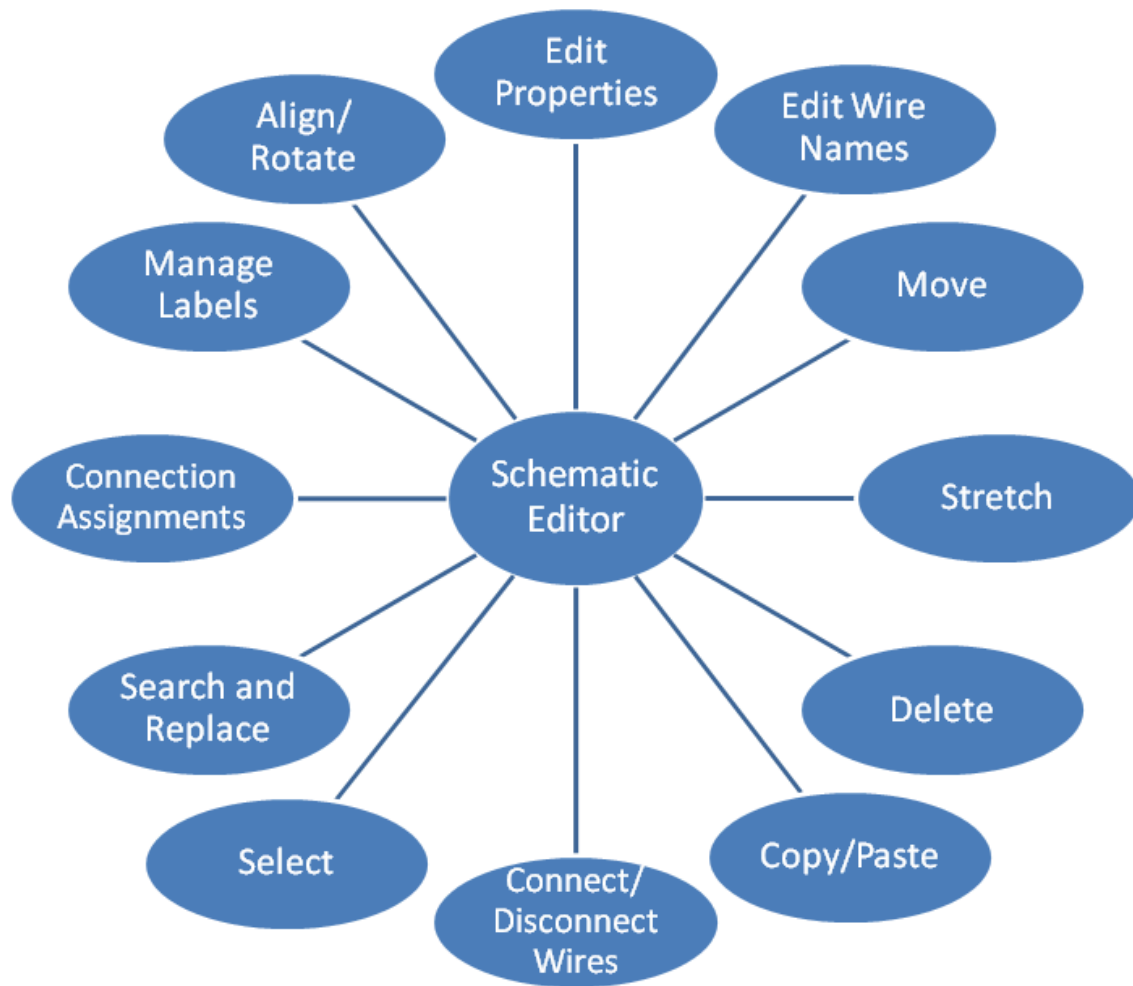
10

Editing Schematics

This chapter describes the features and procedures you can use to modify and manipulate objects in your designs.

The Schematic Editor lets you adjust the position and orientation for all geometrical objects.

The following figure shows the various editing operations available in the editor.



This chapter covers the following topics:

- [Undo and Redo](#)
- [Direct Object Manipulation](#)
- [Moving Objects](#)
- [Stretching Objects](#)
- [Aligning Objects](#)
- [Copying Objects](#)
- [Using the Clipboard](#)
- [Deleting Objects](#)

- [Editing Canvas Text](#)
- [Editing a Cellview in Place](#)
- [Rotating and Flipping Objects](#)
- [Connecting and Disconnecting Crossing Wires](#)
- [Chopping Wires](#)
- [Colorizing Nets](#)
- [Selecting Objects and Areas](#)
- [Renumbering Instances](#)
- [Renaming Instances](#)
- [Ignoring Instances](#)
- [Ordering and Synchronizing Ports](#)

Undo and Redo

The undo and redo features work on all creation and editing commands and allow for an undo or redo depth of 256 levels per design.

Use one of the following commands:

- **Edit > Undo.**
- **Edit > Redo.**

Alternatively, use the **Undo** and **Redo** buttons on the toolbar.

For information on using the `de::undoTransaction` and `de::redoTransaction` Tcl commands, see Design Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

Direct Object Manipulation

Direct object manipulation refers to a simple select-drag-release operation that lets you quickly perform certain edit operations without invoking an editor command.

The object type determines whether the direct manipulation operation results in a move, stretch, or wiring event:

- Non-connectivity objects, such as labels and notes, are moved
- Connectivity objects, such as instances and schematic pins, are stretched

A special case is that of an instance pin; the result of a select-drag-release operation on an instance pin is a wiring event.

If you attempt direct manipulation on an instance pin, the action becomes the start of a wiring operation. Releasing the mouse button completes the wiring operation.

Note:

Direct manipulation ignores the contents of a current selected set; that is, if you have several objects selected, and perform a stretch operation on an unselected object, the selected set is not affected. If instead you directly manipulate an object that is a member of the selected set, the entire selected set is directly manipulated.

To perform a direct manipulation operation, do the following:

1. Move the cursor over an object in the design, click, and hold to select the object.
2. While holding the mouse button, drag the object to a new location in the design.
 - The default operation when you drag the object is a stretch operation.
 - For a move operation, hold the Shift key when you drag the object.
 - For a copy operation, hold the Ctrl key when you drag the object.

The object moves with the cursor.

3. Release the mouse button.

This completes the operation.

Moving Objects

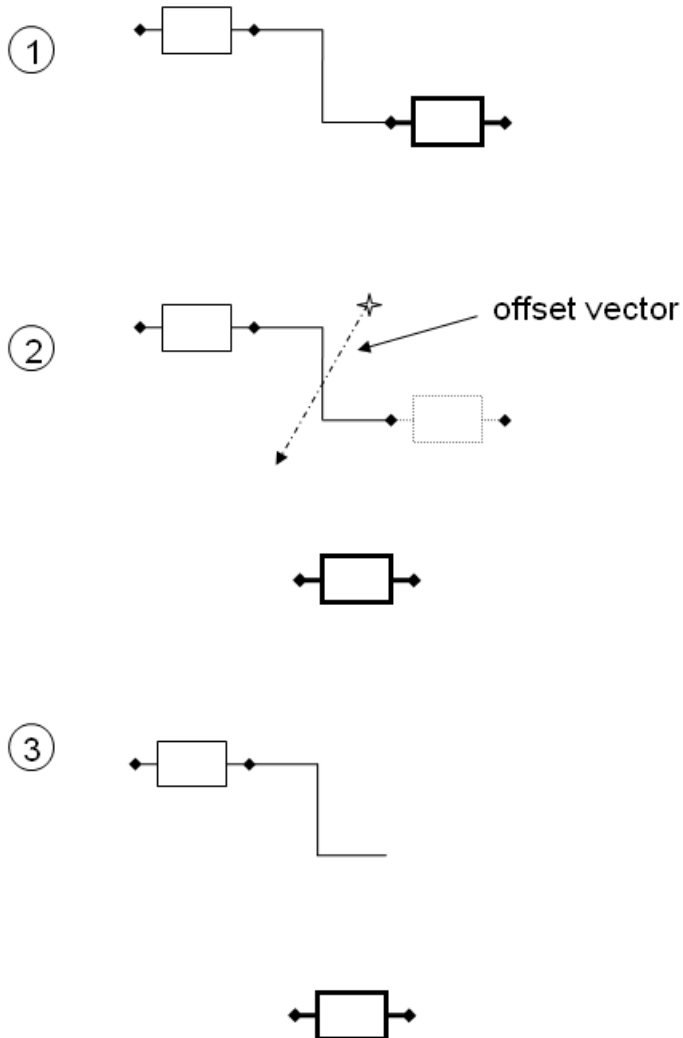
The **Move** command lets you move geometrical objects (including instances, pins, wires, and text) to a new location.

Note:

Parent-child relationships are maintained during a move operation, but not connectivity. If you want to move objects and maintain connectivity, use the **Stretch** command.

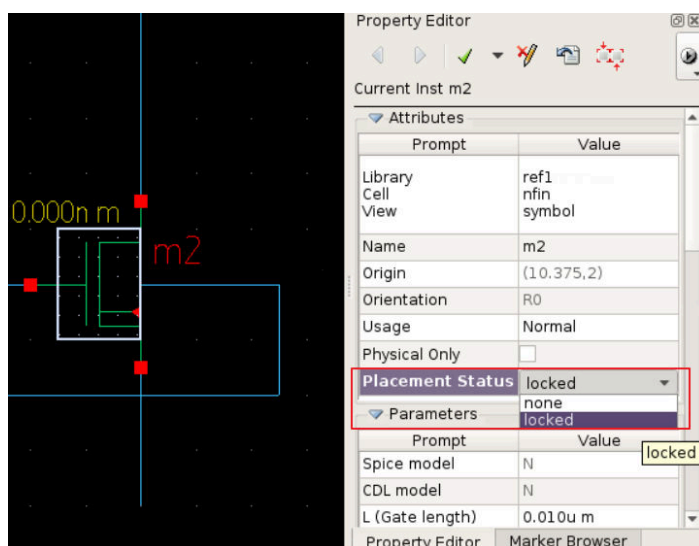
For information on using the `se::move` and `ise::move` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

The following figure illustrates the use of the **Move** command to move a selected instance.



Note:

You can change an instance's **Placement Status** to **locked** in the Property Editor, which can help avoid accidentally moving an object.



Choose one of these move tasks:

- [Moving Objects in the Current Cellview](#)
- [Moving Objects Between Cellviews](#)

Moving Objects in the Current Cellview

To move objects in the current cellview:

1. Select an object on the canvas.

The editor highlights the object.

2. Choose **Edit > Move**.

Alternatively, right-click on an object on the canvas and then choose **Move**.

The **Move** toolbar appears. (To open the **Move** dialog box, use the F3 shortcut key after initiating the interactive command.)

3. Set the toolbar options.

The toolbar contains the following options:

Option	Description
Angle	Controls how you can move the object. See Angle Mode for Creating and Editing Objects .
Transform Icons	These options let you rotate or flip an object during the move operation.

Option	Description
Origin	Specifies the method for defining the point about which to rotate the object.

The menu can be shown during the **Move** command by right-clicking. The menu contains these options:

Option	Description	Default Shortcut Key
Rotate 90	Rotates the object 90 degrees counter clockwise.	R
Flip Horizontal	Flips the object about the Y axis.	Shift+R
Flip Vertical	Flips the object about the X axis.	Ctrl+R

4. Click and drag on the canvas to relocate the object.
5. Click to define the destination.

The command terminates.

Moving Objects Between Cellviews

You can move an object from an original cellview in a schematic design window to a target cellview in another schematic design window.

To move an object to another schematic design window:

1. Select an object in the original schematic design window.

The editor highlights the object.

2. Choose **Edit > Move**.

The **Move** toolbar appears. (To open the **Move** dialog box, use the F3 key after initiating the interactive command.)

3. Set the toolbar options.
4. Move your cursor to the target schematic design window.

The **Move** command and toolbar are activated in the target schematic design window.

5. Click in the target schematic design window to place the object.

On placement:

- The editor deletes the object from the original schematic design window.
- The command terminates.

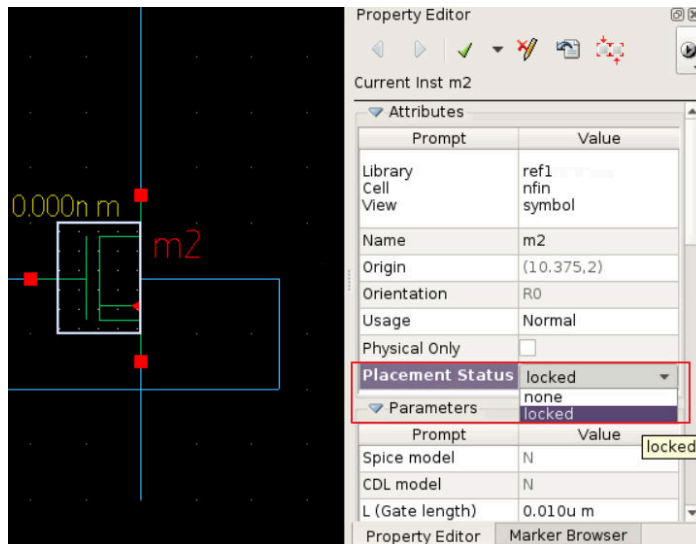
For information about moving objects between cellviews, see [Using the Clipboard](#).

Stretching Objects

The **Stretch** command lets you relocate partially and fully selected objects to new destinations on the current canvas. Wiring objects that are external to the selected set are rewired according to the options you specify in the command toolbar (see [Stretch Toolbar Options](#)).

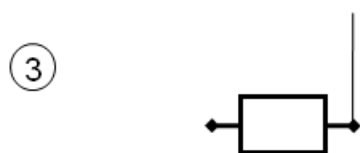
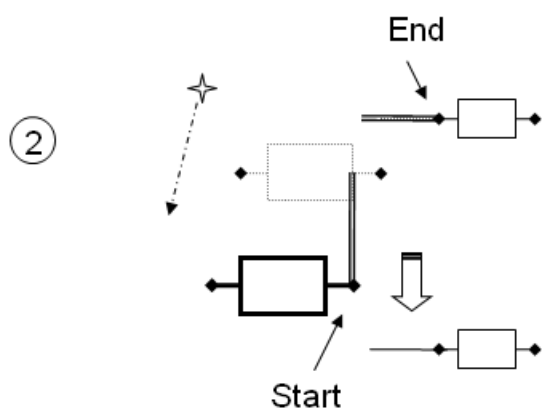
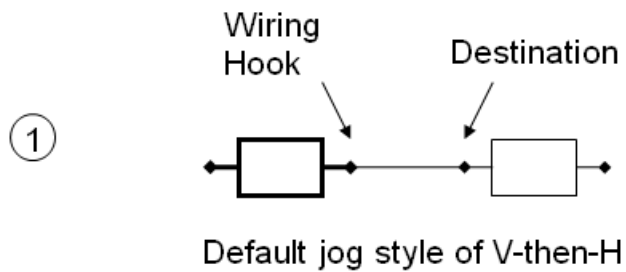
Note:

You can change an instance's **Placement Status** to **locked** in the Property Editor, which can help avoid accidentally stretching an object.

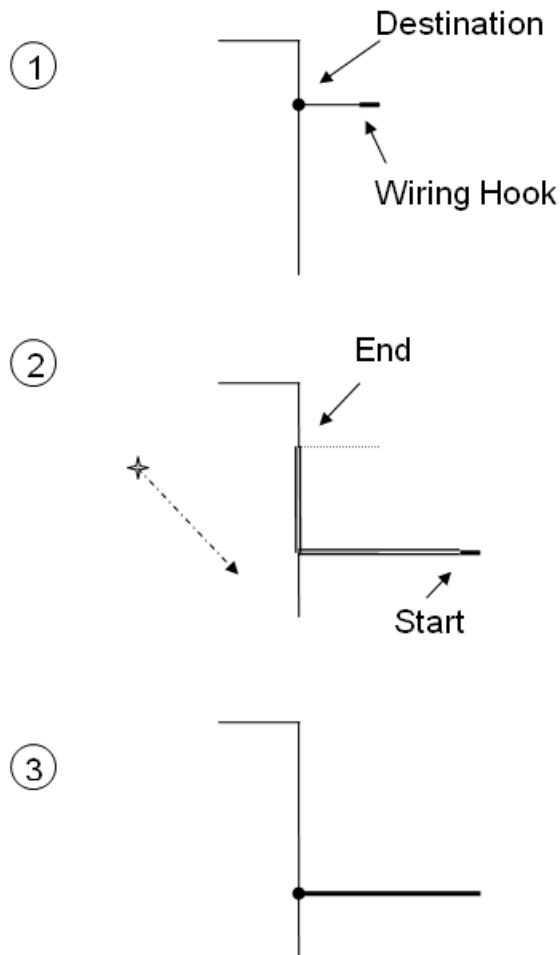


For information on using the `se::stretch` and `ise::stretch` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

The following figure illustrates the use of the **Stretch** command to stretch a selected instance.



The following figure illustrates the use of the **Stretch** command to stretch a partially-selected segment or wire segment.



Note:

Parent-child relationships are maintained during a stretch operation as well as connectivity. If it is not important to maintain connectivity, you can use the **Move** command.

To stretch an object:

1. Select an object on the canvas.

The editor highlights the object.

2. Choose **Edit > Stretch**.

The **Stretch** toolbar appears. (To open the **Stretch** dialog box, use the F3 shortcut key after initiating the interactive command.)

3. Specify the **Stretch** options (see [Stretch Toolbar Options](#)).

4. Click an object to specify the anchor point.
5. Move your cursor to begin stretching the object.

As you move the object, the tool displays the associated net as a dotted line.

6. Click the canvas to specify the destination point.

The operation completes and the command terminates.

Stretch Toolbar Options

The toolbar contains the following options:

Option	Description
Angle	Specifies how to stretch the object. The options are Orthogonal , Diagonal , or Any Angle . See Angle Mode for Creating and Editing Objects .
Transform Icons	These options let you rotate or flip the object while stretching the object.
Origin	Specify method for defining the point about which to rotate the object.
Avoid Symbols	Specifies whether to avoid symbols when creating the route.
Avoid Shorts	Specifies whether to avoid creating name shorts when creating the route.

Stretch Menu

The menu contains these options:

Option	Description	Default Shortcut Key
Rotate 90	Rotates the object 90 degrees counter clockwise.	R
Flip Horizontal	Flips the object about the Y axis.	Shift+R
Flip Vertical	Flips the object about the X axis.	Ctrl+R

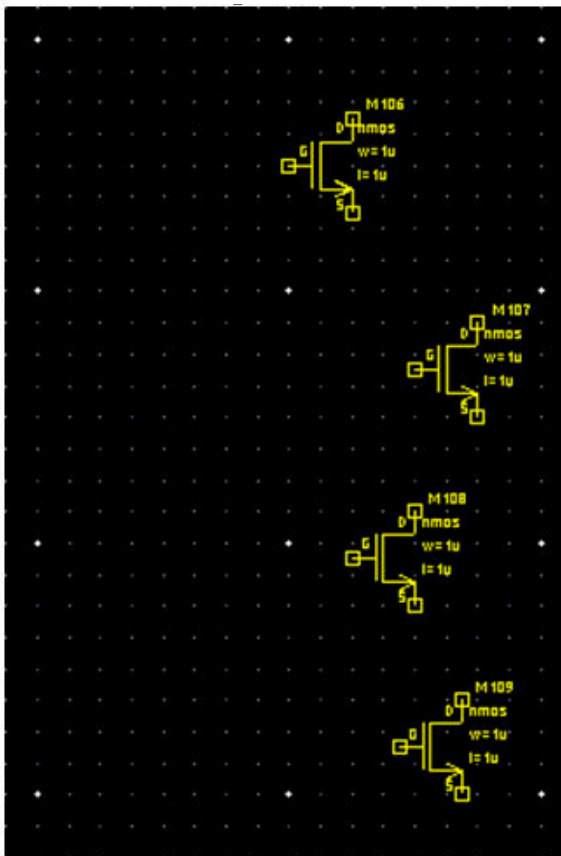
Aligning Objects

The **Align** command lets you:

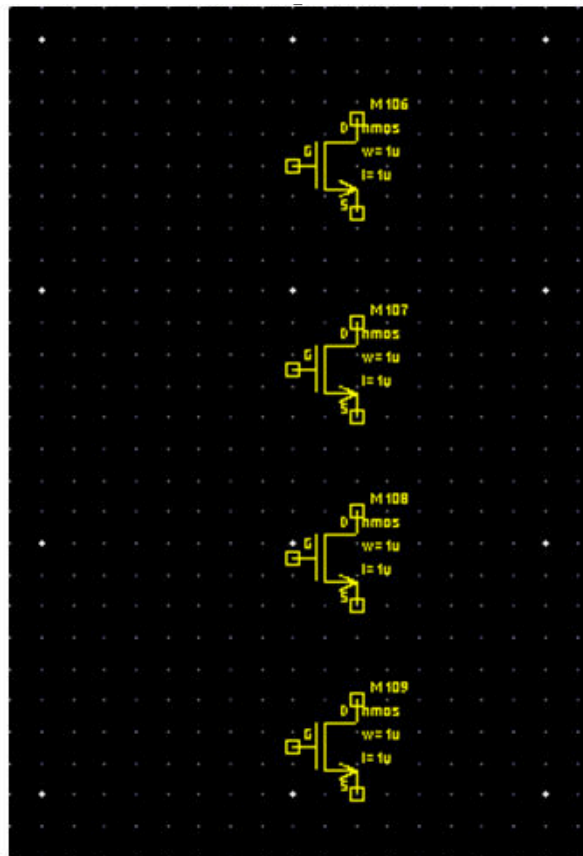
- Arrange instance terminal pin shapes, terminal pin shapes, net wire vertices, and text label (that is, instance label, instance name, instance pin name, wire name, pin name) in the current design to facilitate routing.
- Arrange connected terminal pins or instances to improve the circuit image.

The following figure illustrates alignment of objects (in this case, instance terminals).

Before alignment:

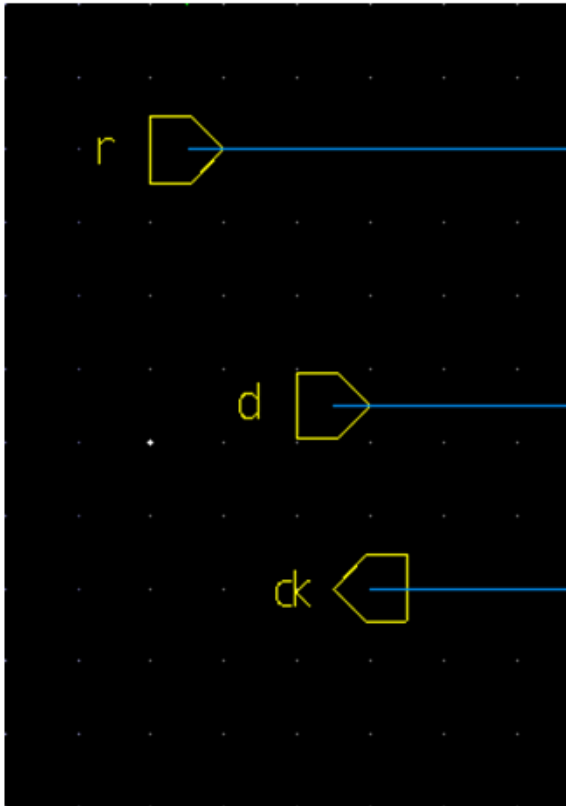


After alignment:

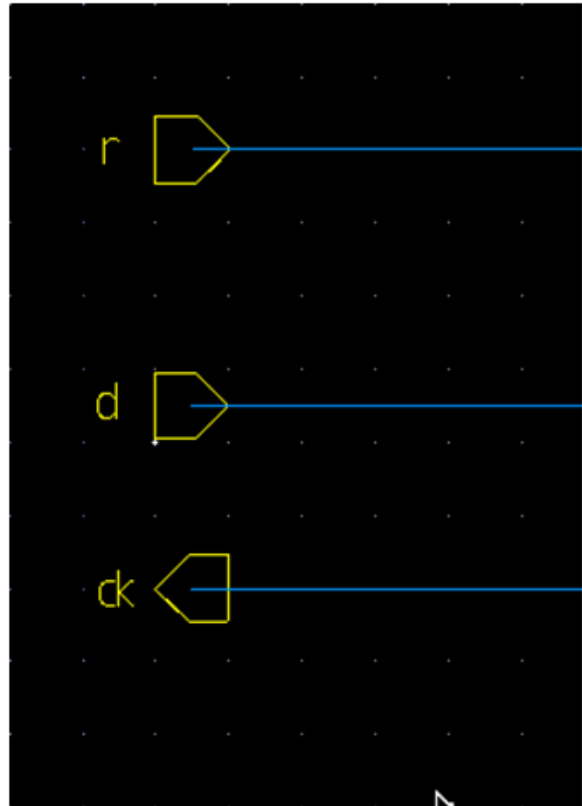


The following figure illustrates aligning connected terminals.

Before alignment:



After alignment:



You can use the **Edit > Align** command to improve design appearance and to make it easier to connect objects in the design. The command lets you align multiple target objects relative to one source object. Selecting a source object displays a reference line prior to committing showing how the target objects align. You can align the following objects: instance terminal pins, terminal pins, and net wire vertices.

For information on using the `se::align` and `ise::align` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

The **Align** command operates in two modes, which are described in the following sections:

- [Aligning Preselected Objects](#)
- [Aligning Post Selected Objects](#)
- [Align Toolbar Options](#)

Aligning Preselected Objects

There are two ways to align preselected objects, as explained in the following sections:

- [Aligning a Set of Preselected Objects](#)
- [Aligning to an Individual Preselected Object](#)

Aligning a Set of Preselected Objects

To align a set of preselected objects:

1. Select a set of objects (instance terminals, terminal pins, or wire vertices).

To filter objects for selection, use the Schematic Object Filter window assistant to make your selections.

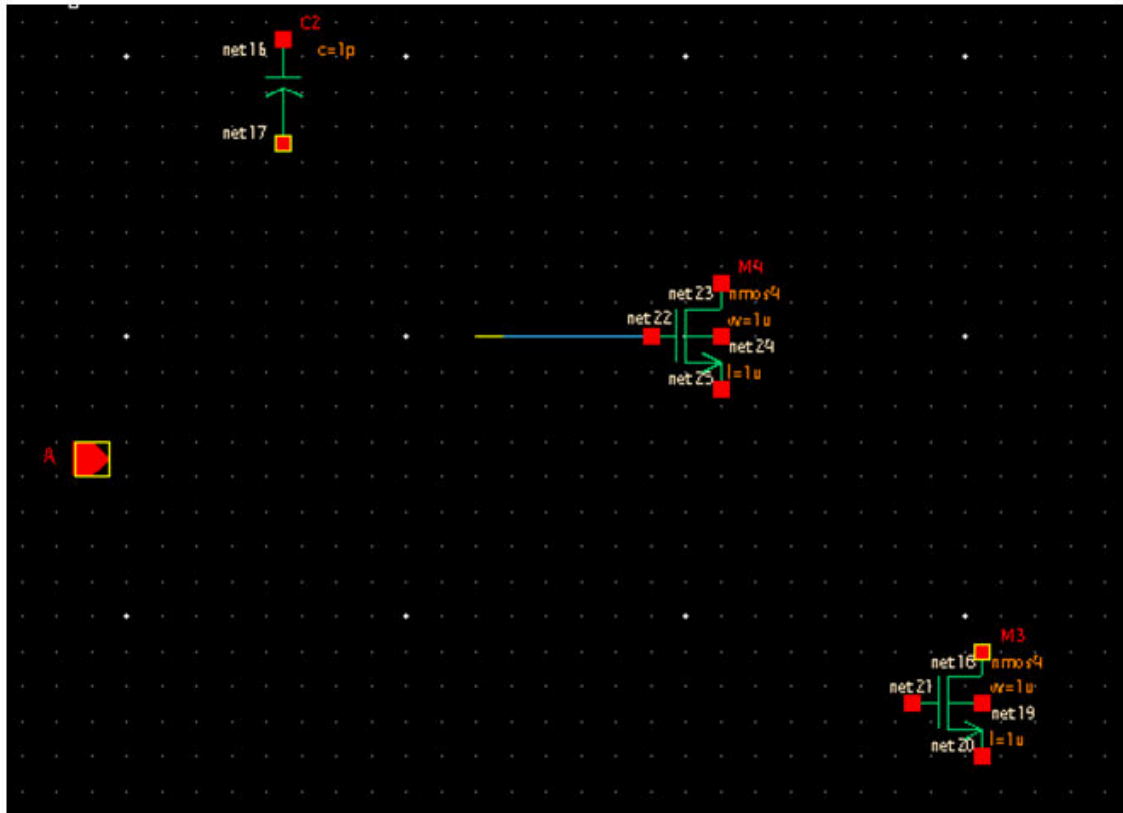
2. Choose **Edit > Align**.

The **Align** toolbar opens.

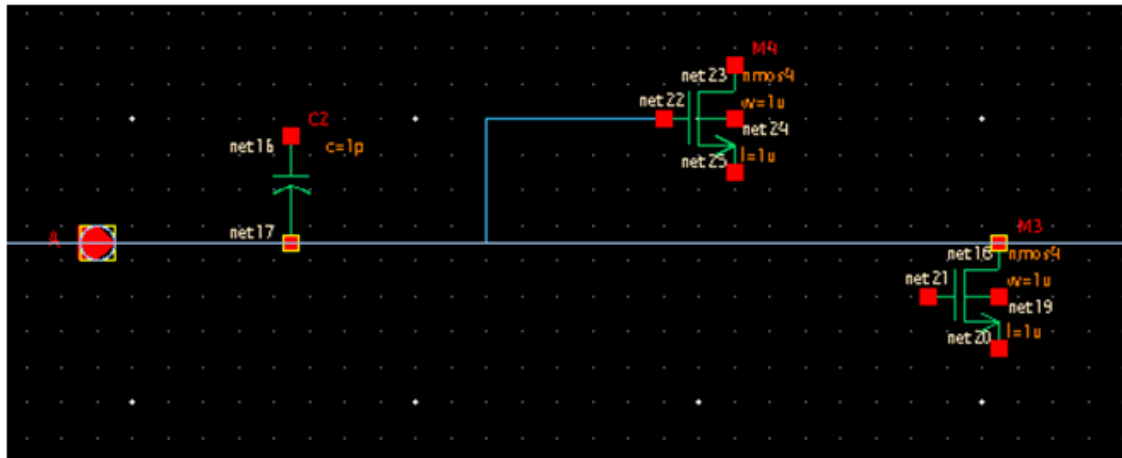
The preselected objects align according to any previously specified option settings. By default, the **Direction** option is set to **Auto** and the **Stretch** option is enabled.

The tool considers the left-most object to be source object when you align objects in a horizontally, and the top-most object to be the source when you align objects vertically. The target objects are aligned relative to the source.

The following figure shows selected objects before alignment.



The following figure shows selected objects after alignment.



3. Press Esc to end the command.

Aligning to an Individual Preselected Object

To align an individual preselected object:

1. Select an object.

The individual object becomes the source object.

To filter objects for selection, use the Schematic Object Filter window assistant to make your selection.

2. Choose **Edit > Align**.
3. (Optional) Change the **Direction** or **Stretch** options.
4. Select a target object.

The target object aligns with the source.

The command remains active. If you select additional objects they are also aligned with the source until you exit the command.

To set a new source for alignment, you must click **Restart** in the toolbar and repeat the above steps to select source and target object.

5. Press Esc to end the command.

Aligning Post Selected Objects

To align post selected objects:

1. Choose **Edit > Align**.
2. (Optional) Change the **Direction** or **Stretch** options.
3. Select a source object.
4. Select a target object.

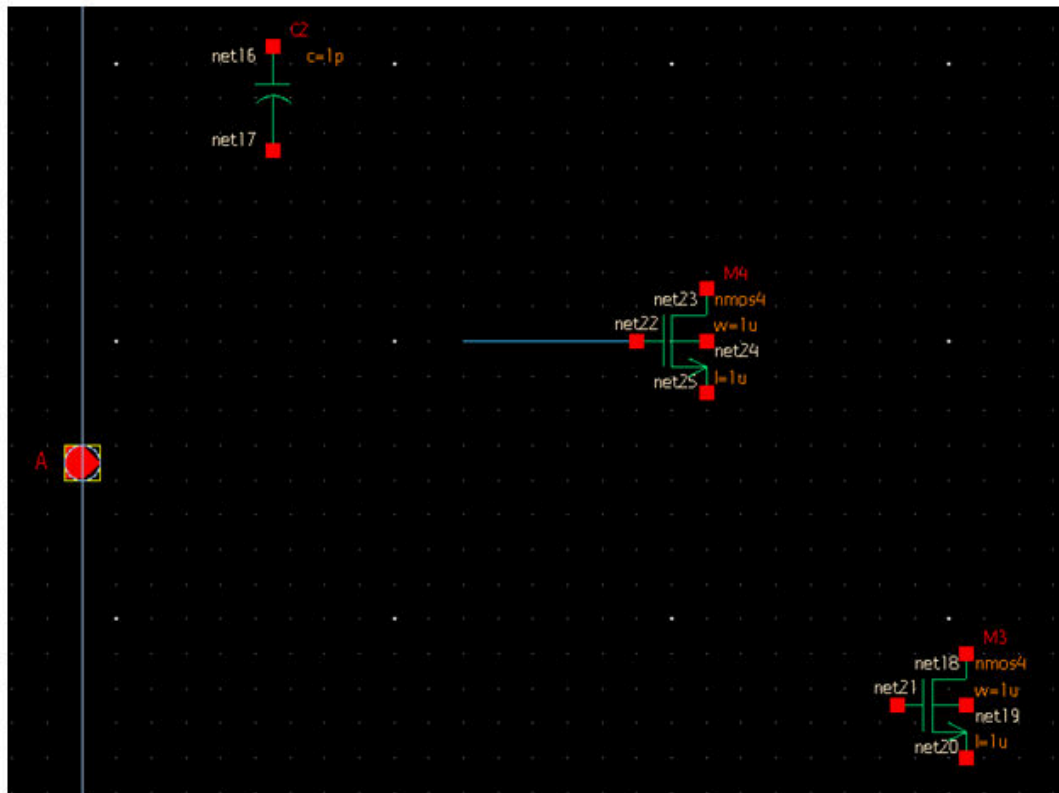
The target object aligns with the source.

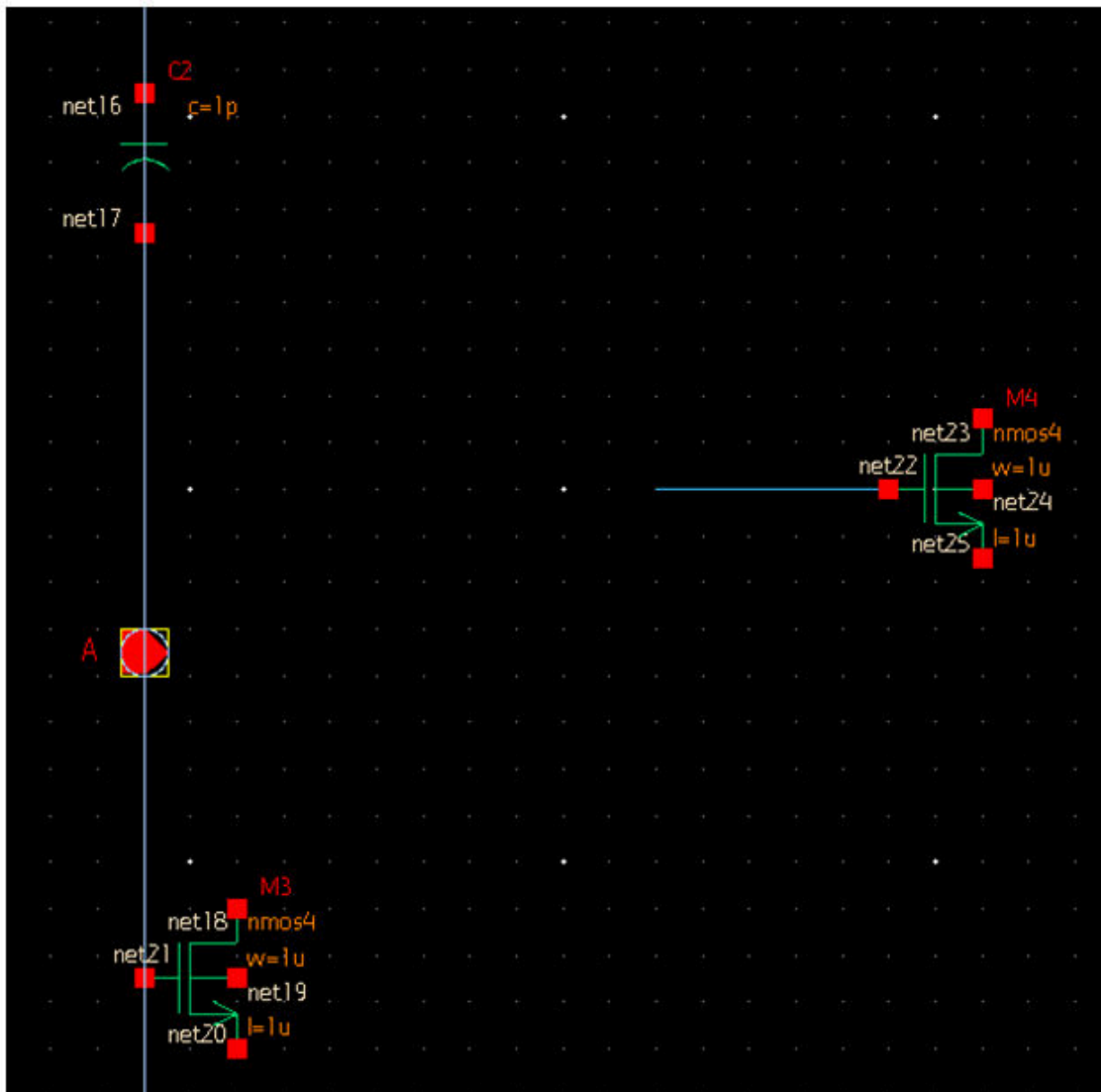
The command remains active. If you select additional objects they are also aligned with the source.

To set a new source for alignment you must click **Restart** in the toolbar and repeat the above steps to select source and target object.

5. Press Esc to end the command.

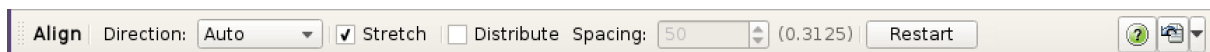
The following figures show the position of objects before and after alignment.





Align Toolbar Options

The toolbar contains the following options:



Option	Description
Direction	Specifies the orientation of the alignment line that is displayed in the canvas. The options are: Auto (default), Vertical, and Horizontal. When set to Auto, the orientation of the alignment line depends on how target objects are selected and the relative positions of the target and source objects. There are two cases: • <i>Targets selected one at a time</i>: In this case, a bounding box comprising the first selected target and the source objects is drawn. If the width is less than or equal to the height, the alignment line is vertical, otherwise it is horizontal. • <i>Multiple targets selected at once</i>: In this case, a bounding box comprising all the selected target and source objects is drawn. If the width is less than or equal to the height, the alignment line is vertical, otherwise it is horizontal. Note: If you toggle between Auto and another direction setting during the same command session, but without committing, the alignment direction is determined by the targets defined after the last toggle to Auto.
Stretch	When enabled (default), wires connected to a target object are stretched when the object moves, so as to maintain connectivity. When disabled, wires connected to a target object are disconnected when the object is moved, breaking connectivity.
Distribute	When enabled, distributes objects by the specified spacing to equal distance from the reference object and along the alignment line. When enabled, the Spacing text field is also enabled.
Restart	Cancels the current alignment operation and starts a new one.

Copying Objects

For information on using the `ise::copy` and `de::copy` Tcl commands, see Schematic Editing Tcl and Design Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To copy an object:

1. Select an object on the canvas.

The editor highlights the object.

2. Choose **Edit > Copy**.

The **Copy** toolbar appears. (To open the **Copy** dialog box, use the F3 shortcut key after initiating the interactive command.)

3. Specify the **Copy** options (see [Copy Toolbar Options](#)).

4. Click to specify the reference point.
5. Move your cursor to the target location.

As you move the cursor, the copied object appears under the cursor.

6. Click to place the copied object.

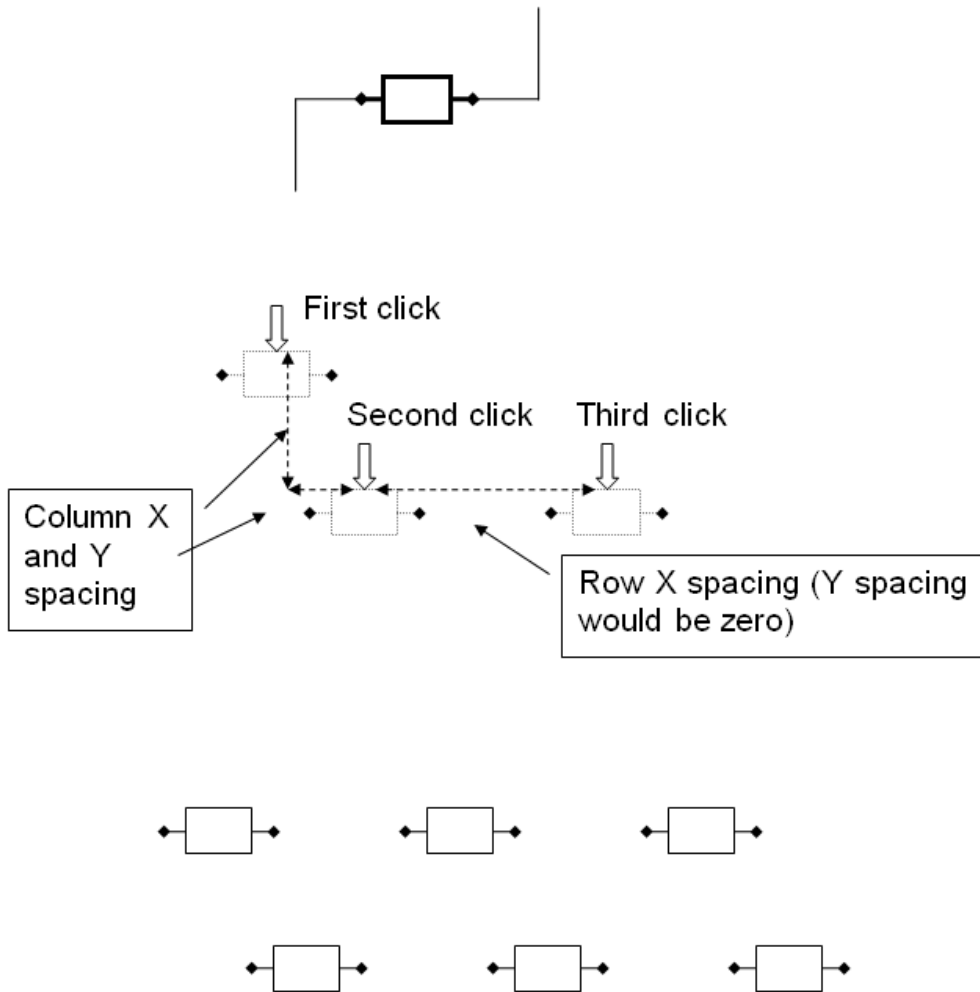
The target location can be in the current schematic design window or in another schematic design window. If you move the cursor over another design window, the command toolbar is activated in that window until you place the object.

- If you preselected the objects, the copied objects remain selected after placement and the **Copy** command completes.
- If you selected the objects during the command, the copied objects are deselected after placement and the command remains active in the origin window.

7. Press Esc to end the command.

Creating Multiple Copies

To create multiple copies, the value for either the **Rows** or **Column** option should be greater than 1. The following figure shows how to create multiple copies of the object when the values for Columns = 3 and Rows = 2.



Copy Toolbar Options

The toolbar contains the following options:

Option	Description
Angle	Controls how to copy the object. See Angle Mode for Creating and Editing Objects .
Transform Icons	These options let you rotate or flip the object while copying the object.
Origin	Specifies method for defining the point about which to rotate the object.
Rows	Specifies the number of rows for an array.

Option	Description
Columns	Specifies the number of columns for an array.
Chop	Chops wires under the instance after the instance is pasted (instance pins are connected).
Keep Nets	These options control how net names are propagated. • Labeled: Copy all attached wire labels of selected wire segments. • All: Create labels for unlabeled nets so the copied objects have the exact same connectivity as the original objects. • None: Do not copy wire labels (even if they are selected). This option does not affect selected segments labeled as global.

Copy Menu

The menu contains these options:

Option	Description	Default Shortcut Key
Rotate 90	Rotates the object 90 degrees counter-clockwise.	R
Flip Horizontal	Flips the object around the Y axis.	Shift+R
Flip Vertical	Flips the object around the X axis.	Ctrl+R

Using the Clipboard

The clipboard feature lets you to copy, cut, and paste blocks of data in a consistent way within a design or between designs.

The clipboard is a container of database objects which serves as the paste buffer for the Custom Compiler editors. There is one clipboard per Custom Compiler session. On session startup, the clipboard is empty. The contents of the clipboard cannot be edited or modified except by replacement with new contents.

For information on using the `ise::paste`, `de::copy`, and `de::paste` Tcl commands, see Design Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

The following sections describe how to use the clipboard:

- [Cutting Objects to the Clipboard](#)
- [Copying Objects to the Clipboard](#)
- [Pasting Objects From the Clipboard](#)

Cutting Objects to the Clipboard

The **Cut** command lets you move selected objects to the clipboard for pasting into either the same cellview or a different cellview.

Note:

When you cut a selected object, the entire logical object is deleted, not just the associated geometry.

To cut an object to the clipboard and paste the object elsewhere:

1. Select the object on the canvas.

The editor highlights the object.

2. Choose **Edit > Cut**.

The object disappears from the canvas and is copied to the clipboard.

3. Select **Edit > Paste** on the target canvas.

The deleted object follows the cursor.

4. Click the target canvas to place the object.

The operation completes.

Copying Objects to the Clipboard

The **Copy to Clipboard** command lets you copy an object to the clipboard for pasting it into the same cellview or a different cellview.

When you copy an object to the clipboard, the entire logical object is copied to the clipboard, not just the associated geometry.

To copy an object to the clipboard:

1. Select an object on the canvas.

The editor highlights the object.

2. Choose **Edit > Copy to Clipboard** or use the Ctrl+C shortcut key.

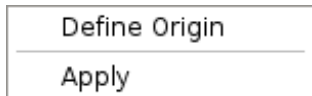
The **Copy to Clipboard** toolbar appears.



3. (Optional) Click **Define Origin** and click in the design to specify a reference anchor point.

When you later paste the clipboard contents into a design, the tool places the anchor point at the redefined location.

4. (Optional) Right-click to show the Cut to Clipboard menu, as illustrated below.



5. Click **Apply** (the green check mark) on the toolbar.

The tool moves the selected objects to the clipboard and then deletes the objects from the original design.

If nothing is selected, the clipboard remains unchanged.

Pasting Objects From the Clipboard

The **Paste** command lets you paste clipboard contents into any valid target cellview (source schematic, symbol, other schematics, or layout designs).

Note:

Schematic instances appear as layout instances when you paste them in the layout design.

To paste an object:

1. Choose **Edit > Paste**.

The **Paste** toolbar options appear.

2. Specify the **Paste** options (see [Paste Toolbar Options](#)).
3. Move your cursor over the canvas.

The contents of the clipboard follow the mouse cursor.

4. Click a location on the canvas to place the objects.

The command completes.

Paste Toolbar Options

The toolbar contains the following options:

Option	Description
Transform Icons	These options let you rotate or flip the object during a paste operation.
Rows	Specifies the number of rows for an array.
Columns	Specifies the number of columns for an array.

Deleting Objects

The **Delete** command lets you selectively delete objects from a design. The command operates only on fully selected objects and ignores any partially selected objects.

For information on using the `se::delete` and `ise::delete` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To delete an object:

1. Select the object in the canvas.

The editor highlights the object.

2. Choose **Edit > Delete**.

The object disappears from the canvas and the command completes.

The following rules apply to object deletion:

- Wires are deleted by wire segment. Connectivity recomputation, including the deletion or addition of solder dots, is performed automatically.
- Deleting instances does not delete any connected wires.
- Partially selected objects are not affected by the deletion operation.

The deletion operation is bound to the Delete key by default.

Editing Canvas Text

The **Edit Canvas Text** command lets you edit net or wire names directly within the schematic canvas by double-clicking on them.

The canvas editing triggers the `ide::editCanvasText` Tcl command, described under Design Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To edit a wire name:

1. (Optional) Double-click the wire name and start editing.
2. Select the wire name in the canvas.

The editor highlights the wire name.

3. Right-click and choose **Edit** to start editing the name.
4. Press Enter to commit the editing action.

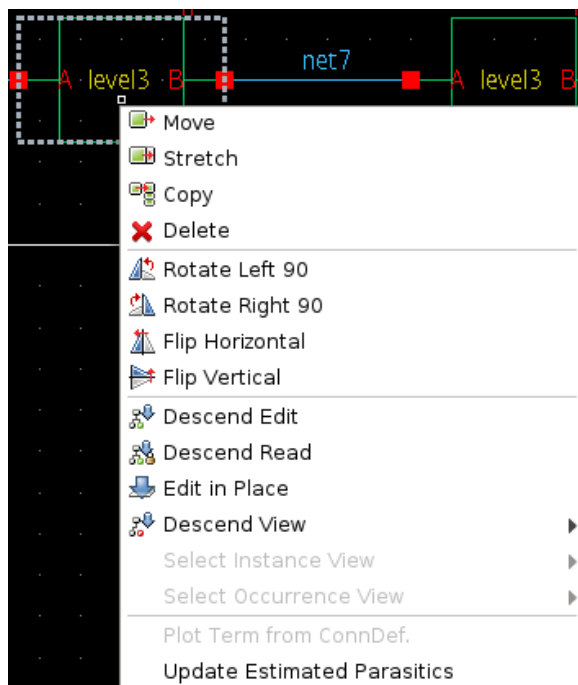
Editing a Cellview in Place

You can edit a cellview in the context of the view in which it is instantiated. For example, it is often desirable to edit a symbol directly in the context of the schematic in which it is placed. This aids in pin alignment as well as placement of labels for property annotation.

For information on using the `ide::descend` Tcl command, see Design Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To edit a cellview in place:

1. Select the instance for the symbol you want to edit in place.
2. Right-click and choose **Edit in Place**.



Alternatively, choose **Design > Hierarchy > Edit In Place**. For more details, see Navigating the Design Hierarchy in the *Custom Compiler Environment User Guide*.

Note:

The **Design > Hierarchy > Edit in Place** command is hidden if the design library is read-only or if you are in a read-only session.

(Optional) When descending to an instance by double-clicking, **Descend Edit**, or **Descend Read**, if the instance is bound to an external net list file, setting the `seDescendUseExternalEditor` Tcl preference to *true* opens the external netlist file by an external text editor, otherwise opens the external netlist file by the Text Viewer. However, if the netlist file is opened in another new window, no change for the current design context.

3. When you have finished editing the symbol in place, right-click and choose **Edit in Place**. Alternatively, choose **Design > Hierarchy > Return** to return to the schematic.

Rotating and Flipping Objects

The **Rotate** command lets you rotate objects in 90-degree increments or flip objects around the X and Y axis.

Note:

Connections are not retained when you rotate or flip objects. Use the **Stretch** command to rotate objects and retain connections (see [Stretching Objects](#)).

To rotate an object:

1. Select an object in the canvas.

The editor highlights the object.

2. Choose **Edit > Rotate > <option>**

The options are **Rotate Left 90**, **Rotate Right 90**, **Flip Horizontal**, and **Flip Vertical**.

3. The object rotates or flips and the operation completes.

Connecting and Disconnecting Crossing Wires

While creating or editing schematic designs, you can connect or disconnect orthogonally crossing wire segments.

For information on using the `se::connectWires`, `se::disconnectWires`, `ise::connectWires`, and `ise::disconnectWires` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

These sections describe the procedures for:

- [Connecting Crossing Wires](#)
- [Disconnecting Crossing Wires](#)

Connecting Crossing Wires

Use the **Connect Wires** command to connect two orthogonal wires that cross.

To connect orthogonally crossing wires:

1. Choose **Edit > Connect Wires**.
2. Click the crossing point of the two orthogonal wires that you want to connect.

The two orthogonally overlapping wires are split into two segments each, and a solder dot is created at the point where the four endpoints meet, forming a connection.

The command remains active.

3. Press Esc to end the command.

Note:

The **Schematic Design Options** dialog box lets you control the size of automatically created solder dots. See [Setting Design Options](#).

Disconnecting Crossing Wires

Use the **Disconnect Wires** command to disconnect four wire segments joined by a solder dot.

Note:

You cannot remove a solder dot from a T-shape connection.

To disconnect orthogonally connected wires:

1. Choose **Edit > Disconnect Wires**.
2. Select the solder dot joining the four orthogonal wires.

The function removes the solder dot and the four wire segments revert to two collinear segments.

The command remains active.

3. Press Esc to end the command.

Chopping Wires

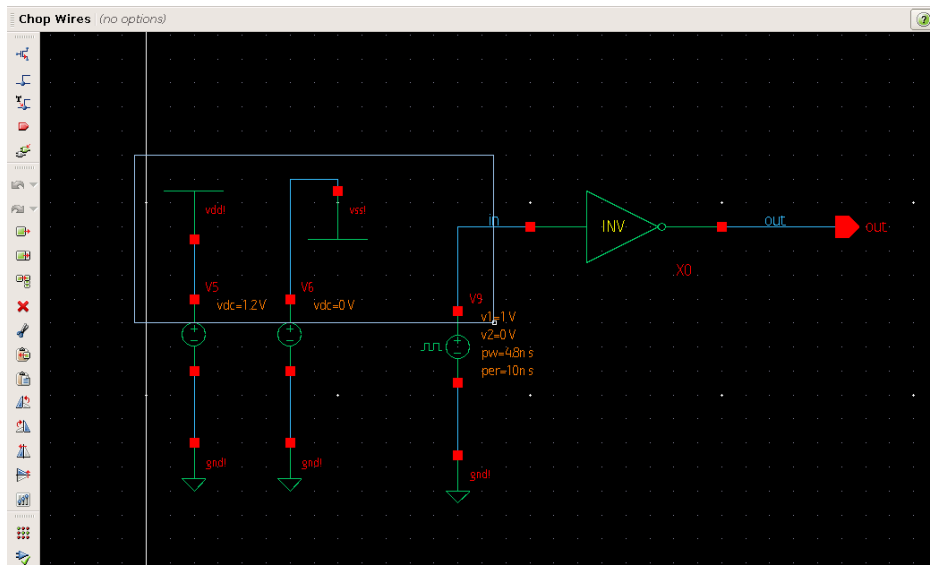
The **Chop Wires** command lets you delete all wiring objects in an area you specify from the design.

For information on using the `se::chopWires` and `ise::chopWires` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

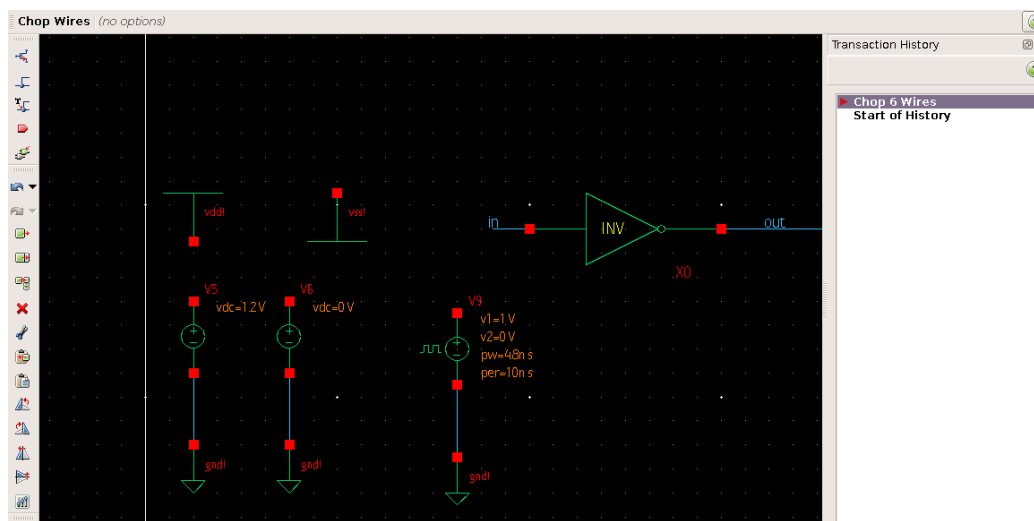
To chop wires:

1. Choose **Edit > Chop Wires**.
2. Click a point in the canvas and drag to select the area in which you want to chop wiring objects. Click a second point to end the selection and chop the wiring objects. The following figures show a design before and after chopping wires.

Note that the Transaction History (available by choosing **Window > Assistants > Transaction History**) indicates the number and type of wiring objects that were chopped.



Before **Chop Wires**



After **Chop Wires**

The following rules apply when chopping wires:

- Wire segments located completely in the chopping area (including area borders) are deleted. The net name labels attached to these segments are also deleted too (as they follow the group).
- Net name labels located completely in the chopping area are deleted, even if their parent segments are not affected.

- c. Wire segments located partially in the chopping area are cut at the point of intersection with the area border; the part in area is deleted, and the parts (one or two) out of area remain.
- d. Net name labels located partially in the chopping area are not deleted (if they are not covered by rule a).
- e. After wire segment partition (rule c), if the initial segment has attached wire segments that are not deleted, the remaining part of the segment inherits undeleted segments. If there are two remaining parts, the segment is attached to the geometrically closest one.
- f. All net renaming actions caused by the aforementioned object deletions are made as expected, similar to how renaming occurs after any other object deletion.

The command remains active.

3. Press Esc to end the command.

Colorizing Nets

The **Colorize Nets** command lets you colorize nets in a schematic design to aid visualization. This color annotation allows you to see and find nets of specific groups quickly. Net groups define colors and line styles and are configured according to color definitions set using the Net Colorization Definitions Editor.

See [Net Colorization Definitions Editor](#) for details on how to set net color definitions.

For information on using the `se::colorizeNets` and `ise::colorizeNets` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To colorize wires:

1. Choose **Edit > Colorize Nets**.

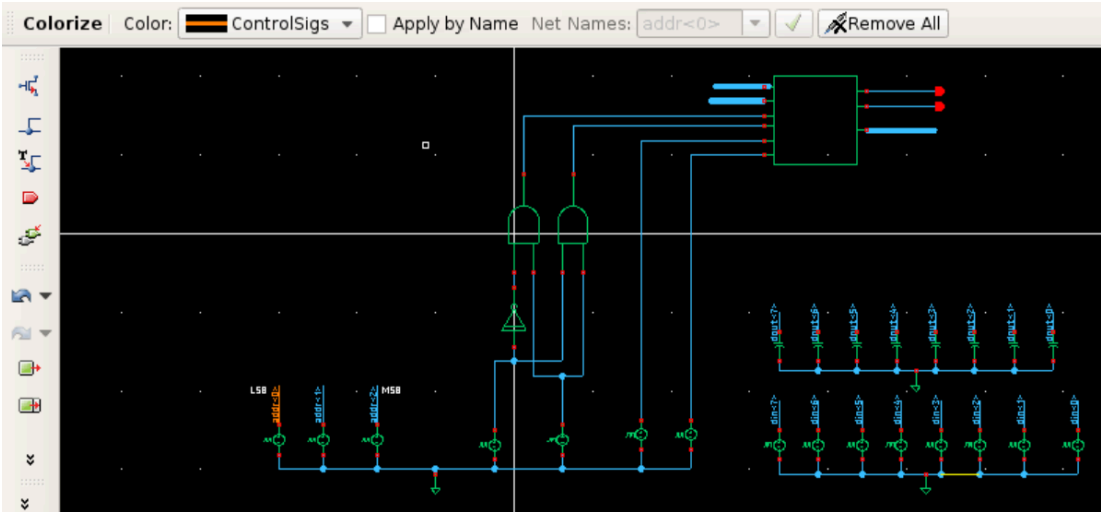
The **Colorize** toolbar appears.

2. Specify the **Colorize** options (see [Colorize Toolbar Options](#)).
3. Choose **Apply by Name** and select or type net names from the drop-down menu, then click  to apply the colorization to the selected nets.

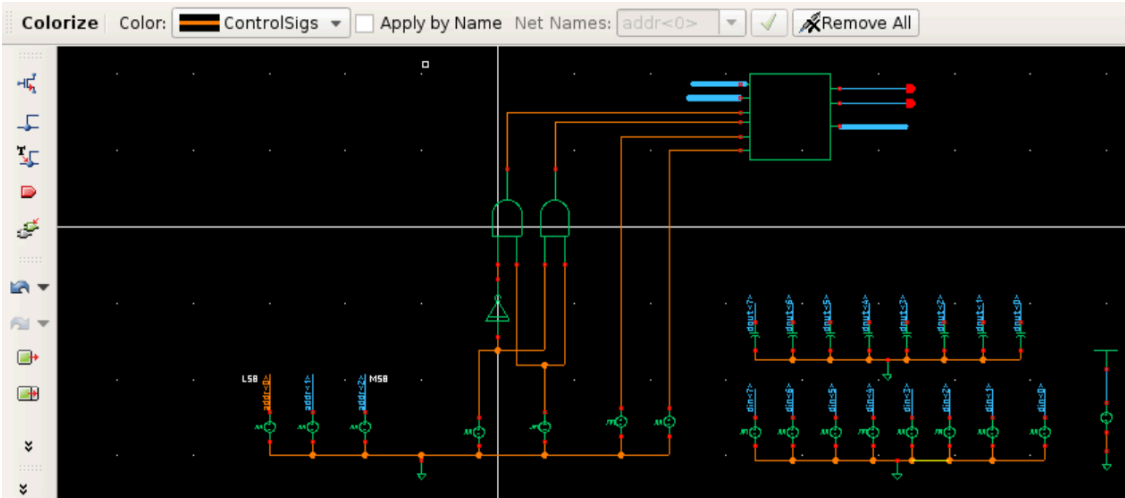
Alternatively, select individual nets or click and drag on the canvas to select a net on which to apply the net colorization.

4. Press Esc to end the command.

The following figures show a design before and after colorizing nets.



Before colorization



After colorization

Colorize Toolbar Options

The toolbar contains the following options:

Option	Description
Color	Predefined color definitions appear in the drop-down menu. These are set in the Net Colorization Definitions Editor. For instructions, see Net Colorization Definitions Editor .

Option	Description
Apply by Name	When selected, allows you to choose specific Net Names to colorize. Click  to apply the colorization to the selected nets.
Remove All	Removes all colorization from the schematic design.

Selecting Objects and Areas

The Schematic Editor supports the following object selection options:

- [Selecting Single Objects](#)
- [Selecting Multiple Objects](#)
- [Selecting All Objects](#)
- [Enabling Partial Area Selection](#)
- [Filtering Selections](#)
- [Enabling Undo and Redo Selection](#)

Selecting Single Objects

You can select single objects regardless whether they are unobstructed, appear to be grouped too close to other objects, or overlap with other objects.

To select a single unobstructed object:

- Move your cursor over an unselected object and click.

The editor highlights the object and deselects any previously selected objects.

To select a single object that appear to be grouped among other objects:

1. Move your cursor over the object grouping and click.

The tool finds and highlights an object.

2. If the highlighted object is not the one you want, continue clicking until the editor highlights the target object.

The software cycles through each object in the grouping with each mouse click. Cycle selection and highlighting lets you identify and select a single object in situations where several objects are grouped too close together to easily select just one.

Note:

By default, your left mouse button is bound to cycle selection, as well as the Ctrl+1 and Ctrl+2 keys.

Selecting Multiple Objects

You can select multiple objects using the following selection functionality.

- [Selecting Objects By Region](#)
- [Selecting Objects By Name](#)
- [Adding and Removing Selected Objects](#)

Selecting Objects By Region

You can select single or multiple objects by specifying regions on the canvas. All the objects within the specified regions are selected.

To select objects by region:

1. Choose **Edit > Select > By Region** or press Shift+A.

The **Select by Region** toolbar appears.

Option	Description
Select by Region	◦ Select: Selects the objects within the selection region ◦ Deselect: Deselects the objects within the selection region
Region	Specifies the selection region.
Angle	Activated when you choose Polyline or Polygon as the region. Specifies how to draw the selection region.
Criteria	Specifies whether objects must be fully enclosed by the selected region.

2. Specify the options.

3. Use one of the following methods to define the region:

- Use your cursor to specify an object or to specify the points of a polyline.
- Click and drag the cursor to draw a rectangle or polygon.

The objects within the region are selected and the command remains active.

4. (Optional) Shift+click to add more objects to the selection set (see also, [Adding Objects to the Selected Set](#)). Use Ctrl+click to deselect objects.

5. Press Esc to end the command.

The objects remain selected.

Selecting Objects By Name

You can select objects based on object type whose name matches a specific string or regular expression.

To select objects by name:

1. Choose **Edit > Select > By Name**.

The **Select by Name** toolbar appears.

Option	Description
Type	Specifies the object type.
Name	Specifies the object name.
Regex	Specifies whether the object name is a regular expression.

2. Specify the options.

3. Click **Apply**.

The named objects are selected and the command remains active.

4. (Optional) Specify another object type or name, and click **Apply** to add objects to the selection set (see also, [Adding Objects to the Selected Set](#)).

5. Press Esc to end the command.

The objects remain selected.

Adding and Removing Selected Objects

The following sections describe how to add and remove selected objects.

- [Adding Objects to the Selected Set](#)
- [Removing Objects From the Selected Set](#)

Adding Objects to the Selected Set

To add a single object to an already selected object set:

- ▶ While holding down the Shift key, select the object.

The object is immediately added to the selection set.

Removing Objects From the Selected Set

To remove a single object from an already selected object set:

- ▶ While holding down the Ctrl key, select the object.

The object is immediately removed from the selection set.

Selecting All Objects

The **Select All** command lets you select all objects not currently excluded by the selection filter.

To select all objects not currently excluded by the selection filter:

- ▶ Choose **Edit > Select > Select All**.

The editor highlights all the selected objects.

For information on using the `de::selectAll` Tcl command, see Design Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

Enabling Partial Area Selection

Certain objects, such as wires and shapes, work with the concept of partial area selection. Partial area selection is a mode that you must explicitly enable as the option is disabled by default.

In Partial Object Selection mode, you can area select the vertices, edges, or centerline of an object, as well as the entire object.

The objects to which you can apply area partial selection are:

- Wire segments

You can partially area select wires at their endpoints, and move the endpoint to a new location. This is the process of extending/contracting the wire so that it is longer/shorter than its original length.

- Shapes
- Pin shapes in the Symbol Editor

Note:

You cannot delete partially selected objects.

To enable partial area selection:

1. Choose **Options > Design**.

The **Schematic Design Options** dialog box appears.

2. In the **Selection** region of the dialog box, choose the **Partial** option.
3. (Optional) Change the **Radius** setting.
4. Click **OK**.

You can also directly enable the partial selection mode by clicking on the **Selection** mode button on the Schematic Editor toolbar. By default the selection mode is set to **full**; clicking on the **Selection** mode button sets the mode to **partial**.

You can visibly distinguish selected objects from unselected objects: selected objects and their selection boxes appear with the select/drawing layer purpose pair. As with all other layer purpose pairs (LPPs), you can configure this layer's display attributes.

In addition, the selection status bar only displays the current number of objects that are wholly or partially selected (a partially selected object is counted as a whole object). An instance, a wire segment, a label, a schematic pin, and a wire endpoint each count as a single selected object.

Filtering Selections

The Schematic Object Filter and the Symbol Object Filter are selection filters that let you filter objects for selection based on the specific object type.

Use the following commands to access the object filters:

- In the Schematic Editor, choose **Window > Assistants > Schematic Object Filter**.
- In the Symbol Editor, choose **Window > Assistants > Symbol Object Filter**.

For more information about working with window assistants, see the *Custom Compiler Environment User Guide*.

Enabling Undo and Redo Selection

You can undo and redo object selection using the **Undo Selection** and **Redo Selection** buttons on the **Selection** toolbar. See [Selection Toolbar](#).

Renumbering Instances

If you add or delete many instances during schematic design, your final design could contain instances with non-sequential or dissimilar names. In such cases, you can use the **Renumber Instances** command to sequentially renumber all, or selected, instances.

For information on using the `de::renumberInstances` and `de::showRenumberInstances` Tcl commands, see Design Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

The Schematic Editor supports two renumbering options:

- Row-based renumbering
Renumbers selected instances in a left-to-right and top-to-bottom pattern.
- Column-based renumbering
Renumbers selected instances in a top-to-bottom and left-to-right pattern.

Note:

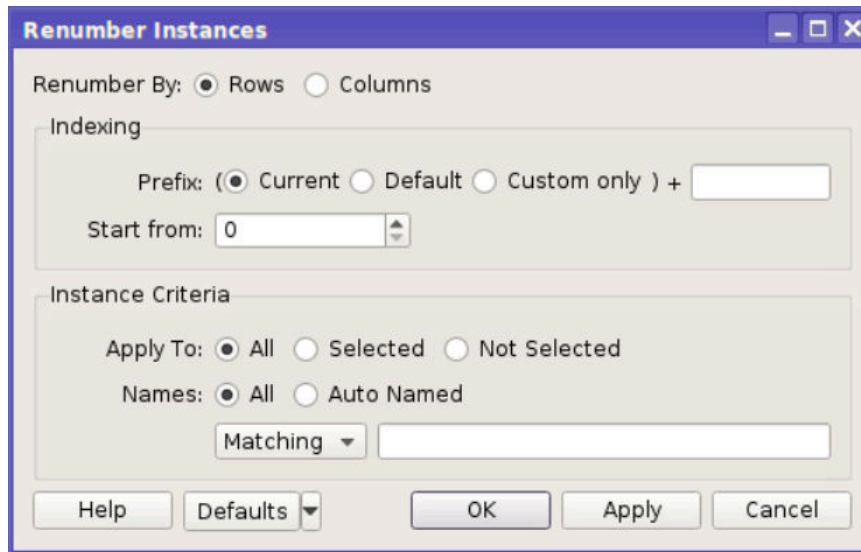
Instance renumbering applies only to the current cellview, not across the schematic hierarchy. Also, renumbering applies to both auto- and user-named instances.

Additionally, user-named instances must have trailing numbers so that they can be included in the renumbering operation. An instance with a name like "MyInst", with no trailing numbers, is not included.

To renumber instances:

1. Choose **Edit > Renumber Instances**.

The **Renumber Instances** dialog box appears.



The dialog box contains the following options:

Option	Description
Renumber By	Specifies the basis for the renumber process. ◦ Rows: Renumbers the selected instances in a left-to-right and then top-to-bottom pattern. ◦ Columns: Renumbers the selected instances in a top-to-bottom and then left-to-right pattern.
Indexing	
Prefix	The number prefix to use when indexing instances. ◦ Current: Uses the current prefix. (This is the default.) If you specify a prefix string in the text box, this string is appended to the current prefix. ◦ Default: Uses the default prefix. The default prefix is retrieved based on the instance's default naming rules (for example, using the <code>instNamePrefix</code> setting for the instance master, and so forth). If you specify a prefix string in the text box, this string is appended to the default prefix. ◦ Custom only: Uses only the custom prefix string you specify in the text box.
Start from	Specifies the start number to use for indexing. The default is 0.
Instance Criteria	
Apply to	Restricts processing based on selection. ◦ All: Processes all instances in the cellview. ◦ Selected: Processes only the instances in the select set ◦ Not Selected: Processes only those instances not in the select set.

Option	Description
Names	Restricts processing depending on how the instances were named. ◦ All: Processes all instances in the cellview. ◦ Auto Named: Processes only those instances whose names were generated automatically. Enter a string in the text field to further restrict processing and select from the following string matching options: ◦ Matching: Processes instances matching the string. ◦ Excluding: Ignores instances matching the string.

2. (Optional) Specify the options.

3. (Optional) Click **Apply**.

The operation completes but the dialog box remains open for further operations.

4. Click **OK**.

The operation completes and the dialog box closes.

Renaming Instances

You can use the **Rename Instance** command to rename instances.

For information on using the `ise::renameInst` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To rename instances:

1. Choose **Edit > Rename Instance**.

The **Rename Instance** toolbar appears.

The toolbar contains the following options:

Option	Description
Names	Specify one or more instance names. See also Naming Instances .
Expand	If the Names field contains a non-scalar name (for example, vectors like <code>A<0:10></code> or bundles like <code>"X,Y"</code>), the names should be split into bits before using them.

Option	Description
Scalarize	Makes the vector bit names scalar by removing the brackets. For example, if you have the name "X<10:20>" in Names field and you have both Expand and Scalarize enabled, clicking two instances renames them to X10 and X11. If only Expand is enabled, the instances would be renamed X<10> and X<11>.

2. Specify the options.

The specified instance name attaches to your cursor.

3. Hover over an instance on the canvas and click to rename it.

The name of the instance changes and the name attached to the cursor updates to the next name in the series.

4. Repeat [Step 3](#) as needed.
5. Press Esc to end the command.

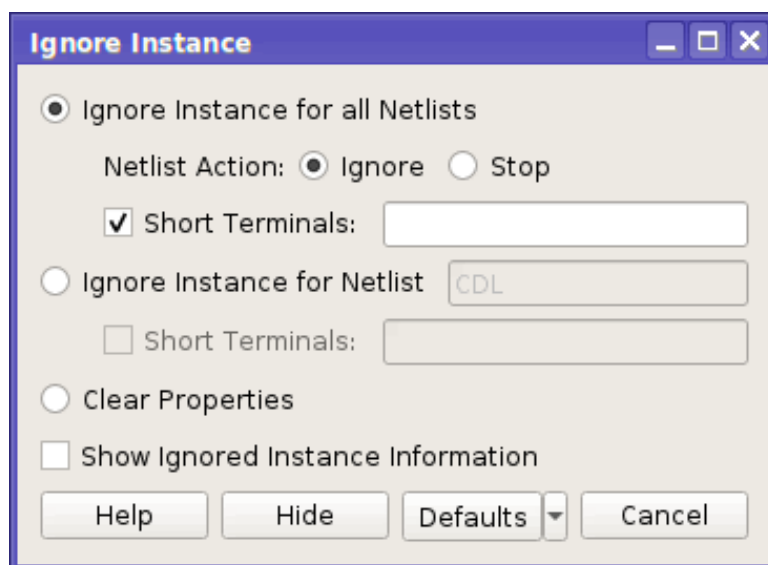
Ignoring Instances

You can use the **Ignore Instance** command to ignore instances.

To ignore instances:

1. Choose **Edit > Ignore Instance** or use the shortcut key Shift+Delete.

The **Ignore Instance** dialog box appears.



2. Specify one of the following options to ignore the instances.

- **Ignore Instance for all Netlists:** When enabled, ignore the instance for all types of netlists. The `nlAction` property of the instance is set to ignore.
 - **Netlist Action: Ignore-** When selected, removes the property for all netlist
 - **Netlist Action: Stop-** When selected, stops the property for all netlist.

The `nlIgnore` property of the instance is set to all netlists.

When the **Short Terminals** option is selected, specify the terminals to be shorted in the text field.

- **Ignore Instance for Netlist:** When enabled, ignore the instance for the specified netlists. The valid netlists are: CDL, Eldo, HSPICE, NanoSim, Spectre, VHDL, VerilogICC, and Verilog.

When the **Short Terminals** is selected, specify the terminals to be shorted in the text field.

The `nlIgnore` property of the instance is set to specified netlists.

3. Select the instance or area select the instances to be ignored.

4. (Optional) Enable the **Clear Properties** option to clear all ignored property created by the **Ignore Instance** command.

5. (Optional) Enable the **Show Ignored Instance Information** option to show persistent highlightings after the command ended.

You can use the `seShowIgnoreInstInfo` Tcl preference to set this option or use the shortcut key Ctrl-Shift+Delete to toggle on/off the option.

6. Press Esc to end the command.

Ordering and Synchronizing Ports

The **Edit > Port Order Edit/Synchronize** command is available in the schematic, symbol, and text editors. The command opens a window that lets you manage and synchronize port ordering with other views.

Port ordering sets the value of the `portOrder` property, which the various netlisters reference when writing out instance statements. Therefore, the port order must be consistent (that is, synchronized) among views to avoid connectivity mistakes when netlisting.

Ordering and synchronizing ports is described in the following sections:

- [Ordering Ports](#)
- [Synchronizing Port Order Between Views](#)

Ordering Ports

Note:

Additional information on port ordering is provided in the *Custom Compiler Tcl Reference Manual* > Netlist Tool Tcl > Netlister Preference Examples.

On starting the command, the tool opens the **Edit/Synchronize Port Order** dialog box and looks for a portOrder property for the design. When the property exists, the tool reads in the terminal names specified by the portOrder property and displays those names in the **Port Order** list box. When the property does not exist, the tool lists the ports alphabetically in the **Unused Ports** list box.

Note:

Names that appear in the portOrder property but which do not exist as ports in the design are displayed in red.

The tool displays in the **Unused Ports** list box any available terminal names that do not appear in the portOrder property.

Once opened, you can perform the following operations to manage the ports:

- [Sorting Individual Port Names](#)
- [Sorting Multiple Port Name Collections](#)
- [Sorting All Port Names](#)
- [Expanding and Creating Bus Names](#)
- [Adding Port Names](#)
- [Editing Port Names](#)
- [Moving Port Names Between Lists](#)

Sorting Individual Port Names

Sort the ports in the **Port Order** list box in one of the following ways:

- Drag and drop port names within the list box.
- Select a port name and right-click to open a menu of ordering commands.

- Select a port name and click the arrow buttons below the list box to move the port up or down in the list.

You can perform the above operations on individual port names or on groups of port names (see [Sorting Multiple Port Name Collections](#)).

Sorting Multiple Port Name Collections

You can select multiple port names and edit their ordering, or move them as a group within the **Port Order** list or between the **Port Order** list and the **Unused Ports** list.

To sort multiple port names:

1. Click to select a port name.
2. Use Ctrl-click to select additional names.

When you right-click on the selection, a menu lets you sort the collection as well as move the collection within the list. If you move a collection within the **Port Order** list, the selection ordering is maintained.

Sorting All Port Names

To sort all port names at once:

- Right-click on the **Port Order** title bar to open a menu that let you sort the entire list of port names in ascending or descending order.

Expanding and Creating Bus Names

Manage bus names in one of the following ways:

- Right-click on an existing bus name and select the **Scalarize** command from the menu to list the individual bit names.
- Right-click on a group of valid individual bit names and select the **Group bits** command from the menu to create a bus name.

Adding Port Names

You can add individual port names, buses, and bundles. On commit, bundled names are listed individually. You can also enter duplicate port names as well as port names that do not exist yet in the design.

Add port names to the **Port Order** list in one of the following ways:

- Double-click on the **click to add** string to add a port name at the end of the list.

The string becomes a text entry field where you can enter a name.

If you enter a port name that currently appears in the **Unused Ports** list, the name disappears from the **Unused Ports** list on commit.

Illegal names are ignored.

- Right-click on a port name and select **Insert New** from the menu to add a port name above the highlighted name.

Note:

You can create an empty row by not entering any text, but the tool removes the row from the list when you click **Apply** or **OK**. Similarly, if you try to drag an empty row to the **Unused Ports** list, the row disappears when you drop it.

Editing Port Names

To edit a port name:

- Double-click an existing port name. The name becomes editable. Enter a new name.

If you enter the name of an unused port, the name disappears from the **Unused Ports** list.

If you enter the name of a port that does not exist in the design, the name appears in red.

Moving Port Names Between Lists

You can move port names from the **Port Order** list to the **Unused Ports** list and vice versa.

To move port names between lists:

1. In a source list, select the port name or a group of names.
2. Drag the name or the group to the target list.

When you drag a group to the **Port Order** list and position your cursor between two port names, a horizontal line indicates where the group will be placed. When you drop the group, the current group ordering is maintained.

When you drag a valid group to the **Unused Ports** list, the ordering is ignored because the **Unused Ports** list is sorted alphabetically.

Note:

If you drag a port name from the Port Order list to the **Unused Ports** list, but the port does not exist in the design, it is automatically moved to the end of the **Unused Ports** list and sorted separately. Similarly, if you move buses with members that do not exist in the design to the **Unused Ports** list, the

members that do not exist in the design are added to the end of the list and sorted separately.

Synchronizing Port Order Between Views

The tool lets you synchronize port ordering between the different views of a cell.

The **Copy To** region of the dialog box lists all the views of the cell, except any text views. Text views are excluded from the destination list because port ordering for text views must remain consistent with the text itself (that is, with the Verilog module statement, the SPICE subcircuit statement, and so forth). However, a text view can be used as the source view from which the port order is copied to other views.

To synchronize a source view with destination views:

1. In the **Copy To** region of the window, select the view to which you want to synchronize.
2. Click **Apply** or **OK**.

The result depends on whether a target view is open or closed and whether an open view is in edit or read-only mode, as described below:

- Closed target view

The tool opens the view, applies and saves the changes, and then closes the view again.

- Target view opened in edit mode

The tool applies the changes, but does not perform a save operation. The save operation must be performed interactively during the session or when prompted on closing the view.

- Target view opened in read-only mode

The tool changes the view mode to edit, applies and saves the changes, and then sets the mode back to read-only.

11

Editing Parameters

This chapter describes the features and procedures you can use to edit object properties, parameters, and attributes.

Note:

For information about using the Property Editor Assistant, see the *Custom Compiler Environment User Guide*.

This chapter covers the following topics:

- [On-Canvas Parameter Editing](#)
- [Filtering Parameters](#)
- [Editing Cellview Properties](#)
- [Editing Instance Attributes and Properties](#)
- [Editing Wire Properties](#)
- [Editing Pin Attributes](#)
- [Editing Text Attributes](#)

 [Show me](#) an example of how to select and edit multiple schematic instances in the Property Editor.

On-Canvas Parameter Editing

You can directly edit a parameter value visible in the schematic. If the parameter does not use some type of enumeration (such as a toggle button or cyclic list) the parameter is available directly in the schematic.

To activate the visible parameter in the schematic:

1. Move your cursor over the parameter.
2. Press Enter.

The parameter value can now be edited directly on the canvas.

Filtering Parameters

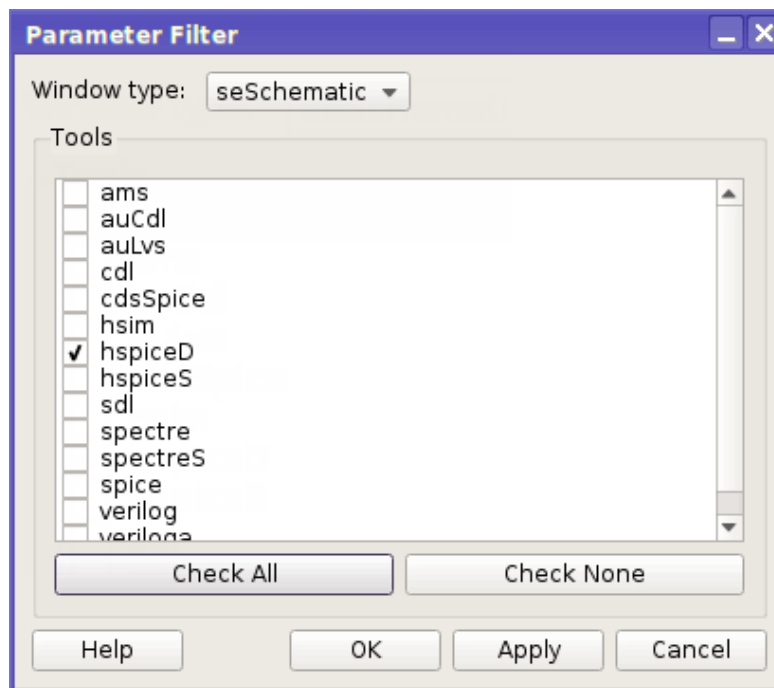
The Schematic Editor provides a parameter filter that lets you filter parameters relevant to a given application. For example, you want to view parameters in the **Add Instance** and **Property Editor** dialog boxes that are relevant only to the certain simulators that you use.

To filter parameters:

1. Choose **Edit > Properties > Parameter Filter**.

Alternatively, in the Property Editor, select **Parameter Filter** from the toolbar menu .

The **Parameter Filter** dialog box appears.



2. Use the check boxes to enable or disable the tools.

Editing Cellview Properties

You can access properties associated with a cellview. Such properties may include the time at which the cellview was last changed and the default prefix used for naming instances of the cellview. Some properties are hidden from being edited in the Property Editor, but are editable from the command line.

For information about using the Property Editor Assistant, see the *Custom Compiler Environment User Guide*.

Editing Instance Attributes and Properties

You can access information associated with an instance, including the instance name, the transform (orientation), and properties defined locally for the instance. You can also override default values for properties inherited from the cell master of the instance. For information about using the Property Editor Assistant, see the *Custom Compiler Environment User Guide*.

As with cellview properties, you can configure these properties to be hidden or editable.

For instances with `cdsParam(N)` labels placed and a **paramLabelSet** value specified for a cell, when annotating them on the schematic canvas, there is a possibility to evaluate only those parameter values which are available in the current selected parameter filter (see in [Filtering Parameters](#)). By setting the Tcl preference `dbFilterParamLabelSet` to *true*, the tool filters the parameter names from cell's **paramLabelSet** prior to draw `cdsParam(1)` ... `cdsParam(N)` labels of an instance in the schematic canvas. The algorithm honors the current selected tool names and based on **instParameters**, **otherParameters**, and **propMapping** values, defined with Parameter Definition Editor Tool, of a particular **SimInfo** section.

Take as an example the *ppulse* cell of *analogLib* library has the following CDF data:

- `paramLabelSet` = "z0 num dc pw per"
- **SimInfo's** parameters (based on **instParameters**, **otherParameters**, and **propMapping**)
 - `spectre` = "num r vdc v1 v2 td tr tf pw per acm acp Fnpairs ..."
 - `hspiceD` = "num z0 power dc v1 v2 td tr tf pw per acm acp ..."
- The resulting parameter names list to annotate in `cdsParam(N)` slots are:
 - If filtered to "spectre" = "num pw per"
 - If filtered to "hspiceD" = "z0 num dc pw per"

Editing Wire Properties

You can change the visual attributes of a wire branch, including the color, thickness, and line style. The changes apply only to the selected wire segment, with an option to apply the changes to the physically connected net or entire net. Net name changes are only possible by changing the attached wire labels. Net name attributes of a wiring object are read-only.

Editing Pin Attributes

You can change the terminal type (direction) and terminal name by using the Property Editor with a pin selected.

For information about using the Property Editor Assistant, see the *Custom Compiler Environment User Guide*.

Editing Text Attributes

You can change the text associated with labels and notes by editing their properties. You can also adjust the color, height, font, transform, and text justification. The **Attach** option lets you specify a shape that is then grouped with the text in a parent/child relationship, where the text is the child.

12

Exploring the Design

This chapter describes the features and procedures you can use to explore your designs.

This chapter covers the following topics:

- [Searching for Objects](#)
- [Displaying Label Attachments](#)
- [Adding Probes](#)
- [Removing Probes](#)
- [Highlighting Signals From WaveView](#)

Searching for Objects

You can search for objects in a design or a hierarchy based on criteria that you specify in the **Search and Replace** dialog box. On completion, the search function returns a list of objects that match your criteria. Selecting an object from the search list opens the cellview that contains the object.

Note:

You can also optionally replace certain object properties in conjunction with a search.

For information on using the `db::showFindReplace` Tcl command, see Database Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To perform a search, do the following:

1. Choose **Edit > Search and Replace**.

The **Search and Replace** dialog box appears. See Search and Replace Tool.

2. Complete the dialog box options.

A list of objects matching the criteria is returned.

3. Select an object from the list of found objects.

Reviewing Search Results

A search returns a list of objects. Clicking an object opens the cellview containing that object.

Color-Coding Search Results

You can assign a color to the search criteria so that the returned objects are highlighted in that color. For objects in child cellviews, the highlight color is displayed through the hierarchy. For example, if a transistor within an instantiated block is returned as part of a search operation, the parent block is highlighted in the color assigned to the search criteria. You can then descend into the block to find the component highlighted in the same color.

Displaying Label Attachments

You can use the Schematic Editor to show and hide graphical representations of the relationships between various schematic objects, such as wire labels that are offset from their parent segments. In such cases, the graphical representation consists of polylines drawn between the label objects and their parent segments.

To toggle the display of polylines between labels and their parent objects:

- Choose **Check > View/Hide Label Attachments**.

Adding Probes

A probe is a logical design bookmark that points to a piece of the design's logical representation. A probe is unique to a specific instance path; that is, two placements of the same design each have a different set of probes.

For information on using the `de::createProbe` and `ide::createProbe` Tcl commands, see Design Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

This section includes the following topics:

- [Using the Add Probe Command](#)
- [Add Probe Dialog Box and Toolbar Options](#)
- [Add Probe Menu](#)

Using the Add Probe Command

To add a probe:

1. In the Schematic Editor, choose **Add > Probe** or **Design > Probe > Add**.

The **Probe** toolbar appears. See [Add Probe Dialog Box and Toolbar Options](#).

2. Complete the toolbar options.
3. Click the object on the canvas that you want to probe.

The probed object is highlighted.

When Tcl preference `deProbeNetShowFlightline` is true, shows flight lines for net probes.

When Tcl preference `deProbeNetShowPinFlightline` is true, shows pin-to-pin highlights for net probes.

The command remains active.

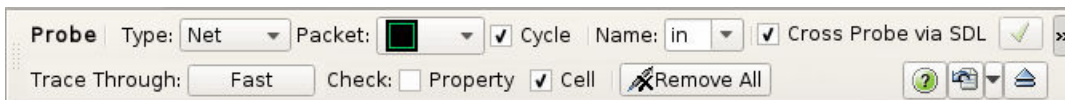
Note:

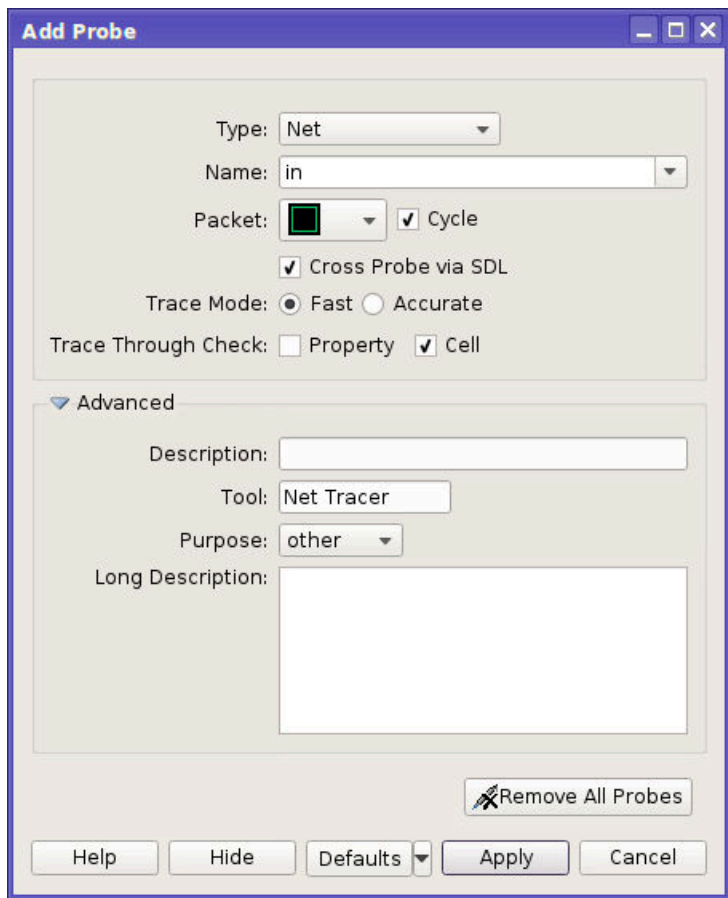
When selecting a net in the Design Navigator, the disjoint wire of the same net is connected with flight lines.

4. When you finished, press Esc to end the command.

Add Probe Dialog Box and Toolbar Options

When the **Design > Probe > Add** or **Add > Probe** command is active, pressing F3 or clicking the **Eject** button on the toolbar shows the **Add Probe** dialog box. This section provides a quick reference of the **Probe** toolbar and the **Add Probe** dialog box.





The following table describes the options available on the **Probe** toolbar and the **Add Probe** dialog box. For each option, an associated Tcl preference is available. For information on how to use Tcl preferences in scripts to automate routing operations, see *Preferences in the Custom Compiler Tcl Reference Manual*.

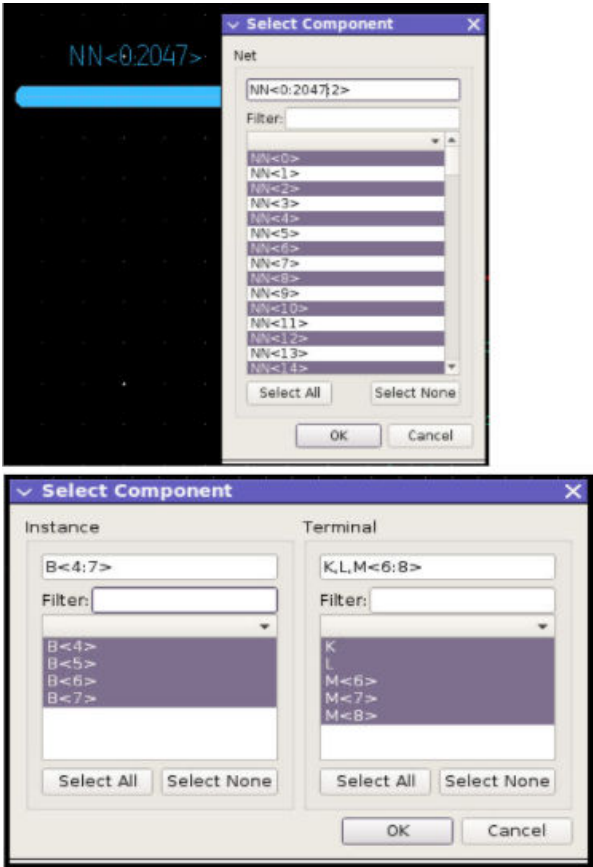
Option	Description
Type	• Any: Specifies that the probe type is dependent on the active object. Instance probes are created for instances, net probes for wires, terminal probes for port pin figures, and instance terminal probes for 1x port pin figures. • Terminal: Specifies a terminal probe, which can be a single element of a multi-bit terminal. • Instance: Specifies an instance probe, which can be one or more elements of a scalar or array instance. • Instance Terminal: Specifies an instance terminal probe, which can be one or more elements of a vectored instance terminal. • Net: Specifies a net probe for a net, bus, or bundle. Tcl preference: deProbeType

Option	Description
Packet	A display resources packet alias used to represent this probe. Note that some clients require the ability to create these dynamically. You use up to 30 packet colors for probing. The default color refers to highlight1 ~ highlight30. Tcl preference: <code>deProbePacket</code>
Cycle	When enabled, automatically displays the next packet in the active set after you place a probe. Tcl preference: <code>deProbeCycle</code>
Name	A probe's object name references an object in the target design. Because the object can be removed, the association is by name rather than object pointer. Interpretation of this element depends on the type of probe, as described below. • <i>For terminal probes</i>, the string represents a scalar or bit terminal name or a vector terminal name. The terminal name may not correspond to an actual terminal in the design but instead to one or more elements of a multi-bit terminal • <i>For instance probes</i>, the string represents a scalar or bit instance, vector instance, or array instance name. The instance name may not correspond to an actual instance in the design but instead to one or more elements of a vector or array instance. • <i>For instance terminal probes</i>, the string represents an instance name and terminal name separated by a '/'. For example, <code>i2/t</code> or <code>i3<0:1>/t</code>, where the latter example indicates two instance terminals. • <i>For net probes</i>, the string represents a scalar or bit net, bus or bundle. The net name might not correspond to an actual net in the design, but instead to one more elements of a bus or bundle. Tcl preference: <code>deProbeName</code>
Cross Probe via SDL	Available when other cross-probing tools, such as SDL, are enabled through SDL > Probe > Add or through Add Probe in the Schematic Assistant. See Adding Probes in the <i>Custom Compiler SDL and Visually Assisted Automation Guide</i>. When selected and you add a probe in one view, this option lets you create a probe on an equivalent object in the corresponding view. Tcl preference: <code>deCrossProbeTool</code>
Trace Mode	Set up the trace mode by Fast or Accurate. See Trace Through options in Setting Display Options for details. Tcl preference: <code>deProbeNetHighlightFullTrace</code>
Trace Through Check	Specifies a trace-through method by the following: • Property: Traces through properties. • Cell: Traces through cells. See Trace Through in Setting Display Options for details.

Option	Description
Remove All (Toolbar only) Remove All Probes (Dialog box only)	Removes all probes in the active context.
<i>Advanced</i>	(Dialog box only) Optional advanced options.
Description	(Dialog box only) A string that allows for further filtering or grouping of probes. Should be contain a very short description of the probe (1-4 words recommended). Tcl preference: <code>deProbeDescription</code>
Tool	(Dialog box only) Tool that is creating this probe. Tcl preference: <code>deProbeTool</code>
Purpose	(Dialog box only) Probe purpose: • error • warning • info • cross • other Tcl preference: <code>deProbePurpose</code>
Long Description	(Dialog box only) Detailed description of the probe. Tcl preference: <code>deProbeLongDescription</code>
Apply	Creates the probe.
Defaults	Resets all of the options to your Custom Compiler environment default values.
Eject	(Toolbar only) Opens the Add Probe dialog box which lets you define options. The Hide button at the bottom of the dialog box closes the dialog box and displays the options on the toolbar. To switch between the toolbar and the dialog box while the command is active, use the Eject button and the Hide button in the dialog box or press the F3 shortcut key.

Add Probe Menu

When the **Add Probe** command is active, right-click to show the menu.



The menu contains this option:

Option	Description
Select Component	Available only for multi-bit objects. Opens the Select Component dialog box, which you can use to select a component object from a vector or array instance, bus, bundle, or multi-bit instance terminal. The necessary components of a multi-bit name can be selected from the list (all are selected by default) or set by typing directly in the component name field (Net, Instance, or Terminal, depending on the type of probe) at the top of the dialog box.

Removing Probes

Probes can be removed by using any of the following:

- The Delete key, which is bound to the **Delete** button on the Probe assistant toolbar, after selecting the probes.

For information about using the Probe assistant, see the *Custom Compiler Environment User Guide*.

- The `db::destroy` Tcl command when removing the probed objects from the design.

When a probe is deleted due to the corresponding probe objects being removed from the design, an undo of that delete operation does not restore the probe.

Highlighting Signals From WaveView

You can highlight signals in schematic designs from the PrimeWave Design Environment WaveView tool. For information on how to use WaveView, the waveform tool for the PrimeWave Design Environment, see *Getting Started with the PrimeWave Design Environment Waveform Viewer*.

13

Assistants for Schematic Designing

Use design window assistants to facilitate design development. Design assistants help with operations on the design in the current window.

This chapter covers instructions for using the Schematic Editor design window assistants.

- [Deciding Which Assistant to Use](#)
- [AMS Visualization Assistant](#)
- [Device Label Editor Assistant](#)
- [Estimated Parasitics Assistant](#)
- [Marker Browser Assistant](#)
- [Net Tracer Assistant](#)
- [Template Manager Assistant](#)

Deciding Which Assistant to Use

The following table lists the Schematic Editor assistants in alphabetical order and describes how each assists with designing.

Assistant	Description
AMS Visualization Assistant AMS Visualization Assistant	Annotates the analog, digital, and mixed signal partitioning of a design to aid in debugging and analysis. Uses view-based or simulation-based partitioning in the Custom Compiler Schematic Editor or a text editor.
Clipboard Assistant	Shows thumbnails of available clipboards when pasting objects in the Layout Editor, Schematic Editor, and Symbol Editor windows. For information, see the <i>Custom Compiler Environment User Guide</i> .

Assistant	Description
Console Assistant	Displays events and error messages, and lets you type Tcl commands. By default, the Console appears at the bottom of the Schematic Editor. For information, see the <i>Custom Compiler Environment User Guide</i> .
Constraint Editor Assistant	Creates and applies design constraints. For further information, see Creating Constraints .
Design Navigator Assistant	Navigates designs by component; operates on nets and net groups. For information, see the <i>Custom Compiler Environment User Guide</i> .
Device Label Editor Assistant	Controls settings for device labeling.
Device Palette Assistant	Automates repetitive instance placement. For information, see the <i>Custom Compiler Environment User Guide</i> .
Estimated Parasitics Assistant	Adds estimated parasitic devices to a schematic design.
Hierarchy Navigator Assistant	Navigates designs by instance or master name. For information, see the <i>Custom Compiler Environment User Guide</i> .
Marker Browser Assistant	Lists and browses to marked design violations in designs.
Net Tracer Assistant	Probes nets in designs.
Probe Assistant	Displays, cycles through, and deletes probes. (Use the Add > Probe command to add probes.) For information, see the <i>Custom Compiler Environment User Guide</i> .
Property Editor Assistant	Creates, and edits, properties, edits parameters, and edits the values of attributes; controls connectivity and connection assignments; displays instance terminal connectivity. For information, see the <i>Custom Compiler Environment User Guide</i> .
Schematic/Symbol Object Filter Assistant	Controls settings for precision canvas selection in schematic and symbol designs. For information, see the <i>Custom Compiler Environment User Guide</i> .
Template Manager Assistant	Finds matches from templates in template libraries and binds them to template intent constraints.

Assistant	Description
Transaction History	Lists committed database transactions; reverts designs to previous states in the current session. For information, see the <i>Custom Compiler Environment User Guide</i> .
VDRC Assistant	Lets you edit voltage annotations and set up voltage heat map views. For information, see the <i>Custom Compiler Electrical Checking and Reporting User Guide</i> .

AMS Visualization Assistant

The AMS Visualization assistant lets you annotate the analog, digital, and mixed-signal partitioning of a design to aid in debugging and analysis. You can use view-based or simulation-based partitioning in the Schematic Editor or a text editor.

View-based partitioning shows the partitions computed by the AMS netlister based on the views found in the bound hierarchy. Simulator-based partitioning is the back-annotation of the domain the simulator used for each level of hierarchy.

The following objects can be annotated: instances, nets, terminals. The instance annotation is made based on the instance type and the net and terminal annotation is made starting from their connection to the instances. In addition, in simulator-based mode, the interface elements found by the simulator's partitioning are back-annotated for viewing the location and parameters of these interface elements. For mixed-signal simulation with System-Verilog, the interface element values can be edited and passed to the PrimeWave Design Environment for future simulation use (see [Editing Interface Element Parameters](#)). See also Computing Interface Elements in the *PrimeWave Design Environment User Guide*.

The following sections describe the tasks you can perform using the AMS Visualization assistant:

- [Opening the AMS Visualization Assistant](#)
- [Highlighting Partitions in View-Based Mode](#)
- [Highlighting Partitions in Simulator-Based Mode](#)
- [Editing Interface Element Parameters](#)
- [Highlighting Partitions in a Text Editor](#)
- [Refreshing Partitions](#)

- [Clearing Partitions](#)
- [Accessing Analog Explanations](#)

Opening the AMS Visualization Assistant

To open the AMS Visualization assistant from the Schematic Editor, use one of these methods:

- Choose **Tools > AMS** in the Schematic Editor.
- Choose **Window > Assistants > AMS Visualization (View Based)** from the Schematic Editor toolbar.

To open the AMS Visualization assistant from the PrimeWave Design Environment:

- Choose **Results > Annotate > Interface Elements**, which opens the AMS Visualization Assistant in the Schematic Editor with current simulation results loaded.

The assistant docks along the window edge in the Schematic Editor. Undock it by clicking and dragging it.

The assistant has two tabs: **General** and **Interface Elements**. The **Interface Elements** tab is only visible in simulator-based mode. Details are provided in the following sections.

Highlighting Partitions in View-Based Mode

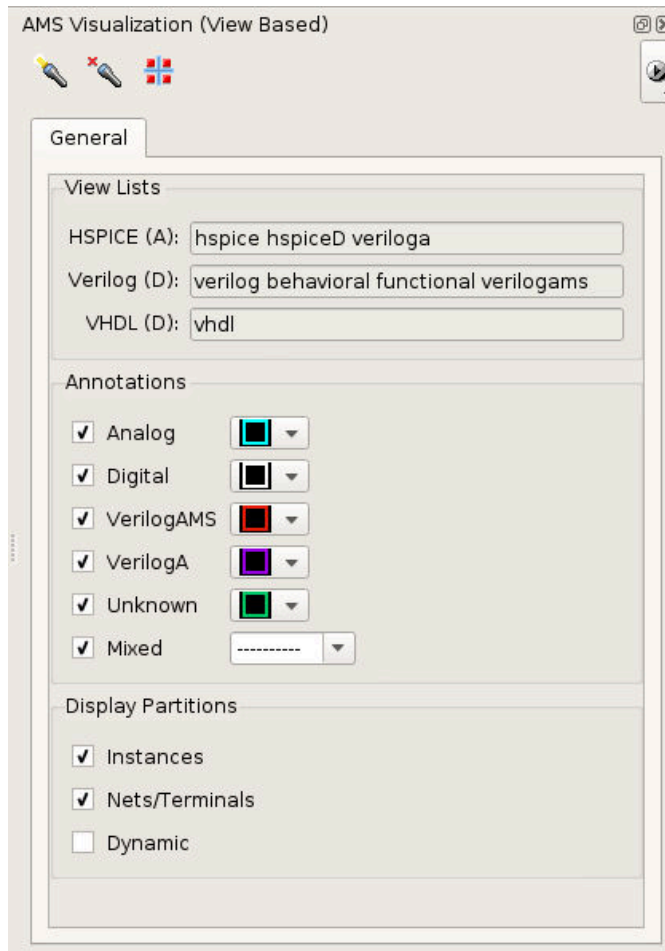
You can choose which design partitions to highlight and which highlight colors to apply. In view-based mode, the instance and net domain is identified according to the partition defined by the AMS netlister.

You can also specify whether to annotate all partitions or only individual partitions as you move your cursor over objects.

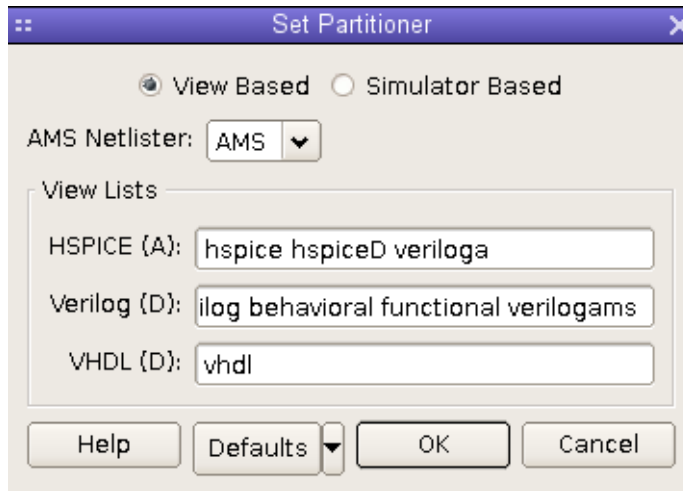
To highlight partitions:

1. Open the AMS Visualization assistant as described in [Opening the AMS Visualization Assistant](#).

The AMS Visualization assistant opens in view-based mode by default.



2. In the **View Lists** section of the **General** tab, the default netlister and view list is HSPICE. To change this, click the **Set Partitioner**  button in the assistant's toolbar to open the **Set Partitioner** dialog box.



When an AMS netlister is selected in the **Set Partitioner** dialog, the language netlisters are listed below the selection and their associated views are shown in the corresponding text fields. You can edit the view lists to add or delete the views you want netlisted. This information populates the **View Lists** section in the **General** tab of the AMS Visualization assistant when you click **OK**.

3. Close the **Set Partitioner** dialog box. The **General** tab shows the following options in view-based mode:

Table 17 *AMS Visualization Assistant General Tab Options (View-Based)*

Option	Description
View Lists	Read-only information from the Set Partitioner dialog. (See Step 2.)
Annotations	Specifies the highlight color for each of the following partitions, and whether to display the partition. ◦ Analog: The branch is only analog. ◦ Digital: The branch is only digital. ◦ Unknown: Unable to determine the complete contents of the branch. ◦ Mixed: The branch contains both analog and digital.
Display Partitions	Specifies which objects to highlight. ◦ Instances ◦ Nets/Terminals ◦ Dynamic: When enabled, applies the specified annotations only when your cursor hovers over an object. (Off by default.)

4. Specify the **Annotations** options you want to apply.
5. Specify the **Display Partitions** options you want to apply.

- 6. To highlight only individual partitions as you move your cursor over objects, enable the **Dynamic** option (off by default).
- 7. The assistant highlights all of the specified partitions. Use the toolbar buttons to clear or refresh current highlighting.

Table 18 *AMS Visualization Assistant General Toolbar Buttons and their Actions*

Button	Tooltip	Action
	Show/refresh partition visualization	Highlights partitions as currently specified in the General tab options. For more details, see Refreshing Partitions .
	Hide partition visualization	Clears all highlighted partitions. For more details, see Clearing Partitions .
	Open Set Partitioner dialog	Opens the Set Partitioner dialog. For more details, see Step 2 .

Note:

Right-click over a non-pin instance to access the **AMS Explain** command (see [Accessing Analog Explanations](#)).

Highlighting Partitions in Simulator-Based Mode

While you are running simulations with the PrimeWave Design Environment, you can choose which design partitions to highlight and which highlight colors to apply. In simulator-based mode, the only type of nets you can view are mixed nets and they are hidden by default. The instances' and nets' domain and interface element information is obtained from the reports of the executed simulation.

Note:

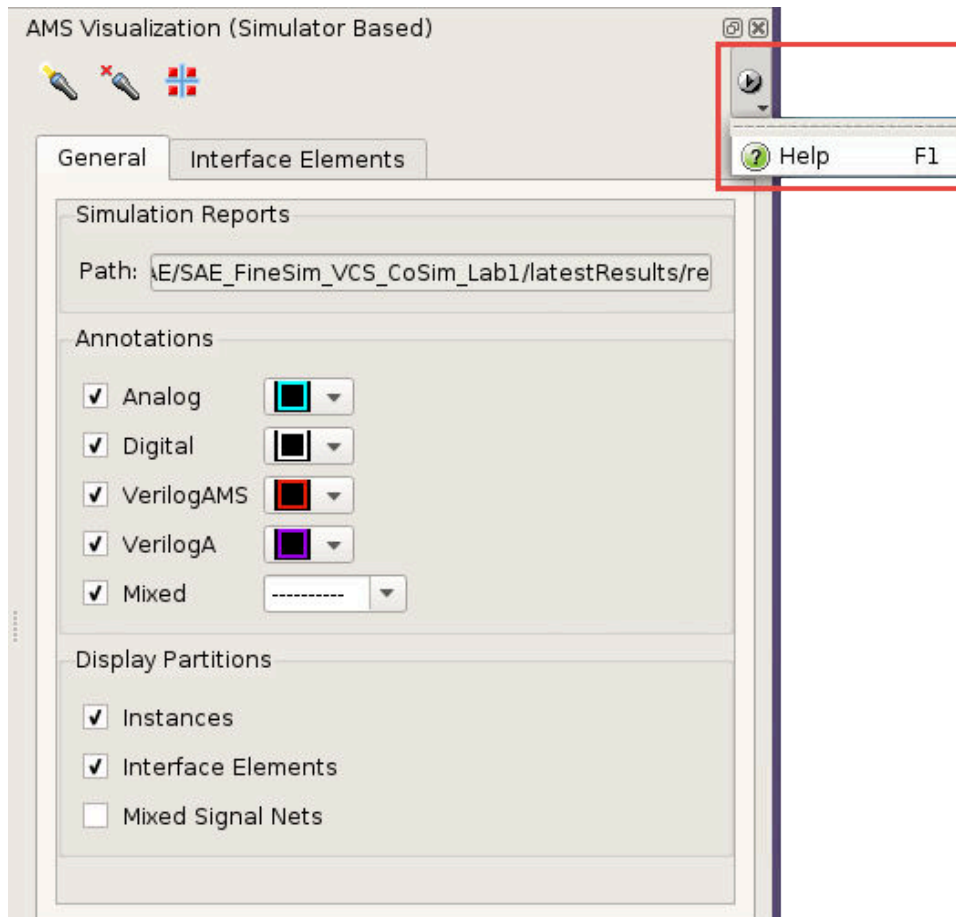
Before you start, simulation data must be generated using the PrimeWave Design Environment. See the *PrimeWave Design Environment User Guide* for further information.

To highlight all partitions:

- 1. Generate simulation data using the PrimeWave Design Environment. (See the *PrimeWave Design Environment User Guide* for further information.)
- 2. In your active PrimeWave Design Environment session, choose **Results > Annotate > Interface Elements** to open the AMS Visualization Assistant in the Schematic Editor.

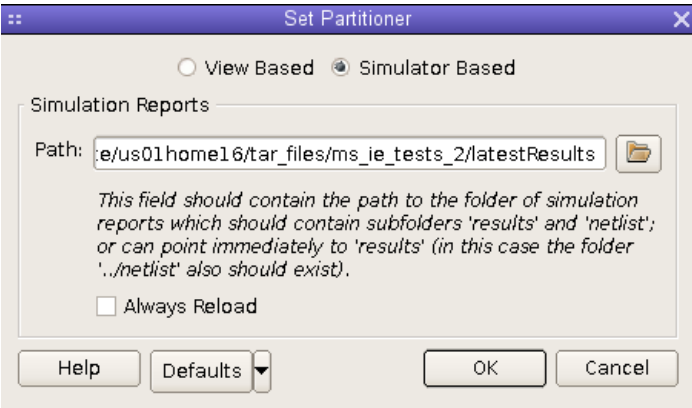
Alternatively, open the design in the Schematic Editor and choose **Tools > AMS or Window > Assistants > AMS Visualization (Simulator Based)**.

The AMS Visualization assistant opens in simulator-based mode. Notice that the Help is available from the assistant's toolbar menu  (outlined in red in the following figure).



- Note that if you exit the PrimeWave Design Environment before opening the AMS Visualization assistant, the assistant automatically opens in view-based mode.

To change to simulator-based mode, click the **Set Partitioner**  button in the assistant's toolbar to open the **Set Partitioner** dialog box.



- a. Choose **Simulator Based** mode. **Path** is automatically set by the active PrimeWave Design Environment session to the currently selected simulation results. If the PrimeWave Design Environment is not running, or if you want to annotate based on other results, browse to the **Path** of the simulation results directory with the annotation data.
- b. Select **Always Reload** if you want to force a reload of the simulation partition file on each **Show** action before updating the annotation.

The **Set Partitioner** dialog box's information populates the **Simulation Reports** section in the **General** tab of the AMS Visualization assistant when you click **OK**.

4. Notice that the **General** tab contains the following options in simulator-based mode:

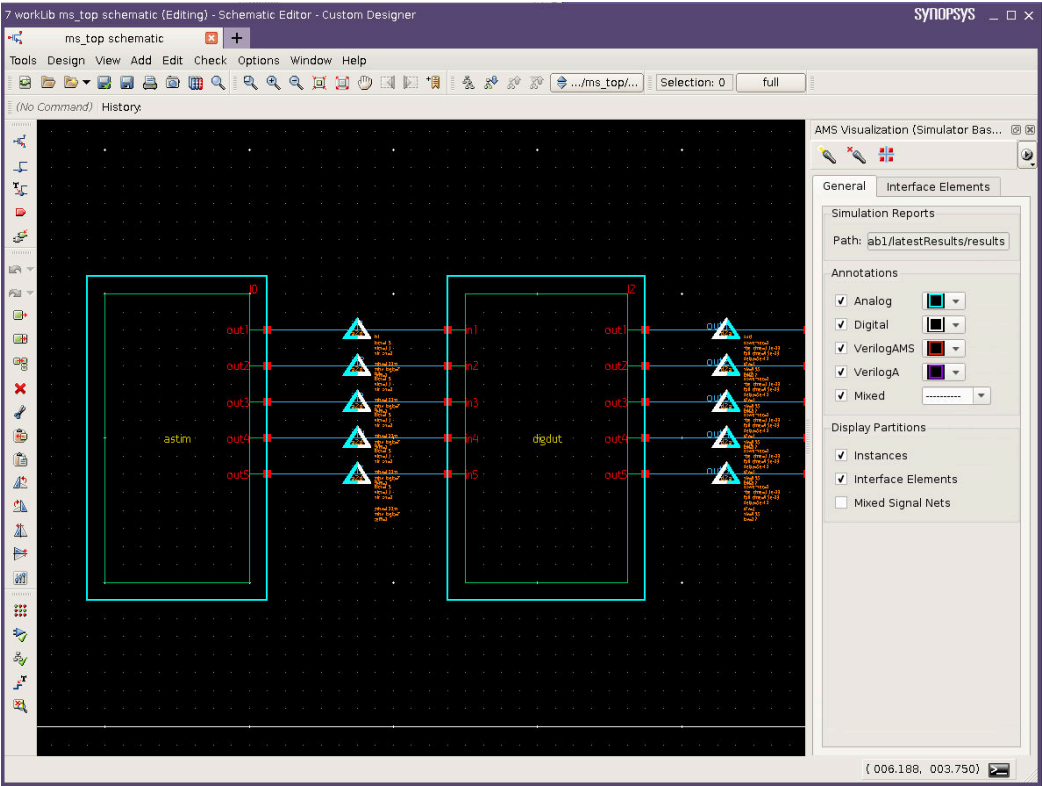
Table 19 AMS Visualization Assistant General Tab Options (Simulator-Based)

Option	Description
Simulation Reports	The Path field should contain the path to the simulation report folder (simv.msv). This is found in the PrimeWave Design Environment result directory hierarchy and you should select the “results” directory where the simulator writes its results. When a valid path is specified, the AMS Visualization assistant's title automatically shows the simulation type (for example, “Simulator Based (Co-Sim)” or “Simulator Based (Verilog-AMS)”.

Option	Description
Annotations	Specifies the highlight color for each of the following partitionings, and whether to display the partitioning. ◦ Analog Cell ◦ Digital Cell ◦ VerilogAMS Cell ◦ VerilogA Cell ◦ Mixed Nets ◦ Mixed Branches Cell annotation shows the language and simulator engine used for the cell for checking that the right view was used. Mixed Nets let you know that there is an Interface Element placed on this net at its highest scope. Mixed Branches brings in a hierarchical view to let you see whether the content below a cell contains both analog and digital.
Display Partitions	Specifies which objects to highlight. ◦ Instances ◦ Interface Elements ◦ Mixed Signal Nets

5. Specify the **Annotations** options you want to apply.
6. Specify the **Display Partitions** options you want to apply.

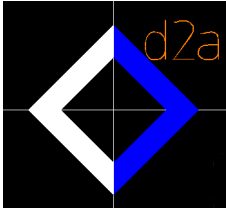
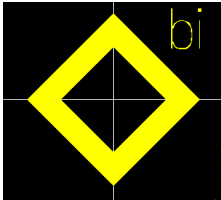
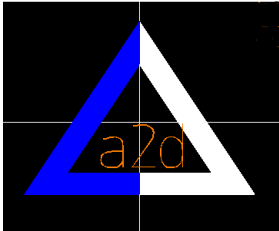
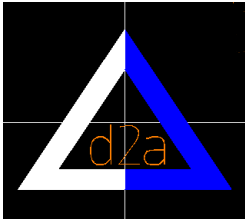
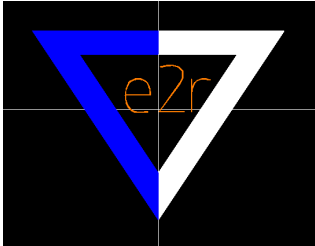
The tool highlights all of the specified partitions on the design canvas.

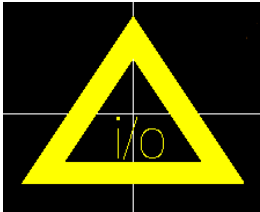


The Schematic Editor indicates the different interface element types using different icons, as explained in [Table 20](#). Note that the icon colors (except for bidirectional icons) you see in your session depend on your current settings for analog and digital visualization colors.

Table 20 *Interface Element Types and Icons*

Interface Element Type and Abbreviation	Icon
Verilog AMS	
Analog-to-Digital a2d	

Interface Element Type and Abbreviation	Icon
Digital-to-Analog d2a	
Bidirectional bi	
Co-Sim	
Analog-to-Digital a2d	
Digital-to-Analog d2a	
Electrical-to-Real e2r	

Interface Element Type and Abbreviation	Icon
Real-to-Electrical r2e	
Electrical-to-Net e2n	
Net-to-Electrical n2e	
In/Out i/o	

7. Use the toolbar buttons (described in [Table 18](#)) to clear or refresh current highlighting.

Editing Interface Element Parameters

In simulation-based mode, the interface elements found by the simulator's partitioning are back-annotated for viewing the location and parameters of these interface elements. For mixed-signal simulation with System-Verilog, you can edit the interface element values and pass them to the PrimeWave Design Environment for future simulation use.

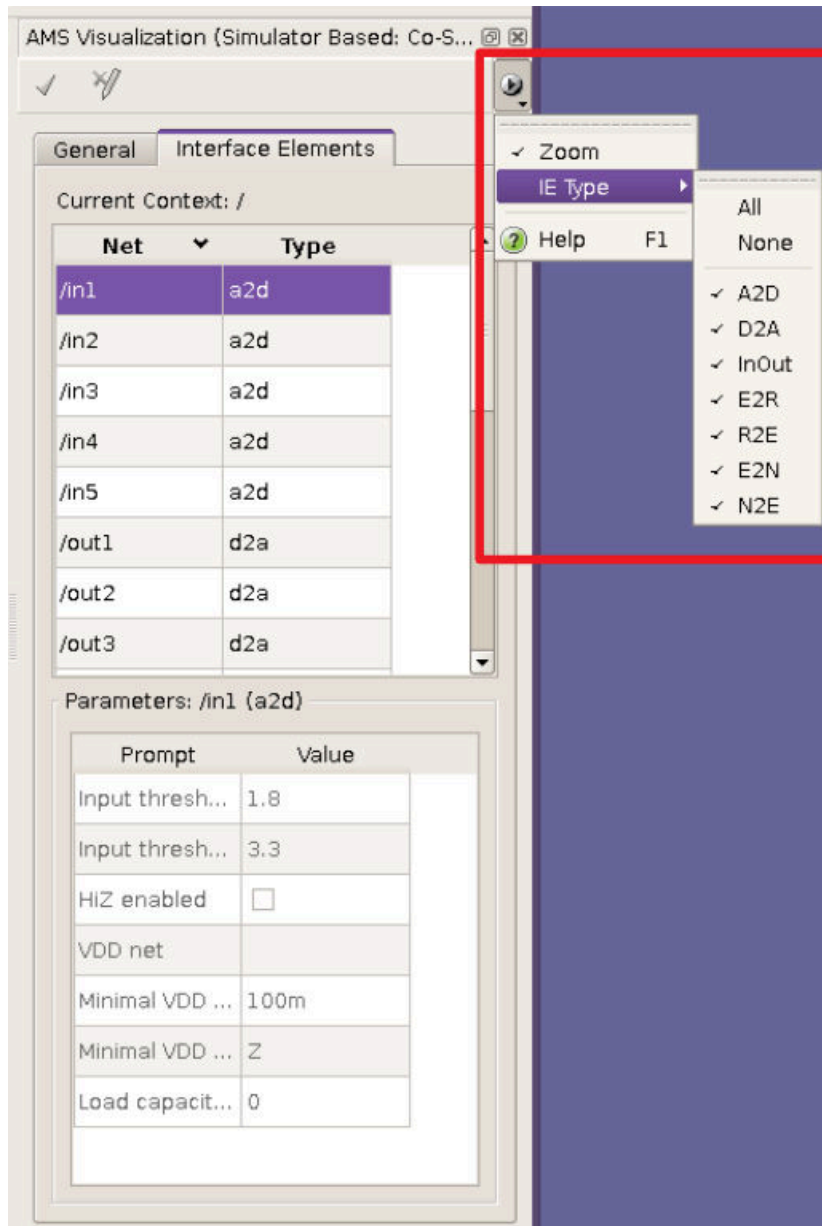
Note:

Before you start, simulation data must be generated using the PrimeWave Design Environment. See the *PrimeWave Design Environment User Guide* for further information.

1. Open the AMS Visualization assistant in simulator-based mode, as described in [Highlighting Partitions in Simulator-Based Mode](#).
2. Switch to the **Interface Elements Parameters** tab to edit interface elements (shown in the following figure). (Alternatively, right-click on the net or instance terminal object with the interface element annotation to bring up the context-sensitive menu and choose **Edit Interface Element Parameters** to open the **Edit Interface Element Parameters** dialog box from the design canvas.)

Note:

If the PrimeWave Design Environment is not active, this tab only allows viewing of the results. Editing in Verilog-AMS simulation mode is currently not supported.



In this tab, it is only possible to edit interface element parameters if the parent PrimeWave Design Environment session is currently active.

3. In the **Interface Elements** tab, click the assistant's drop-down menu  to see the available viewing options (outlined in red in the figure above).

Table 21 AMS Visualization Assistant Interface Elements Tab Drop-Down Menu

Menu Item	Action
Zoom	Double-clicking on a specific interface element in the table traverses to the occurrence in the design and highlights the interface element. For schematic views, Zoom zooms closer to the selected interface element. (On by default.)
IE type	Filters the interface elements listed in the Current Context section by type, All (the default), or None . (For a list of interface element types, see Table 20 .)
Help	Opens the online help documentation for this topic.

The **Interface Elements** tab contains the following sections:

Table 22 AMS Visualization Assistant Interface Elements Tab Options

Section	Description
Current Context	Lists the current view's interface elements in tabular form. The table contains the following columns: ◦ Net (if simulation type is Co-Sim) or Instance Terminal (if simulation type is Verilog-AMS): The hierarchical name of the object to which the interface element is set. ◦ Type: The corresponding interface element abbreviation is shown in this field. (For a list of interface element types and their abbreviations, see Table 20.) Click on a column header to view the sort arrow () and sort the column's data alphabetically or reverse alphabetically.
Parameters	Displays the parameters of the interface element selected in the interface elements table. The currently editable interface element populates the title of this table (for example, Parameters: /in1 (a2d) as shown in the figure above). The table contains the following columns: ◦ Prompt: The display name for the interface element parameter. ◦ Value: The value used in the simulation. If edited, the results are sent back to the PrimeWave Design Environment for use in the next simulation.

4. Select the interface element of interest in the **Current Context** table.
5. Edit the selected interface element's parameters in the **Parameters** table. Once any parameter is edited, its name is marked in bold in the **Parameters** table.

6. Use the toolbar buttons to apply your parameter value changes.

Table 23 *AMS Visualization Assistant Interface Elements Toolbar Buttons and their Actions*

Button	Tooltip	Action
	Apply pending changes of IE parameters	Applies pending parameter value changes. Only enabled when there are pending changes (marked in bold in the Parameters table).
	Discard pending changes of IE parameters	Discards pending parameter value changes. Only enabled when there are pending changes (marked in bold in the Parameters table).
	Open IE parameters editing in PrimeWave	Opens the active PrimeWave Design Environment session Mixed Signal Option dialog box, which allows you to set defaults as well as occurrence-specific values for interface elements. Values modified in the Schematic Editor's AMS Visualization Assistant are annotated to this dialog. Only enabled when an interface element is selected in the Parameters table.

If you change the current interface element selection or attempt to close the assistant while there are pending changes, you are prompted to apply or discard your changes.

When an interface element (IE) location is not accessible because it is in the lower hierarchy of the occurrence which is the leaf of the design, this interface element is nevertheless created and shown in the AMS Visualization assistant. This kind of interface element is defined as *partial lineage* IE, which is not visualized in the Schematic Editor or Text Editor.

Warnings are reported in the Console as follows:

Warning: Interface element on the instance terminal "/I1/bu1/drain" cannot be completely loaded from simulation because the instance terminal "bu1/drain" is not found in design "alsi_hdlLib/BUF/verilog". The interface element with partial lineage "/I1" will be created. (PARTIAL)
Warning: Interface element on the instance terminal "/I1/FF1/FF2/bu1/drain" cannot be completely loaded from simulation because the lineage "/I1/FF1/FF2" is not found in design "CosaLab/TOP_S012/schematic". The interface element with partial lineage "/I1" will be created.

The *partial lineage* IE is created (marked in italic) in the AMS Visualization assistant as follows:

General

Interface Elements

Current Context: /

tance termi ▾	Type
/I1/buf1/drain	a2d
/I1/FF1/FF2/buf1/drain	a2d
/I4/bot1/in	a2d

IE instTerm lineage '/I1/buf1/drain' is partially accessible. Highlight will not be available.

Note:

The red warning label is shown when the *partial lineage* IE row is selected in the table.

When double-click on the IE row to highlight it, only the hierarchy is changed to show the accessible lineage of that IE, but the IE object is not highlighted.

Supports nominal, results, simv.msv, or connectmodule.rpt file as a source of interface elements. Supports showing the expected parameter values in the interface elements at analog to digital or digital to analog conversion point reading connect module report from results path instead of the reading default values from the connect module.

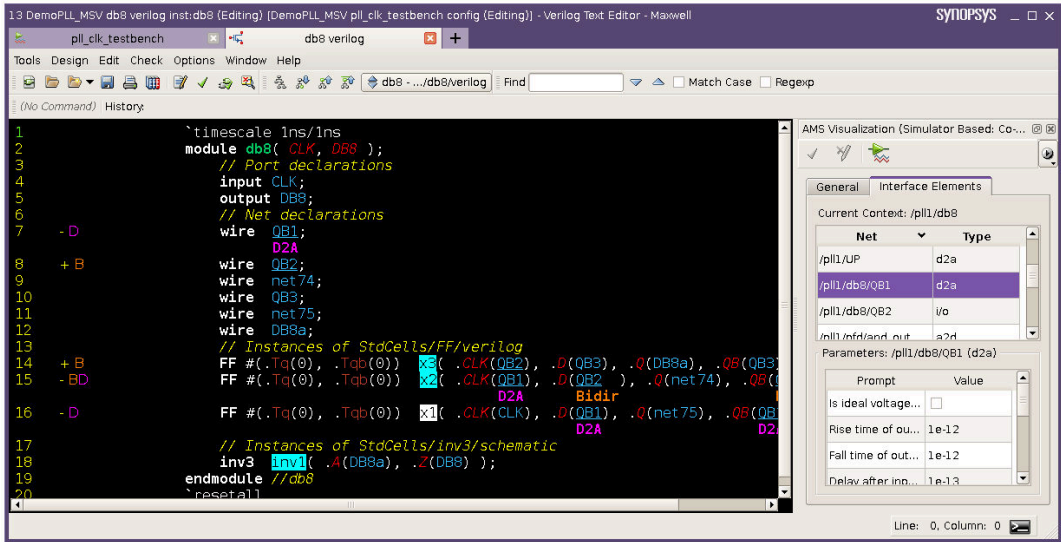
[EditDeletePin](#)

- <https://jira.internal.synopsys.com/secure/AddComment!default.jspx?id=5589330>

Highlighting Partitions in a Text Editor

You can highlight partitions in the Custom Compiler text editor using the AMS Visualization assistant.

1. Repeat the procedure described in [Highlighting Partitions in Simulator-Based Mode](#) using a text file. You can also move back and forth between the two editors for designs that have both analog and digital information.
2. The tool highlights all of the specified partitions in the text editor and adds annotations to the text file.



The following table provides a key to the annotations added to the right of the line numbers.

Text Annotation	Direction
A	Analog-to-Digital a2d
B	Bidirectional bi
D	Digital-to-Analog d2a
E	Electrical-to-Real e2r
R	Real-to-Electrical r2e
N	Net-to-Electrical n2e

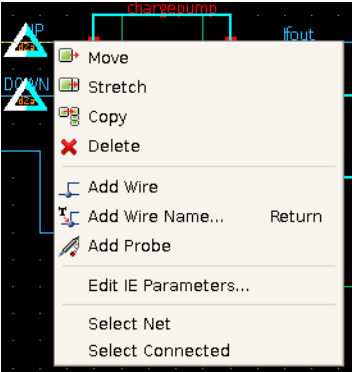
Text Annotation	Direction
T	Electrical-to-Net e2n

The underline of the nets indicates the nets on which the interface elements are placed (shown in the figure above).

There can also be situations when more than one letter is listed. For example, “ABD” indicates that that particular line has a2d, bidirectional, and d2a interface elements present.

If there are multiple interface element types for the same instance, you can expand by clicking on the interface element annotation, which adds a row below the line showing exactly which nets have interface elements and the interface element type for that net.

- 3. Right-click on the net or instance terminal object with the interface element annotation to bring up the menu.



- 4. Choose **Edit IE Parameters** to open the **Edit Interface Element Parameters** dialog box from the Text Editor.

Net: **out1**

Simulation: **Co-Sim**

Direction: **Bidirectional**

Parameters

Input threshold for output 0:

Input threshold for output 1:

☐ HiZ enabled

☐ Is ideal voltage source

Rise time of output voltage:

Fall time of output voltage:

Delay after input change:

Rule for input X translation:

Output voltage for input 0:

Output voltage for input 1:

VDD net:

Minimal VDD voltage:

Minimal VDD logic:

Load capacitance:

In this dialog box, it is only possible to edit interface element parameters if the parent PrimeWave Design Environment session is currently active.

5. Make any desired changes to the available parameter options for the selected interface element.
6. Click **OK** or **Apply** to apply your edits.

Refreshing Partitions

If changes are made to the database after you partition your design, the partitionings could now be invalid. The **Show/Refresh Partition** button  lets you reapply valid partitionings instead of clearing and reapplying them.

To refresh the partitions:

- Click the **Show/Refresh Partition** button .

The highlighting is reapplied.

Clearing Partitions

You can clear all highlighting.

To clear partitions:

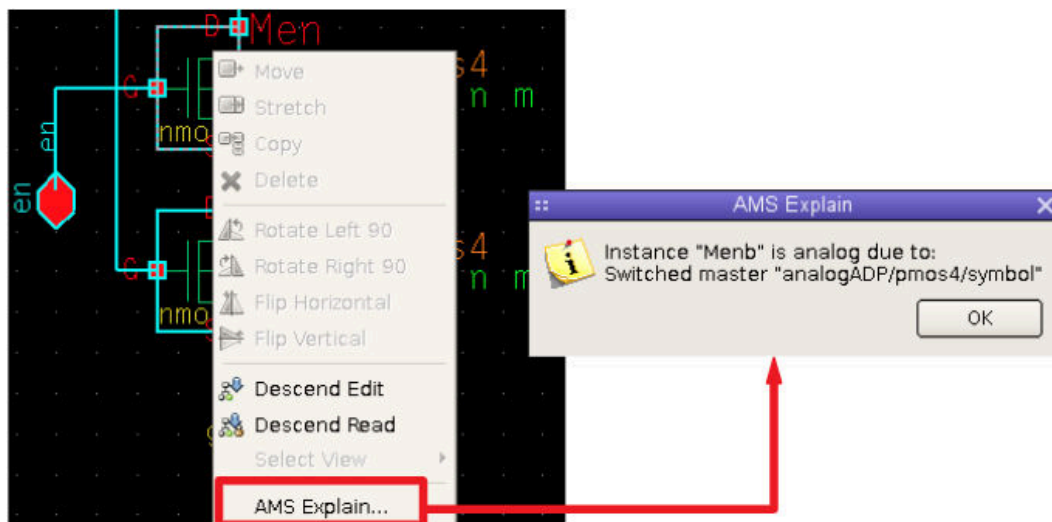
- Click the **Hide Partition** button .

The highlighting is removed from all partitioning.

Accessing Analog Explanations

You can get more information about why an instance is determined to be analog using the **AMS Explain** command.

1. Open the AMS Visualization assistant as described in [Opening the AMS Visualization Assistant](#).
2. Highlight partitions as described in [Highlighting Partitions in View-Based Mode](#) or [Highlighting Partitions in Simulator-Based Mode](#).
3. The **AMS Explain** command is added to the drop-down menu when you right-click on instances.



Device Label Editor Assistant

You can use the Device Label Editor assistant to control the display of evaluated labels associated with components. Through this assistant, you can choose to display default or alternate labels from the component description format (CDF) parameter set. You can use the device label editor with both open, and unopened, designs.

These sections cover:

- [Opening the Device Label Editor Assistant](#)
- [Controlling Device Label Display](#)
- [Editing Attribute Labels](#)
- [Editing Parameter Labels](#)
- [Saving and Loading Device Label Settings](#)

Opening the Device Label Editor Assistant

Open the Device Label Editor to work with evaluated device labels.

To open the Device Label Editor:

- In a schematic design, choose **Window > Assistants > Device Label Editor**.

The assistant docks along the window edge. Undock it by clicking, and dragging it.

To show device label details here, select a device in the design.

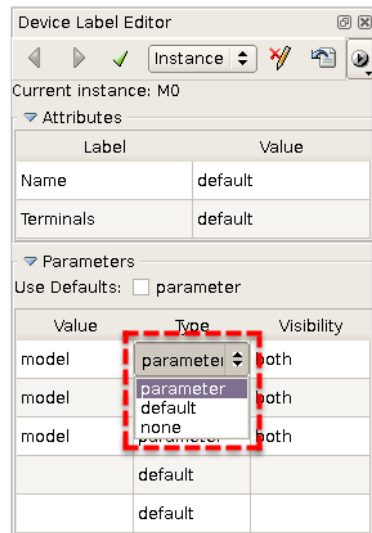
Controlling Device Label Display

You can control device label display to limit label information shown on the canvas.

Use the Device Label Editor assistant to control device label display.

Begin by opening the Device Label editor assistant. In a schematic, choose **Window > Assistants > Device Label Editor**.

- You can right-click any column title, and choose which column to expose. You can expose the label type, the parameter to be shown in the label, and the label visibility.



- To force the device to use parameters from the component description format paramLabelSet field, check **Use Defaults**.

Editing Attribute Labels

You can edit attribute labels to adjust the labeling information for a name or terminal attribute in a design.

Use the Device Label Editor assistant to edit attribute labels.

To edit device label attributes:

1. Begin by opening the Device label Editor assistant. In a schematic, choose **Window > Assistants > Device Label Editor**.
2. In the Device Label Editor **Attributes** section, click the name or terminal whose label you want to change.
3. From the list box, make a selection.

Refer to [Table 24](#) and [Table 25](#) to predict the result of this change.

4. Apply changes (click the green check mark ).

[Table 24](#) describes Device Label Editor name values.

Table 24 *Device Label Editor Name Values*

Name Value	Description
Instance	Displays the name attribute of the current instance
Cell	Displays the cell name attribute of the current instance's placed master
Default	Inherits the display type from a more global scope
None	Suppresses the display

Table 25 describes Device Label Editor terminal values.

Table 25 *Device Label Editor Terminal Values*

Terminal Value	Description
Name	Displays the name attribute of the instance's placed master's terminal
Net Name	Displays the name of the net attribute of the instance terminal
Default	Inherits the display type from a more global scope
None	Suppresses the display

Editing Parameter Labels

You can edit parameter labels to control which component description format parameter is shown for a device in a design.

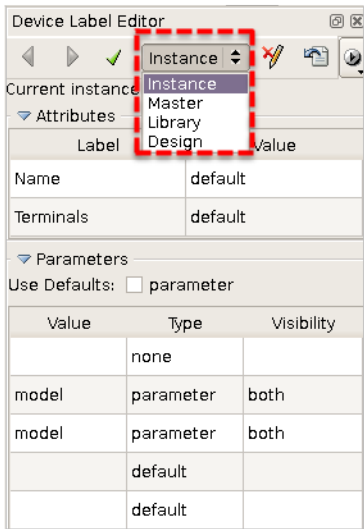
Use the Device Label Editor assistant to edit parameter labels.

To edit parameter labels:

1. Begin by Device Label Editor assistant. In a schematic, choose **Window > Assistants > Device Label Editor**.
2. In the design, select the device.
3. In the Device Label Editor under **Parameters**, right-click the current name of the label you want to change.
4. From the list box, make a selection.

To hide parameter labels on the canvas, set the type to **none**.

5. In the toolbar, choose the scope to which the change will be applied.



For scope details, refer to the table that follows.

6. Apply changes (click the green check mark ).

Table 26 describes Device Label Editor scope settings.

Table 26 Device Label Editor Scope Settings

Scope	Application of settings
Instance	Applies settings only to the selected/current instance. A setting of Default will cause the corresponding label evaluation to be inherited from a higher scope (Design, Master, or Library).
Design	Applies settings to all instances in the design whose master matches that of the current/selected instance. A setting of Default will cause the label display to be inherited from a higher scope (Master or Library).
Master	Applies settings to all instances whose master matches that of the current instance. All designs in the current session will be impacted, whether they are opened or closed at the time the change is made. A setting of Default will allow the cdsName and cdsTerm label displays to be inherited from the Library scope. A setting of Default for the parameter labels will cause the display to be determined by the information in the cell component description format.
Library	Applies settings to all instances in the same library as the current instance's master, again across all designs in the current session. A setting of Default will restore the cdsName and cdsTerm labels to the default settings.

Saving and Loading Device Label Settings

These sections cover:

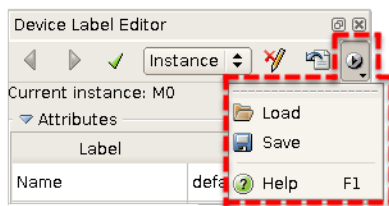
- [Saving Device Label Display Settings](#)
- [Loading Device Label Display Settings](#)

Saving Device Label Display Settings

You can save current label display settings to store them for future sessions. Use the Device Label Editor assistant to save device label settings. By default, the system saves device label settings on session end in the file `device_displays.xml`, and reloads this file on next session startup. You can save them to the file of your choice instead.

To save device label display settings:

1. Begin by opening the Device Label Editor assistant. In a schematic, choose **Window > Assistants > Device Label Editor**.
2. From the **Device Label Editor** toolbar menu, choose **Save**.



3. In the **Save Device Display** window, enter a name for the file you want to save.
4. Click **Save**.

Loading Device Label Display Settings

You can load device label settings to make them available in this session. By default, the system saves device label settings on session end in the file `device_displays.xml`, and reloads this file on next session startup. You can load them from a file of your choice instead.

To load device label display settings:

1. Begin by opening the Device Label Editor assistant. In a schematic, choose **Window > Assistants > Device Label Editor**.
2. From the **Device Label Editor** toolbar menu, choose **Load**.
3. In the **Load Device Display** window, enter the name of the existing file.
4. Click **Open** to load the file.

Estimated Parasitics Assistant

You can use the Estimated Parasitics assistant to add estimated parasitics to the schematic design to emulate parasitic effects and simulate the design with estimated parasitics.

The following topics are described:

- [Opening the Estimated Parasitics Assistant](#)
- [Adding Devices to the Schematic](#)
- [Registering Custom Parasitic Devices and Device Libraries](#)
- [Specifying Parasitic Options to Calculate Capacitance](#)
- [Calculating Capacitance Values and Adding Capacitance Devices](#)
- [Specifying Parasitic Options to Calculate Resistance](#)
- [Calculating Resistance Values and Adding From Layout](#)
- [Calculating Pi Resistance/Capacitance Values](#)
- [Ignoring Estimated Parasitic Devices in a Simulation](#)
- [Adding EMag Devices to the Schematic After Extracting EM Data](#)

 [Show me](#) how to use the Estimated Parasitics Assistant.

Opening the Estimated Parasitics Assistant

To open the Estimated Parasitics assistant:

- In a schematic view, choose **Window > Assistants > Estimated Parasitics**.

The assistant docks along the window edge. Undock it by clicking and dragging it.

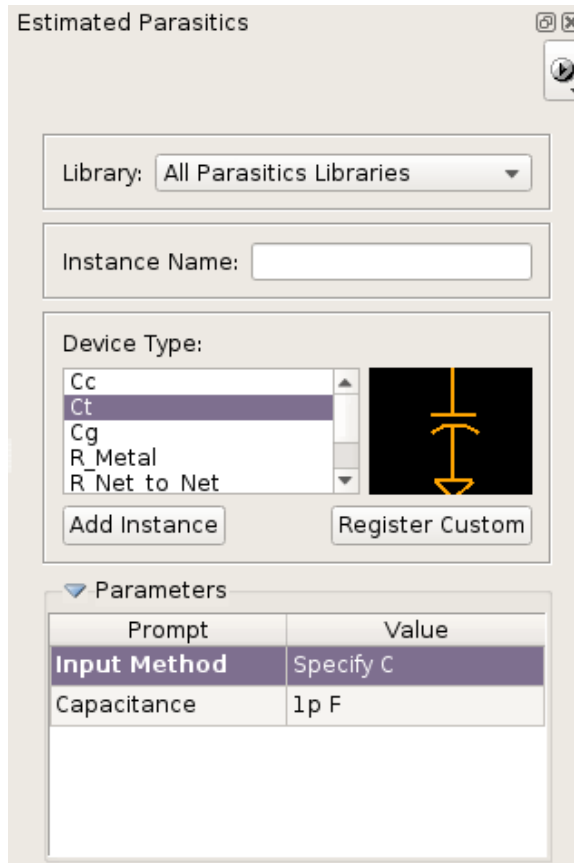
Adding Devices to the Schematic

Use the Estimated Parasitics assistant to add a parasitic device to your schematic design. You can add multiple devices of the same type.

It is useful to use the Estimated Parasitics assistant in conjunction with the Property Editor to edit properties of the parasitic devices as you create them. You must use the Property Editor if you want to edit the properties of an already placed parasitic device instance.

To add a device to the schematic design:

1. Begin by opening the Estimated Parasitics assistant. In a schematic view, choose **Window > Assistants > Estimated Parasitics**. The Estimated Parasitics Assistant opens.



2. Select one or more parasitics libraries using the **Library** option. You can select multiple libraries using Ctrl+click on each desired library. The default is **All Parasitics Libraries**.

Use the Tcl preference `seEstimatedParasiticsCustomDeviceLibName` to specify additional libraries to display in the EPA and set the default set of libraries for display. The default value is `parasitics`. See [Registering Custom Parasitic Devices and Device Libraries](#).

3. Choose a device type from the **Device Type** list. When you choose a device type, a thumbnail to the right of the list shows what the device will look like in the schematic.

To add a custom device to the list of device types, see [Registering Custom Parasitic Devices and Device Libraries](#).

4. You are prompted to specify any necessary parameter values in the **Parameters** section. The available parameters in the **Parameters** section vary depending on which device is selected. See the following sections for specific usages.
5. (Optional) Fill in the **Instance Name**. If you do not choose an instance name, the default naming convention is C<number> for a Cap and R<number> for a Res, where <number> is the next available number that is not used.
6. Click **Add Instance**. The device is now attached to the cursor and you can place it in your design by clicking in the location you want it.

Parasitic devices can be distinguished from other elements of the schematic in that they are smaller (half-size) and a different color (orange).

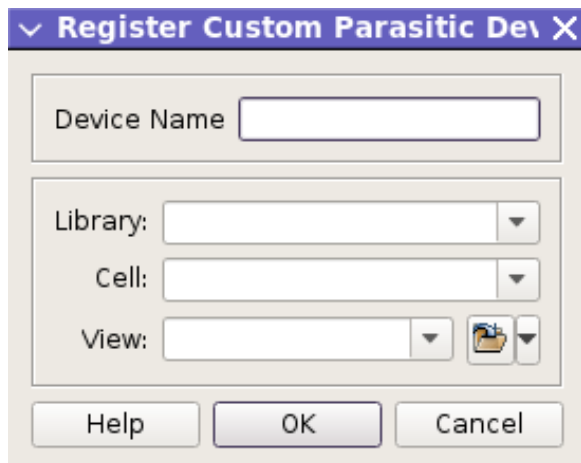
7. Run the simulation as usual to include this schematic showing estimated parasitics. You can compare simulations with and without estimated parasitics by controlling the netlist.

Registering Custom Parasitic Devices and Device Libraries

A number of devices are provided in the Estimated Parasitics assistant **Device Type** list from the parasitic library that is delivered with Custom Compiler. You can also create custom devices.

To add a custom device to the list of device types:

1. Choose **Register Custom** in the Estimated Parasitics assistant and fill in the **Register Custom Parasitic Device** dialog box.

The image shows a dialog box titled "Register Custom Parasitic Device" with a close button (X) in the top right corner. The dialog box has a light gray background. It contains three input fields: "Device Name" at the top, followed by "Library:" and "Cell:" which are grouped together. Below these is a "View:" field with a folder icon and a dropdown arrow. At the bottom of the dialog box are three buttons: "Help", "OK", and "Cancel".

2. Click **OK** to close the **Register Custom Parasitic Device** dialog box.

The device appears in the list of device types.

Alternatively, use the `se::registerParasiticTopology` command as part of your startup scripts. Place your startup-related commands in the `SYNOPSIS_CUSTOM_SITE/estimated_parasitics/startup.tcl` file to add the device permanently to all sessions. See `se::registerParasiticTopology` for an example, including how to associate an image preview with your custom device.

By default, the Estimated Parasitics assistant (EPA) uses all parasitic devices from Custom Compiler's parasitics library. You can specify additional libraries to display in the EPA through using the Tcl `seEstimatedParasiticsCustomDeviceLibName`. The default value is `parasitics`.

For example: `db::setPrefValue seEstimatedParasiticsCustomDeviceLibName -value "custom_parasitics_lib1 custom_parasitics_lib2 parasitics"`

Optionally, you can specify a prefix for each custom library of parasitic devices using the Tcl preference `seEstimatedParasiticsCustomDeviceNamePrefix`.

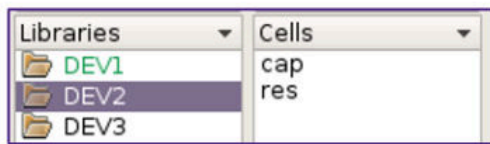
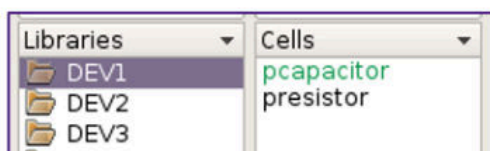
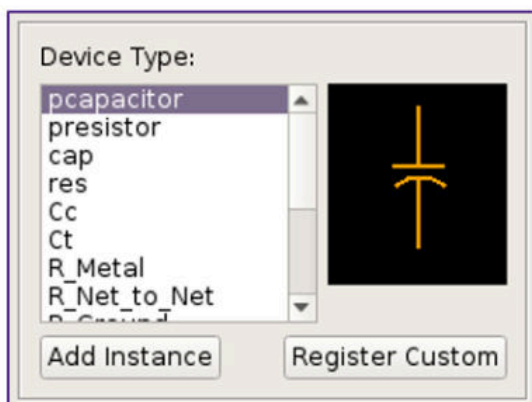
Note:

You cannot specify a prefix for the devices of the parasitics library.

For example: `db::setPrefValue seEstimatedParasiticsCustomDeviceNamePrefix -value "custom1_ custom2_"`

Note:

The lengths of `seEstimatedParasiticsCustomDeviceLibName` and `seEstimatedParasiticsCustomDeviceNamePrefix` must be consistent.



You can add a preview image for each added device. The image file must adhere to the following rules:

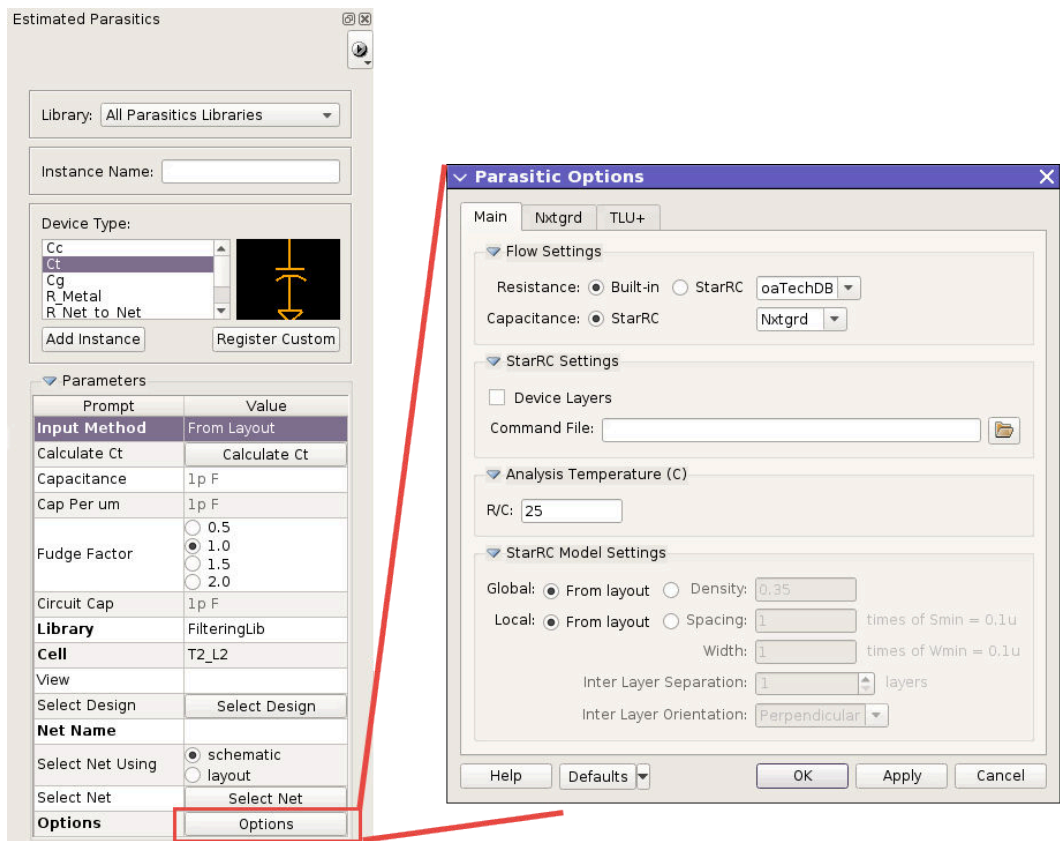
- Format is PNG
- Name is lower-case (no matter of the case of the device name)
- Name fits the device type name
- Location is `$SYNOPSIS_CUSTOM_SITE/images/<device_type_name>.png`

Specifying Parasitic Options to Calculate Capacitance

For capacitance-type parasitic devices (such as Cc and Ct), you can specify parasitic options to calculate the capacitance from the wire geometry and from a layout.

To do this, you must first ensure you have the proper setup for using StarRC inside Custom Compiler. See [Preparing for Capacitance Reporting](#).

In the **Parameters** section of the Estimated Parasitics assistant, change the **Value** from **Specify C** to **Specify Geometry** or **From Layout** and click the **Options** button to open the **Parasitic Options** dialog box.



You can modify the instance's capacitance (specifying new values for the geometry) once it has been instantiated using the Estimated Parasitics assistant.

For descriptions of the options in the **Parasitic Options** dialog box, see About the Electrical Options Dialog Box in the *Custom Compiler Electrical Checking and Reporting User Guide* for details.

Calculating Capacitance Values and Adding Capacitance Devices

For capacitance-type parasitic devices, the **Input Method** parameter has the values listed in the following table.

Input Method is especially helpful when used incrementally. You can start with an early design by adding a device using either **Specify C** or **Specify Geometry** as the **Input Method** and then update it using **From Layout** as the **Input Method** to route the associated net to the partial (or complete) layout.

Input Method	Description
Specify C	Specify a fixed capacitance value.

Input Method	Description
Specify Geometry	Specify physical parameters, such as length, width, and metal shape that the connection should have in the layout.
From Layout	Select a net from a layout design and use its capacitance value (described below).

You can use the Estimated Parasitics assistant to calculate the capacitance value for a selected net from an existing partial or complete layout and add that capacitance value to the design.

To get the capacitance value for a selected net:

1. Begin by opening the Estimated Parasitics assistant. In a schematic view, choose **Window > Assistants > Estimated Parasitics**.
2. Choose **Ct**, **Cg**, or **Cc** from the **Device Type** list.
3. In the **Parameters** section, choose **From Layout** for the **Input Method**. A set of parameters appears.

Estimated Parasitics

Library: All Parasitics Libraries

Instance Name:

Device Type:

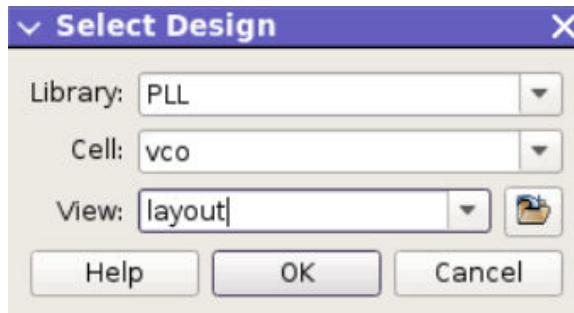
Cc
Ct
Cg
R_Metal
R_Net to Net

Add Instance Register Custom

Parameters

Prompt	Value
Input Method	From Layout
Calculate Ct	Calculate Ct
Capacitance	1 p F
Cap Per um	1 p F
Fudge Factor	☐ 0.5 ☒ 1.0 ☐ 1.5 ☐ 2.0
Circuit Cap	1 p F
Library	FilteringLib
Cell	T2_L2
View	
Select Design	Select Design
Net Name	
Select Net Using	☒ schematic ☐ layout
Select Net	Select Net
Options	Options

4. Select the **Library**, **Cell**, and **View** for the design you want to use. By default, the **Library** and **Cell** fields are autopopulated in the **Parameters** section with the same library and cell as your schematic design. Select the **View**, or click **Select Design** to open the **Select Design** dialog box and make your selections there.



5. Choose a **Fudge Factor** value for modeling how a neighboring wire's length and switch state (static, switching out of phase, or switching in phase) could affect the calculated capacitance. Select from the following values:

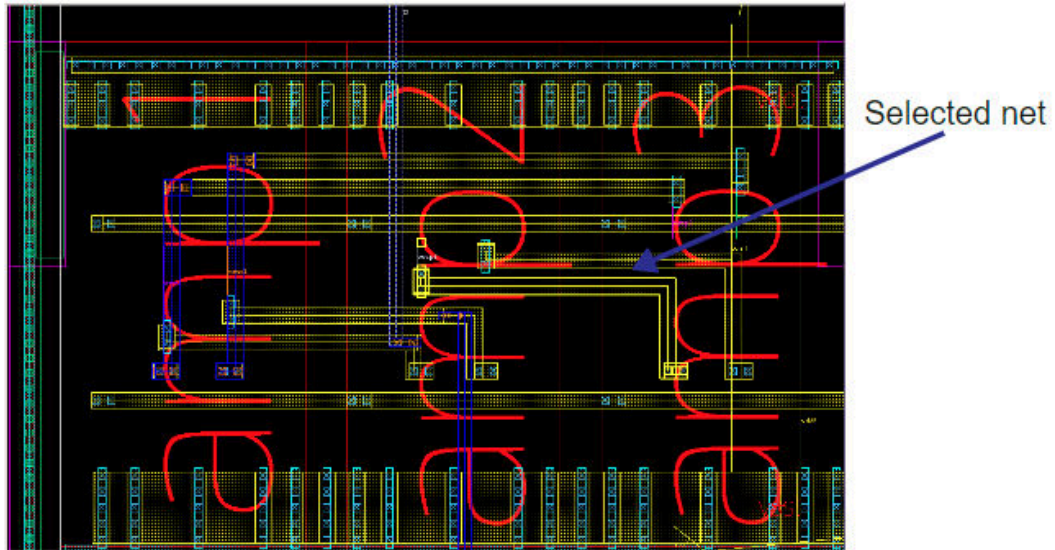
- 0.5: For in-phase switching.
- 1.0: For shielded nets (the default).
- 1.5 and 2.0: For out-of-phase switching. The Miller effect is such that if the signals are out of phase, the C values are higher. If signals are in phase, the coupling C values are lower. Use these two values to adjust C values of out-of-phase signal nets depending on the context.

6. Specify the net name. To do this manually, click in the **Value** field for **Net Name** and type the net name.

Alternatively, choose whether to select the net from the **schematic** or the **layout**, then click **Select Net** in the **Parameter Options** section.

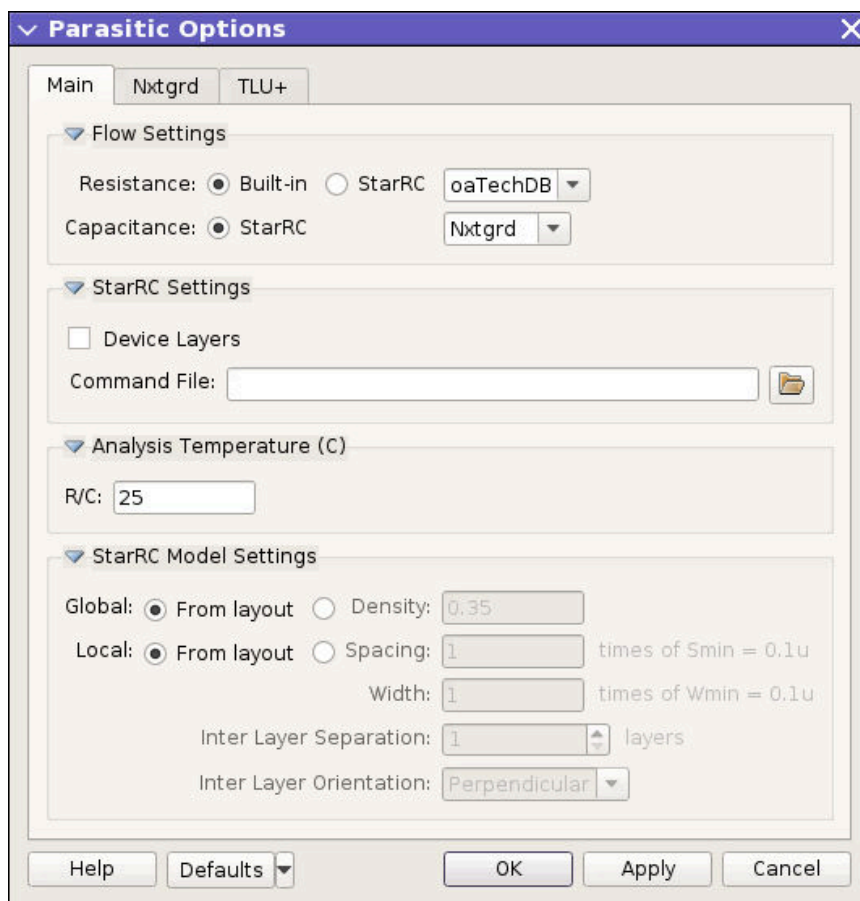
If you chose schematic, select the net from the schematic design.

If you chose layout, the Layout Editor opens. (You can ignore any design open warnings.) Select the net from the layout design.



Once you have selected the net, you are returned to the Schematic Editor and the Estimated Parasitics assistant.

7. Click **Options** in the Estimated Parasitics assistant to select parasitic options. See [Specifying Parasitic Options to Calculate Resistance](#).



For descriptions of the options in the **Parasitic Options** dialog box, see About the Electrical Options Dialog Box in the *Custom Compiler Electrical Checking and Reporting User Guide* for details.

8. Click **OK** to close the **Parasitic Options** dialog box.
9. In the Estimated Parasitics assistant, click **Calculate Ct** (or **Calculate Cc** or **Calculate Cg**) in the **Parameters** section. StarRC begins to run. When StarRC is finished, the assistant automatically calculates the capacitance-to-ground value, shown in the **Capacitance** box.

Estimated Parasitics

Library:

Instance Name:

Device Type:

- Ct
- Cg
- R_Metal
- R_Net_to_Net
- R_Ground

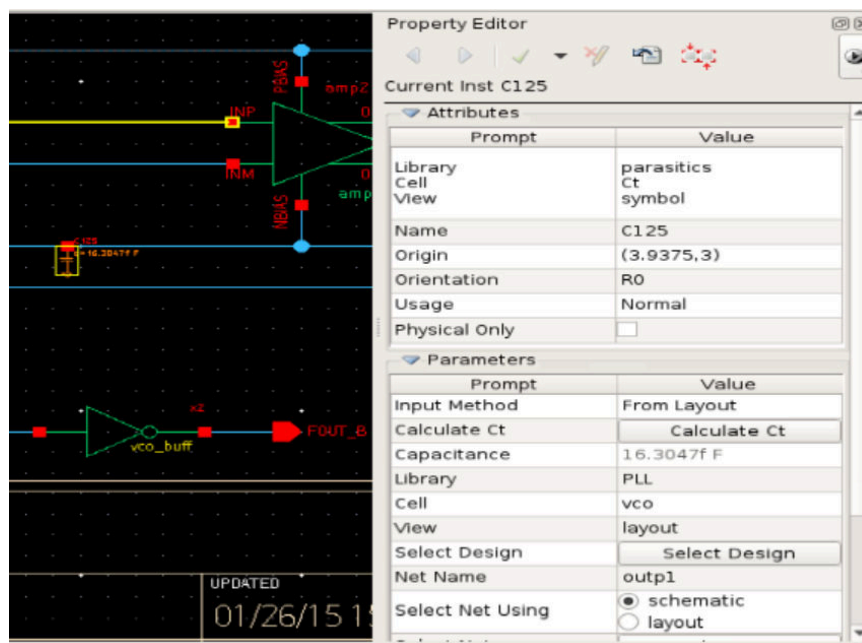
Parameters

Prompt	Value
Input Method	From Layout
Calculate Ct	Calculate Ct
Capacitance	1p F
Cap Per um	1p F
Fudge Factor	☐ 0.5 ☒ 1.0 ☐ 1.5 ☐ 2.0
Circuit Cap	1p F
Library	FilteringLib
Cell	T2_L2
View	
Select Design	Select Design
Net Name	
Select Net Using	☒ schematic ☐ layout
Select Net	Select Net
Options	Options

In addition to the total capacitance, the capacitance per unit length (**Cap Per um**) is also shown in the **Capacitance** box.

- (Optional) Name the instance you plan to place by filling in the **Instance Name**.
- Click **Add Instance** to place the instance in the design. The instance attaches to your cursor. Click in the canvas to place the instance. Press Esc to exit the **Add Instance** command.
- Close the Estimated Parasitics assistant.
- With the new instance selected in the schematic design, open the Property Editor (**Window > Assistants > Property Editor**) and observe that the capacitance

parameters generated in the Estimated Parasitics assistant are now included in the design.



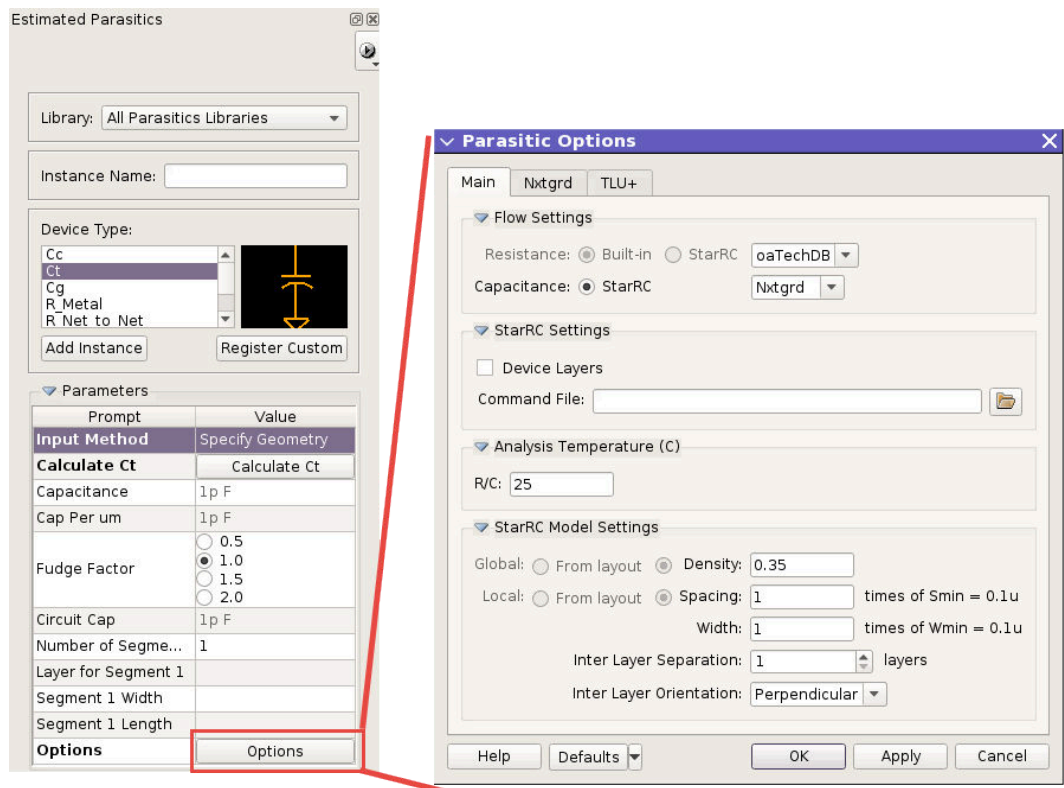
You can make changes to the net selection parameters in the layout design or directly in the Property Editor and click **Calculate Ct** to recalculate the net capacitance value.

Specifying Parasitic Options to Calculate Resistance

For R_metal devices, you can specify parasitic options to calculate the resistance from the wire geometry.

Before you start, see Preparing Data for Checking Resistance.

In the **Parameters** section of the Estimated Parasitics assistant, change the **Value** from **Specify R** to **Specify Geometry** or **From Layout** and click the **Options** button to open the **Parasitic Options** dialog box.



You can modify the instance's resistance (specifying new values for the geometry) once it has been instantiated using the Estimated Parasitics assistant.

For descriptions of the options in the **Parasitic Options** dialog box, see About the Electrical Options Dialog Box in the *Custom Compiler Electrical Checking and Reporting User Guide* for details.

Calculating Resistance Values and Adding From Layout

For resistance-type parasitic devices, the **Input Method** parameter has the values listed in the following table.

Input Method	Description
Specify R	Specify a fixed resistance value.
Specify Geometry	Specify physical parameters, such as length, width, and metal shape that the connection should have in the layout.
From Layout (available for R_Metal only)	Select a net from a layout design and measure its resistance value (described below).

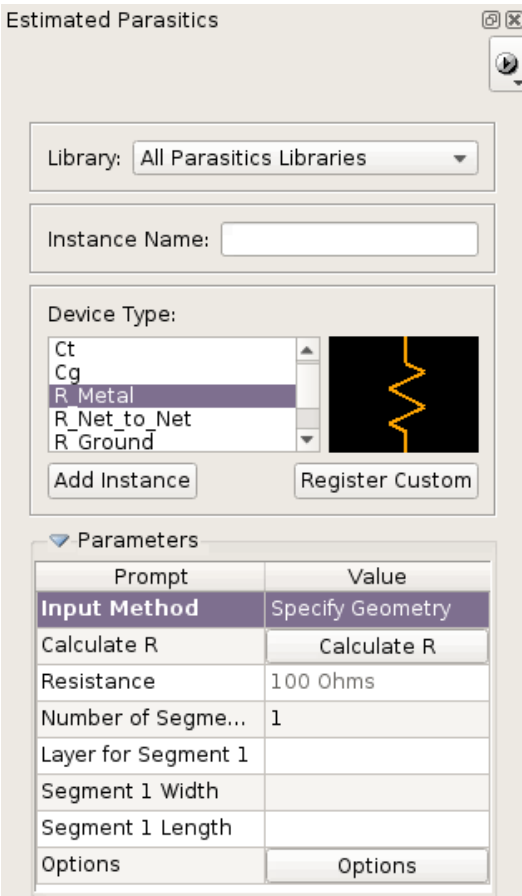
You can use the Estimated Parasitics assistant to calculate the resistance value for a selected net from an existing partial or complete layout and add that resistance value to the design.

Note:

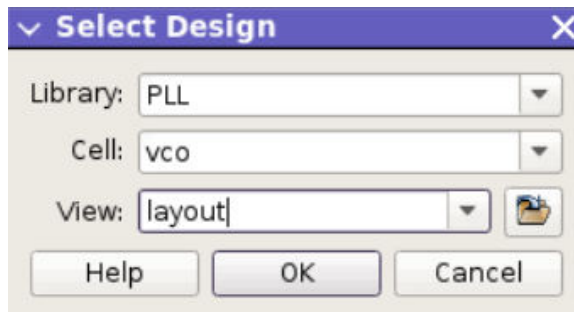
The Estimated Parasitics assistant uses the technology library of the schematic design, unless another technology library is defined via the Tcl preference `seEstimatedParasiticsTechLib`.

To get the resistance value for a selected net:

1. Begin by opening the Estimated Parasitics assistant. In a schematic view, choose **Window > Assistants > Estimated Parasitics**.
2. Choose **R_Metal** from the **Device Type** list.
3. In the **Parameters** section, choose **From Layout** for the **Input Method**. A set of parameters appears.

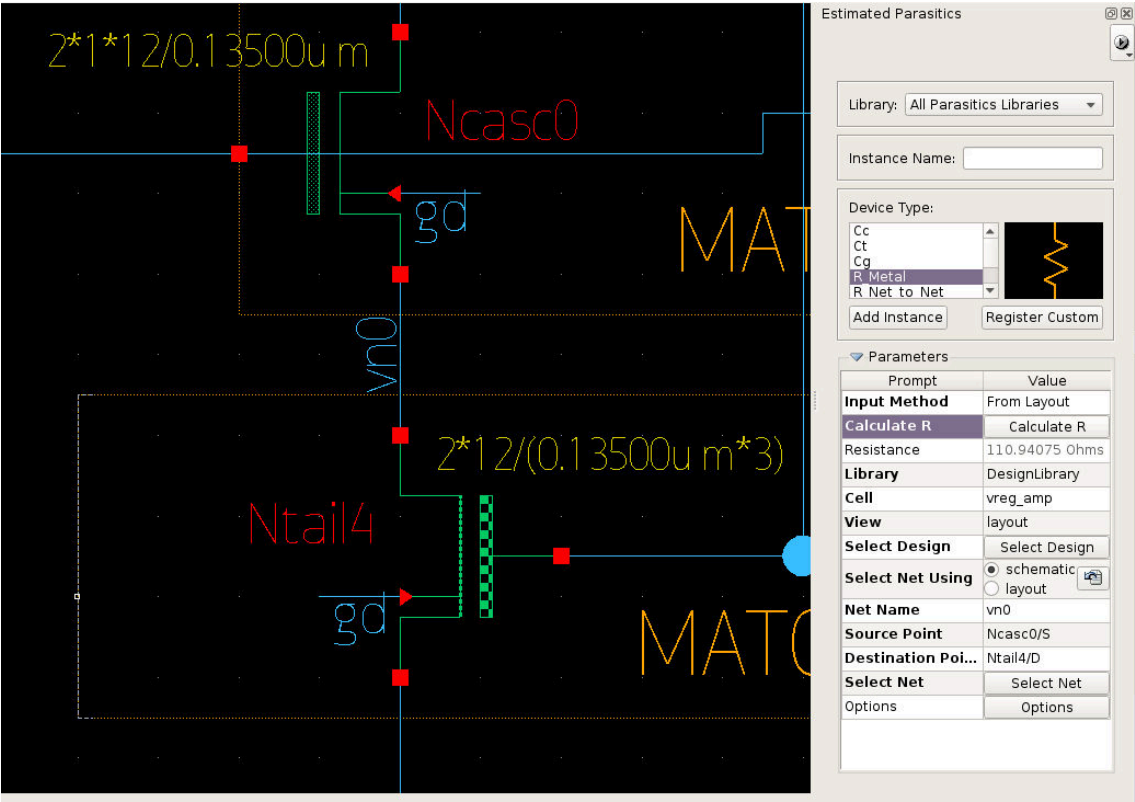


4. Select the **Library**, **Cell**, and **View** for the layout you want to use. By default, the **Library** and **Cell** fields are autopopulated in the **Parameters** section with the same library and cell as your schematic design. Select the **View**, or click **Select Design** to open the **Select Design** dialog box and make your selections there.



5. In the **Parameter Options** section, choose **Select Net Using** either **schematic** or **layout**. The selected editor opens. (You can ignore any design open warnings.)
6. If you chose **layout**, choose two points on the net in the design to select the net for which you want to calculate the resistance value.

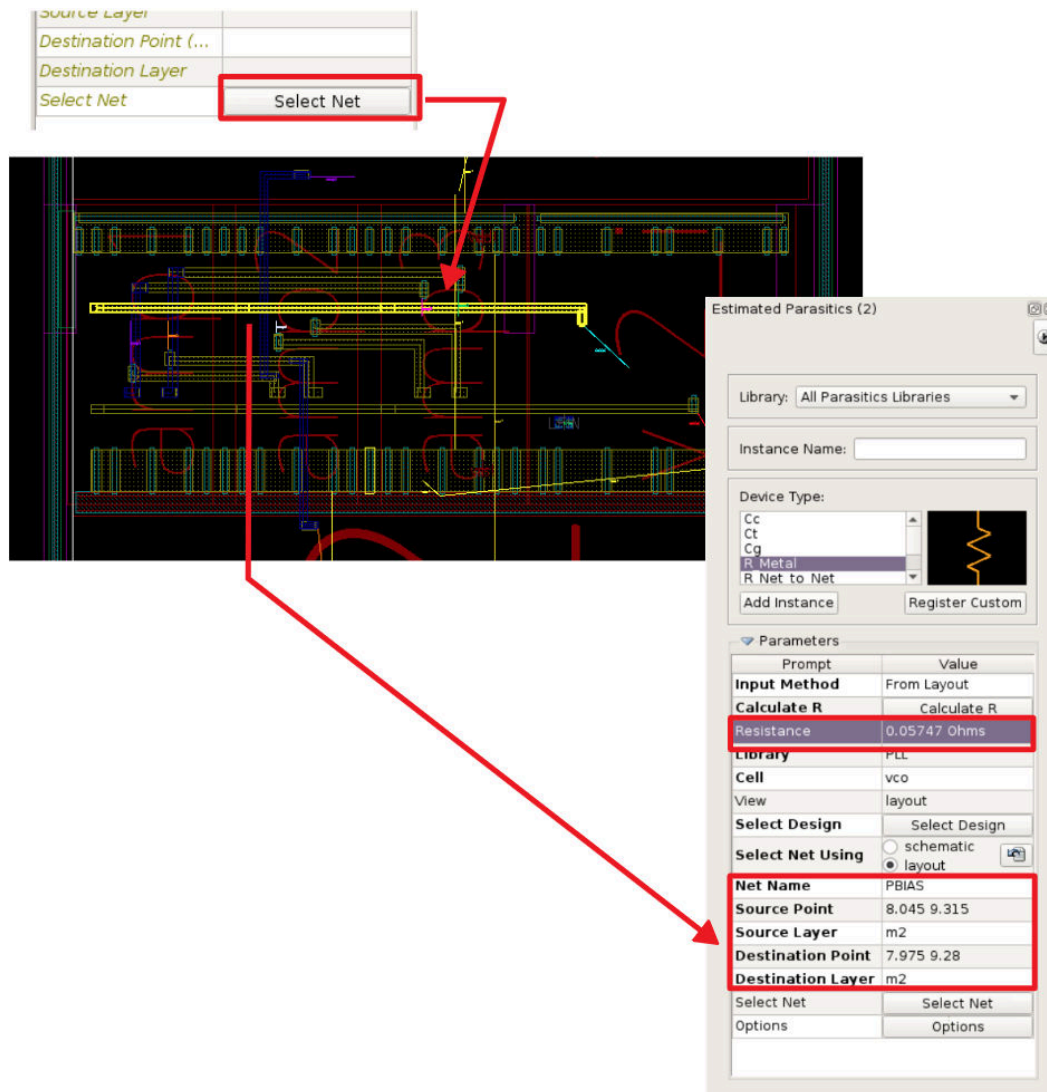
If you chose **schematic**, choose source and destination instance terminals on the same net for which you want to calculate the resistance value.



Note:

Before you can select instance terminals from the schematic, you must have performed SDL to ensure a correspondence between the layout and schematic designs.

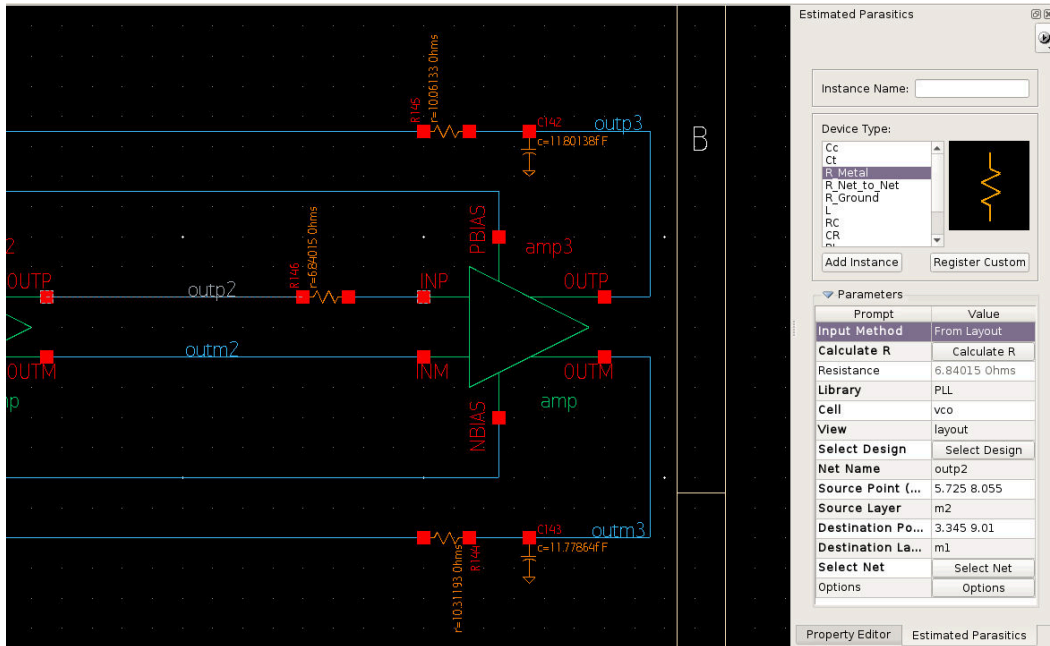
Once you have selected the second point, you are returned to the Schematic Editor and the Estimated Parasitics assistant (see figure).



In the Estimated Parasitics assistant, the net information is populated in the **Parameters** section. The assistant automatically calculates the resistance value, shown in the **Resistance** box.

7. (Optional) Add the **Instance Name**.
8. Click **Add Instance** to place the instance in the design. The instance attaches to your cursor. Click in the canvas to place the instance. When inserted on a wire, the resistance breaks the wire properly. Press Esc to exit the **Add Instance** command.
9. Close the Estimated Parasitics assistant.

- With the new instance selected in the schematic design, open the Property Editor (**Window > Assistants > Property Editor**) and observe that the resistance parameters generated in the Estimated Parasitics assistant are now included in the design.



You can make changes to the net selection parameters in the layout design or directly in the Property Editor and click **Calculate R** to recalculate the net resistance value.

Note that resistance calculation can be done using the built-in engine or the StarRC engine, depending on your selection in the **Parasitic Options** dialog box.

Calculating Pi Resistance/Capacitance Values

For the Pi and Pi2 parasitic devices, the **Input Method** parameter has the values listed in the following table.

Input Method	Description
Specify RC	Specify fixed resistance and capacitance values.
Specify Geometry	Specify physical parameters, such as length, width, and metal shape that the connection should have in the layout.
From Layout	Select a net from a layout design and measure its PiRC (or Pi2RC) value (described below).

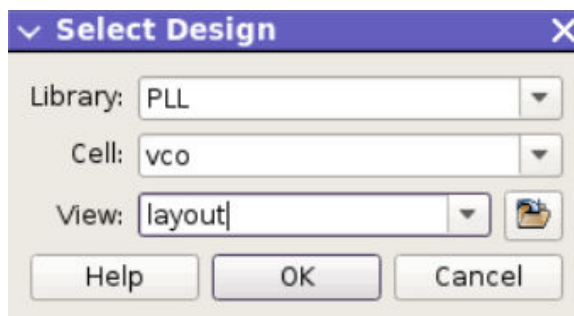
You can use the Estimated Parasitics assistant to calculate the PiRC (or Pi2RC) value for a selected net from an existing partial or complete layout and add that PiRC (or Pi2RC) value to the design.

Note:

The Estimated Parasitics assistant uses the technology library of the schematic design, unless another technology library is defined via the Tcl preference `seEstimatedParasiticsTechLib`.

To get the PiRC (or Pi2RC) value for a selected net:

1. Begin by opening the Estimated Parasitics assistant. In a schematic view, choose **Window > Assistants > Estimated Parasitics**.
2. Choose **PI** or **PI2** from the **Device Type** list.
3. In the **Parameters** section, choose **From Layout** for the **Input Method**. A set of parameters appears.
4. Select the **Library**, **Cell**, and **View** for the layout you want to use. By default, the **Library** and **Cell** fields are autopopulated in the **Parameters** section with the same library and cell as your schematic design. Select the **View**, or click **Select Design** to open the **Select Design** dialog box and make your selections there.



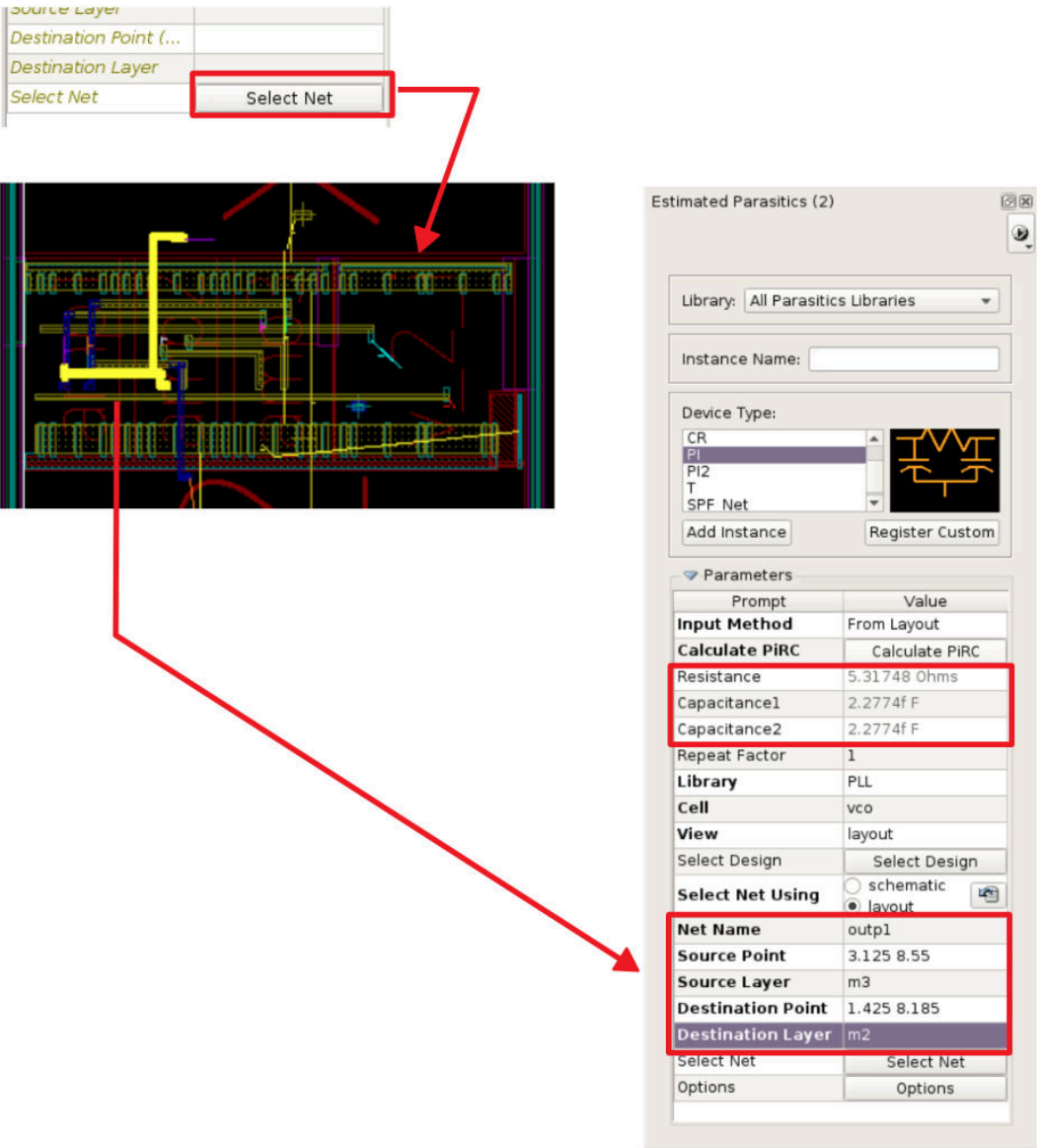
5. In the **Parameter Options** section, choose **Select Net Using** either **schematic** or **layout**. The selected editor opens. (You can ignore any design open warnings.)
6. If you chose **layout**, choose two points on the net in the design to select the net for which you want to calculate the resistance value.

If you chose **schematic**, choose source and destination instance terminals on the same net for which you want to calculate the resistance value.

Note:

Before you can select instance terminals from the schematic, you must have performed SDL to ensure a correspondence between the layout and schematic designs.

Once you have selected the second point, you are returned to the Schematic Editor and the Estimated Parasitics assistant (see figure).



In the Estimated Parasitics assistant, the net information is populated in the **Parameters** section. The assistant automatically calculates the PiRC values, shown in the **Resistance**, **Capacitance1**, and **Capacitance2** boxes.

7. (Optional) Add the **Instance Name**.

8. Click **Add Instance** to place the instance in the design. The instance attaches to your cursor. Click in the canvas to place the instance. When inserted on a wire, the resistance breaks the wire properly. Press Esc to exit the **Add Instance** command.
9. Close the Estimated Parasitics assistant.
10. With the new instance selected in the schematic design, open the Property Editor (**Window > Assistants > Property Editor**) and observe that the resistance parameters generated in the Estimated Parasitics assistant are now included in the design.

You can make changes to the net selection parameters in the layout design or directly in the Property Editor and click **Calculate PiRC** to recalculate the PiRC values.

Ignoring Estimated Parasitic Devices in a Simulation

You can run a simulation that ignores estimated parasitic devices. By default, parasitic devices are part of the schematic netlist and they can easily be filtered out. This is useful when working with the post-layout parasitic netlist if you do not want to get both estimated and post-layout parasitics at the same time during the simulation.

To ignore estimated parasitic devices:

- In the PrimeWave Design Environment, choose **Setup > Environment Options** and enable the **Ignore Estimated Parasitic Devices** option in the **Environment Options** dialog box. See Setting Environment Options.
- Alternatively, from the Custom Compiler console, choose **Export > Netlist** and enable the **Ignore Estimated Parasitic Devices** option in the **Netlister Options** tab. See Exporting Design Data in Netlist Format.

Adding EMag Devices to the Schematic After Extracting EM Data

You can use the Estimated Parasitics assistant to extract net parasitics to a S-param file containing the EM data, and then add the EMag device to the schematic with pointer to the S-param file.

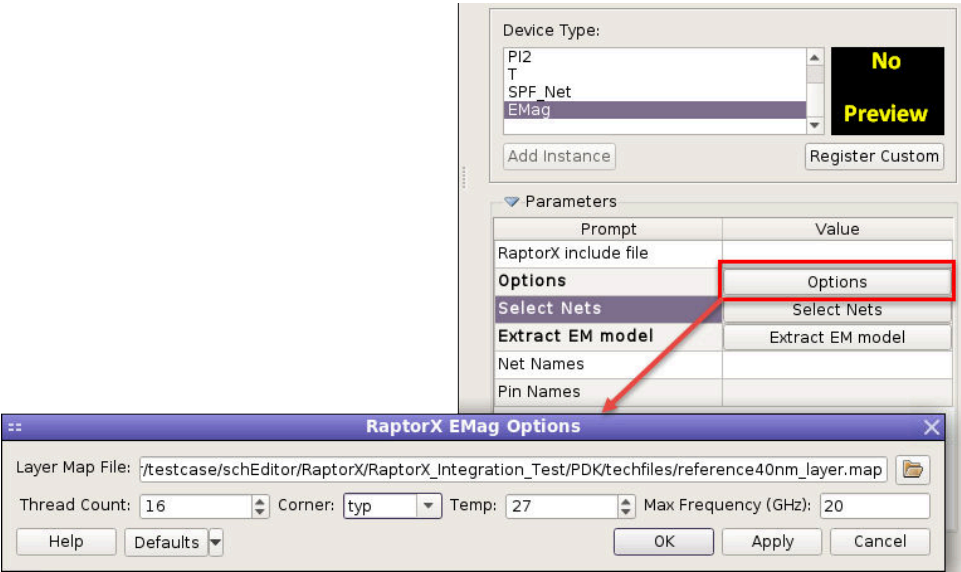
Note:

Before you start, make sure you open the schematic design in the SDL mode.

To extract the EM data for selected nets and add the EMag device:

1. Begin by opening the Estimated Parasitics assistant. In a schematic view, choose **Window > Assistants > Estimated Parasitics**.
2. Choose **EMag** from the **Device Type** list.

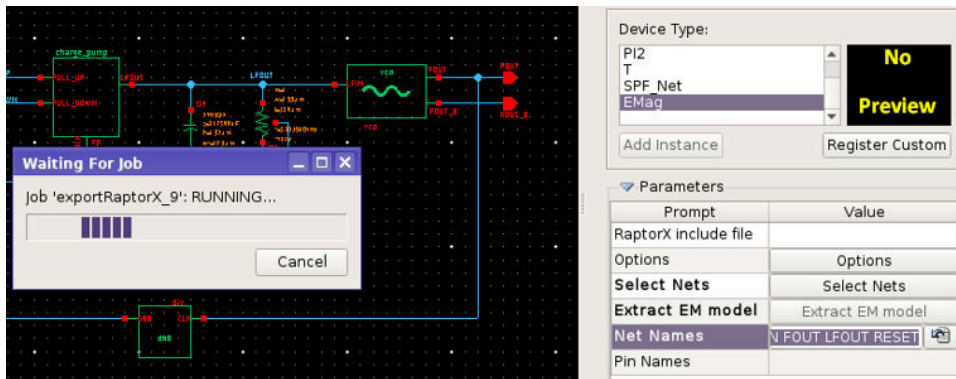
3. In the **Parameters** section, click **Options** to show the **RaptorX EMag Options** dialog box.



In the **RaptorX EMag Options** dialog box, set up the following options:

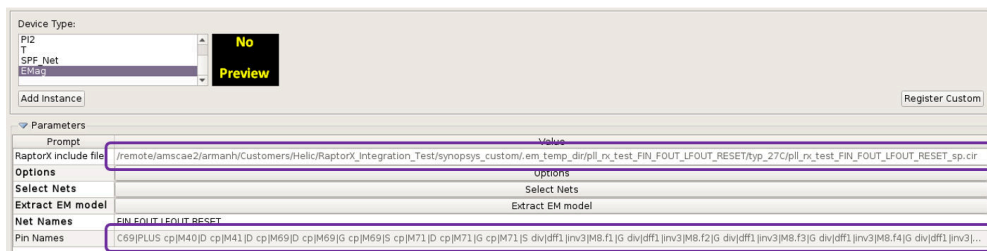
Option	Description
Layer Map File	Layer mapping file from PDK.
Thread Count	The number of possible threads for the machine.
Corner	Corner specified in the Helic PDK setup.
Temp	Extraction temperature setup.
Max Frequency	Maximum frequency for extraction.

4. Click **Select Nets** to select nets in the schematic canvas.
- (Optional) You can select more than one net by Shift-click and deselect each net by Ctrl-click.
- Selected net names are appended in **Net Names**.
5. When you are done with the net selection, click **Extract EM model** to run the extraction.
- A dialog box appears showing the progress of extraction job. Click **Cancel** if you want to cancel the job.



6. When the extraction job is completed:

- Pointer to the S-param file appears in the **EMag** device's **RaptorX Include File** field.
- Extracted ports appears in the **Pin Names** field.

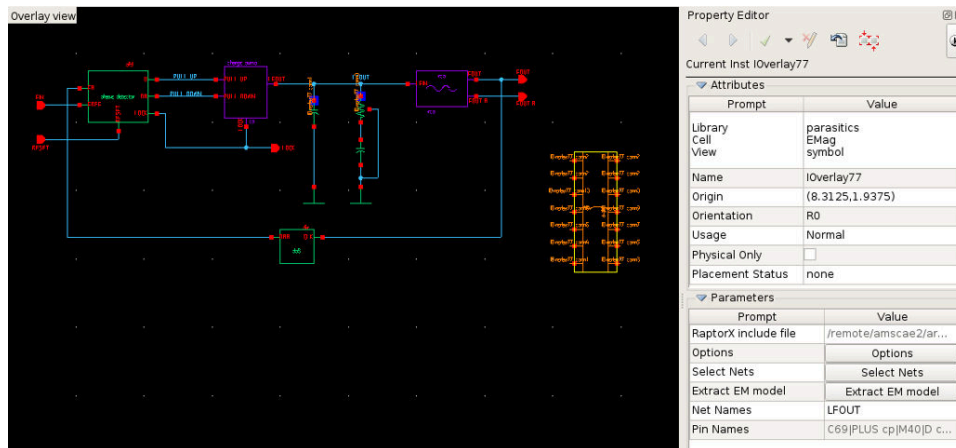


7. Click **Add Instance** to place the selected EMag device with extracted data into the schematic canvas.

The Schematic Editor creates an overlay emag design, which is available as a separate *emag* view for the same cell in the Library Manager, on the schematic canvas.



Extraction parameters are available in the Property Editor for the selected EMag device.



The EMag device represents as an overlay instance and appropriately includes netlist when netlisted.

```
.subckt pll_rx_test fin fout fin_lock reset
.include /remote/amscae2/armah/Customer/Helic/RaptorX_Integration_Test/PLL/pll_rx_test/emag/overlay/pll_rx_test/emag/pll_rx_test_LFOUT/typ_27C/pll_rx_test_LFOUT_sp.cir
xioverlay77 ioverlay77_conn1 ioverlay77_conn0 ioverlay77_conn3 ioverlay77_conn4
+ ioverlay77_conn5 ioverlay77_conn6 ioverlay77_conn7 ioverlay77_conn8
+ ioverlay77_conn9 ioverlay77_conn10 ioverlay77_conn2 ioverlay77_conn2
+ ioverlay77_conn2 ioverlay77_conn2 pll_rx_test_LFOUT
x176 net871 ioverlay77_conn0 vss! rmpoly w=1.88u l=60.8u m=1
xc69 ioverlay77_conn1 vss! nmoscap lr=1.97u wr=12.9u nf=2 nrep=2 m=1
xc70 net871 vss! nmoscap lr=1.97u wr=12.9u nf=12 nrep=2 m=1
xdiv fout net861 db8
xpfd pull_down net861 fin_lock reset pull_up phase_detector
xvco fout fout_b lfout ioverlay77_conn2 vco
xcp lfout lock pull_down pull_up ioverlay77_conn3 ioverlay77_conn4
+ ioverlay77_conn5 ioverlay77_conn6 ioverlay77_conn7 ioverlay77_conn8
+ ioverlay77_conn9 ioverlay77_conn10 charge_pump
.ends pll_rx_test
```

Marker Browser Assistant

Note:

The Marker Browser assistant only works in the Schematic Editor tool.

You can use the Marker Browser assistant to show design violations in design schematics or symbols.

The Marker Browser assistant lists all violations in the design, along with the severity of the violation. Each violation has an associated short and long message. The more detailed messages are grouped by type, and listed under the shorter messages in a tree format.

These sections cover:

- [Opening the Marker Browser Assistant](#)
- [Viewing Error Markers](#)
- [Showing Error Markers by Severity](#)
- [Showing Error Markers Added by Integrated Tools](#)
- [Updating Error Markers](#)

Opening the Marker Browser Assistant

Open the Marker Browser to work with design violations.

Some operations, such as running a schematic rules check resulting in violations, will open the Marker Browser automatically.

To open the Marker Browser assistant:

- In any schematic or symbol view, choose **Window > Assistants > Marker Browser**.

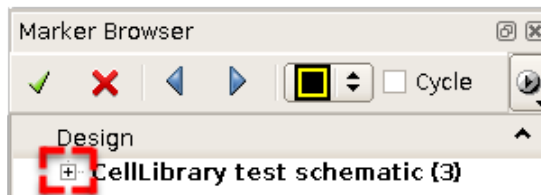
The assistant docks along the window edge. Undock it by clicking and dragging it.

Viewing Error Markers

You can view error markers associated with violations.

To view error markers:

1. Begin by opening the Marker Browser assistant. In any schematic or symbol view, choose **Window > Assistants > Marker Browser**.
2. In the Marker Browser, click the **Prev/Next**  buttons on the toolbar to navigate to the previous or next marker relative to the selected marker.
3. Click the **+** symbol next to marker groups to expand groups, and see the underlying markers in the hierarchical tree.



Click the  symbol next to marker groups to collapse groups in the hierarchical tree.

4. Click on a marker's message to highlight the marker in the design, and also show the more detailed message associated with it. Find the longer messages in the lower region of the assistant.

To show more than one marker or a marker group, use Ctrl+click and select each marker individually.

(Longer messages are shown only when single markers are selected.)

5. If the marker is not in the current design open for editing, you can optionally double-click the marker to open the design containing the marker. The system then highlights the marker in the newly opened design.

The Marker Browser displays error markers of the following types:

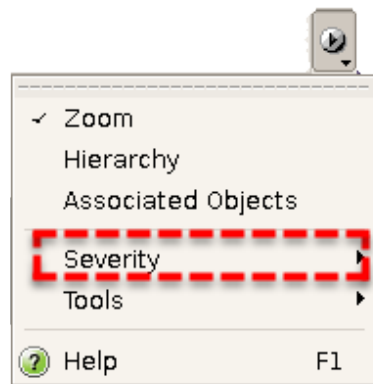
- Color shorts appear as precoloring conflict violations.
- Violations of Predefined Rules: Markers created due to the SRC/ERC violations of the rules defined in the Check Options dialog box, when this dialog box is available. (When it is available, you can open it by choosing **Check > Setup**.) You can control markers shown by severity. See [Showing Error Markers by Severity](#).
- Violations Found by External Tools: Markers created due to the violations found by external tools integrated with the Custom Compiler tool, such as a DRC engine like Hercules DRC or a parasitic extraction tool, such as StarRC. You can control the display of markers added by other tools. See [Showing Error Markers Added by Integrated Tools](#).

Showing Error Markers by Severity

You can show error markers by severity for a more targeted evaluation process.

To show error markers by severity:

1. Begin by opening the Marker Browser assistant. In any schematic or symbol view, choose **Window > Assistants > Marker Browser**.
2. From the Marker Browser toolbar menu, choose **Severity**, and then specify a severity type from the list, or **All** or **None**.



Severity types are fixed values as defined by OpenAccess:

- Error
- Warning
- Signed Off Error
- Information
- Annotation
- Fatal Error
- Critical Error
- Acknowledge Warning
- Signed Off Critical Error

Showing Error Markers Added by Integrated Tools

You can selectively show error markers added by integrated tools. You can do this to filter the violation set displayed, and limit errors shown to those errors and warnings associated with a specific tool or tools.

Use the Marker Browser to show error markers added by integrated tools.

To show error markers added by integrated tools:

1. Begin by opening the Marker Browser assistant. In any schematic or symbol view, choose **Window > Assistants > Marker Browser**.
2. From the Marker Browser toolbar menu, choose **Tools**, and specify the tool or tools from the list, or **All**, or **None**.

Markers can be generated by any tool that generates violations, and then writes them to the design's data in the OpenAccess database.

Updating Error Markers

The Marker Browser updates error markers when you perform one of the following operations:

- Checking the current view with **Check > Current View**.
- Checking and saving the design with **Design > Check and Save**.
- Changing the design lineage.

For information about general designing, see General Designing in the *Custom Compiler Environment User Guide*.

Net Tracer Assistant

You can use the Net Tracer assistant to probe nets interactively throughout the design hierarchy.

These sections cover:

- [Opening the Net Tracer Assistant](#)
- [Creating a Probe](#)
- [Viewing a Probed Net](#)
- [Setting Probe Color](#)
- [Clearing Probes](#)
- [Filtering Nets by Name](#)
- [Tracing Through Ignored Devices](#)
- [Tracing Through by Cell Configuration](#)

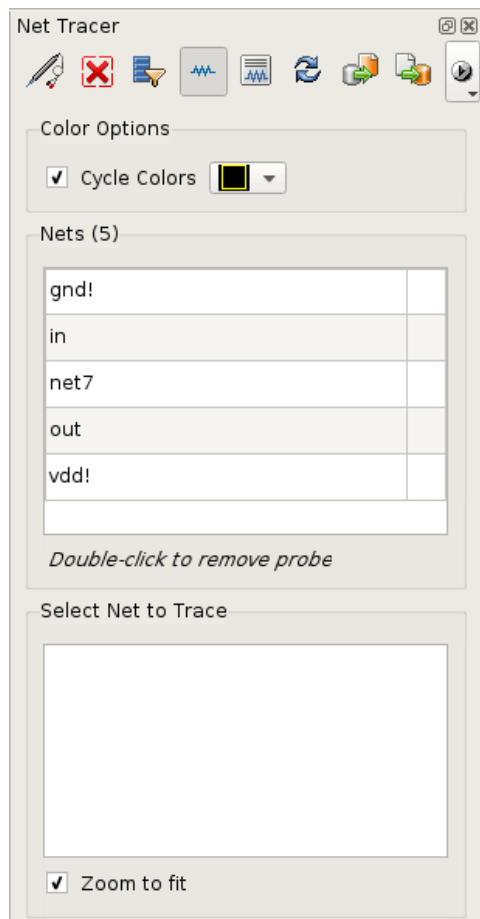
- [Saving Probe Nets](#)
- [Loading Probe Nets](#)

Opening the Net Tracer Assistant

Open the Net Tracer to work with nets.

To open the Net Tracer assistant:

- In any design window, choose **Window > Assistants > Net Tracer**.



The assistant docks along the window edge in the Schematic Editor. Undock it by clicking and dragging it.

Creating a Probe

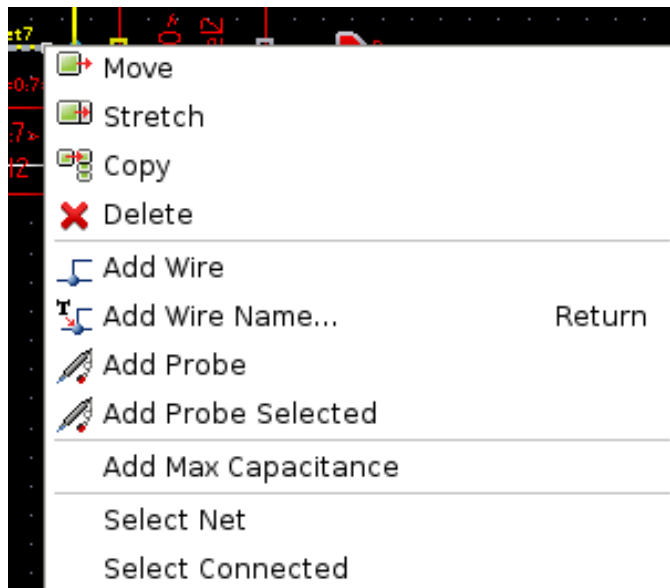
You can click on nets in the top region of the Net Tracer assistant, or select nets directly in the schematic and text editors to create a probe.

To create a probe:

1. In the Net Tracer, click **Probe nets by selecting in the design**  on the toolbar to launch `ide::createProbe` for selecting nets.
2. Click an unprobed net in the design window. As each net is selected, a probe is created, the net and its probe color are added to the **Nets** section of the Net Tracer assistant, and the **Equivalent Nets** region is updated to show the result of the selection.

In addition to creating a probe, clicking an unprobed net in the design window populates the Net Tracer results with the list of electrically connected nets.

To create a probe for an active object, hover on a net on the canvas and then right-click to choose **Add Probe**. Alternatively, select more than one net on the canvas and then choose **Add Probe Selected** to probe multiple objects.



Right-clicking on empty space in `ide::createProbe` shows the context-sensitive menu with all actions disabled.

Viewing a Probed Net

You can browse to the associated design editor window to view a probed net.

To view a probed net:

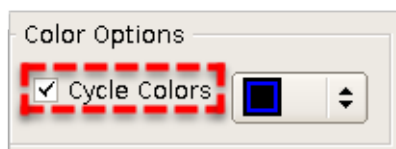
- In the Net Tracer, double-click a net in the **Equivalent Nets** section. If **Zoom to fit** is selected, the design editor is sized to view the extent of the probed net.

Setting Probe Color

You can choose the color of the next probe to be created.

To set probe color:

1. In the **Color Options** section, click the color selector arrow to access the drop-down menu of predefined highlight sets.
2. The **Color Options** section always displays the highlight set for the next created probe. If **Cycle Colors** is checked (the default), the current probe color is changed after each probe is created.



Clearing Probes

You can clear an individual probe you created with the Net Tracer.

To clear a probe:

- Double-click the net name in the **Nets** section of the Net Tracer assistant.

You can also clear all probes at once.

To clear all probes:

- Click **Clear all probes**  on the toolbar.

Filtering Nets by Name

You can filter nets by name with the Net Tracer.

To filter nets by name:

- Click **Filter**  on the toolbar.

This adds a **Filter** box above both the Nets and the Equivalent Nets tables. These filter boxes can be used to limit and search through their respective tables. Clicking **Filter** a second time hides the filter boxes and updates the tables to show their full contents. Note that the equivalent nets are only shown for the currently selected net.

When filtering is turned on, **Filter**  is shown sunken to indicate the state. For filtering settings, refer to the **Interface** tab in the **General Options** dialog box, described in Setting General Options.

Tracing Through Ignored Devices

By default, the Net Tracer passes through ignored devices.

- Click **Trace through ignored devices**  on the toolbar to toggle this setting on and off. When toggled, all the existing probes are removed, and only the probe on the currently selected net (if it existed before) is re-created with the new setting.

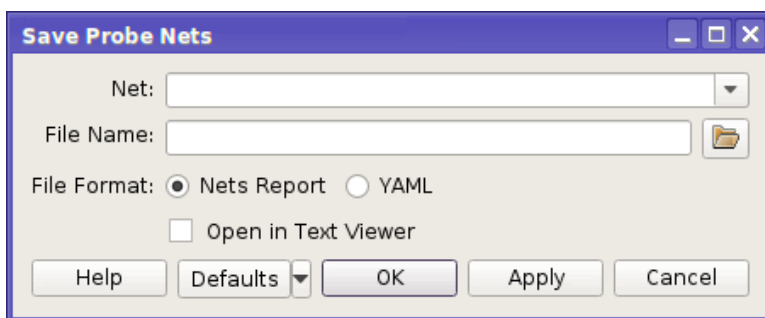
Tracing Through by Cell Configuration

- Click **Trace through by cell configuration**  on the toolbar to toggle this setting on and off. When toggled on, the Net Tracer assistant traces through cell configurations according to the settings in the **Check Cells** cell configuration table under General Options dialog box **Schematic > Probe** tab.

Saving Probe Nets

To save probing results:

1. Begin by opening the Net Tracer assistant. In any design window, choose **Window > Assistants > Net Tracer**.
2. From the Net Tracer toolbar, click **Save Probe Nets**  on the toolbar to save the probing results of a specified net or all nets (denoted by *) in **Nets Report** or **YAML** format using the **Save Probe Nets** dialog box. Note that only the YAML format is reloadable.

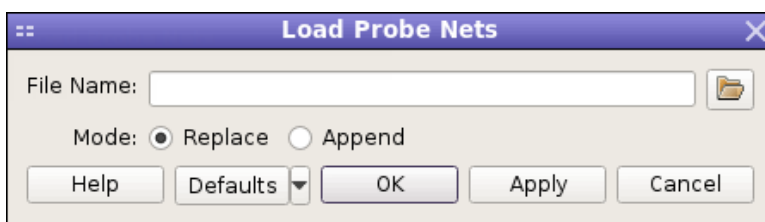


3. Enable **Open in Text Viewer** to open the net information in a text viewer after saving the file.
4. Click **OK** to close the **Save Probe Nets** dialog box.

Loading Probe Nets

To load probing results:

1. Begin by opening the Net Tracer assistant. In any design window, choose **Window > Assistants > Net Tracer**.
2. From the Net Tracer toolbar, click **Load Probe Nets**  on the toolbar to load the probing results from a **YAML** file using the **Load Probe Nets** dialog box.



3. When the **Replace** (default) mode is enabled, removes all probes when loading probing results to the Probe Assistant.
4. Click **OK** to close the **Load Probe Nets** dialog box.

Template Manager Assistant

This topic describes the following:

- [About the Template Manager](#)
- [Displaying the Template Manager Assistant](#)

- [Finding Template Matches](#)
- [Binding Templates to Constraints](#)
- [Managing Bound Templates](#)
- [Recognizing Built-In Circuit Structures in Template Cells](#)
- [Adding Notes to Template Matches](#)

About the Template Manager

The Template Manager assistant in the Schematic Editor allows the schematic user to find matches for template cells that already have layout information associated with them (for example, a set of layout shapes/polygons), or that have constraint information associated with them. By binding these template cells in the Schematic Editor using the Template Manager assistant, the information is automatically fed forward to the layout user in the next SDL session. Hence, the schematic designer is effectively instructing the layout designer how to do the layout in an efficient flow of information through template constraints.

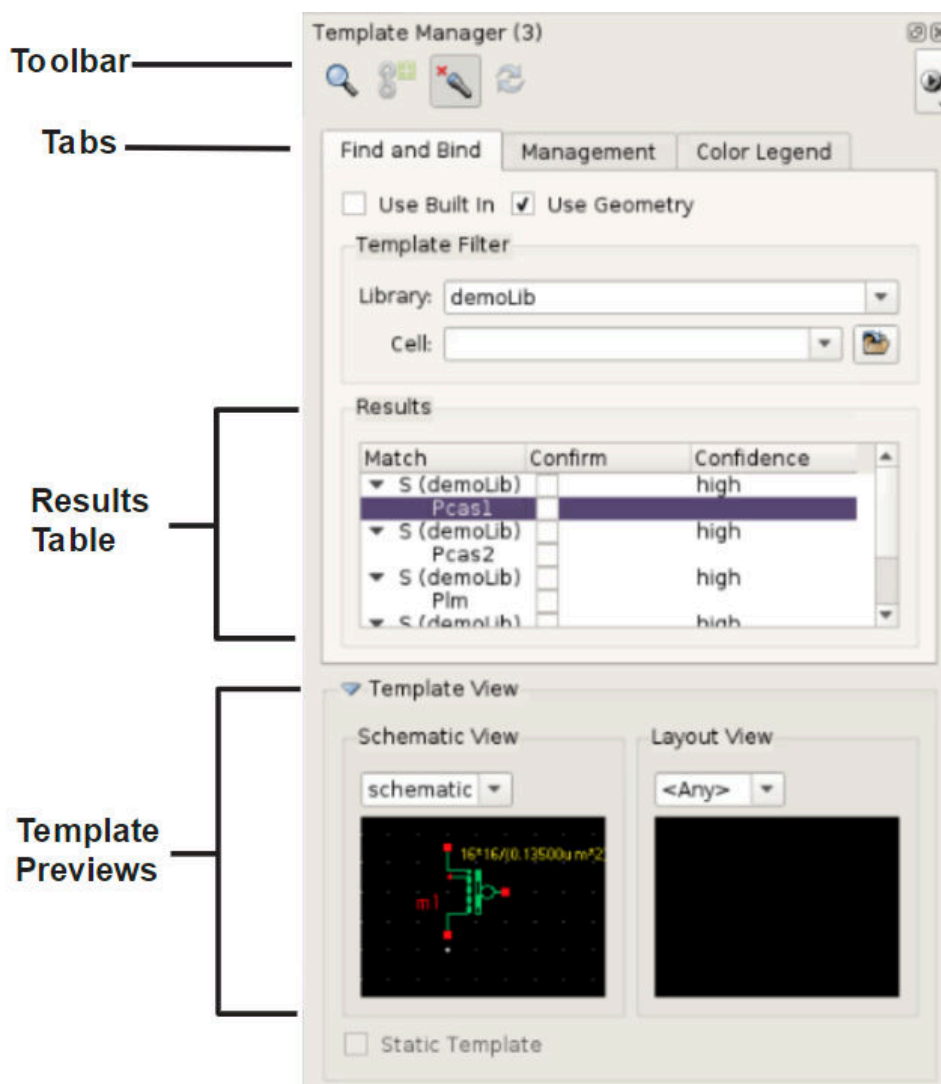
For information on Using Templates and the Template Manager, see the *Custom Compiler SDL and Visually-Assisted Automation Guide*.

You use the Template Manager in the Schematic Editor to:

- Find matches from templates in template libraries
- Bind template matches to template intent constraints
- Use template intent constraints from the Schematic Editor to drive an SDL layout (template intent constraints are automatically propagated into the next SDL session)
- Show which template structures in schematic are already bound to constraints
- Delete template bindings/constraints
- Recognize circuit structures and identify known analog building blocks
- Identify templates on the schematic, color-coded according to basic functionality

For more information about templates in the Schematic Editor, see [Creating Templates](#).

The Template Manager consists of the following:

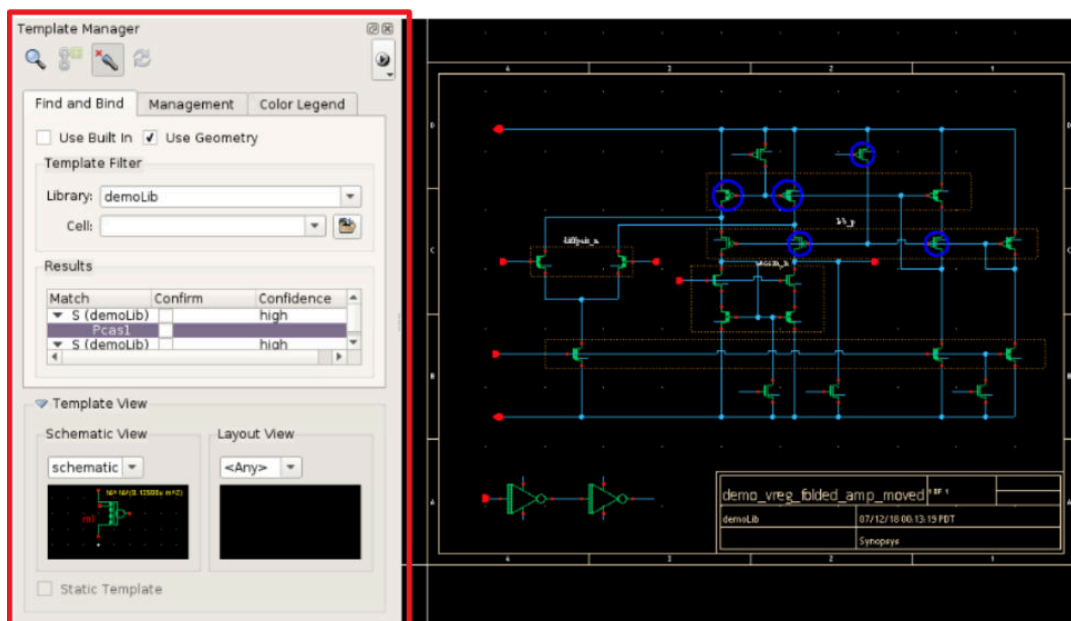


Displaying the Template Manager Assistant

To display the Template Manager assistant:

- From any schematic design, choose **Window > Assistants > Template Manager**.

By default, the Template Manager appears on the left side of the design window. You can undock and move it by dragging it to another location.



Finding Template Matches

To find template matches from the **Find and Bind** tab:

1. Enter a template filter **Library** name.
2. (Optional) Enter a template filter **Cell**.
3. (Optional) Uncheck **Use Geometry** to search the entire schematic design without filters.

By default, **Use Geometry** is selected to filter search results by geometrical alignments, such that for two-transistor matches of known built-in circuit structures, a match is discarded if the two transistors are not either horizontally or vertically aligned in the schematic design.

Note:

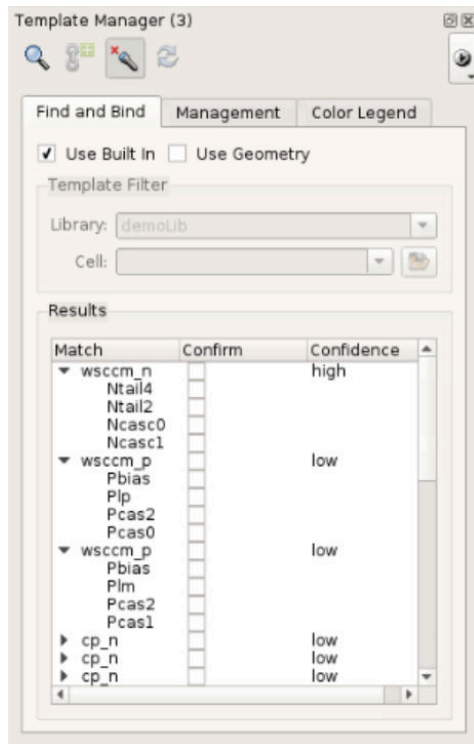
Unchecking **Use Geometry** could result in many false positive matches.

4. Click **Search**  on the toolbar.

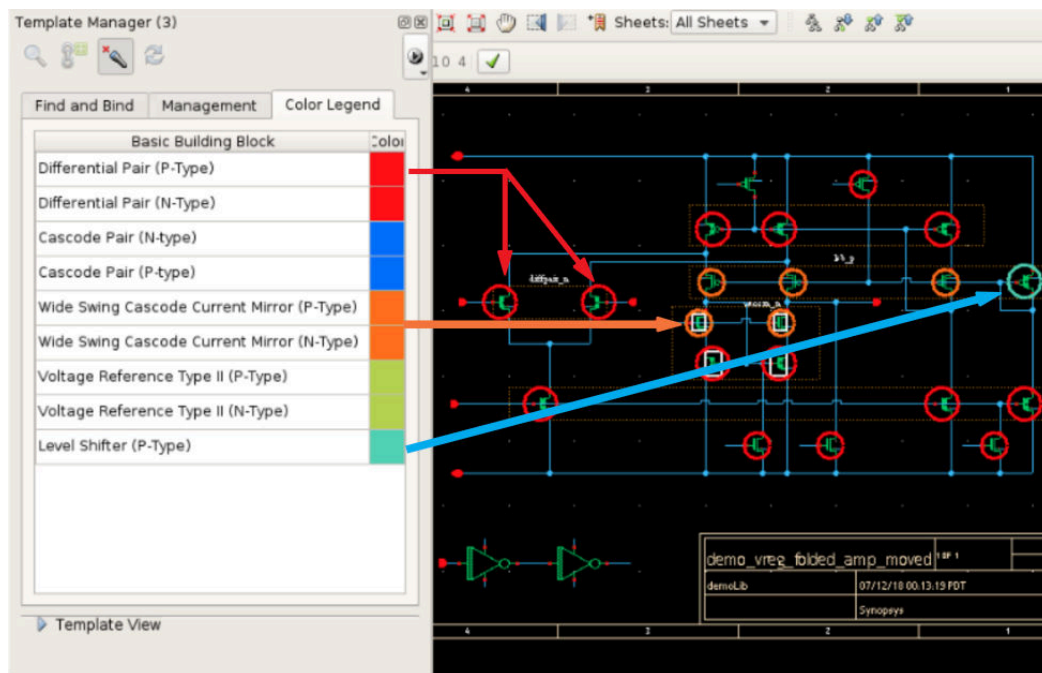
Results are displayed in the Results table on the **Find and Bind** tab.

Chapter 13: Assistants for Schematic Designing

Template Manager Assistant



Color coding appears in the schematic design according to underlying circuit functionality, as shown in the **Color Legend** tab.



5. (Optional) Check **Confirm** in the Results table next to matches you want to confirm are indeed matches. Any conflicting matches are automatically removed.

Caution:

The confirm operation cannot be undone.

6. (Optional) Remove any unwanted matches in the Results table by selecting the row for that match, right-clicking, and choosing **Remove Match**.

7. (Optional) Hide color annotations on the canvas by clicking **Hide**  on the toolbar.

8. (Optional) Show color annotations on the canvas again by clicking **Annotate**  on the toolbar.

Binding Templates to Constraints

To bind a template matching result to a template intent constraint:

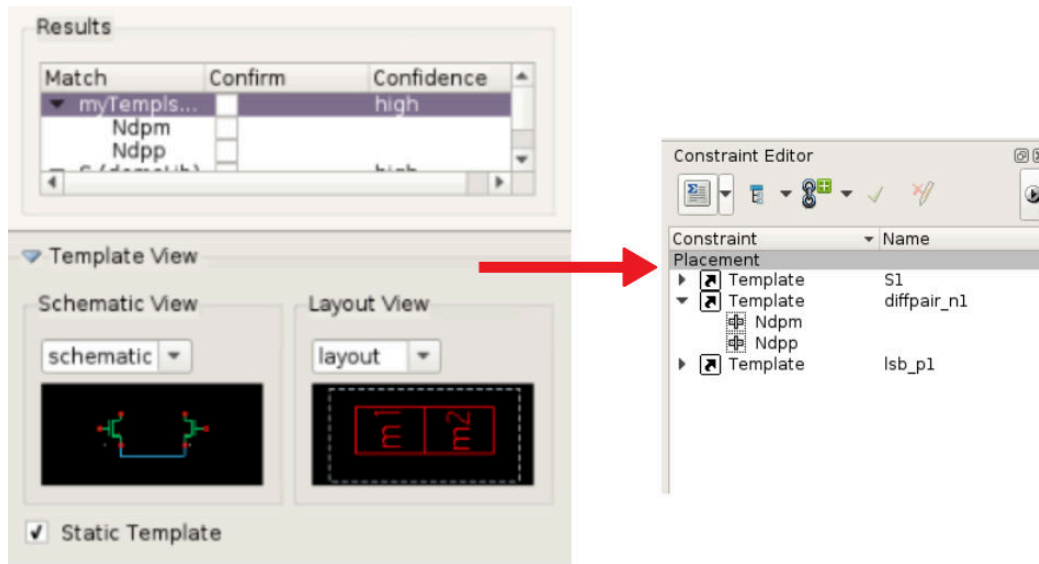
1. On the **Find and Bind** tab, select a template matching result from the Results table.
2. Expand the **Template View** section to see the schematic and layout previews.
3. Select a **Layout View** (the default is **<Any>**).

Selecting a **Layout View** of **<Any>** for the bind operation creates a template intent constraint that specifies only the template library and cell (LC pair). Selecting an actual **Layout View** for the bind operation creates a template intent constraint that specifies the template Library/Cell/View (LCV). The view can actually be a layout constraint view that indicates how the design should be laid out in SDL.

4. (Optional) If you selected an actual layout view and not **<Any>**, check **Static Template** to make this a static template.

5. Click **Create Template Intent Constraint**  on the toolbar.

A template intent constraint is automatically created and appears in the Constraint Editor (**Window > Assistants > Constraint Editor**), as shown in the following figure.

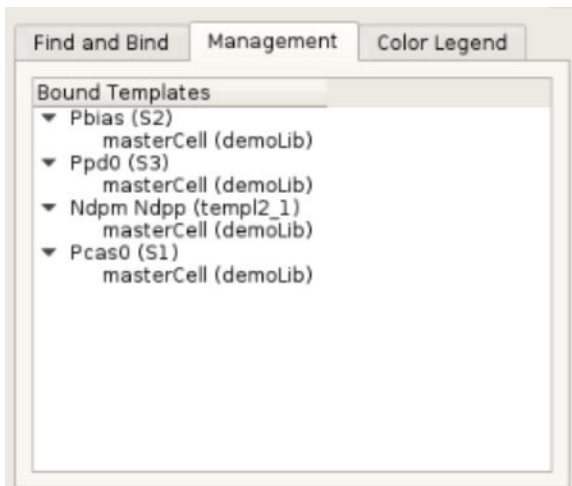


Such template constraints automatically transfer to a subsequent SDL session. For information about Introduction to SED, SDL and Visually Assisted Automation, see the *Custom Compiler SDL and Visually-Assisted Automation Guide*.

The selected template match disappears from the Results table in the Template Manager assistant. The Results table only shows unbound template matches.

Managing Bound Templates

The **Management** tab displays all template intent constraints in the Bound Templates table.



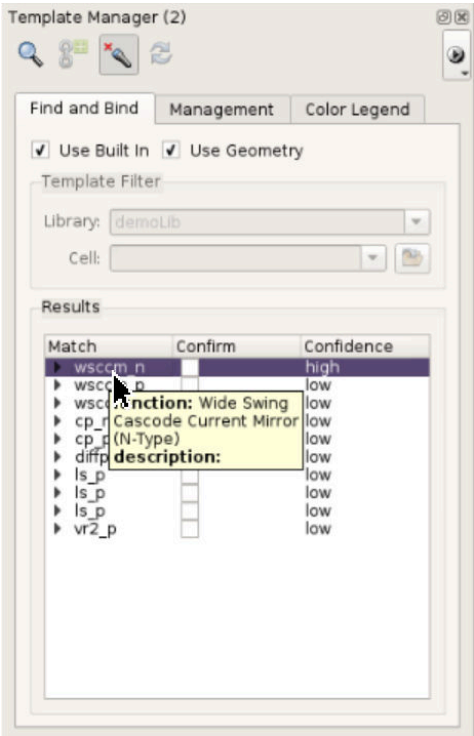
To unbind a template from its template intent constraint, select a row in the Bound Templates table, right-click, and choose **Unbind**.

Recognizing Built-In Circuit Structures in Template Cells

Instead of entering template filters, you can turn on **Use Built In** on the **Find and Bind** tab to find matches of known, built-in, hard-coded circuit structures such as diffpairs, current mirrors, level shifters, and so forth. This does not require a template library.

To find template matches with built-in circuit structures:

1. Follow the procedure in [Finding Template Matches](#), but instead of entering template filters for **Library** and **Cell**, check **Use Built In** before clicking **Search**  on the toolbar.
2. Select a match in the Results table to cross-select it on the schematic canvas.
3. Hover over any result in Results table to see a tooltip describing the function, as shown in the following figure. The Schematic Editor also recognizes built-in circuit structures in your template library matches.



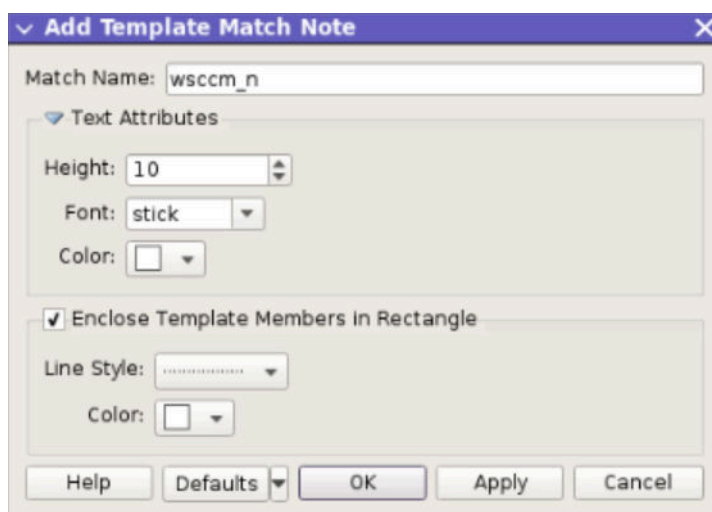
When **Use Built In** is checked, color coding (shown in the **Color Legend** tab) still represents the underlying template functionality; unrecognized circuit structures have no associated colors.

Adding Notes to Template Matches

You can add a note to template matches.

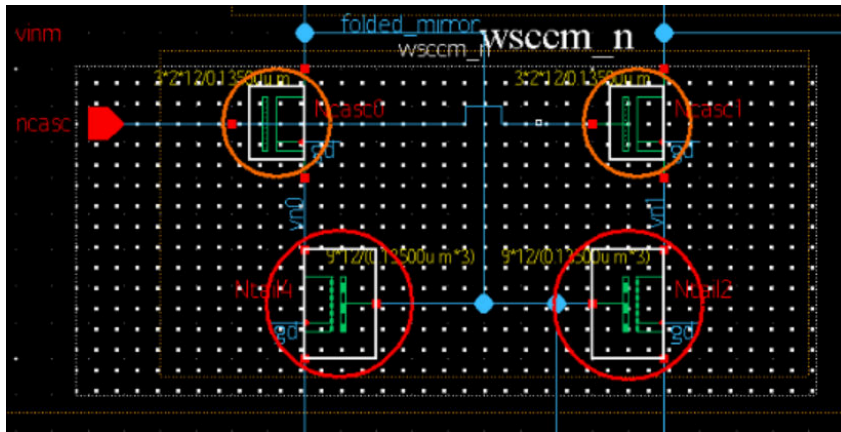
To add a note:

1. Select the template match in the Results table.
2. Right-click and choose **Add Note**. The **Add Template Match Note** dialog box opens.



3. (Optional) Enter the **Match Name**. By default, this is the template function, if known, or the name of the cell, if the template function is not known.
4. (Optional) Adjust the **Text Attributes** if necessary.
5. (Optional) Select **Enclose Template Members in Rectangle** to draw a rectangle around the template instances. Adjust the **Line Style** and **Color** of the rectangle if necessary.

- Click **OK** to create the template match note. The note is displayed in the schematic design.



14

Analyzing and Debugging Schematic Designs

Use design tools and utilities to analyze and debug designs. Design tools and utilities include editors and features that operate independently.

This chapter covers instructions for using the Schematic Editor design tools and utilities.

- [Deciding Which Tool or Utility to Use](#)
- [Annotate Operating Point Utility](#)
- [Compare Designs Utility](#)
- [Connections Navigator](#)
- [Net Colorization Definitions Editor](#)
- [Plot Signal Utility](#)
- [Power Domain Analyzer Utility](#)
- [Update Estimated Parasitics Utility](#)

Deciding Which Tool or Utility to Use

The following table lists tools and utilities in alphabetical order and describes how each helps with analyzing or debugging designs.

Tool	Design Development
Annotate Operating Point Utility	Allows you to annotate operating points onto the schematic design in the Schematic Editor.
Bookmark Manager Tool	Sets new bookmarks, and loads and saves existing bookmarks. For information, see the <i>Custom Compiler Environment User Guide</i> .
Compare Designs Utility	Compares the differences in logic between two schematic designs.

Tool	Design Development
Connections Navigator	Analyzes designs to determine the actual connectivity when parameterized connections are used in the design.
Display Resource Editor Tool	Assigns display resources; creates display colors, stipples, and line styles; loads and merges display resource files; defines packets and packet aliases. For information, see the <i>Custom Compiler Environment User Guide</i> .
Job Monitor Tool	Tracks running job processes. For information, see the <i>Custom Compiler Environment User Guide</i> .
Library Manager Tool	Creates and manages libraries, cells, cellviews, and cell categories. For information, see the <i>Custom Compiler Environment User Guide</i> .
Net Colorization Definitions Editor	Allows you to create color definitions for coloring nets in the Schematic Editor.
Parameter Definition Editor Tool	Displays, defines, and sets values for parameters. For information, see the <i>Custom Compiler Environment User Guide</i> .
Plot Signal Utility	Plots signals in the Schematic Editor that are simulated outside of the PrimeWave Design Environment.
Power Domain Analyzer Utility	Allows you to see all the global net connections throughout the hierarchy.
Search and Replace Tool	Finds and optionally replaces components in a design. For information, see the <i>Custom Compiler Environment User Guide</i> .
Technology Manager Tool	Imports and exports technology data, technology files; resets referenced technologies. For information, see the <i>Custom Compiler Environment User Guide</i> .
Text Viewer Tool	Displays text output for all running jobs that produce it. For information, see the <i>Custom Compiler Environment User Guide</i> . (If you are looking for information about text views of designs, and not text views of job processes, see instead Editing HDL Formats With the Text Editor .)
Update Estimated Parasitics Utility	Recalculates the R/C values automatically when there is a technology change.

Annotate Operating Point Utility

The Annotate Operating Point utility allows you to annotate operating points onto the schematic design in the Schematic Editor.

The first step toward debugging analog circuits is to display operating point information for circuit components. Operating point information depends on the simulator as well as the device being used. For MOSFET devices, operating points could include, for example:

- operation region
- threshold voltage (vth)
- saturation voltage (vdsat)
- drain current (id)
- transconductance (gm)

For resistors, operating points could include values for:

- current
- voltage drop
- power
- effective resistance

You can use the functionality provided for back-annotating operating points from simulation results. Each device master must include one or more `cdsParam` interpreted labels for the display of operating points. The number of labels in the placed master determine the number of available slots for annotation. The default operating point parameters to display are defined in the CDF property `opPointLabelSet`.

You can use the Device Label Editor to override the default `opPointLabelSet`. For devices that use primitive models (no subcircuits), this capability usually works without any changes to the default component description format information.

The subcircuit models contain a primary instance, which represents the component in the circuit. The process of back-annotation requires accessing the operating points of a device that is one or more levels down in the results hierarchy.

The following example of N-channel MOSFET *macromodel* device adds a `scale` parameter and uses this parameter to calculate the width and length of the `nch_mac` device.

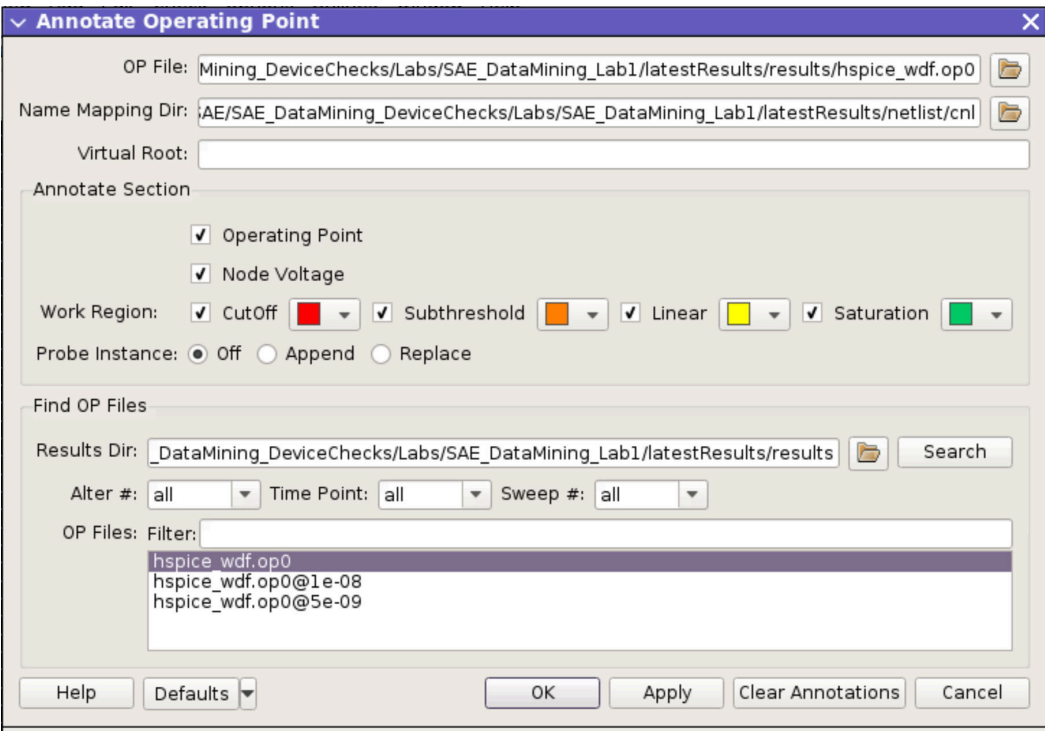
```
.subckt nch_mac n1 n2 n3 n4 l=length w=width multi='1' nf='1'  
  scale='scale_mos'  
main n1 n2 n3 n4 nch w='w*scale' l='l*scale' m=multi
```

```
.ends
.model nch NMOS level=54
```

After simulation, operating point information is available for the primitive device main, but not for the nch_mac subcircuit. The information is stored as XY.main:z, where XY is the instance name that calls nch_mac, and z is the operating point parameter. For example, X1.main:gm will be the transconductance of the instance X1 that uses nch_mac as its model. One way to solve a macro model mismatch is to use simulation options such as .FLAT or .AUTOFLAT. These options help the simulator flatten the macro device and store the operating point information at the same level as a macro definition. (Refer to the documentation for the simulator you are using for detailed usage information about these simulation settings.)

To use the Annotate Operating Point utility to back-annotate operating points on the schematic design:

- 1. Choose **Utilities > Annotate Operating Point** from the Schematic Editor main menu bar. The **Annotate Operating Point** dialog box appears.



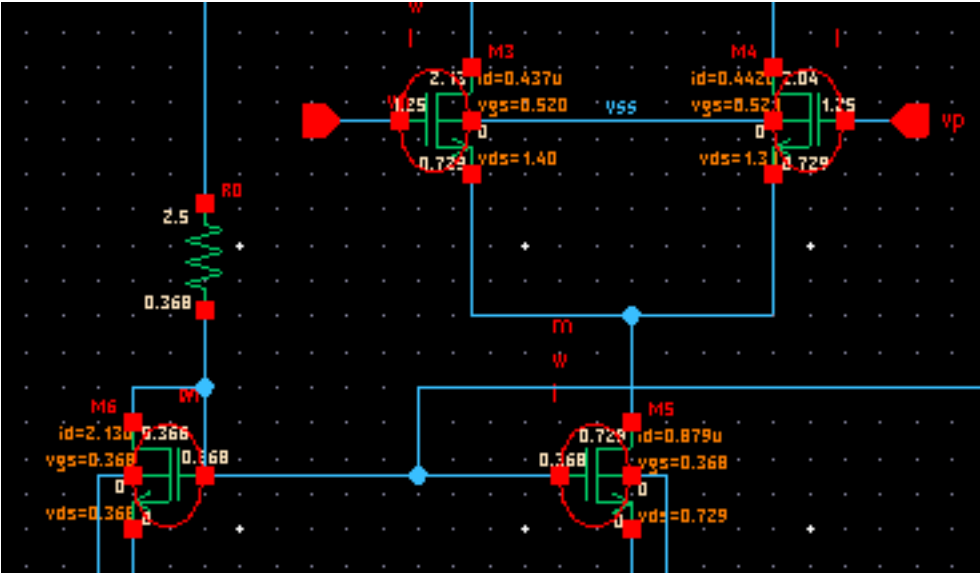
The **Annotate Operating Point** dialog box has the following options:

Option	Description
OP File	Defines the path to the operating point file.

Option	Description
Name Mapping Dir	Defines the path to the name mapping directory.
Virtual Root	Allows you to set a virtual root, which enables a flow where your design is in a different hierarchy than its testbench. This is useful, for example, to create an instance for the top design in the testbench.
Annotate Section	
Operating Point	When selected, operating points are annotated to the schematic design.
Node Voltage	When selected, node voltages are annotated to the schematic design.
Work Region	Defines the work region to annotate. You can select different highlight colors for each region. Available regions are: CutOff , Subthreshold , Linear , and Saturation .
Probe Instance	When set to Append or Replace , opens the Instance OP dialog box when you click an instance in the design. This allows you to see all of the parameters of an instance at once rather than the limited set visible on the schematic design. (To change the parameters being annotated, use the Device Label Editor assistant.) Options are: Off (do not open the Instance OP dialog box when an instance is clicked), Append (append new data to the Instance OP table), Replace (replace data in the Instance OP table).
	
Find OP Files	Create condition filters in this section of the dialog box to target OP files quickly.
Results Dir	Defines the path to the results directory. You can browse to search for a results directory.

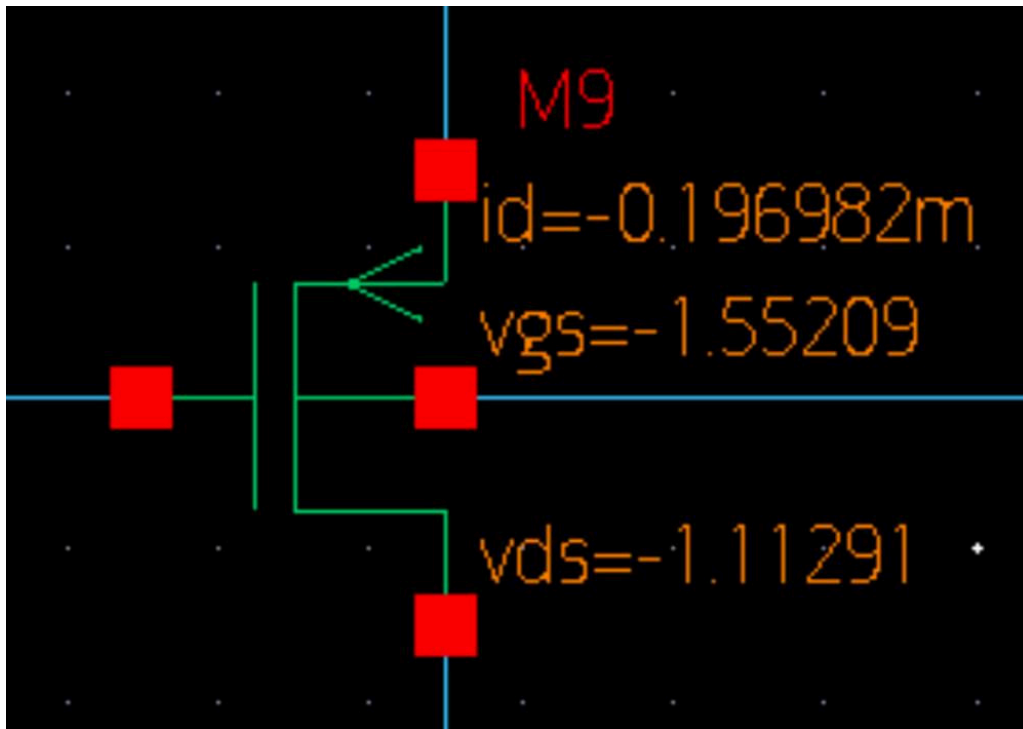
Option	Description
Alter #	Filters OP files generated with an .alter setting in the simulation. The OP file with a different .alter setting follows the naming convention [filename].op[alter #][@Time Point][@Sweep #].
Time Point	Filters OP files generated with timing points in the simulation. The OP file with a different timing point setting follows the naming convention [filename].op[alter #][@Time Point][@Sweep #].
Sweep #	Filters OP files generated with sweep indexes in the simulation. The OP file with a different sweep follows the naming convention [filename].op[alter #][@Time Point][@Sweep #].
OP Files	Shows OP files that matched the Alter # , Time Point , Sweep # , and Filter settings.

2. Click **Apply** to annotate the design according to the settings in the **Annotate Operating Point** dialog box.



In the above figure, circles on MOS devices define the region in which the device operates. The circles' colors correspond to the settings in the **Annotate Operating Point** dialog box for **Cutoff**, **Subthreshold**, **Linear**, and **Saturation**.

3. View the operating point values by zooming into a device. Notice the values for the defined cdsParam() properties, id, vgs, and vds. You can use the Device Label Editor assistant to change the parameters assigned to each cdsParam() property.



4. Click **Clear Annotations** to clear annotations from the schematic.
5. Click **OK** to annotate the design and close the dialog box.

Compare Designs Utility

The Compare Designs utility allows you to compare the differences in logic between two schematic designs. It can be used for comparing either single designs or design hierarchies. The comparison serves as a special instances-oriented check, mainly used to show the differences in netlisting. The following items are checked:

- Instances (added and removed)
- Nets connected to the instances
- Values of instance parameters

Other details are ignored (such as labels, any shapes, and nets not connected to any instance).

When comparing designs, any hierarchical context changes you make in one design are also made in the other design. For example, if you descend into Instance I0 of CellView 1, the context is also descended in CellView 2 and the **Compare Designs** window is updated accordingly.

If there is no split window showing a side-by-side view of the differences open, the **Compare Designs** window is synchronized with the context changes in its parent window. In other words, if the context of the parent window changes, the same change is done in a headless context of CellView 2, and both CellView 1 and CellView 2 in the **Compare Designs** window are updated to the new values taken from the switched contexts. If the occurrences of the switched contexts are still aligned, the **Compare Designs** window is automatically populated with the new difference data. Otherwise, a warning pops up in the dialog box and the **Run** button is enabled to allow you to compare unmatched designs on demand. The same message is shown when the contexts in the split window do not align.

When **Hierarchical Comparison** is checked, running the **Compare Designs** command checks the whole hierarchy. **Compare Designs** in hierarchical comparison mode follows the same name instances in **CellView 2** to dive into the subhierarchy. If the config view is specified in **CellView 1** or **CellView 2**, hierarchical comparison follows the config view to search for switched masters and compare them. If the specified cellview is not a config view, hierarchical comparison follows the `seViewSearchList` preference.

Note:

If you descend into a context of one design that is not available in the other design, a button appears on the canvas indicating that there is an occurrences mismatch. Click it to sync the hierarchy levels to the nearest occurrence that is at the same level in both designs.

In certain cases, the dialog is not able to sync with context changes, such as when the parent design is not the top design and the context in the parent window appears. In those cases, context changes are ignored.

For information on using the `se::compareDesigns` Tcl command, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

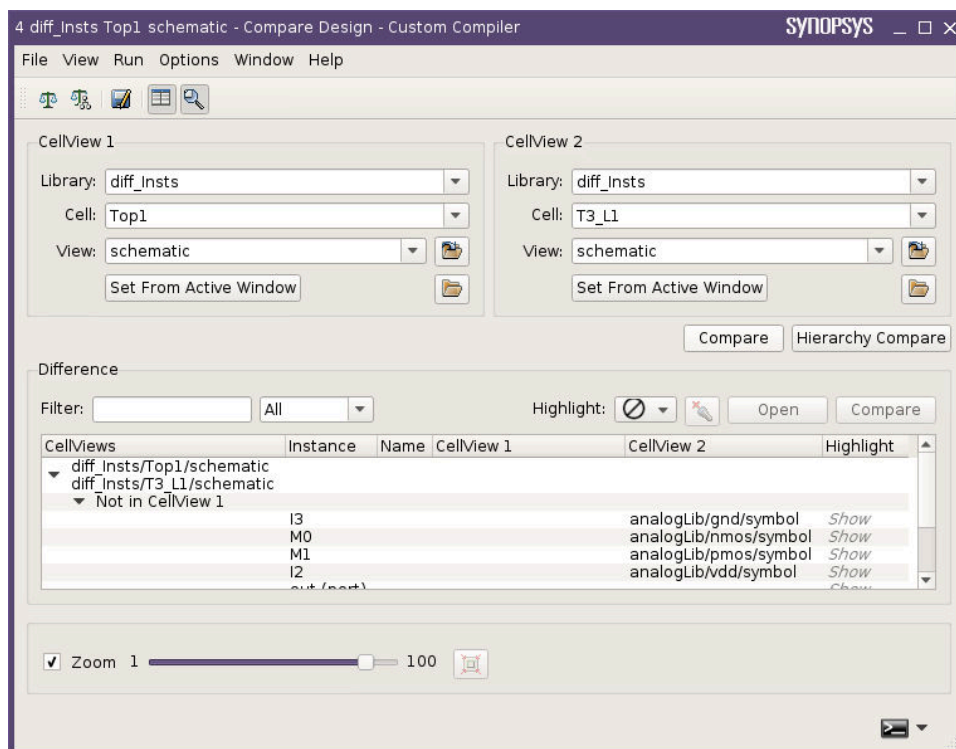
This section contains the following topics:

- [Using the Compare Designs Utility](#)
- [Using Compare Design Menus](#)

Using the Compare Designs Utility

To use the Compare Designs utility:

1. In the Schematic Editor, choose **Utilities > Compare Designs**. The **Compare Designs** window appears. **CellView 1** is automatically populated with the current design's **Library/Cell/View** information.



For information on using the menus in the **Compare Design** window, see [Using Compare Design Menus](#).

2. (Optional) Click **Set From Active Window** to select an instance from the Schematic Editor and fill up the Library, Cell, Views by its switched master. If no switched master is found or the master design is not valid view type, the error message is displayed.
3. (Optional) Click **Open Cellview**  to manually open a new cellview window, if needed. The tool shows a new Schematic Editor window for schematic view and a new Hierarchy Editor window for config view.
4. For **CellView 2**, select the **Library**, **Cell**, and **View** for the second design you wish to compare.
5. (Optional) To compare the whole hierarchy of the selected designs, check **Hierarchical Comparison**. **CellView 1** and **CellView 2** become renamed **TopCellView 1** and

TopCellView 2 to indicate the starting point of the hierarchical comparison. The top cellviews' comparison results are expanded in the **Difference** table; others are collapsed.

The results are shown in the table grouped by master.

6. Click **Compare** or **Hierarchy Compare**.

The **Compare Designs** window updates the **Difference** table to show the detailed differences between the two designs.

The **Difference** table has the following columns:

Column	Description
CellViews	For each compared cellview pair, mismatch results are categorized into 5 major parts: ◦ Net: compare nets connected to instance terminals that belong to same name instances in CellView 1 and 2. ◦ Param: compare parameters of same name instances in CellView 1 and 2. ◦ Master: compare place masters of same name instances in CellView 1 and 2. For hierarchical comparison, switched masters are compared if place masters are the same. ◦ Not in CellView 1: instances that only exist in CellView 1 but not found in CellView 2. ◦ Not in CellView 2: instances that only exist in CellView 2 but not found in CellView 1.
Instances	List of instances in both designs, with matching instances at the top of the list. This column is hierarchical (a two-level tree). The zero-level titles are: ◦ Net: names of instances for which any connected net name is different ◦ Param: names of instances for which any parameter value is different ◦ Not in < >: names of instances absent in corresponding design
Name	This column contains: ◦ For Net rows: names of instance terminals to which the nets are connected. ◦ For Param rows: names of instance parameters. For Not in < > rows, this column is not used.
CellView 1	For CellView 1 , this column contains: ◦ For Net rows: net names in the corresponding design. ◦ For Param rows: parameter values in the corresponding design. For Not in < > rows, this column is not used.
CellView 2	For CellView 2 , this column contains: ◦ For Net rows: net names in the corresponding design. ◦ For Param rows: parameter values in the corresponding design. For Not in < > rows, this column is not used.

Column	Description
Highlight	Double-click a cell in the Highlight column to highlight that instance in the design, or in both designs if it occurs in both. For Net instances the nets are highlighted, for the rest the instances are highlighted. Use the Highlight selection field to set the highlight color of selected mismatch results manually.
Open	Select a cell and then click the Open button to open the schematic design of the master Library/Cell/View .
Compare (available when Hierarchical Comparison is not selected)	Select a cell under the Master CellViews and then click the Compare button to compare the master design of CellView 1 with the master design of CellView 2 . This runs the Compare Designs command and opens another Compare Designs dialog box showing the new Difference table and a new visualization window displaying the two designs side by side.
Go to (available when Hierarchical Comparison is selected)	Select a cell under the Master CellViews and then click the Go to button to jump to the sub hierarchy comparison result in the table.

7. (Optional) To filter the comparison results, type your filter criteria in the **Filter** text field.

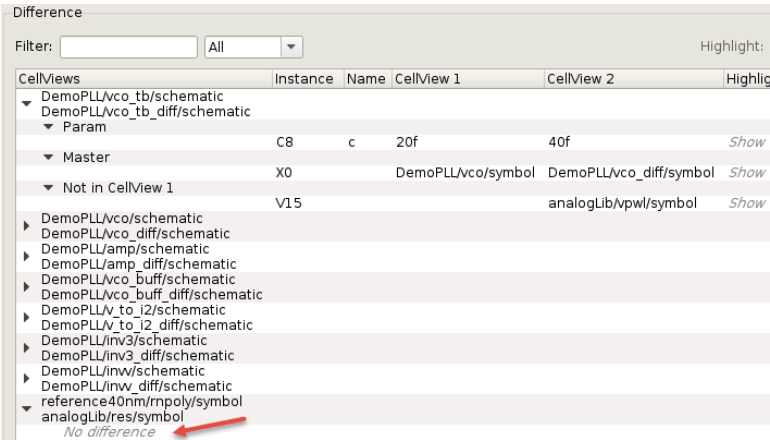
For a non-empty keyword, all matched entries are expanded and grouped to the top of the list. Also, unmatched hierarchies are hidden.

You can filter columns in the **Difference** table by choosing **All**, **Instance**, **Name**, **CellView 1**, and **CellView 2**.

To filter the mismatches caused by system nets, turn off the **Options > Show System Nets** option. The net section is then filter if both the instance terminals in CellView1 and CellView2 connect to system nets. When the option is toggled on, all results are shown.

To filter parameter values that are scientific equal (for example: 1k is equal to 1000), turn off the **Options > Match Parameters by String** option.

To hide the hierarchy with empty differences in the table, turn on the **Options > Show Differences Only** option. Otherwise, turn off the option to show the hierarchy with empty differences, as shown in the following figure.



Difference

Filter: All Highlight:

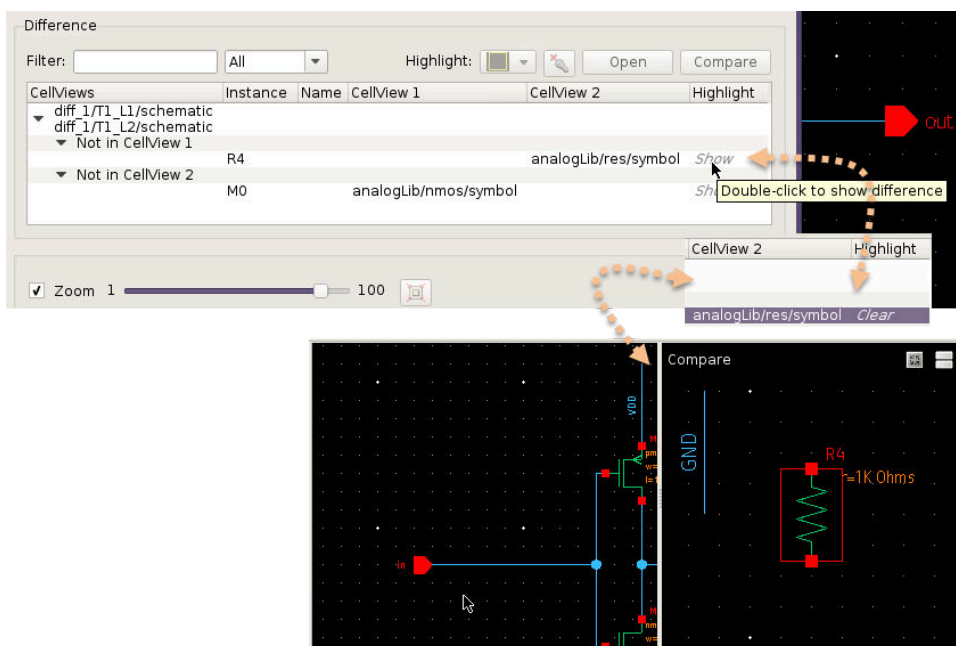
CellViews	Instance	Name	CellView 1	CellView 2	Highlight
▼ DemoPLL/vco_tb/schematic					
DemoPLL/vco_tb_diff/schematic					
▼ Param					
▼ Master	C8	c	20f	40f	Show
▼ Not in CellView 1	X0		DemoPLL/vco/symbol	DemoPLL/vco_diff/symbol	Show
▼ V15				analogLib/vpwl/symbol	Show
DemoPLL/vco/schematic					
DemoPLL/vco_diff/schematic					
DemoPLL/amp/schematic					
DemoPLL/amp_diff/schematic					
DemoPLL/vco_buff/schematic					
DemoPLL/vco_buff_diff/schematic					
DemoPLL/v_to_i2/schematic					
DemoPLL/v_to_i2_diff/schematic					
DemoPLL/inv3/schematic					
DemoPLL/inv3_diff/schematic					
DemoPLL/invw/schematic					
DemoPLL/invw_diff/schematic					
reference40nm/mpoly/symbol					
analogLib/res/symbol					
No difference					

Note:

Filtering adheres to the **Filters** preference setting of the **General Options** dialog box for partial match, wildcard, regular expression, and match start. See Setting General Options.

8. Select rows in the **Difference** table.
- (Optional) To select more than one row in the table, use Ctrl-click to incrementally select a row. Alternatively, use Shift-click to select multiple rows. Note that the selection should be within the same hierarchy to display one context at a time.
- (Optional) Use the shortcut key Ctrl+A to select all rows in the table.
9. Show instances in the designs by double-clicking **Show** in the **Highlight** column of the **Difference** table.

The canvas shows a split view of both designs you are comparing. Note that the **Highlight** column changed from **Show** to **Clear**. The visualization window is not the same window from which you called **Compare Designs**.

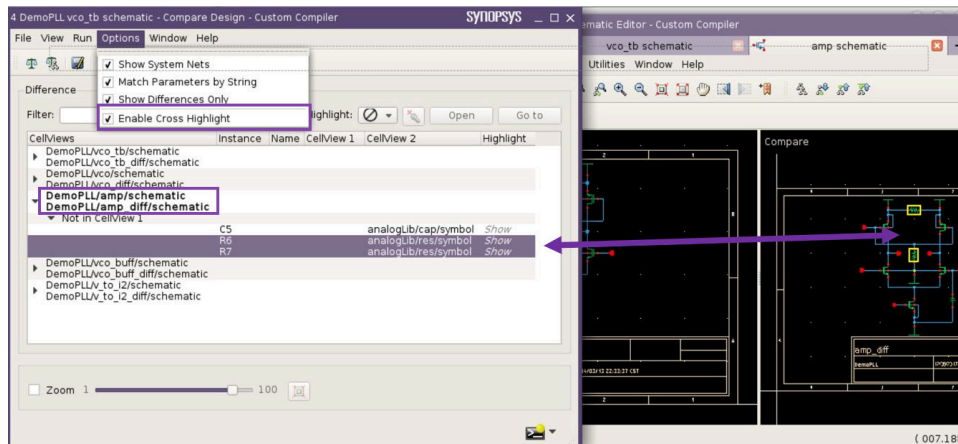


The canvas displays the nets or instances you highlighted. The color of the highlighting in the table rows corresponds to the color of the highlighting in the designs.

To hide any highlighting, do one of the following:

- Double-click **Clear** in the **Highlight** column.
- Choose  from the **Highlight** selection field.
- Choose **Clear All Highlights** , which is next to the **Highlight** selection field, to clear all existing highlights.
- Right-click in the **Difference** table and choose:
 - **Highlight Selected** to clear all existing highlights under selection and then highlight selected nets or instances with auto-picked colors, which are triggered by `de::createHighlightSelector -showAuto`.
 - **Clear Highlights** to clear all existing highlights under selection.
- Click **Compare** or **Hierarchy Compare** to update the entire table's contents; the highlights are removed.

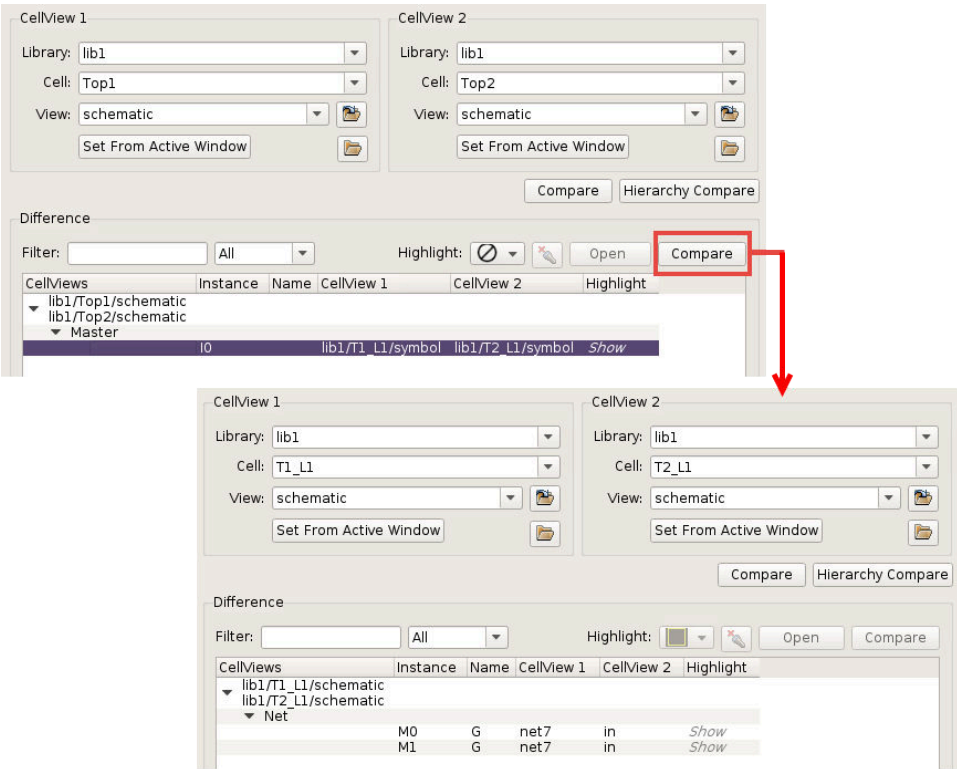
10. (Optional) Turn on the **Enable Cross Highlight** toggle option to enable cross highlighting between the **Difference** table and the schematic canvas.



11. (Optional) When the **Zoom** option is enabled, the difference is zoomed by the selected percentage.

Click **Fit to highlighted differences** so that nets or instances in the highlighted set fit in the viewport.

12. Click **Compare** to compare the master design of **CellView 1** with the master design of **CellView 2**. This runs the **Compare Designs** command and opens another **Compare Designs** window showing the new Difference table and a new visualization window displaying the two designs side by side.



13. Click **Open** to open the schematic design of the master Library/Cell/View in a new window.
14. (Optional) In the **Compare Designs** window, click **File > Export** to open the **Save Result to File** dialog box. Enter a **File name** and click **Save** to export the design comparison results to a `.rpt` text file.
15. Click **Cancel** to exit the **Compare Designs** window.

Using Compare Design Menus

The following tables describe the Compare Design menus:

- [Table 27](#) File Menu
- [Table 28](#) View Menu
- [Table 29](#) Run Menu
- [Table 30](#) Options Menu

Table 27 File Menu

Menu	Command	Description
File		Opens the Save Result to File dialog box. Enter a File name and click Save to export the design comparison results to a <code>.rpt</code> text file. See Step 14 in Using the Compare Designs Utility .
	Export	
	Close	Closes the Compare Design.

Table 28 View Menu

Menu	Command	Description
View		Show or hide the CellView section. When disabling CellView , only displays the Difference section in the window.
	CellView	
		Show or hide the Zoom section. When disabling CellView and Zoom , only displays the Difference section in the window.
	Zoom	

Table 29 Run Menu

Menu	Command	Description
Run		Click the button to compare two schematic design.
	Zoom	
		Click the button to compare two schematic design with hierarchical comparison.
	Zoom	

Table 30 Options Menu

Menu	Command	Description
Options	Show System Nets	When toggled (on by default), shows mismatches caused by system nets in the Difference table. The net section is then filtered if the instance terminals in both CellView1 and CellView2 connect to system nets. See Step 7 .
	Match Parameters by String	When toggled (on by default), compares parameter values by string match. For example, 1k and 1000 are considered mismatches. See Step 7 .
	Show Differences Only	When toggled (off by default), shows the details of matched cellviews in the Difference table and also exports the details to the report file. See Step 7 .
	Enable Cross Highlight	When toggled (off by default), enable cross highlighting between the Difference table and the schematic canvas. See Step 10 .

Connections Navigator

You can use the Connections Navigator to analyze your designs to determine the actual connectivity when parameterized connections are used in the design.

For information on using the `de::showConnectionsNavigator Tcl` command, see Design Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

This section contains the following topics:

- [About the Connections Navigator](#)
- [Opening the Connections Navigator](#)
- [Setting Connections Navigator Options](#)
- [Editing Assignments](#)
- [Traversing the Design](#)

About the Connections Navigator

You can use the Connections Navigator to:

- Show all connection definitions (connectDefs) in a hierarchy
- Determine which assignments are available for an instance, in addition to which assignments have been overridden (that is, a list of all connectDefs in that hierarchy)

branch and where the connectDef exists, along with any assignments hiding the connectDef)

- Identify where an assignment exists in the current block — for both instance level and cellview-wide assignments
- Identify where an assignment exists in the hierarchy
- Determine the current connected net for each connection definition on an instance
- Determine, from the set of selected instances, whether there are any parameters that are not in all instances, as well as which instance(s) are missing which parameter(s)
- Determine the connected net for each parameter and each instance

When invoked, the Connections Navigator computes the connection definition (connectDef) information for the current edit design in the associated window. The connect definition information includes the set of all parameters found in the hierarchy along with hierarchical assignment information for each parameter.

Opening the Connections Navigator

To open the Connections Navigator:

- Choose **Design > Connections Navigator**.

The Connections Navigator window opens and populates the three list box regions.

The Connections Navigator has three list box regions, toolbar icons, and toolbar menus, which are described in the following sections:

- [Parameter Region](#)
- [Assignments Region](#)
- [Blocking Assignments Region](#)
- [Connections Navigator Toolbar Icons](#)
- [Connections Navigator Toolbar Menus](#)

Parameter Region

The parameterized connections region lists all connection definitions in the complete hierarchy of the edit design, including oaAssignAssignments (connectDef overrides). This list represents all the connection definitions that can be or have been assigned. The definitions are grouped by parameter name. Same named parameters are grouped together with the specific definitions in a sublevel. By default, all sublevels are collapsed.

Assignments Region

The Assignments region:

- Displays the summary information
- Contains controls for setting assignments for the filtered instances in the edit design

The summary information appears in a table where each row represents an instance (see the following figure). The first two columns display the instance name and the master L/C/V. Additional columns are appended, one for each unique parameter name. Only those parameters selected from the Parameter list are considered. Instances that contain at least one of those parameters in their hierarchy are added to the table.

Double-clicking a definition automatically displays the source design in an editor window and highlights the associated source net, pin, or instance containing the definition.

Note:

Pressing Esc while editing reverts the cell to its previous value.

Clicking the **Refresh** button displays the most recent data set.

Instance	Master	bulk_n	main
sym1	L/C/V	<i>gnd!</i>	
sym2	L/C/V		mynet
sym3	L/C/V		new:myprwl
sym4	L/C/V	<i>gnd!</i>	
sym5	L/C/V	mynet	
sym6	L/C/V		
sym7	L/C/V		badnet

The table entries under each parameter name represent the state of that parameter relative to the specific instance. The background color, text formatting, and the font style allow for visual classification as defined in [Table 31](#), [Table 32](#), and [Table 33](#).

Note:

For table entries that have an assignment state of *full*, but no assignment exists on that instance, the table entry is empty; conversely, if an assignment exists, that text appears in bold type. This indicates that the assignment for that instance does not affect connectivity because the parameter is already fully assigned down the hierarchy.

You can use each column as a sort key. Clicking a column header sorts all the rows in that column using the column's data as the sorting key. Sorting can be in ascending and descending alphanumeric order for the first two columns. For the parameter columns, sorting is based on the assignment state (as the primary key) and the alphanumeric (as the secondary key).

Table 31 Background Colors

Assignment State of Background Color Master	
none	Default
partial	Yellow
full	Red

Table 32 Text Format

Format	Description
<string>:<string>	Indicates an override parameterization (oaAssignAssignment)
<string>	Indicates the explicit net assignment (oaAssignValue)

Table 33 Text Fonts

Text Font of Entry	Description
Normal	The parameter corresponds to an assignment on that specific instance
Italic	No assignment exists for that instance
Bold	Assignment state is <i>full</i> and an assignment exists on that instance.

Blocking Assignments Region

The Blocking Assignments region lists all assignments to the selected parameter in the selected instances' hierarchy. This lets you see assignments that either are, or would be, blocking an assignment on that instance.

Connections Navigator Toolbar Icons

The toolbar contains the following icons:

Option	Description
Open	Enabled when a single row is selected in the Blocking Assignments pane. When pressed, causes the following sequence of events in the associated window: • Descends to the location of the selected assignment • Selects the appropriate row in the assignments table • Performs a Zoom to operation
Zoom to	Causes the associated window to set the view to fit the currently selected instances. This is useful for identifying one or more instances in the design from information in the table.
Descend Into	Traverses the hierarchy downwards one level at a time. See Traversing the Design .
Return	Traverses the hierarchy upwards one level at a time. See Traversing the Design .
Return to top	Returns to the top level of the design.
Refresh	Displays the most recent data set.

Connections Navigator Toolbar Menus

The toolbar contains the following menus:

Option	Description
File	• Close: Closes the Connections Navigator window.
View	• Options: Opens the Options dialog box. See Setting Connections Navigator Options. The following options let you control which regions appear in the Connections Navigator window. For information about the regions, see Opening the Connections Navigator. • Show Parameters • Show Blocking Assignments • Show Parameter Occurrences
Window	• Toolbars: Lists the toolbars that you can display. • Workspaces: Lists the windows you can open to create, load, or save a workspace. • Customize: Opens the Customize window. For information about customizing the window, see the <i>Custom Compiler Environment User Guide</i>.

Option	Description
Help	• Custom Compiler Help • Connections Navigator Help • About Custom Compiler

Setting Connections Navigator Options

To configure the information the Connections Navigator displays:

1. Choose **View > Options**.

The **Connections Navigator Options** dialog box appears.

2. Specify a switch list.
3. Specify any other options you want to apply (see [Connections Navigator Dialog Box Options](#)).
4. Click **OK** or **Apply**.

Connections Navigator Dialog Box Options

The dialog box contains the following options:

Option	Description
View Search List	Specifies the search list.
Highlight in Window	Enables highlighting in the editor window; that is, highlighting of those instances containing the selected parameters. The instances are highlighted with a specific color depending on the assignment state relative to the selected parameters, as shown in the background colors table under Assignments Region .
Hide Fully Assigned Parameters	Hides fully assigned parameters.

Editing Assignments

To edit an assignment:

1. In the **Parameter** region of the Connections Navigator window, select one or more parameter entries.

The **Assignments** region redraws to display the Instance, Master, and connectDef information for the selected entries.

2. On the canvas, click one of the selected cells.
 - The **Assignments** region redraws again to display the connection definition information.
 - The table entry becomes editable.
3. In the **Assignments** region, enter the assignment text.

To create an override assignment, use this format:

`<newName>:<defaultNet>`

Caution:

The colon character (:) is not allowed in the namespace for parameter names and net names.

4. When you change focus or press Enter, the current text in the cell is accepted, if valid.

Traversing the Design

You can use the **Return** and **Descend into** icons to traverse the hierarchy one level at a time. The **Return to top** icon brings you to the top level.

Selecting a single instance enables the **Descend into** icon. Clicking the **Descend into** icon causes a descend operation in the editor window and repopulates the Connections Navigator window with information relative to the new edit design. Double-clicking on a parameter table entry also causes a descend operation. You can use the **Return** icon to go up one level of hierarchy. When you are at the top of the design, the icon is disabled.

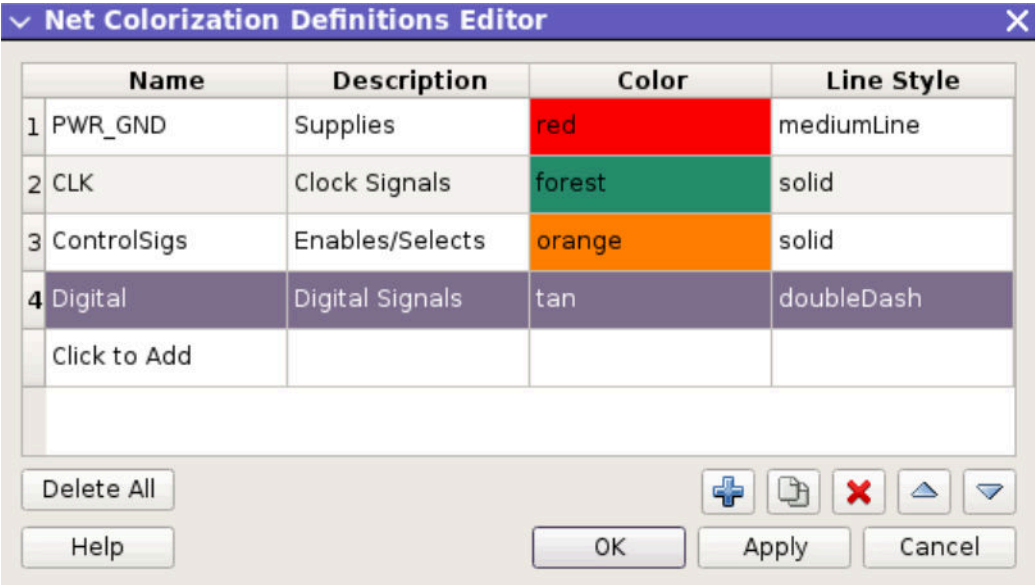
Net Colorization Definitions Editor

The Net Colorization Definitions Editor utility allows you to create color definitions for coloring nets in the Schematic Editor.

For information on using the `se::getColorizationDefinitions`, `se::createNetColorizationDefinition`, and `se::clearNetColorizations` Tcl commands, see Schematic Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To use the Net Colorization Definitions Editor:

1. From the Schematic Editor, choose **Utilities > Net Colorization Definitions**. The **Net Colorization Definitions Editor** dialog box appears.



The **Net Colorization Definitions Editor** dialog box has the following options:

Option	Description
Name	The user-defined name for the color definition.
Description	The user-defined description of the color definition.
Color	The color that is used for the selected nets.
Line Style	The line style for the color definition.
Delete All	Clears all data from the table.
	Adds a row to the table.
	Duplicates the selected row.
	Deletes the selected row.
	Moves the selected row up.
	Moves the selected row down.

2. Click **Click to add** to add a new color definition to the table and enter your options.
3. Click **OK** or **Apply** to set the color definitions.

The color definitions are now available when you use the **Edit > Colorize Nets** command. See [Colorizing Nets](#) for information.

Plot Signal Utility

The Plot Signal utility allows you to plot signals in the Schematic Editor that are simulated outside of the PrimeWave Design Environment. The Plot Signal utility supports:

- Multiple plot modes
- Plotting of multiple files
- Plotting of voltages and currents
- Consistent color scheme between objects in the Schematic Editor and plots in PrimeWave Design Environment WaveView
- PrimeSim HSPICE, PrimeSim Pro, PrimeSim XA, VCS, and VCS PrimeSim AMS waveform formats
- Some digital features, such as logic and bus plotting

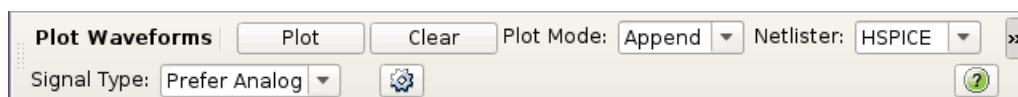
The following sections describe how to use the Plot Signal utility:

- [Opening the Plot Signal Utility and Using the Toolbar](#)
- [Choosing Plot Settings](#)
- [Plotting Signals From the Schematic Editor](#)

Opening the Plot Signal Utility and Using the Toolbar

To open the Plot Signal utility:

1. In the Schematic Editor, choose **Utilities > Plot Signal**. The **Plot Waveforms** toolbar appears.



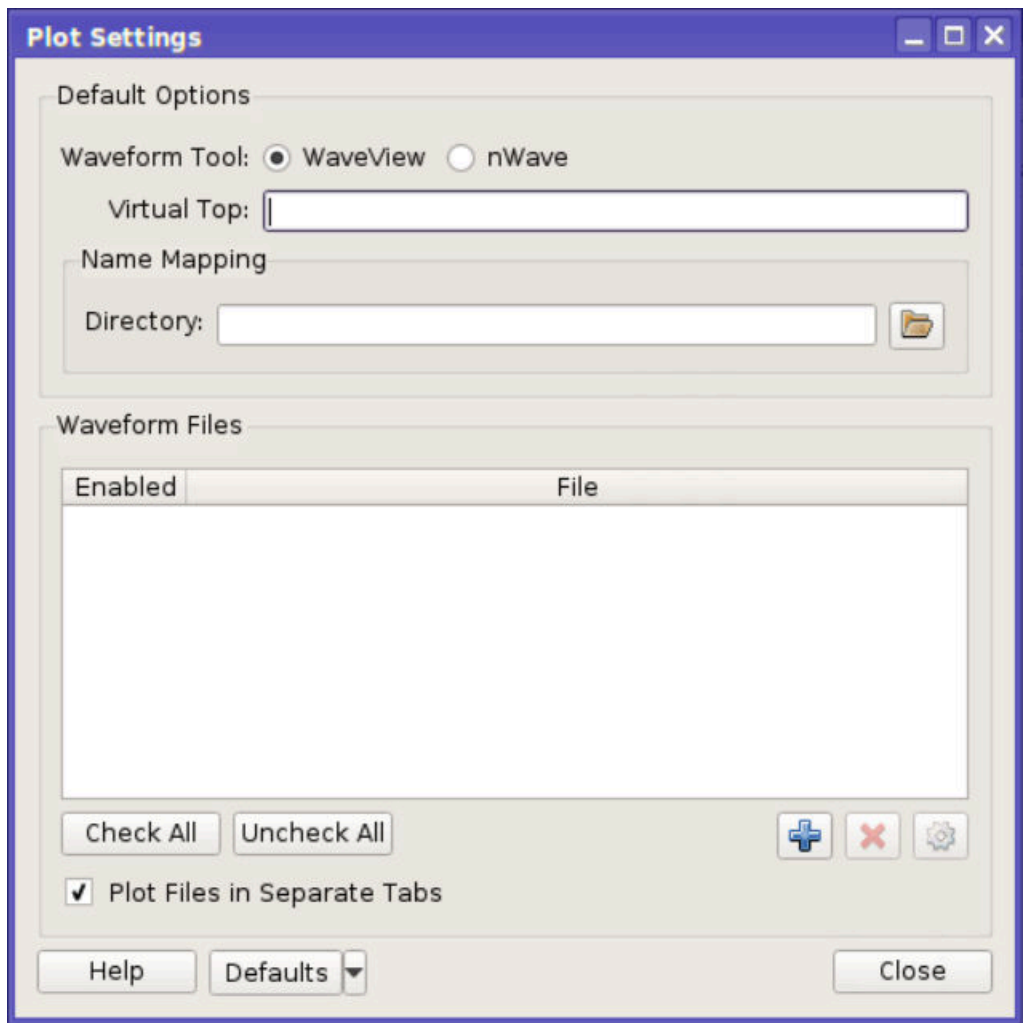
2. Set the following options on the **Plot Waveforms** toolbar:

Option or Button	Description
Plot	Plots waveforms for the selected objects in PrimeWave Design Environment WaveView using the current settings. If you have not previously configured your plot settings, such as selecting waveform files, Plot opens the Plot Settings dialog box. See Choosing Plot Settings .
Clear	Clears all plots.
Plot Mode	Determines whether to Append new plots to previous set in PrimeWave Design Environment WaveView or Replace the previous set of plots with the current ones. Note that plots in WaveView have the same colors as the probes you create in the Schematic Editor using the Probe assistant, for ease of tracking. Tcl preference: <code>sePlotSignalMode</code>
Netlister	Sets the default name-mapping algorithm to use if the signal cannot be found in the waveform database. HSPICE makes identifiers lower-case and Verilog preserves case. Tcl preference: <code>sePlotSignalNetlister</code>
Signal Type	Sets the preferred signal type by Analog Only , Digital Only , Prefer Analog (default), Prefer Digital , or All . Tcl preference: <code>sePlotSignalSignalType</code>
	Opens the Plot Settings dialog box.

Choosing Plot Settings

The **Plot Settings** dialog box appears when:

- you click **Plot** in the **Plot Waveforms** toolbar but no waveform files are specified for plotting
- you click the **Show Settings Dialog** button  in the toolbar.



Set up your **Plot Settings** options and click **OK** or **Apply** before you plot signals from the Schematic Editor. See [Plotting Signals From the Schematic Editor](#) for instructions on plotting.

The **Plot Settings** dialog box has the following options:

Option or Button	Description
Default Options	
Waveform Tool	Selects the WaveView tool by PrimeWave Design Environment WaveView (default) or Verdi nWave . Shows error messages if no Verdi command exists when you select nWave and Plot . Tcl preference: sePlotSignalWaveformToolName

Option or Button	Description
Virtual Top	Allows you to set a virtual root, which enables a flow where your testbench is in text but your design is in schematic. Note: If the top of your netlisted design matches the top of the simulation results you are using, you do not need to set Virtual Top .
Name Mapping	Browse to the name-mapping Directory . The name-mapping directory should correspond to the <code>cn1</code> directory used for generating the simulation's structural netlist.
<i>Waveform Files</i>	
	Click  to open the Select waveform files dialog box and browse to the waveform file you want to load. You can load multiple files. More than one file can be enabled for plotting at the same time.
	Delete a file by selecting that file's row and clicking  .
	Use  to open the Configure Selected files dialog box, which allows you to set up name mapping on a per file basis. After configuring files, any differences in name mapping are shown in the Waveform Files window.
	Check All enables all the files loaded in the Waveform Files window. Uncheck All disables them all.
Plot Files in Separate Tabs	Instructs PrimeWave Design Environment WaveView to create a separate tab for each waveform file's plots. On by default. Tcl preference: <code>sePlotSignalPlotFilesInSeparateTabs</code>

Plotting Signals From the Schematic Editor

Once you have chosen your plot settings (described in [Choosing Plot Settings](#)), you can begin using the Plot Signal utility. To plot signals from the Schematic Editor:

1. Select one or more objects on the canvas.

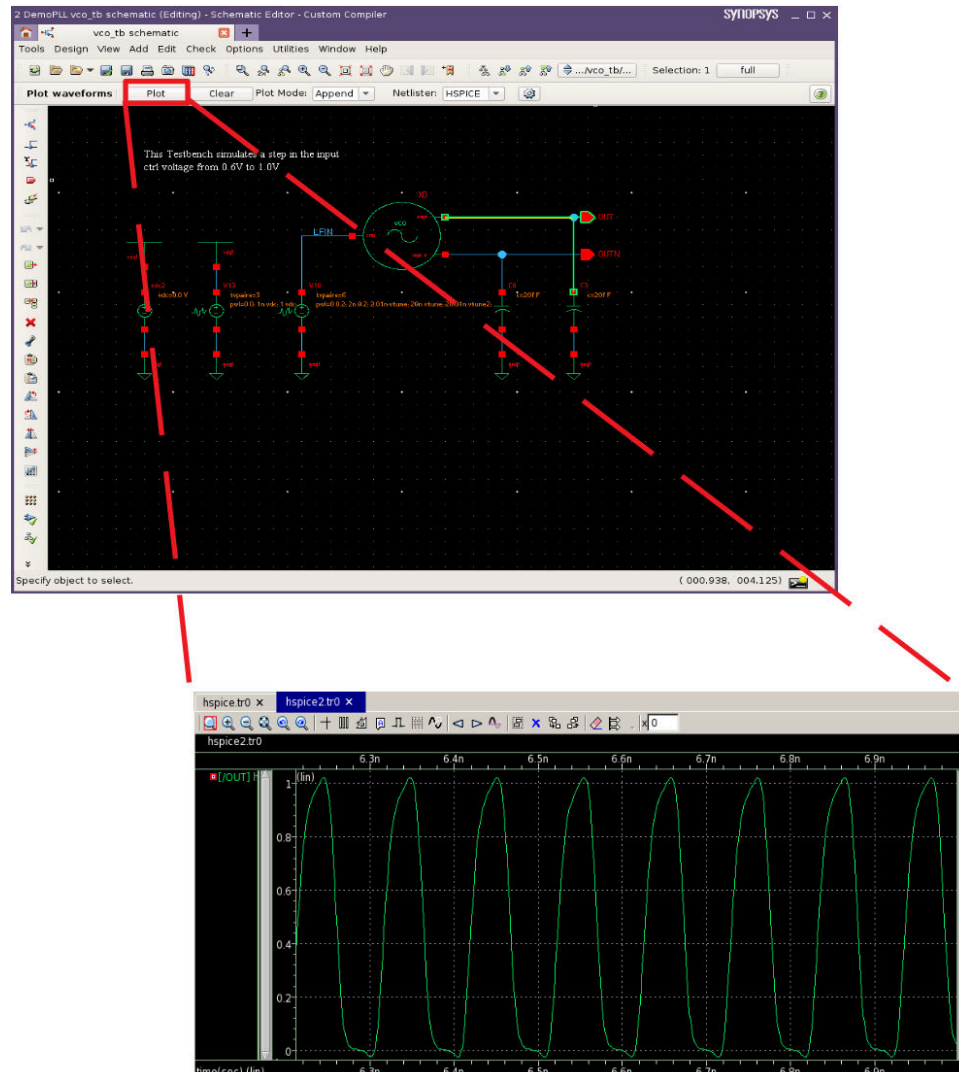
When you select a net to plot signals, the tool searches the signals from the selected net. If no signal is found, it looks up equivalent nets from the top level to other upper level.

2. On the **Plot Waveforms** toolbar click **Plot**. The signals are plotted in WaveView according to your settings.

Note:

If you have not already specified the waveforms to use for plotting, the **Plot** command opens the **Plot Settings** dialog box to allow you to set the plot files. See [Choosing Plot Settings](#) for information.

The following figure demonstrates plotting analog signals. The OUT net is selected and highlighted in green. Two waveform files are enabled to plot the signals; they are displayed in separate tabs in WaveView. The waveforms are shown in green to match the highlighting in the schematic.



3. (Optional) Plot another set of signals. Depending on whether you are in **Append** or **Replace** mode, the waveforms are either appending to the original set in WaveView or the original set is cleared and replaced with the new set.
4. (Optional) Clear the plots by clicking **Clear**.
5. Press Esc to end the command.

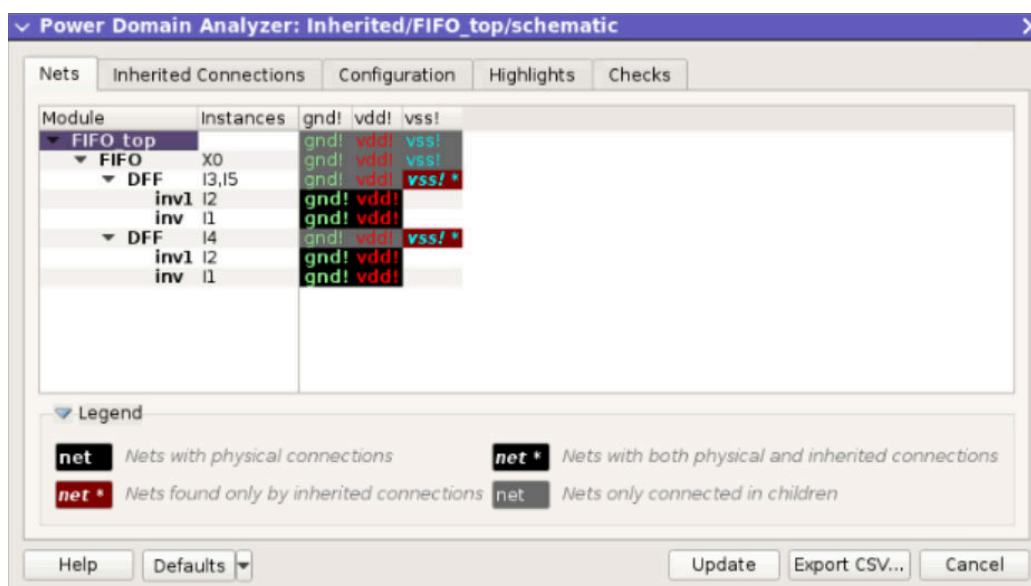
Power Domain Analyzer Utility

The Power Domain Analyzer utility allows you to see all the global net connections throughout the hierarchy. This can be useful when designs in the hierarchy use different power/ground supplies. The Power Domain Analyzer also helps you observe any non-supply and non-global net(s). You can also use the Power Domain Analyzer to analyze and debug any problems related to inherited connections.

Opening the Power Domain Analyzer

To open the Power Domain Analyzer:

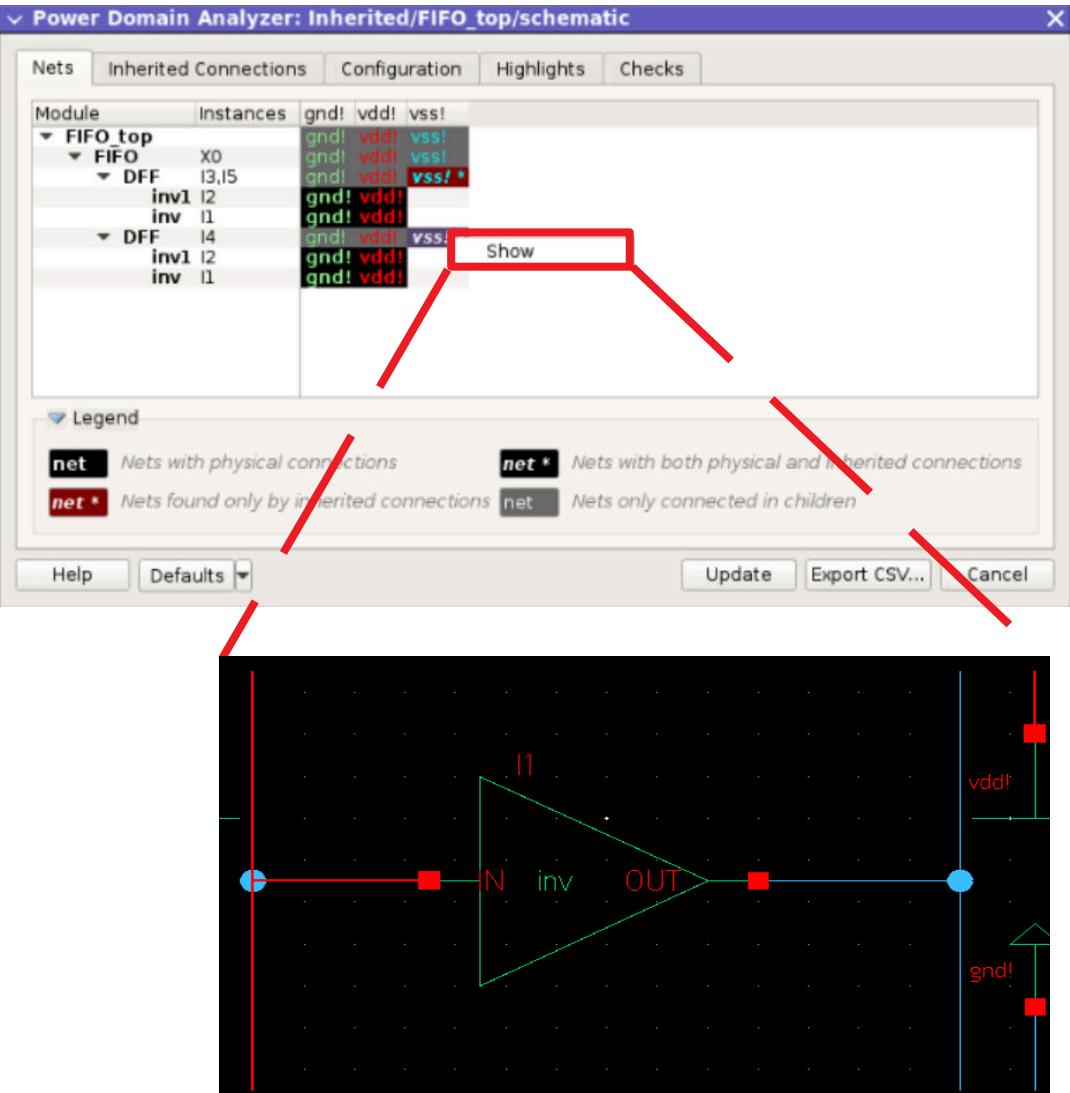
- Choose **Utilities > Power Domain Analyzer** from the Schematic Editor main menu bar. The **Power Domain Analyzer** dialog box appears.



Viewing Power Domain Results

You can view power domain results in the **Nets** tab.

Right-click on any cell in the table and click **Show** to see all the nets related to that level in the design canvas.



If the active cell contains a net name, only this net is highlighted in the canvas. Multiple cell selection is allowed for highlighting more than one selected net.

When you select several different rows, only one row is considered in effect; the rest will be unselected when you click **Show**.

The **Nets** tab has the following columns:

Column	Description
Module	Cell names in the design hierarchy. Hover over a cell to see a tooltip of the Library/Cell/View and occurrence information for that cell.

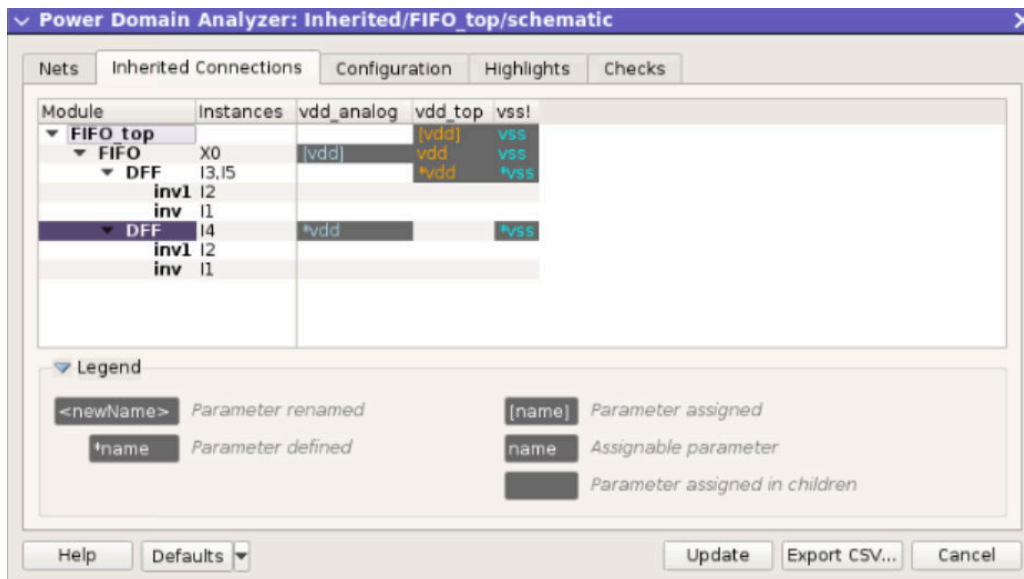
Column	Description
Instances	The list of instances for that level.
<list_of_nets>	The power nets related to that level of the design. The nets shown in this column reflect all the nets propagated up from the lower levels of the design.

The legend at the bottom of the **Nets** tab indicates the status of each net. You can change the colors of this legend using the **Configuration** tab options. (See [Configuring View Options](#).)

-  Nets with physical connections.
-  Nets with both physical and inherited connections.
-  Nets found only by inherited connections.
-  Nets only connected in children.

Analyzing and Debugging Inherited Connections

Inherited connections are parameterized connections composed using connection definitions (connectDefs), assignment values (oaAssignvalues), and assign assignments (oaAssignAssignments). The **Inherited Connections** tab provides options for you to analyze and debug inherited connections.



The **Inherited Connections** tab has the following columns:

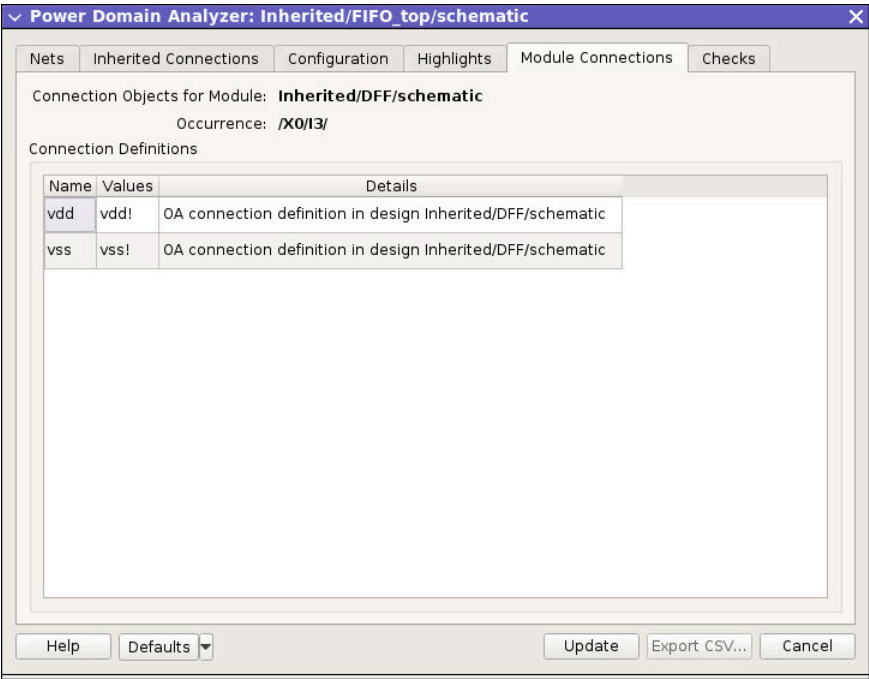
Column	Description
Module	Cell names in the design hierarchy. Hover over a cell to see a tooltip of the Library/Cell/View and occurrence information for that cell. The Module column could have multiple records for the same module. Different records have different assignments.
Instances	The list of instances for that level.
<list_of_effective_nets>	The effective nets related to that level of the design. The table cells show the virtual terminal names. The names shown in this column reflect all the nets propagated up from the lower levels of the design.

The legend at the bottom of the **Inherited Connections** tab indicates the status of each virtual terminal name. You can change the colors of this legend using the **Configuration** tab options. (See [Configuring View Options](#).)

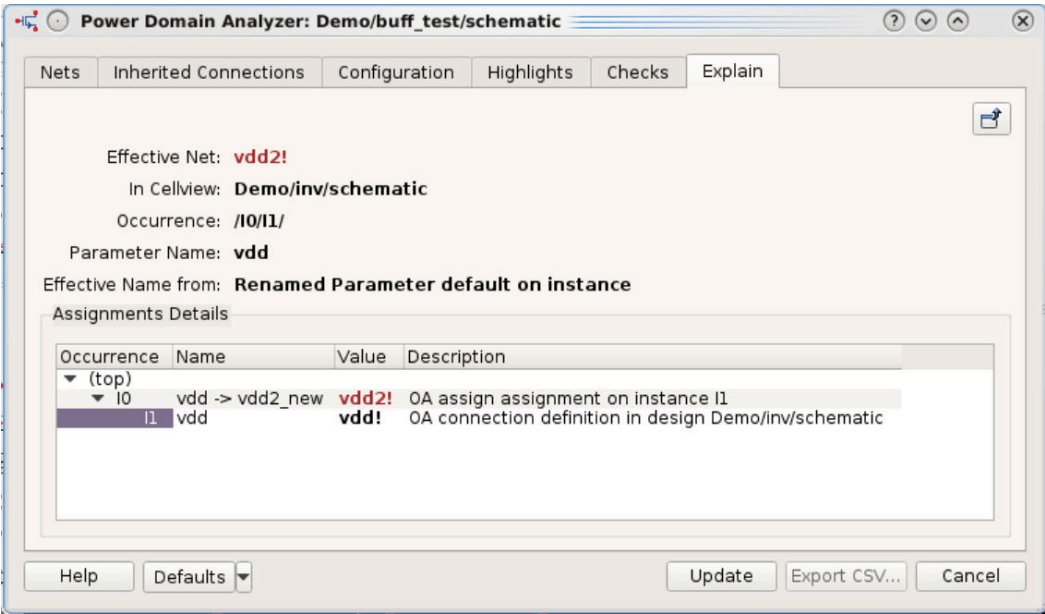
-  Parameter defined.
-  Parameter renamed.
-  Parameter assigned.
-  Assignable parameter.
-  Parameter assigned in children.

To analyze inherited connection results:

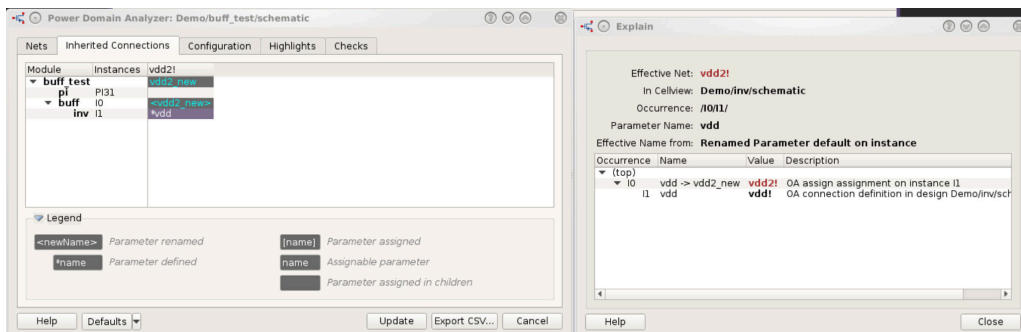
1. Right-click any cell in the **Instances** column and choose **Show Instances** to open the design in the schematic window where those instances are placed.
2. Right-click any cell in the <list_of_effective_nets> and choose **Show <virtual terminal name>** to open the design in the schematic window where that connection definition was created.
3. Right-click any module and choose **Module Connections** to open a new tab, **Module Connections**, showing all the connection objects for the selected design.



4. Right-click any net and choose **Explain** to open a new tab, **Explain**, which gives the reason the connection was formed at this level.

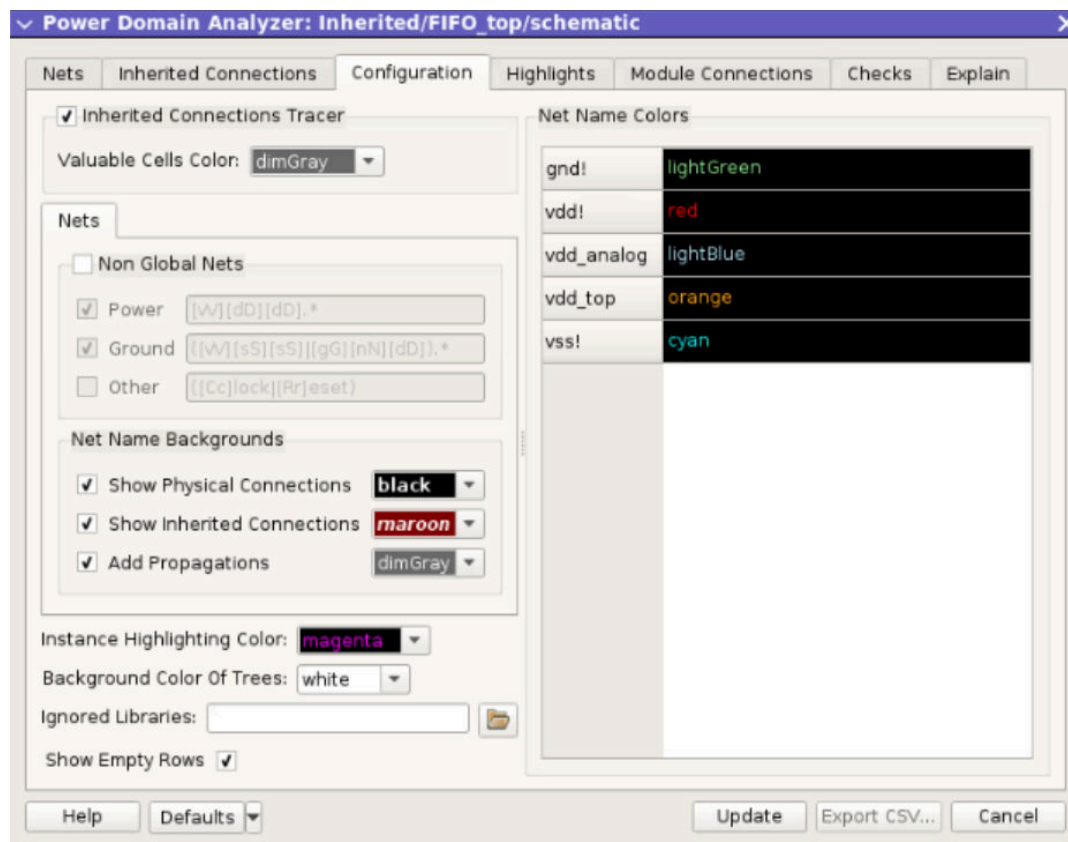


Click the **Detach** button  in the upper-right corner to detach the **Explain** tab from the Power Domain Analyzer window and view them side by side.



Configuring View Options

The **Configuration** tab provides options for you to configure the visual appearance of the nets and inherited connection parameter names in the design.

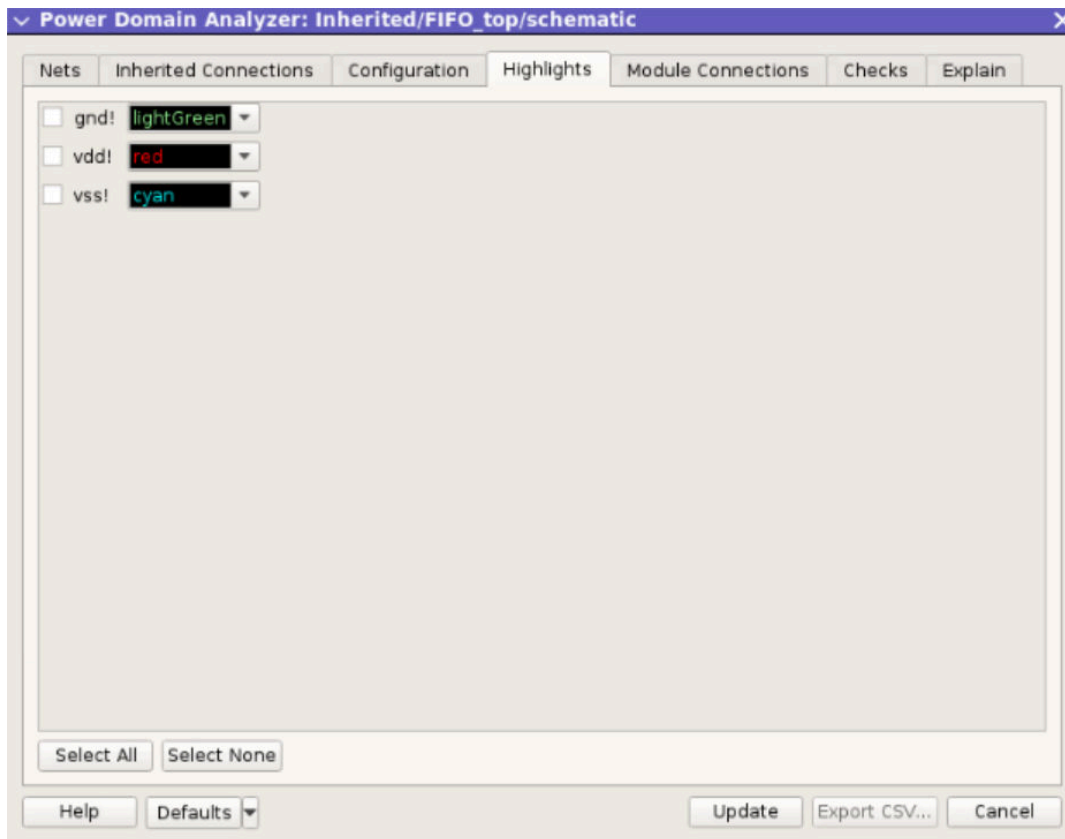


The **Configuration** tab has the following options:

Option	Description
Inherited Connections Tracer	Enables the inherited connections tracer (on by default). When unchecked, the Inherited Connections and Checks tabs no longer appear in the Power Domain Analyzer dialog box after you click Update .
Non Global Nets	Specifies which nets to show in the design: Power , Ground , Other . By default, the nets shown for Power are [vV] [dD] [dD] .* and for Ground are ([vV] [sS] [sS] [gG] [nN] [dD]) .*. Edit these fields as necessary if your design has non-standard net names. Enable the checkbox to show non-global nets.
Net Name Backgrounds	Specifies the background color for net names. Enable the checkboxes to Show Physical Connections , Show Inherited Connections , and/or Add Propagations . Possible background colors include black , dimGray , blue , maroon , darkGreen , and purple .
Instance Highlighting Color	Specifies the highlighting color of instances.
Background Color Of Trees	Specifies the background color of trees.
Ignored Libraries	Specifies a list of libraries from which the cells will not be processed, together with their entire lower hierarchy.
Show Empty Rows	Exposes hidden empty rows in the trees.
Net Name Colors	Lists the net names in the design and allows you to edit their color.

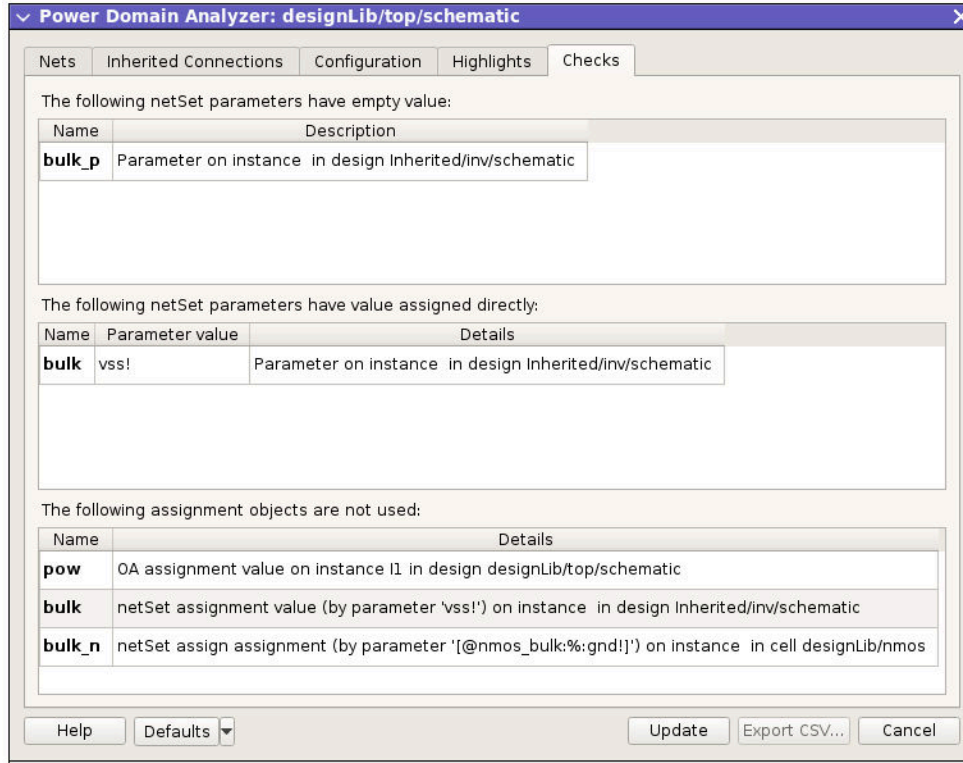
When you make changes to the checkboxes in the **Configuration** tab, you will be prompted to click **Update** to apply those changes. Color and background changes are executed immediately and do not require you to click **Update**.

The **Highlights** tab lists the net names from the **Nets** tab and allows you to edit their color.



Debugging Problems With Inherited Connections

Each time the Power Domain Analyzer is initiated, the entire hierarchy of the design is traced. If there are any issues related to inherited connections, they are tabulated in the **Checks** tab. If there are no issues, the **Checks** tab does not appear.



The **Checks** tab can contain up to three tables; each table is shown only if there are items in it.

- The first table shows errors in which the cdfParam objects of type "netSet" have an empty default value. It has the following columns:

Column	Description
Name	The name of the parameter.
Description	Explanation of where the parameter occurs.

- The second table shows cases in which the cdfParam objects of type "netSet" have a default value defined as one word (the Assignment Value object). It has the following columns:

Column	Description
Name	The name of the parameter.
Parameter Value	The current value of the parameter.

Column	Description
Details	Explanation of where the parameter occurs.

- The third table shows errors in which Assignment Values and Assign Assignments correspond to parameters that are never defined in their lower hierarchy. In other words, there is assignment in the hierarchy, but there is no connection definition for that assignment. It has the following columns:

Column	Description
Name	The name of the parameter.
Details	Explanation of where the parameter occurs and the type of object (AV/AA).

Exporting Results to a File

To export results to a `.csv` file, from the **Configuration** tab, click **Export CSV**. The **Export CSV to File Format** dialog box opens. Choose your file name and **Save** the file.

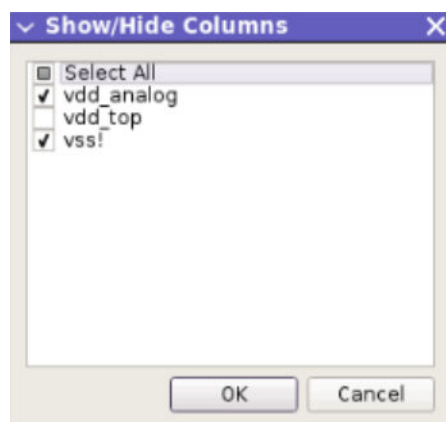
The first fields in the rows of the `.csv` file are digits showing the hierarchy level.

Changing Column Display

You can adjust the columns displayed in the tree views of the **Nets** and **Configuration** tabs.

Hide an individual column by right-clicking the column header and choosing **Hide**.

Right-click a column header and choose **Show/Hide Columns** to select columns to display or hide in the **Show/Hide Columns** dialog box.



Update Estimated Parasitics Utility

The Update Estimated Parasitics utility allows you to recalculate the R/C values automatically when there is a technology or layout change.

To use the Update Estimated Parasitics utility:

1. Choose **Utilities > Update Estimated Parasitics** from the Schematic Editor main menu bar. The **Update Estimated Parasitics** toolbar appears.



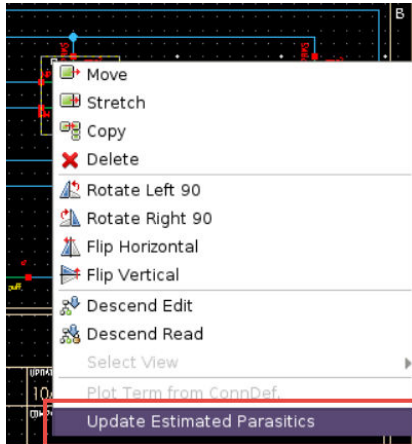
2. Select the necessary options from the toolbar.

The **Update Estimated Parasitics** toolbar has the following options:

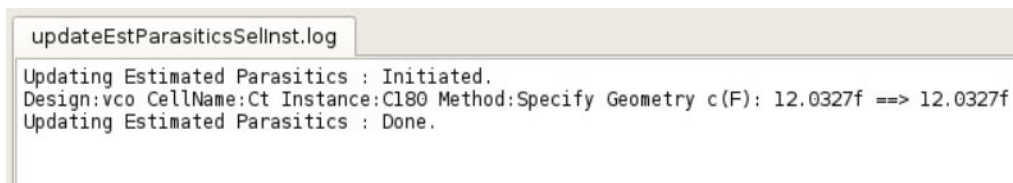
Option or Button	Description
Scope	◦ 0x: Considers parasitic devices at 0x level. ◦ Entire Library: Considers parasitic devices for the entire design library. ◦ Full Hierarchy: Considers parasitic devices for the entire design hierarchy. ◦ Selected Instances: Considers selected instances of parasitic devices.
Options	Displays the Parasitic Options dialog box. See Estimated Parasitics Assistant .
Save	Saves the designs. When the option is unchecked, the utility works like a checker and reports possible changes but does not commit them. (Tcl preference: seUpdateEstParasiticsSaveAndClose)
Calculate	Calculates parasitics.
Close	Closes the dialog box.
Help	Opens the Custom Compiler help.

You can also select an instance on the canvas and recalculate estimated parasitics for the selected instance. To do this:

1. Select an instance (or multiple instances) and right-click to bring up the menu.



2. Choose **Update Estimated Parasitics**. The parasitics are recalculated for the selected instance(s). The Text Editor opens in a new tab showing the log file.



15

Editing HDL Formats With the Text Editor

You can edit the HDL (hardware description language) format for your design using the HDL Text Editor.

Edit a design's HDL format by using the Text Editor.

You can use the Text Editor to view, edit, and perform basic syntactic checks on designs represented as text. This includes designs expressed as SPICE, CDL, VHDL, Verilog, and Verilog-A modules. You can find more information about Verilog-A cells in the *Verilog-A Language Reference Manual*. Find this manual on the internet.

You can use the Text Editor to edit netlists and other text-based views while utilizing a synchronized OpenAccess netlist design database known as *shadow OA*.

You can also use the Text Editor to view instance parameters in the Property Editor in read-only mode. See the *Custom Compiler Environment User Guide* for information on the Property Editor Assistant.

This chapter covers the following:

- [Opening an HDL Text Editor View](#)
- [Using Text Editor Toolbars](#)
- [Setting HDL Text Editor View Display Options](#)
- [Setting HDL Text Editor View Design Options](#)
- [Traversing the Hierarchy of HDL Text Editor Views](#)
- [Editing HDL Text Editor Views](#)
- [Annotating HDL Text Editor Views after Running Simulation](#)

Opening an HDL Text Editor View

Create a Text Editor view when you want to work with the logical view of a design. You can import the textual netlist of a design, such as a VHDL, Verilog, Verilog-A, or Verilog-AMS netlist, for use as a text view.

To create a text view, use one of the following methods:

- From the Library Manager, create the new cellview, and choose an associated Text Editor from the list.

See Creating Designs in the *Custom Compiler Environment User Guide*.

- From a symbol view, create a cellview from cellview.
- Use the Tcl command `db::importText`.

Open a Text Editor view to display formatted netlist text describing the design intent.

To open a text view:

- Open the design cellview, and specify the view associated with the Text Editor, either SPICE, CDL, VHDL, Verilog, Verilog-A, or Verilog-AMS.

For help on opening a design, see Opening Designs in the *Custom Compiler Environment User Guide*.

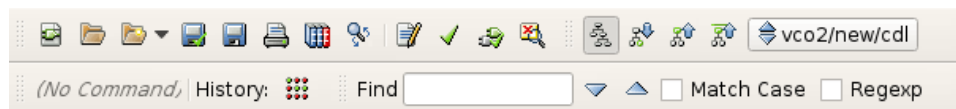
The Text Editor canvas displays the textual netlist using text packets for specific language elements. Each text packet is configurable. In addition, each language element can be bound to a command or action. For example, when the cursor is over an instance name, you can double-click or use the P shortcut key to invoke the `teDescend` action. This causes the current edit design to change to the switched master of that instance.

The Hierarchy Navigator launches automatically when you open a text view.

Using Text Editor Toolbars

Text Editor toolbars offer quick access to text editing operations.

You can display or hide the toolbars in the Text Editor window by choosing **Window > Toolbars**.



The following table lists and describes the Symbol Editor toolbars.

Toolbar	Use this toolbar to...
Command Options	Shows options for the currently enabled interactive command.
Find	Finds text in the editor.

Toolbar	Use this toolbar to...
Hierarchy	Descends in edit or read-only mode, edits the design in place, returns to previous hierarchy level or to top, selects any cellview from the current hierarchy traversal.
PVE	Runs layout versus schematic (LVS) comparisons on the IC Validator.
Standard	Creates new cellviews, opens existing or recent designs, saves and prints designs, opens the Library Manager, as well as searches for and replaces cellviews.

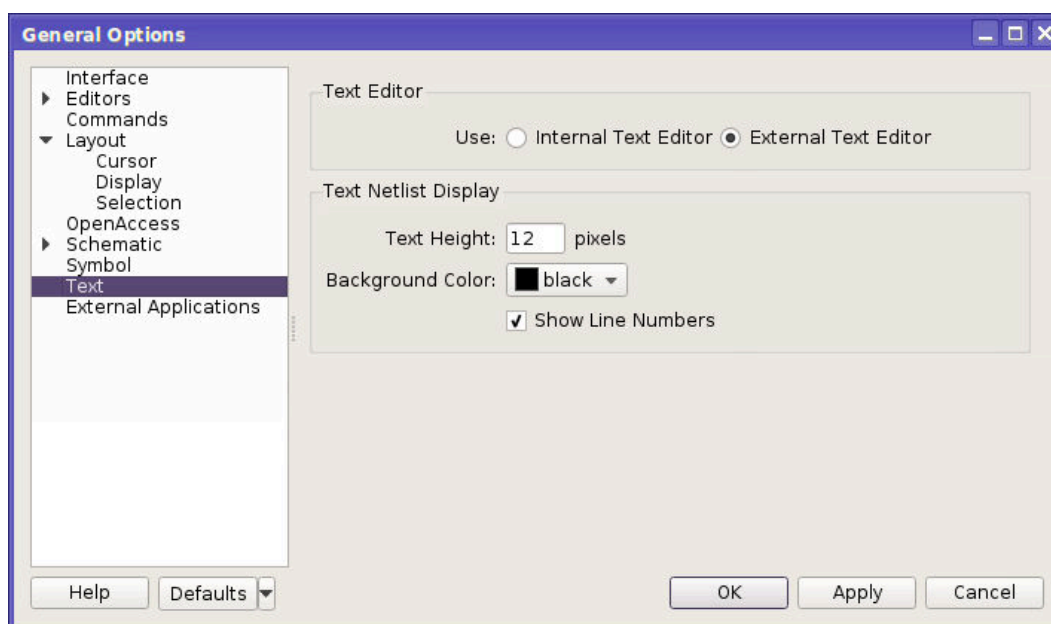
Setting HDL Text Editor View Display Options

View and set Text Editor display options for the current design or view.

For information on using the `te::showDesignOptions` Tcl command, see Text Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To set text view display options:

1. Begin by [Opening an HDL Text Editor View](#).
2. Choose **Options > General** to open the **General Options** dialog box, then switch to the **Text** tab.



3. In the **Text** tab, specify the following options:

Option	Description
Text Editor	
Use	Choose one of the following: ◦ Internal Text Editor ◦ External Text Editor (default) See Editing HDL Text Editor Views . Tcl preference: <code>teEditExternal</code>
Text Netlist Display	
Text Height	Specifies the display text height (the default is 12 pixels).
Background Color	Sets the display background color, black or white (the default is black).
Show Line Numbers	Adds line numbers to the text display (the default is on). Tcl preference: <code>teShowLineNumbers</code>

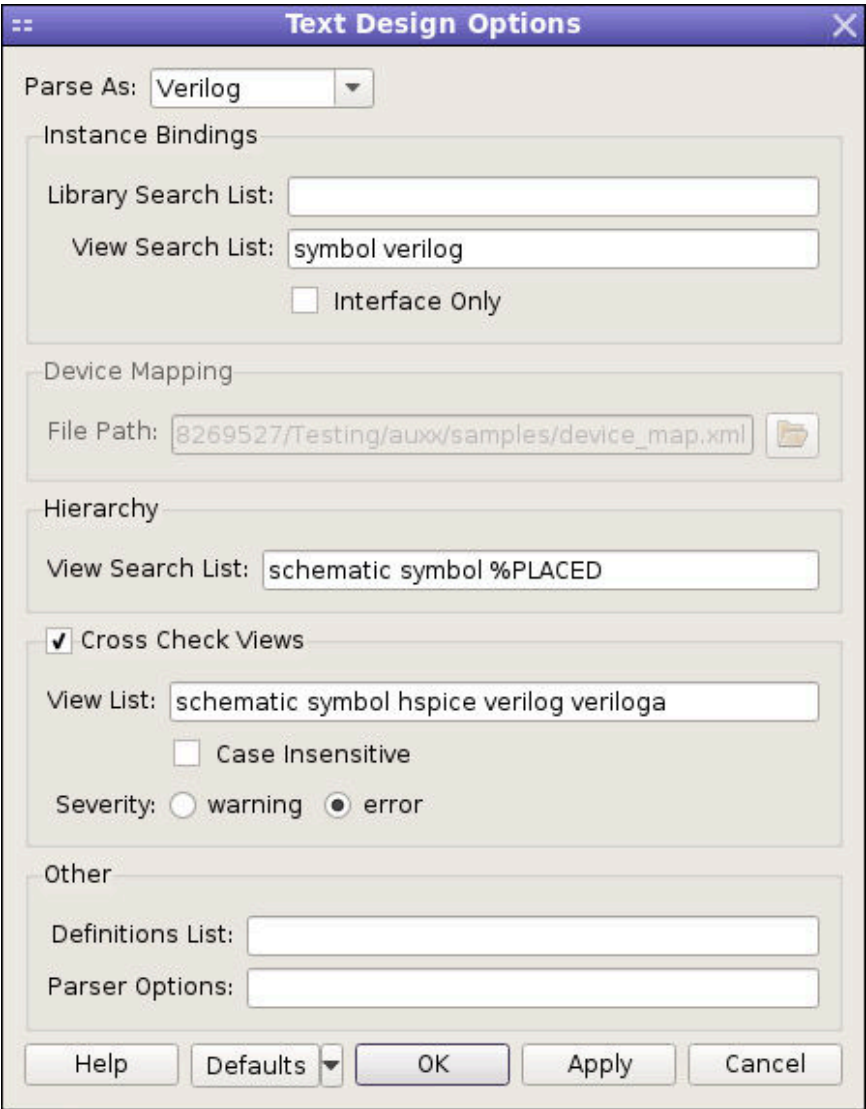
Setting HDL Text Editor View Design Options

View and set Text Editor design options for the current design or view.

For information on using the `te::showDesignOptions` Tcl command, see Text Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

To set text view design options:

1. Begin by [Opening an HDL Text Editor View](#).
2. Choose **Options > Design**. The **Text Design Options** dialog box opens.



3. In the **Text Design Options** dialog box, specify the following options:

Option	Description
Parse As	Specifies parser as Verilog , VerilogA , VerilogAMS , or SystemVerilog .

Option	Description
Instance Bindings	Library Search List is the space-separated list of library names to search when determining a master for a netlist instance. The keyword %PLACED uses the current library name. View Search List is the space-separated list of view names to use when searching for a master for a netlist instance. Interface Only extracts only the interface of the current text view. This is useful for improving performance for very large designs, which are typically extracted.
Device Mapping	(For SPICE/CDL only) File Path specifies the path to the device mapping file.
Hierarchy	View Switch List specifies the views to be considered and order that will be used when descending into a cell. In order for a view to be considered it must exist in the view search list. Note that applications such as Hierarchy Editor (configs), the PrimeWave Design Environment, and SDL ignore this list because they have their own view lists.
Cross Check Views (when check marked)	View List specifies views to use when running cross-view checks with text views. Case Insensitive ignores case differences when comparing objects, such as terminal names, between views. Set Severity with one of the following • warning shows cross-check violations as warnings. • error shows cross-check violations as errors.
Other	Definitions List adds an additional list of definitions that is be predeclared for Verilog, VerilogA, and VerilogAMS views. Parser Options (for VCS parser flow only) adds different options to parse the netlist after a Verilog text view is created. For example, adding -v2005 or -v2k can support different types of construction in the Verilog netlist.

Traversing the Hierarchy of HDL Text Editor Views

Traverse the hierarchy of a Text Editor view to see the various levels of the design as represented in text format.

To traverse a text view:

1. Begin by [Opening an HDL Text Editor View](#).

Opening a text view opens the Text Editor in non-editing mode.

2. Use the **Design > Hierarchy** menu commands to traverse the hierarchy.

Editing HDL Text Editor Views

Edit a Text Editor view to edit the text version of a design.

To edit a text view:

1. Begin by [Opening an HDL Text Editor View](#).

Opening a text view opens the Text Editor in non-editing mode.

(Optional) To open this view in an external text editing tool, set the `teEditExternal` Tcl preference to *true* on startup using the `db::setPrefValue` Tcl command. See [Setting HDL Text Editor View Display Options](#) for setting Text Editor display options.

(Optional) You can specify the external editor you want to set as your default on **External Applications** in the **General Options** dialog box, or set the `deTextEditor` Tcl preference in the Console as follows:

```
db::setPrefValue deTextEditor -value "xterm -e \${EDITOR}"
```

2. On the **Standard** toolbar, click **Edit Text** .

Color-coding disappears while in edit mode. (During editing, the system does not parse the text to determine the object types.)

3. Using keyboard shortcuts, edit the text to make changes. (For example, you can use Ctrl+X to cut text, Ctrl+C to copy, Ctrl+V to paste, and Ctrl+A to select all.) All editing operations are logged.

4. (Optional) After editing, validate the syntax changes by clicking **Check Syntax**  on the **Standard** toolbar.

If any errors are found, the Marker Browser opens and lists all found errors in their hierarchy locations.

See the [Marker Browser Assistant](#).

5. (Optional) You can perform a cross-view check by clicking **Cross View Check** .

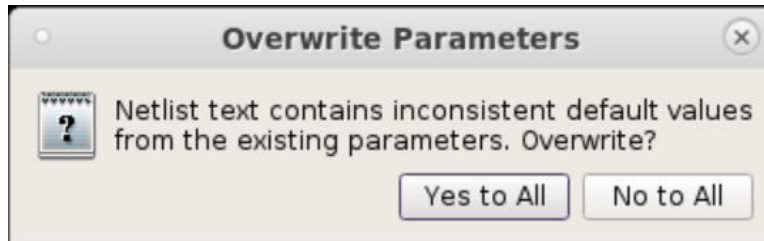
Cross-view checking compares the current text view with the list of views specified with Text Editor design options.

See the console for the completion message, and any violations.

The tool reports a warning if multiple module definitions in one cell are found.

6. Return to non-editing mode by clicking **Edit Text** .

When re-parsing CDF parameters after you modified a netlist file in the Text Editor, the **Overwrite Parameters** question box appears, which can be set through the `dbImportTextOverwriteCDFValue` Tcl preference.



Click **Yes to All** to always update the CDF parameters. Click **No to All** to avoid making any changes to the CDF parameters.

Annotating HDL Text Editor Views after Running Simulation

After running simulation in the PrimeWave Design Environment, it is possible to back-annotate those results to the text editor view.

1. Begin by [Opening an HDL Text Editor View](#).
2. Run simulation in the PrimeWave Design Environment. See the *PrimeWave Design Environment User Guide* for instructions.
3. Back-annotate the simulation results from the PrimeWave Design Environment. See Plotting, Printing, and Reviewing Simulation Results in the *PrimeWave Design Environment User Guide* for instructions.

After simulation and back-annotation from the PrimeWave Design Environment, net voltage value is shown below net names.

16

Editing Netlist Format Designs

You can work with netlist design representations when you do not need to use the canvas.

Use the Netlist Editor to work with a design in netlist format.

This editor shows the logical components of a design in a non-graphical representation. For example, you can use the *verilog2oa* utility to import a design to the Custom Compiler tool, and then use the Netlist Editor to view the design's instances, terminals, and nets. You can also use this editor to cross-check the netlist with other views.

This chapter covers:

- [Opening Netlist Views](#)
- [Setting Netlist Editor Design Options](#)
- [Working With a Netlist View](#)

Opening Netlist Views

Open a netlist view to see the instances, terminals, and nets contained in a design's non-graphical representation.

To open a design netlist view:

- Open the design, and specify the netlist view.

The netlist view is associated with the **NE - netlist** editor.

To open a design, see Opening Designs in the *Custom Compiler Environment User Guide*.

Setting Netlist Editor Design Options

Set netlist editor design options to select the view search list for hierarchical navigation of the design, and to set options for cross-checking the netlist view against other views of the cell.

To set netlist editor design options:

1. Begin by [Opening Netlist Views](#).
2. Choose **Options > Design**.
3. In the dialog box that opens, specify the options as described in the table that follows.

The following table lists Netlist Editor design options.

Option	Description
Hierarchy	View Switch List specifies the list of top level views you want to switch into.
Cross Check Views (when check marked)	View List specifies these views for cross checking comparison with netlist views. Case Insensitive ignores case differences when comparing objects, such as terminal names, between views.
Severity	Warning shows cross check violations as warnings. Error shows cross check violations as errors.

4. Click **OK**.

Working With a Netlist View

Work with the netlist view of a design to visually check the logical components at the top logic level, to navigate to switched views, and to perform on-demand cross view checks between netlist view, and other views for the cell.

To work with a netlist view:

1. Begin by [Opening Netlist Views](#).

Opening the netlist view of a design opens the Netlist Editor, and shows the design logic in three columns listing instances, terminals, and nets.

2. Evaluate the schematic logic through the netlist view:

- To open an instance, double-click it by name.

Navigation uses the view switch list set through Netlist Editor design options. See [Setting Netlist Editor Design Options](#).

- To navigate, click the **Hierarchy Navigator** buttons on the toolbar.

(For navigation details, see Hierarchy Navigator in the *Custom Compiler Environment User Guide*.)

- To work with properties for a selected instance, click the Property Editor button in the toolbar.

(For property editing details, see Property Editor in the *Custom Compiler Environment User Guide*.)

- To perform a cross view check between the current netlist view, and the list of views specified as Netlist Editor design options, click the **Cross View Check** button.

If any errors are found, the Marker Browser opens, and lists the first found error in its hierarchy location.

See the [Marker Browser Assistant](#).

17

Checking Designs

This chapter describes the kinds of checks you can perform on your designs and check procedures.

This chapter covers the following topics:

- [Electric and Semantic Rule Checking Overview](#)
- [Specifying ERC and SRC Rules](#)
- [Running Checks](#)
- [Cross-View Checking](#)
- [Viewing Violation Information](#)
- [Working With Hierarchy](#)

Electric and Semantic Rule Checking Overview

The Schematic Editor lets you perform electrical and semantic rule checks, commonly referred to as ERC and SRC checks, as described in the following table.

A design is considered *connectivity valid* when all the checks have been executed and there are no violations with a severity level of error. A timestamp property is generated for valid designs.

Checks	Description
ERC	Ensure that the connectivity of the schematic is correct; for example, verifying that two output pins are not electrically connected.
SRC	Ensure that the data is semantically correct; for example, verifying that instance and net names are correct, that global net usage is valid, or that physically connected wires do not have a name short.

Specifying ERC and SRC Rules

At any time in the schematic capture process, you can launch the ERC/SRC checker from the **Check** menu. The default behavior is to check within the bounds of the current cellview, including cross-view checking.

The **Check Options** dialog box consists of a category tree and an options display area that changes dynamically as you expand and select categories. See [Configuring Check Options](#) for instructions.

You can configure:

- **General** options that let you configure the check operation for the following behaviors:
 - Start the Marker Browser automatically, see [Managing Markers](#) for more information
 - Display violations
 - Prompt you to save if errors are found

Note:

By default, if you opt to save a cellview that contains errors, you are warned. Some downstream tools (such as netlisters) might not be able to handle errors such as schematic name shorts.

- Enable the cross-view checker, see [Cross-View Checking](#) for more information
 - Which view list to use when performing the cross-view checking
- **Advanced** options that let you configure the check operation for filters to libraries and views, and also additional rule options.
- **Rules** setup for various types of checks, see [Setting Up Check Rules](#) for more information.

This section describes the following topics:

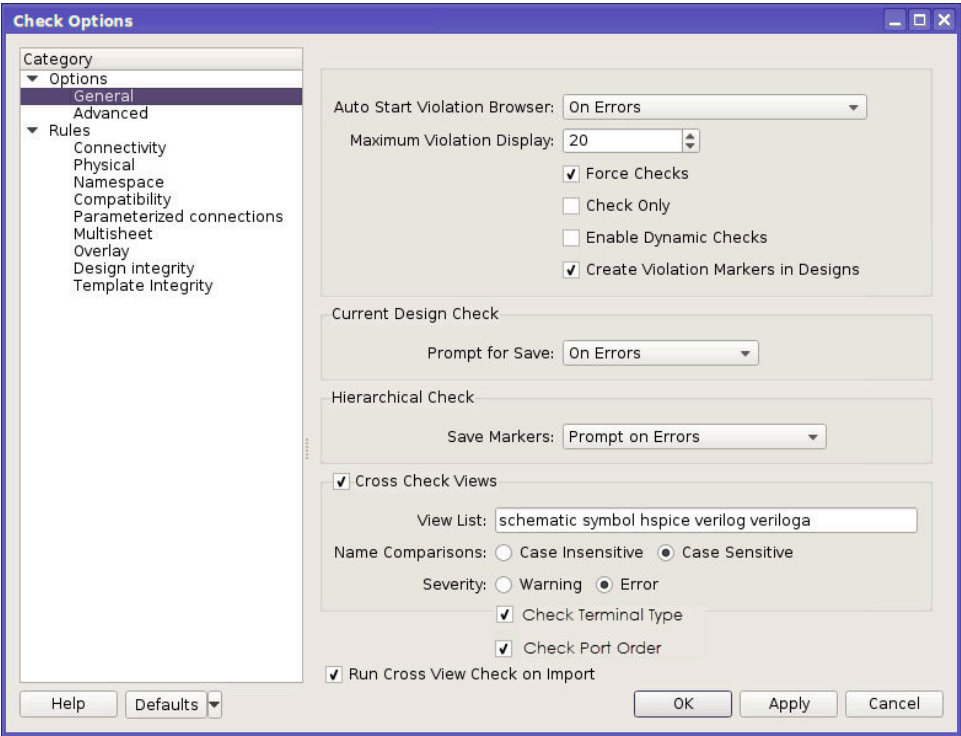
- [Configuring Check Options](#)
- [Managing Markers](#)
- [Setting Up Check Rules](#)

Configuring Check Options

To configure the check options:

1. Choose **Check > Setup**.

The **Check Options** dialog box appears.

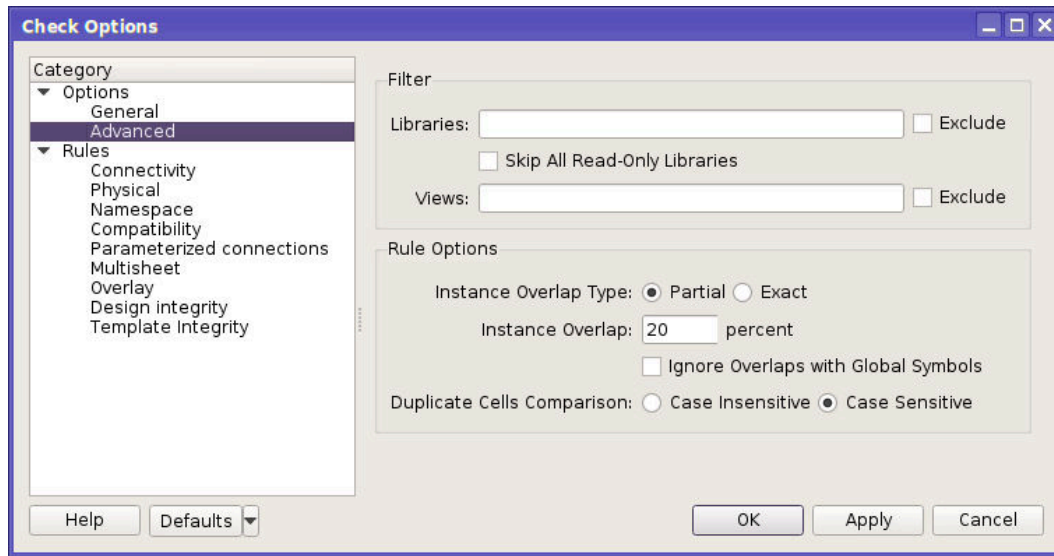


2. Configure the general options as described in the following table.

Option Group	Description
Auto Start Violation Browser	Controls whether the Marker Browser assistant launches automatically. ◦ On Errors (default): Starts when the top design contains errors. ◦ On Warnings or Errors: Starts when the top design contains warnings or errors. ◦ Never: Do not start the marker browser automatically. Tcl preference: <code>seCheckAutoStartBrowser</code>
Maximum Violation Display	Sets the maximum number of violations to display in the Console per rule. The default is 20. Tcl preference: <code>seCheckMaximumViolationDisplay</code>
Force Checks	Checks designs first to see if rules need to be executed. Tcl preference: <code>seCheckForced</code>

Option Group	Description
Check Only	Checks designs without creating any markers. Tcl preference: <code>seCheckNoMarkers</code>
Enable Dynamic Checks	Enables any dynamic checking that performs when the end transaction happens. Tcl preference: <code>seCheckDynamic</code>
Create Violation Markers in Designs	When enabled, creates OA markers in checked designs. When disabled, creates violation markers in an overlay to avoid modifying designs. Tcl preference: <code>seCheckCreateMarkerInDesign</code>
Delete Preexisting Markers	Choose one of the following to decide the removal of error markers after performing Check or Check and Save : All (default): Deletes error markers created by all tools. SRC/ERC Only : Deletes error markers created by SRC/ERC only. Tcl preference: <code>seCheckDeletePreexistingMarkersMode</code>
Current Design Check	Prompt for Save decides when to save designs in non-hierarchical checking. ◦ On Errors (default): Saves when you prompted with errors. ◦ On Warnings or Errors: Saves when you prompted with warnings or errors. ◦ Always: Always saves designs with violations. ◦ Never: Do not save designs with violations. Tcl preference: <code>seCheckPromptCurrent</code>
Hierarchical Check	Save Markers decides when to save designs with violations during hierarchical checking. ◦ On Errors (default): Saves when you prompted with errors. ◦ On Warnings or Errors: Saves when you prompted with warnings or errors. ◦ Always: Always saves designs with violations. ◦ Never: Do not save designs with violations. Tcl preference: <code>seCheckAutoSave</code>
Cross Check Views	Performs cross-view checking before running the rules for a design. See Cross-View Checking in the Schematic Editor . Tcl preference: <code>seCheckCrossViews</code>
Run Cross View Check on Import	Performs cross-view checking when importing netlist.

3. Switch to **Advanced** to show the advanced options in the **Check Options** dialog box.



4. (Optional) Configure the advanced options as described in the following table.

Option Group	Description
Filter	Decides when to start the marker browser automatically. ◦ Libraries and Views let you enter lists of libraries and view names. Each list filters designs during processing. The elements of the list can be given in exact match format. ◦ Skip All Read-Only Libraries lets you add all the read-only libraries to the exclusion list. ◦ When Exclude is disabled (default), the list represents libraries or view names to use. Only those designs that have a matching library or view name are considered for processing. If a list is empty, all names are considered. ◦ When Exclude is enabled, the library or view names listed become an exclusion list. Designs that match a library or view name from an exclusion list are skipped and are not processed. Tcl preferences: <code>seCheckLibraryFilter</code>, <code>seCheckViewFilter</code>, <code>seCheckLibraryFilterIsExclude</code>, <code>seCheckSkipReadOnlyLibraries</code>

Option Group	Description
Rule Options	Configures the following options: ◦ Instance Overlap Type uses Partial (default) or Exact overlap type in instance overlap checking. ◦ Instance Overlap lets you modify the preference value used by rule implementations. The check flags instance bounding boxes that overlap one another by the specified percentage. The default is 20. ◦ Ignore Overlaps with Global Symbols ignores instance overlaps with global symbols. ◦ Duplicate Cells Comparison compares cell names in Case Insensitive or Case Sensitive (default) manner. Tcl preferences: <code>seCheckOverlapTypeIsPartial</code>, <code>seCheckInstOverlap</code>, <code>seCheckIgnoreOverlapsWithGlobals</code>, <code>seCheckDuplicateCellsCaseSensitive</code>

5. (Optional) Click **Apply**.

The dialog box remains open and active.

6. Click **OK** to apply the settings and close the dialog box.

Managing Markers

Failed ERC and SRC checks generate markers near the offending objects. You can locate and delete these markers using the Marker Browser assistant. See [Marker Browser Assistant](#).

For read-only designs, you can disable the **Create Violation Markers in Designs** option in the **Check Options** dialog box to run ERC or SRC checks. In this case, set the `seUseErrorViewer` Tcl preference to true to use the Error Viewer assistant to display the violation markers. See Error Viewer Assistant in the *Custom Compiler Environment User Guide*.

Setting Up Check Rules

Choose **Check > Setup** to open the **Check Options** dialog. Access the check options from the **Rules** branch of the **Category** tree. The rules that you can set are categorized as follows:

- [Connectivity Checks](#)
- [Physical Checks](#)
- [Namespace Checks](#)
- [Compatibility Checks](#)

- [Parameterized Connection Checks](#)
- [Multisheet Checks](#)
- [Overlay Checks](#)
- [Design Integrity Checks](#)
- [Template Integrity Checks](#)
- [User Checks](#)

Connectivity Checks

The connectivity checks shown in the following table let you specify checks for the following:

Check	Description
Floating Input Pins	Flags schematic pins of type input that are not connected to: • A schematic output pin, I/O, switch or tristate pin • An instance input pin, I/O, or switch pin • A global net • A net with a connection definition
Floating Input Instance Pins	Flags instance (symbol) pins of type input that are not connected to: • A schematic output pin, I/O, switch or tristate pin • An instance input pin, I/O, or switch pin • A global net • A net with a connection definition
Signal Without Source	Flags signals that do not have a connection to output instance terminals or input schematic terminals.
Floating Output Pins	Flags schematic pins of type output that are not connected to: • A schematic output pin, I/O or switch pin • An instance input pin, I/O, switch, or tristate pin • A global net • A net with a connection definition
Floating Output Instance Pins	Flags instance (symbol) pins of type output that are not connected to: • A schematic input pin, I/O, switch or tristate pin • An instance output pin, I/O, or switch pin • A global net • A net with a connection definition
Floating I/O Pins	Flags schematic pins of type inputOutput that are not connected to: • Any schematic pin type: input, output, I/O, switch, or tristate • Any instance pin type: input, output, I/O, switch, or tristate • A global net • A net with a connection definition

Check	Description
Floating I/O Instance Pins	Flags instance (symbol) pins of type inputOutput that are not connected to: - Any schematic pin type: input, output, I/O, switch, or tristate - Any instance pin type: input, output, I/O, switch, or tristate - A global net - A net with a connection definition
Floating Switch Pins	Flags schematic pins of type switch that are not connected to: - A schematic input pin, I/O, or switch pin - An instance output pin, I/O, switch, or tristate pin - A global net - A net with a connection definition
Floating Switch Instance Pins	Flags instance (symbol) pins of type switch that are not connected to: - A schematic input pin, I/O, or switch pin - An instance output pin, I/O, switch, or tristate pin - A global net - A net with a connection definition
Floating Tristate Pins	Flags schematic pins of type tristate that are not connected to: - A schematic input pin, I/O or switch pin - An instance output pin, I/O, switch, or tristate pin - A global net - A net with a connection definition
Floating Tristate Instance Pins	Flags instance (symbol) pins of type tristate that are not connected to: - A schematic input pins, I/O or switch pin - An instance output pins, I/O, switch, or tristate pin - A global net - A net with a connection definition
Floating Nets	Flags any wire that lacks a physical connection or a connect by name to a schematic or instance pin.
Shorted Output Pins	Flags a symbol output pin that is connected to any other symbol output pin.
Name Shorts	Checks for direct connections of two nets that are explicitly named either by wire label or global connection.
Connections by Name	Flags all by-name connected nets in a design, excluding global net connections
Schematic Pin on Conflicting Subnet	Flags all connections between a wire segment on a named net and a schematic pin on a terminal name that is different from the net name
Wire Label on Conflicting Net	Checks for wire names that do not dictate the net of the parent object.

Check	Description
Terminal/Global Net Short	Flags connections between non-global terminals and global nets, or between global terminals and global nets whose names conflict.
Connection Width Mismatch	Flags pins with connection width mismatches. Connection widths match if the number of bits in the pin is evenly divisible by the number of bits in the connecting net. For example, a pin "a<0:3>" can be connected to net "b<0:1>" because the number of bits in the pin (4) is evenly divisible by the number of bits in the net (2). In this case the connectivity of the pin is: a<0> => b<0>, a<1> => b<1>, a<2> => b<0>, a<3> => b<1>.
Pin BitTerm Inconsistency	For old OA to prevent bus terminal (ex: A<1:1>) and bit terminal (ex: A<1>) connected to different nets. Default: Enabled
"portOrder: Property Mismatch	Flags pins or the entire design if there is a mismatch between the terminals found in the design and the portOrder property of the design. For example: the portOrder property contains invalid (non-OA-compliant) terminal namesnon-empty portOrder while no terminals in designempty "portOrder" while there are terminals in designexisting terminal names are absent in port ordernonexistent terminal names are mentioned in port order

Physical Checks

The physical options shown in the following table let you specify checks for the following:

Check	Definition
Multiple VFlag Connections	Flags connections to multiple Vflag symbols on the same net.
Unconnected Wires	Flags wire segments that are either not labeled or that have endpoints that do not intersect other segments or symbol/schematic pins.
Solder on Crossing Wires	Flags 4-way connections formed by a solder on a pair of crossing wires. Such connections can be inadvertently formed by starting a route from a solder on a T-junction.
Overlapping Instances	Flags instance bounding boxes that overlap one another, and, if so, by what percentage.

Check	Definition
Implicit Vectored Nets	Flags the case where the width of a net is connected to a terminal of equal width on an iterated instance. In order to make the connection, the Schematic Editor must increase the width of the net via a multiplication factor equal to the number of iterations of the instance. For example, if a symbol pin whose terminal is A<0:2> (width of 3) is on a vectored instance with dimension 2, then the width of the instance specific symbol pin is $3 * 2 = 6$. If the connecting net is of dimension 3, for example net<4:6>, then the oalnstTerm is connected to a bundle named <*2>net<4:6>. This in effect connects all corresponding terms of the vectored instance group; that is, net<4> from the first groups is connected to net<4> from the second group, net<5> from the first groups is connected to net<5> from the second group, and so on.
Ambiguous Connections	Checks for connections whose logic cannot be reliably interpreted, as illustrated in the following example:
Failed Connections	Checks for non-orthogonal routing over symbol pins, schematic pins, and solder dot bounding boxes.
Orphan Wire Labels	Flags wire labels that do not belong to a wire segment in the design.
Hidden Wire Labels	Checks for hidden wire labels.
Unbound Instances	Flags instances in the design for which the placed master does not exist.
Off Grid Object	Flags a design object (can be an instance, pin, wire, graphical or text object, and so forth) that is placed off-grid.

Namespace Checks

The namespace options shown in the following table include specific checks for a given application.

Check	Description
Verilog	Flags any identifiers (instance, net or pin names) that are illegal in the Verilog namespace and are name mapped.
HSPICE	Flags any identifiers (instance, net or pin names) that are illegal in the HSPICE namespace, and are name mapped.

Compatibility Checks

The compatibility options shown in the following table let you specify the following checks:

Check	Description
Global Net Names	Checks for the existence of global signals whose names do not end in "!". In the Schematic Editor there are no restrictions as to what a global signal can be named, although the same is not true of tools from other providers.
Global Instances	Checks for any instances of components that globalizes a net. In the Schematic Editor, the analogLib/makeGlobal component is provided for such a purpose; any net that is attached to the pin of this component has its isGlobal attribute set to true regardless of the net name. Because this promotion is done with properties on the component's terminal, devices other than analogLib/makeGlobal might be used for this purpose.
Cellview Wide Assignments	Checks for the presence of assignments that are placed on a cellview as opposed to one of the instances within the cellview. These are supported in OpenAccess, but are not supported in every tool that uses it.
ConnectDefs As Assignments	Checks for the presence of oaAssignAssignments in the design. These are connection assignments that are themselves connection definitions. These are supported in OpenAccess, but are not supported in every tool that uses it.
ConnectDefs Default Values	Checks for connection definitions in a design that have default values that do not match the nets upon which they are placed, such as placing a connection definition [@pwr:%:vdd3v] on a labeled wire myvdd. This is supported in OpenAccess, but not allowed by some tools.
ConnectDefs on Non Global Nets	Checks for connection definitions placed on nets that are not global. This is supported in OpenAccess but is not allowed by some tools.
ConnectDefs Non Global Defaults	Checks for connection definitions that have a default net that is not a global net. This is supported in OpenAccess, but is not allowed by some tools.

Check	Description
Terminal Expressions	Checks for parameterized connections that specify a terminal expression as the default value.
PEL Expressions	Checks for Parameter Expression Language in the design. These are the functions: parent(), instance(), and lineage.
Shorted Terminals	Checks for shorted terminals in the design.
Unsupported Parameters	Checks for unsupported parameters.

Parameterized Connection Checks

The parameterized connection options shown in the following table let you specify the following checks:

Check	Description
ConnectDefs On Named Nets	Flags connection definitions on named nets.
Terminal ConnectDef Default Net	Flags invalid terminal connectDef defaults.
Terminal ConnectDef Net Conflict	Flags terminal connectDef net conflicts.

Multisheet Checks

The multisheet options shown in the following table let you specify the following checks:

Check	Description
Same Pin on Different Sheets	Checks whether pins of the same terminal exist on different sheets of the schematic.
By Name Connections Between Sheets	Checks whether by name connection exists between different sheets.
Off Sheet Objects	Checks for existence of objects placed outside sheet frames.

Overlay Checks

The overlay options shown in the following table let you specify the following checks:

Check	Description
Invalid Mask	Flags invalid mask.
Invalid Clip Mask	Flags invalid clip mask.
Overlay Masked Pin	Flags overlay masked pin.

Design Integrity Checks

The design integrity options shown in the following table let you specify the following checks:

Check	Description
Duplicate Cells in Current View	Checks whether the design has two instances pointing to the same named cells from different libraries.
Circular Reference	Checks whether a circular reference exists in the given design hierarchy. All views are traversed to detect the circular reference to make sure all possible hierarchy configurations are validated against the circular reference.
Duplicate Cells	Checks whether cells with the same name exist in the design hierarchy. The check traverses the design hierarchy through switched masters, collects all lib/cell pairs, and then checks whether same named cells are encountered during the traversal. A specific case of a violation would be when a design has two instances with switched masters that have the same cell names, but are in different libraries.
Globals Shorted thru Hierarchy	Covers the cases when global nets are physically connected through hierarchy Example 1: Two terminals are physically connected in the child design. In the upper level of hierarchy, they are connected through instance terminals to differently named global nets. Example 2: A global net is connected to the terminal in the child design. In the parent design, the term is connected to a differently named global net through an instance terminal. Note that if the cell which has shorted terminals is connected in the upper level to non-global nets, it is not an ERC/SRC conflict because one of those nets is mapped to the other during netlisting.

Template Integrity Checks

The template integrity options shown in the following table let you specify the following checks:

Check	Description
Missing Master	Flags placed templates where the template master LCV no longer exists (in other words, has been deleted or corrupted due to operations on the template library after the template was placed in the schematic). Treated as a warning by default.
Connectivity Mismatch	Flags placed templates where the connectivity in the template master LCV is different from the connectivity in the placed devices (for example, the template was edited in the template library after it was placed in the schematic). Treated as an error by default.
Out-of-Date	Flags placed templates where the template master is newer than the template that was placed in the schematic. (Specifically, the date stamp on the template master LCV is newer than the timestamp recorded when the template was placed in the schematic.) Treated as a warning by default.
Out-of-Date-Intent	Flags placed templates in which Templatize Layout was checked and used to specify a template library/cell (or library/cell/view) used to guide layout, in which the specified library/cell or library/cell/view is now newer than the timestamp recorded when the template was placed in the schematic. (For example, the template LC or LCV specified in Templatize Layout was edited in the template library after it was placed in the schematic.) Treated as a warning by default.
Template Self-Coherence	Flags template schematic views where the connectivity or instance names are no longer the same as those specified in the template cell's own netlist view (this can occur if you hand-edit the template schematic view after it was originally created, modifying the connectivity or names). Treated as an error by default.

User Checks

The user-defined checks shown in this section are defined using the `-options` argument in the `se::createRule` Tcl command. See `se::createRule` for examples of how to set up these user-defined checks.

Running Checks

After you configure the check options, there are several check operations you can perform:

- [Checking and Saving the Current Design](#)
- [Checking the Current View](#)
- [Checking the Hierarchy](#)

Checking and Saving the Current Design

The **Check and Save** command lets you run all of the enabled ERC and SRC checks on the current design, including the cross-view check, and save the design.

Saving the design in conjunction with the **Check** operation saves existing markers as well as changes made to the design before the check operation. If you enable the **Prompt for save** option in the **Check Options** dialog box, then additional prompting could occur before the save operation completes, depending on the option value you select.

To check and save a design:

- Select the **Design > Check and Save** command.

The design is checked and then saved.

For more information about defining ERC and SRC checks, see [Electric and Semantic Rule Checking Overview](#).

For more information about the Check and Save operation, see Checking and Saving Individual Design Schematics in the *Custom Compiler Environment User Guide*.

Checking the Current View

At any time in the schematic capture process, you can launch the ERC/SRC checker from the **Check** menu. All of the enabled ERC and SRC checks are run on the current design, including the cross-view check, if enabled.

To check the current cellview:

- Choose **Check > Current View**.

If no violations are found a confirming message appears in the console.

If violations exist:

- A message box appears prompting whether you want to save the design with errors (Default). The **Prompt for save** option in the **Check Options** dialog box controls prompting.
- For each violation, a message appears in the console.
- The Marker Browser assistant opens and lists the violation markers (Default). The **Auto Start Marker Browser** option in the **Check Options** dialog box controls whether the assistant launches automatically. For information about using the Marker Browser assistant, see [Marker Browser Assistant](#).

Checking the Hierarchy

The **Check > Hierarchy** command lets you run the predefined ERC and SRC checks on all cellviews in the design hierarchy.

To check the hierarchy:

- Choose **Check > Hierarchy**.

If no violations are found, a confirming message appears in the console for each design in the hierarchy.

If violations exist at the top level:

- A message box appears prompting whether to save the design with errors (Default). The **Prompt for save** option in the **Check Options** dialog box controls prompting.

If violations exist at any level:

- For each violation, a message appears in the console.
- Markers are created in each design, if you have not enabled the **Check Only** option in the **Check Options** dialog box.
- The Marker Browser assistant opens and lists the violations (default). The **Auto Start Marker Browser** option in the **Check Options** dialog controls whether the assistant launches automatically. For information about using the Marker Browser assistant, see [Marker Browser Assistant](#).

Cross-View Checking

Use the Cross-View checker to check the consistency of terminals among the views of a given cell. Consistency of terminal names is checked, as well as the terminal type (direction).

The following sections describe how to perform cross-view checks:

- [Cross-View Checking in the Schematic Editor](#)
- [Cross-View Checking in the Symbol Editor](#)

Cross-View Checking in the Schematic Editor

The following sections describe how to set up and run a cross-view check from the Schematic Editor.

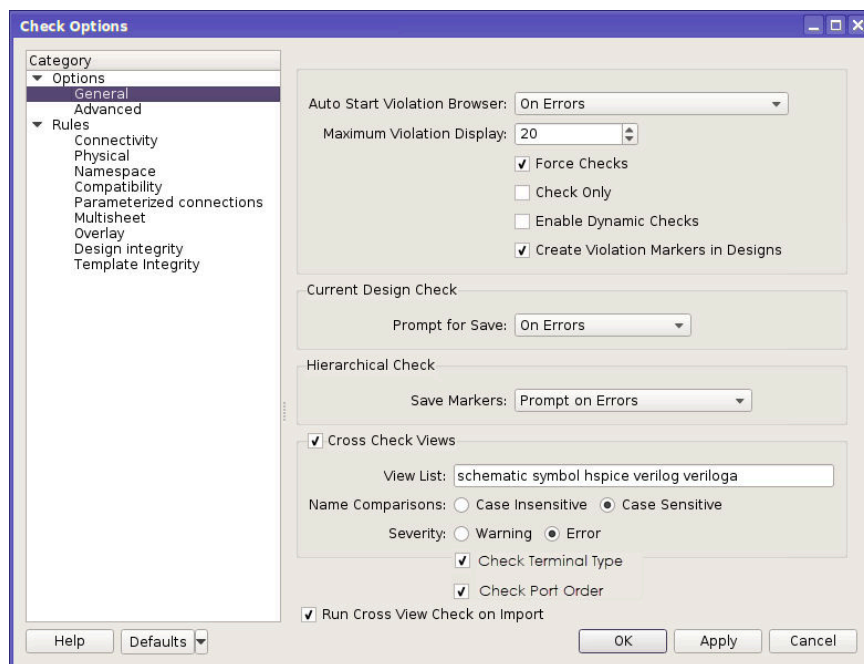
- [Configuring Check Options](#)
- [Running Checks](#)

Configuring Check Options

To configure the cross-view check options in the Schematic Editor:

1. Choose **Check > Setup**.

The **Check Options** dialog box opens.



2. Enable **Cross Check Views** in the dialog box.
3. Specify the options you want to apply.

Option	Description
View List	The list of views to compare against the current view.
Name Comparisons	Determines whether the comparison of terminal names is done case-sensitive or case-insensitive. ◦ Case Sensitive ◦ Case Insensitive
Severity	Specifies whether violations are treated as warnings or errors. ◦ Warning ◦ Error
Check Terminal Type	Controls whether terminal types should be checked in cross view checking. Default: True. Tcl preference: <code>deCrossCheckTermType</code>
Check Port Order	Controls whether port orders should be checked in cross view checking. Default: True. Tcl preference: <code>deCrossCheckPortOrder</code>

4. Click **OK** or **Apply**.

The setup is complete.

Running Checks

To run a cross-view check from a schematic:

1. Enable the **Cross Check Views** option in the **Check Options** dialog box. See [Configuring Check Options](#).
2. Choose **Check > Current View**.

If no violations are found, a confirming message appears in the console.

If violations exist:

- A message box appears prompting whether to save the design with errors (default). The **Prompt for save** option in the **Check Options** dialog box controls prompting.
- If you have not enabled the **Check Only** option in the **Check Options** dialog box, a marker is created for each missing or extra terminal, as well as for terminals that have types that are inconsistent with the other checked views. Note that markers are only created in the view that the cross check command is launched from.

Note:

Markers are created only in the view that the cross check operation is launched from.

- For each violation, a message appears in the console.
- The Marker Browser assistant opens and lists the violations (Default). The **Auto Start Marker Browser** option in the **Check Options** dialog box controls whether the assistant launches automatically. For information about using the Marker Browser assistant, see [Marker Browser Assistant](#).

Cross-View Checking in the Symbol Editor

The following sections describe how to set up and run a cross-view check from the Symbol Editor.

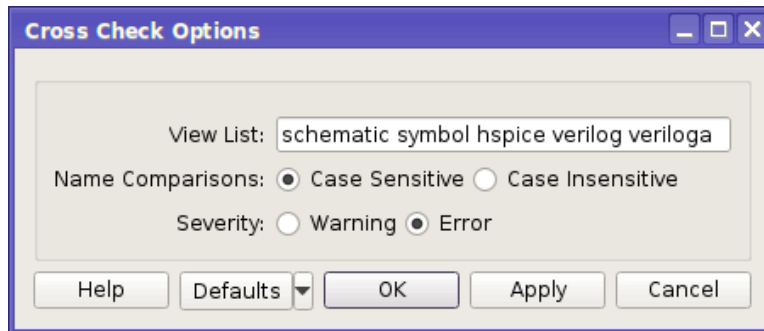
- [Configuring Check Options](#)
- [Running Checks](#)

Configuring Check Options

To configure the cross-view check options in the Symbol Editor:

1. Choose **Check > Setup**.

The **Cross Check Options** dialog box opens.



2. Specify the options you want to apply.

Option	Description
View List	The list of views to compare against the current view.
Name Comparisons	Determines whether the comparison of terminal names is done case-sensitive or case-insensitive. ◦ Case Sensitive ◦ Case Insensitive
Severity	Specifies whether violations are treated as warnings or errors. ◦ Warning ◦ Error

3. Click **OK** or **Apply**.

The setup is complete.

Running Checks

To run a cross-view check in the Symbol Editor:

- Choose **Check > Cross View Check**.

If no violations are found, a confirming message appears in the console.

If violations exist:

- If you have not enabled the **Check Only** option in the **Check Options** dialog box, a marker is created for each missing or extra terminal, as well as for terminals that have types that are inconsistent with the other checked views. Note that markers are only created in the view that the cross check command is launched from.

Note:

Markers are created only in the view that the cross check operation is launched from.

- For each violation, a message appears in the console.
- The Marker Browser assistant opens and lists the violations (default). The **Auto Start Marker Browser** option in the **Check Options** dialog box controls whether the assistant launches automatically. For information about using the Marker Browser assistant, see [Marker Browser Assistant](#).

Viewing Violation Information

The following sections describe how to view violation information:

- [Using the Marker Browser](#)
- [Using Probes](#)

Using the Marker Browser

Failed ERC and SRC checks result in the generation of markers placed near the offending objects. You can access information about the markers using the Marker Browser dockable assistant.

For more information on using the Marker Browser, see [Marker Browser Assistant](#).

For information on using the `db::getMarkers` Tcl command, see Database Editing Tcl in the *Custom Compiler Tcl Reference Manual*.

Using Probes

Probes help you visualize a net throughout the hierarchy.

A probe is a logical design bookmark that points to a logical element of a design and is unique to a specific instance path.

You can probe for the following logical elements

- Instances
- Instance Terminals
- Nets
- Buses
- Bundles
- Cellviews
- Terminals

For more information on using probes, see [Adding Probes](#), [Removing Probes](#), and Probe Assistant in the *Custom Compiler Environment User Guide*.

Working With Hierarchy

You can check the design hierarchy using:

- Hierarchy Editor: A top-level editor.
- Hierarchy Navigator: A dockable assistant that displays the design lineage with options to navigate the lineage by double clicking on entries in the assistant. The Hierarchy Navigator does not provide the ability to configure the design.

For more information on using the Hierarchy Editor and Hierarchy Navigator, see the *Custom Compiler Environment User Guide*.

A

Schematic Editor Shortcut Keys

This appendix provides the binding set available with the default Schematic Editor installation.

The following sections describe the Schematic Editor shortcut keys.

- [Schematic Editor Function Shortcut Keys](#)
- [Schematic Editor Keypad Shortcut Keys](#)
- [Schematic Editor Number Pad Shortcut Keys](#)

Schematic Editor Function Shortcut Keys

The following table describes the available function shortcut keys when using the Custom Compiler (default) or Maestro binding sets. For more information about customizing the binding set, see the *Custom Compiler Environment User Guide*.

For the list of keypad and number pad (numpad) shortcut keys, see [Schematic Editor Keypad Shortcut Keys](#) and [Schematic Editor Number Pad Shortcut Keys](#).

Table 34 *Schematic Editor Function Shortcut Keys*

Shortcut Key	Description
F1	Help
F2	Saves design
F3	Shows or hides one of the following: • Toolbar • Dialog box that corresponds to the toolbar, if applicable (only when the pointer is resting on the schematic canvas)
F4	Shows or hides the Schematic Object Filter
F5	Opens the Open Design dialog box
F6	Toggles between Angle modes during creation and editing operations

Table 34 Schematic Editor Function Shortcut Keys (Continued)

Shortcut Key	Description
F7	Runs ERC check on the current view
F8	Checks and saves the design
F9	Add Note
F10	Shows or hides the Schematic Object Filter
F11	Displays or hides the window title bar to expand the Custom Compiler window to full screen
F12	Prints the design
Shift+F3	Shows or hides command toolbar
Shift+F11	Displays or hides the title bar, menu bar, status bar, and all toolbars and assistants within a window tab

Schematic Editor Keypad Shortcut Keys

The following table describes the available main keypad shortcut keys when using the Custom Compiler (default) or Maestro binding sets. For more information about customizing the binding set, see the *Custom Compiler Environment User Guide*.

For the list of function and number pad (numpad) shortcut keys, see [Schematic Editor Function Shortcut Keys](#) and [Schematic Editor Number Pad Shortcut Keys](#).

Table 35 Schematic Editor Keypad Shortcut Keys

Shortcut Key	Description
[	Zoom In
]	Zoom Out
1	Cycle Active Figure Next
2	Cycle Active Figure Previous
3	Pans the viewport
4	Invokes the Align command
A	Selects by clicking on objects
B	Returns one level up

Table 35 Schematic Editor Keypad Shortcut Keys (Continued)

Shortcut Key	Description
C	Copy
D	Deselects by clicking on objects
E	Descend Read
F	Zooms to fit the whole design
G	Toggles on the Marker Browser assistant
H	Descend Read in a new window
I	Add Instance
L	Add Wire Name
M	Stretch
O	Opens Schematic Design Options dialog box
P	Add Pin
Q	Shows or hides Property Editor assistant
R	Rotates counterclockwise 90 degrees
S	Stretch
T	Peek
U	Undo
V	Prints the design
W	Add Thin Wire
X	Runs ERC check on the current view
Y	Opens the Generate CellView from CellView dialog box
Z	Area zoom
Backspace	Removes point during wiring operation
Delete	Deletes selected objects
Enter	Enables on-canvas editing for active objects like label or note OR Adds a wire name if an active object is wire

Table 35 *Schematic Editor Keypad Shortcut Keys (Continued)*

Shortcut Key	Description
Esc	Ends the active command
Space	Adds point during wiring operation
Tab	Focuses input entries in dialog boxes and command toolbars
Down Arrow	Pan down
Left Arrow	Pan left
Right Arrow	Pan right
Up Arrow	Pan up
Page Down	Cycle to previous sheet
Page Up	Cycle to next sheet
Alt+Space	Opens the window management menu (for GNOME and some KDE installations)
Alt+A	Shows or hides Alignment Aids
Alt+C	Colorize Nets
Alt+F	Selects by providing object name
Alt+I	Rename Instance
Alt+P	Plot Signal
Ctrl+A	Select All
Ctrl+B	Opens the Create Bookmark dialog box
Ctrl+C	Copy to Clipboard
Ctrl+D	Deselect All
Ctrl+E	Returns one level up
Ctrl+F	Shows only the top hierarchy level
Ctrl+G	Selects the view to descend into
Ctrl+H	Opens or closes the Hierarchy Navigator assistant
Ctrl+I	Add Image

Table 35 *Schematic Editor Keypad Shortcut Keys (Continued)*

Shortcut Key	Description
Ctrl+K	Removes all probes from the design
Ctrl+L	Shows Library Manager
Ctrl+M	Opens Bookmark Manager
Ctrl+N	New Window
Ctrl+O	Opens the Open Design dialog box
Ctrl+P	Add Pin
Ctrl+Q	Opens the Design Summary dialog box
Ctrl+R	Redraws the canvas
Ctrl+S	Save As
Ctrl+T	Zooms to fit selected objects
Ctrl+V	Paste
Ctrl+W	Closes the design
Ctrl+X	Cuts selected objects to clipboard
Ctrl+Z	Zoom In
Ctrl+Space	Opens the Console pane
Shift+A	Select by Region
Shift+B	Return to Top
Shift+Tab	Focus input entries in dialog boxes and command toolbars
Shift+C	View Label Attachments
Shift+D	Deselect by Rectangle
Shift+E	Descend in Edit mode
Shift+F	Shows all hierarchy levels
Shift+H	Descend Edit in new window
Shift+I	Renumber Instances
Shift+K	Toggles design read/write mode

Table 35 Schematic Editor Keypad Shortcut Keys (Continued)

Shortcut Key	Description
Shift+L	Add Note
Shift+M	Move
Shift+N	Add Note
Shift+O	Opens the Schematic Display Options dialog box
Shift+P	Add Probe
Shift+R	Flip Horizontal
Shift+S	Opens the Search and Replace dialog box
Shift+T	Shows or hides the Hierarchy Navigator assistant
Shift+U	Redo
Shift+V	Opens PrimeWave EMR
Shift+W	Add Fat Wire
Shift+X	Checks and saves the design
Shift+Y	Opens the Generate CellView from Pin List dialog box
Shift+Z	Zoom Out
Shift+Delete	Ignore Instance
Ctrl-Shift+Delete	Toggle on/off the Show Ignored Instance Information option in the Ignore Instance dialog box
Ctrl+Shift+Space	Opens the Console pane
Ctrl+Shift+A	Adds sheet to the design
Ctrl+Shift+F	Shows or hides the Schematic Object Filter assistant

Schematic Editor Number Pad Shortcut Keys

The following table describes the available number pad (numpad) shortcut keys when using the Custom Compiler (default) or Maestro binding sets. For more information about customizing the binding set, see the *Custom Compiler Environment User Guide*.

For the list of function and main keypad shortcut keys, see [Schematic Editor Keypad Shortcut Keys](#) and [Schematic Editor Keypad Shortcut Keys](#).

Table 36 *Schematic Editor Numpad Shortcut Keys*

Shortcut Key	Description
1	Pan downward to the left
2	Pan down
3	Pan downward to the right
4	Pan left
5	Move the viewport to the cursor location and center it
6	Pan right
7	Pan upward and to the left
8	Pan up
9	Pan upward and to the right